

System Modelling and Design

COMP2111



Johannes Åman Pohjola

+ tutors:

Zhuo (Zoey) Chen

Raphael Douglas Giles

Charran Kethees

Formal

System Modelling and Design

COMP2111



Johannes Åman Pohjola

+ tutors:

Zhuo (Zoey) Chen

Raphael Douglas Giles

Charran Kethees

Formal

System Modelling and Design COMP2111

Credit for the material
also goes to:

Paul Hunter,
Christine Rizkallah,
Liam O'Connor,
and Carroll Morgan



Johannes Åman Pohjola

+ tutors:

Zhuo (Zoey) Chen
Raphael Douglas Giles
Charran Kethees

We'll learn to

model systems in a way that's unambiguous and mathematically precise.

We'll be able to

say what it means for a system to satisfy its specification,
and prove that it does so.

We'll need

a substantial toolbox of discrete math and formal logic.

Don't worry; we'll teach it, not assume it.

Non-examples

September 1981

Transmission Control Protocol
Functional Specification

3.9. Event Processing

The processing depicted in this section is an example of one possible implementation. Other implementations may have slightly different processing sequences, but they should differ from those in this section only in detail, not in substance.

The activity of the TCP can be characterized as responding to events. The events that occur can be cast into three categories: user calls, arriving segments, and timeouts. This section describes the processing the TCP does in response to each of the events. In many cases the processing required depends on the state of the connection.

Events that occur:

User Calls

OPEN
SEND
RECEIVE
CLOSE
ABORT
STATUS

Arriving Segments

SEGMENT ARRIVES

Timeouts

USER TIMEOUT
RETRANSMISSION TIMEOUT
TIME-WAIT TIMEOUT

The model of the TCP/user interface is that user commands receive an immediate return and possibly a delayed response via an event or pseudo interrupt. In the following descriptions, the term "signal" means cause a delayed response.

Error responses are given as character strings. For example, user commands referencing connections that do not exist receive "error: connection not open".

Please note in the following that all arithmetic on sequence numbers, acknowledgment numbers, windows, et cetera, is modulo 2^{32} the size of the sequence number space. Also note that " $=<$ " means less than or equal to (modulo 2^{32}).

Non-examples

This RFC is a specification in English.

Natural language specs tend to have:

- Ambiguities
- Room for interpretation
- Important details in the writer's head absent from actual text.

September 1981

Transmission Control Protocol
Functional Specification

3.9. Event Processing

The processing depicted in this section is an example of one possible implementation. Other implementations may have slightly different processing sequences, but they should differ from those in this section only in detail, not in substance.

The activity of the TCP can be characterized as responding to events. The events that occur can be cast into three categories: user calls, arriving segments, and timeouts. This section describes the processing the TCP does in response to each of the events. In many cases the processing required depends on the state of the connection.

Events that occur:

User Calls

OPEN
SEND
RECEIVE
CLOSE
ABORT
STATUS

Arriving Segments

SEGMENT ARRIVES

Timeouts

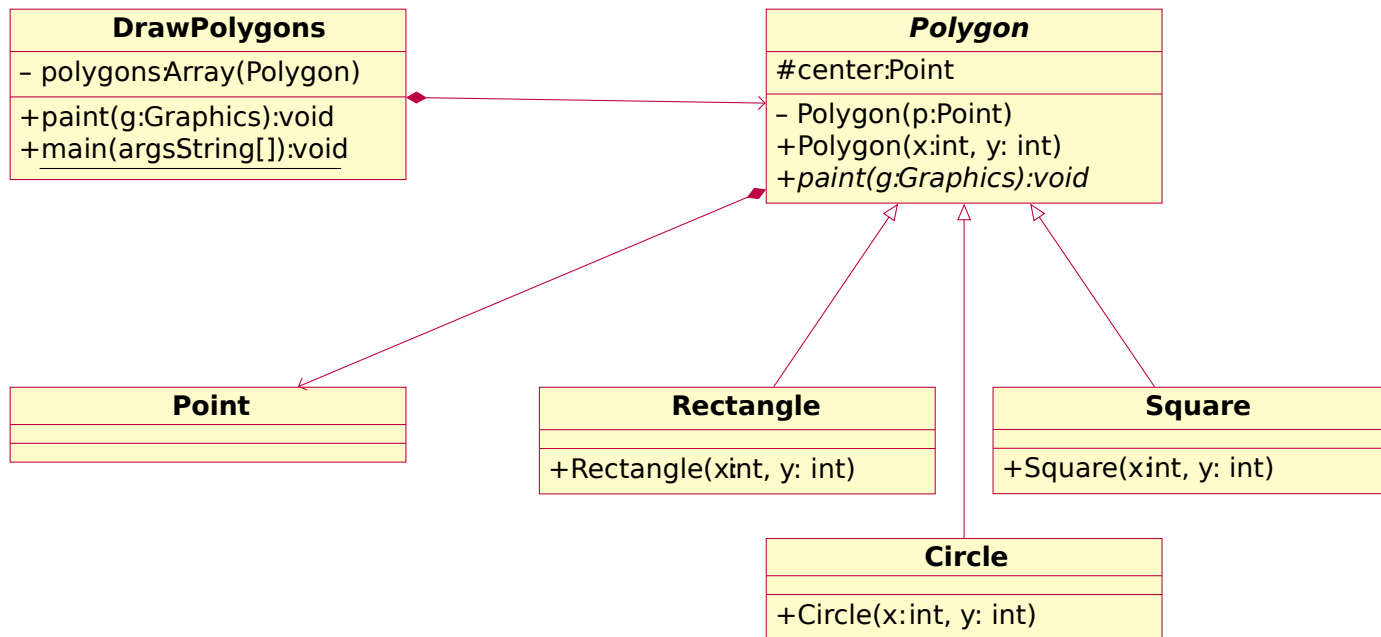
USER TIMEOUT
RETRANSMISSION TIMEOUT
TIME-WAIT TIMEOUT

The model of the TCP/user interface is that user commands receive an immediate return and possibly a delayed response via an event or pseudo interrupt. In the following descriptions, the term "signal" means cause a delayed response.

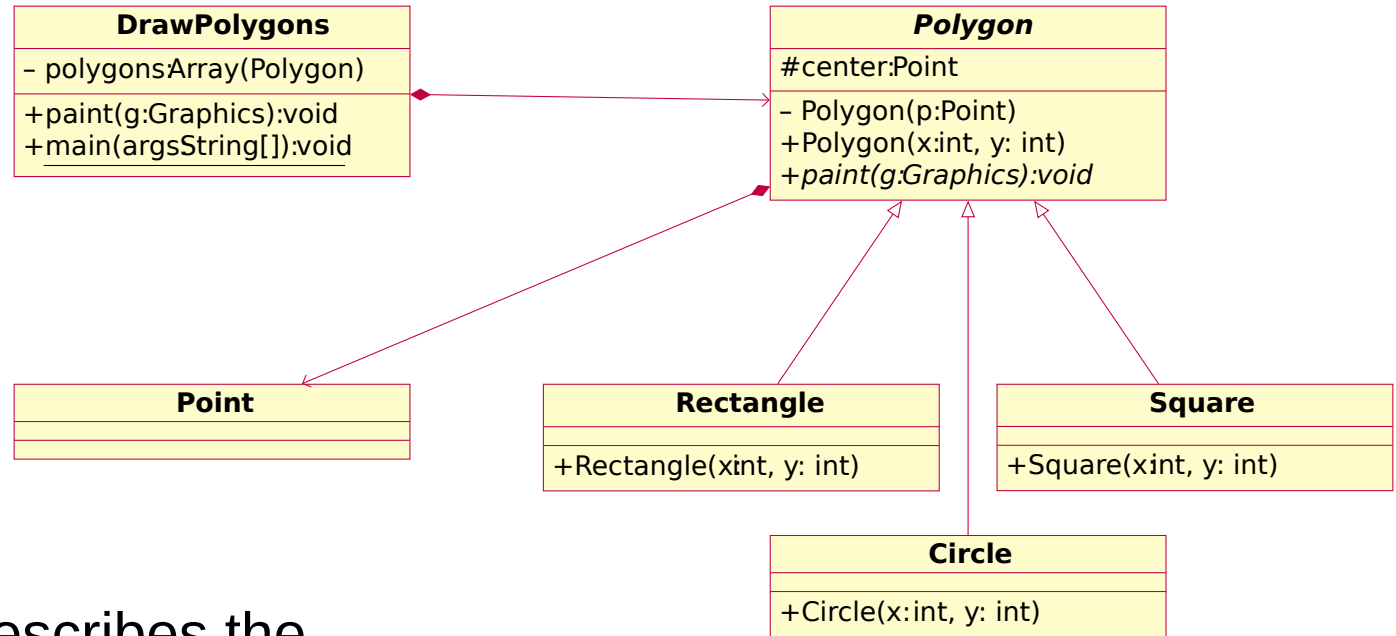
Error responses are given as character strings. For example, user commands referencing connections that do not exist receive "error: connection not open".

Please note in the following that all arithmetic on sequence numbers, acknowledgment numbers, windows, et cetera, is modulo 2^{32} the size of the sequence number space. Also note that " $=<$ " means less than or equal to (modulo 2^{32}).

Non-examples



Non-examples



This UML diagram describes the *structure* of the system, not its behaviour.

Resources

Course website: <http://www.cse.unsw.edu.au/~cs2111>

- Lecture slides
- Assignment instructions
- ...

Ed forum: <https://edstem.org/au/courses/7621>

- General announcements
- Class discussion, announcements
- E-mail cs2111@cse.unsw.edu.au if you haven't been invited!

Moodle: <https://moodle.telt.unsw.edu.au/>

- Lecture recordings
- Weekly quizzes

Examination

- Weekly quizzes: 15 credits total
 - After the lectures of every week (except W6 and W10).
 - Will appear on Moodle.
 - Deadline: Monday 10AM (before start of next week's lectures)
- Three assignments (individual or pair, written): $10 \times 3 = 30$ credits
- Final exam (online, format TBA): 55 credits

Introduction to “Formal” Logic

Start here

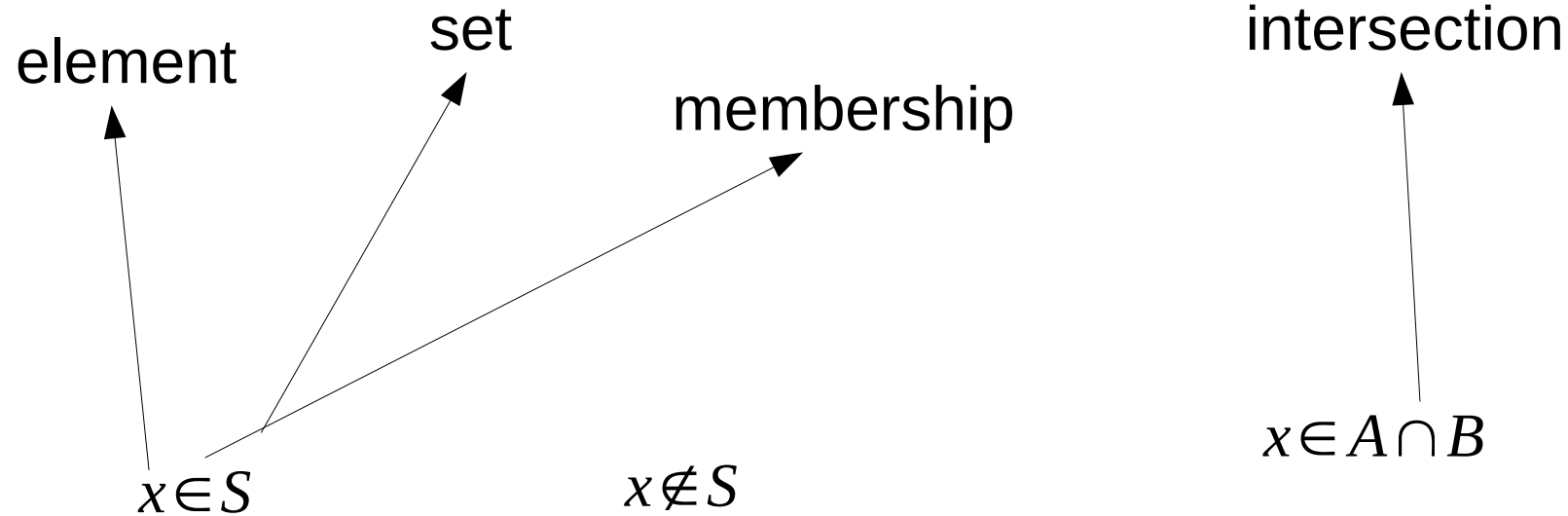


This is a (draft) textbook for
COMP6721 (In-)Formal
Methods by Carroll Morgan

It's on the course website.

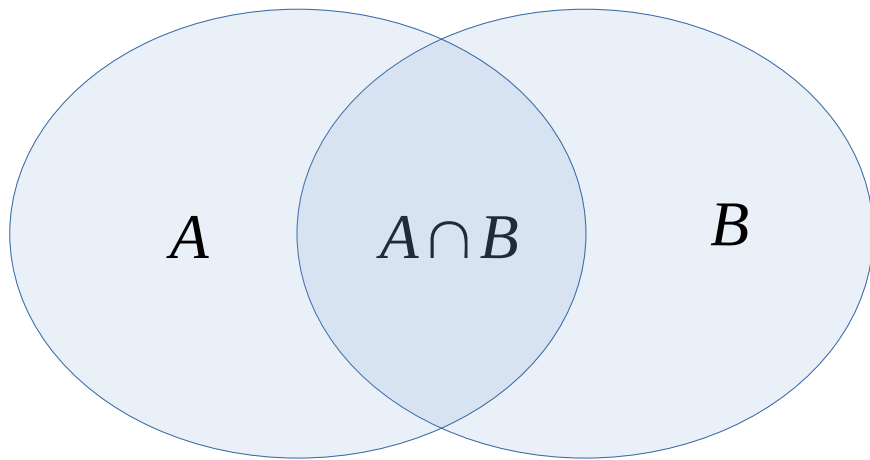
D	The arithmetic of conditions	207
D.1	Introduction and rationale	207
D.2	Why is my program correct?	207
D.3	How do I write my program in the first place?	209
D.4	Calculating with conditions	210
D.5	Simple calculations in logic	212
D.6	Terms	213
D.7	Simple formulae	214
D.8	Propositions, and propositional formulae	215
D.9	Operator precedence	215
D.10	Calculation with logical formulae	217
D.11	<i>Exercises</i> on propositions	218
D.12	Quantifiers	221
D.13	<i>Exercises</i> on quantifiers	222
D.14	(General) formulae	223
E	Some helpful logical identities	225
E.1	Some basic propositional rules	225
E.2	Some basic predicate rules	228
E.3	<i>Exercises</i> on rules for logic	232
E.4	Epilogue on notation and terminology	232

Introduction to “Formal” Logic



defined?

Introduction to “Formal” Logic



$x \in A \cap B$

if and only if

$x \in A$

and $x \in B$

Introduction to “Formal” Logic

intersection $A \cap B$

union $A \cup B$

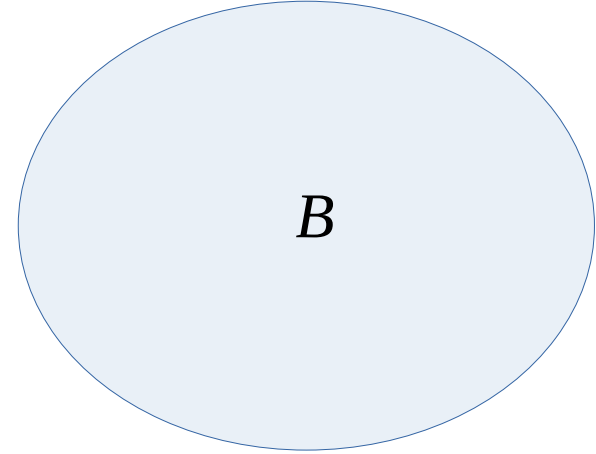
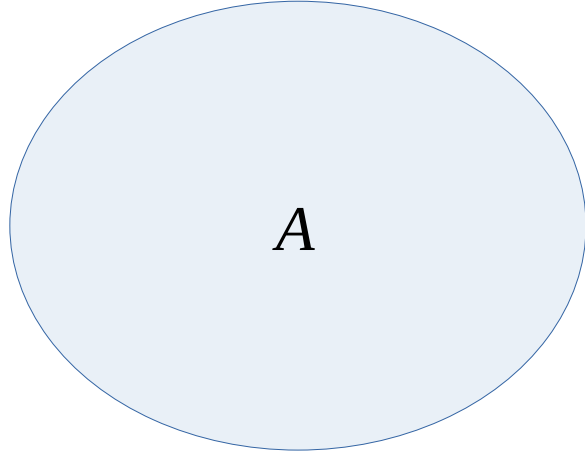
subset $A \subseteq B$

Q: Subset is not like others
in an important way. How?

$x \in A \cup B$ if and only if $x \in A$ or $x \in B$

$A \subseteq B$ if and only if $x \in A$ implies $x \in B$

Introduction to “Formal” Logic



Where is $A \cap B$ now?

Introduction to “Formal” Logic

Let's prove $A \subseteq A \cup B$

Why so pedantic?

$\{y \mid y \subseteq x\}$ The set of all subsets of x

$\{y \mid y \in x\}$ x

Why so pedantic?

$$x \in x$$

You can write it,
But it's never true

Does it make sense to write?

Is it ever true?

$$\{ x \mid x \in x \}$$

Empty set

Why so pedantic?

$$\{ x | x \notin x \}$$

$$y = \{ x | x \notin x \}$$

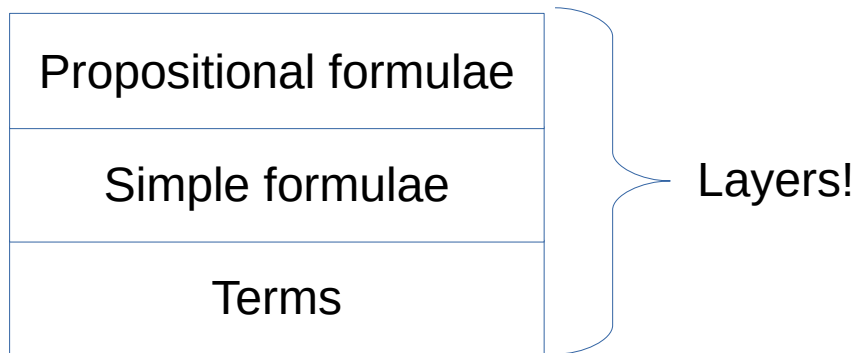
$$y \in y \Rightarrow y \notin y$$

$$y \notin y \Rightarrow y \in y$$

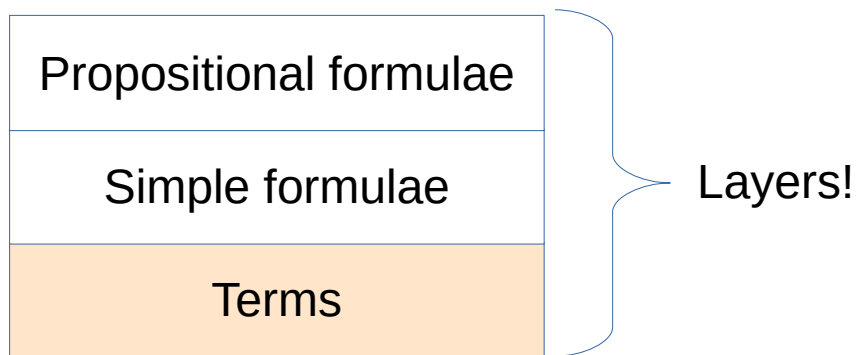
“There is just one point where I
have encountered a difficulty”
- Bertrand Russell

Q: Why does this matter?

The language of logic



The language of logic

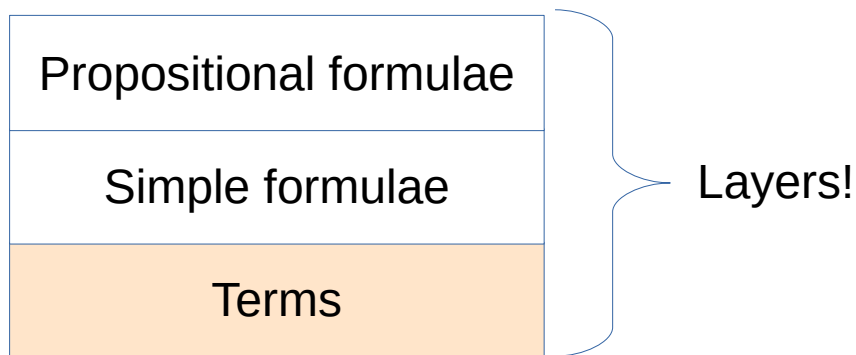


A *term* is either

- (a) a variable, or
- (b) a constant symbol, or
- (c) a function symbol applied to the correct number of other terms.

A function's number of arguments is its *arity*.

The language of logic



A *term* is either
(a) a variable, or
(b) a constant symbol, or
(c) a function symbol applied to the correct number of other terms.

A function's number of arguments is its *arity*.

variables

$x, y, \dots$

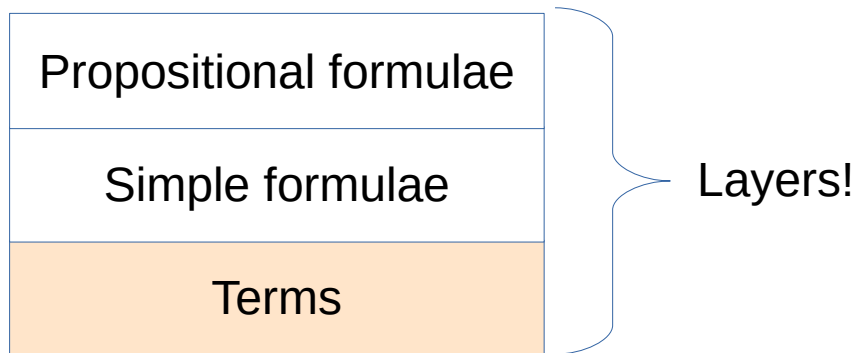
Constants

$1, \{\}, \pi$

functions

element, subset,
 $+, -, !$

The language of logic



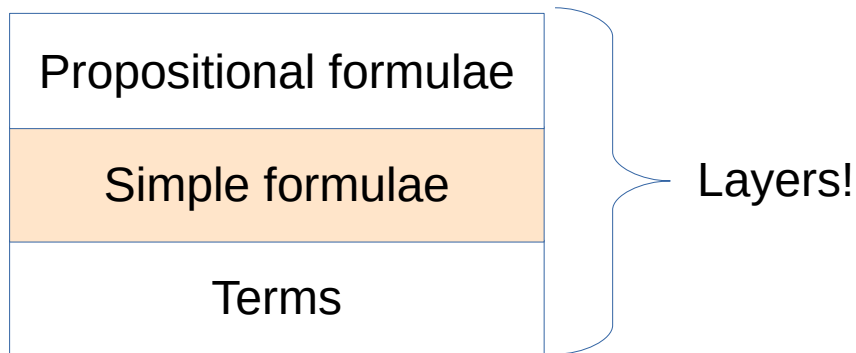
0
 $x+1$
 x
 $\sin(x/2)$

Terms have *values*

$x\ y$
 $x+$
 $x++$

Not terms

The language of logic



A *simple formula* is a predicate symbol applied to the correct number of (term) arguments.

$t < u$

$t = u$

$t \geq u$

$even(t)$

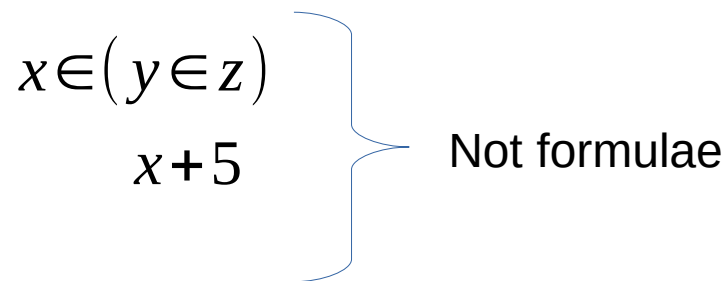
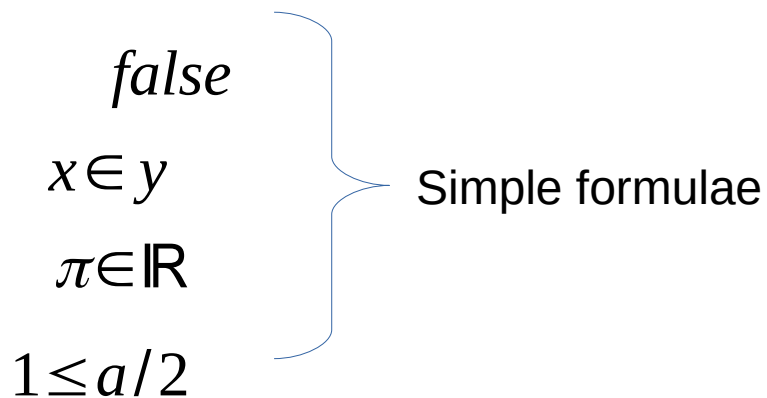
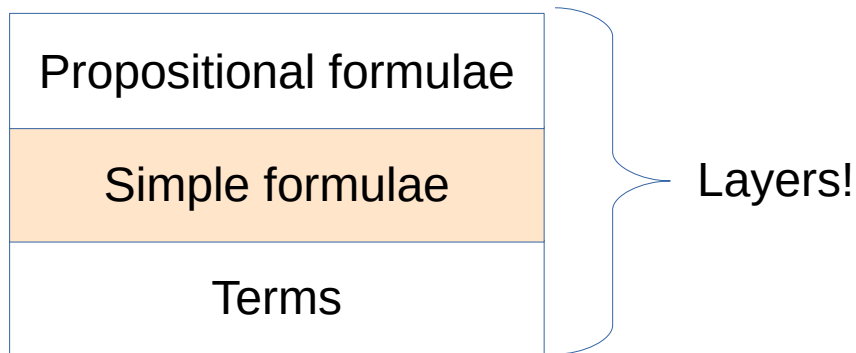
$t \in u$

$false$

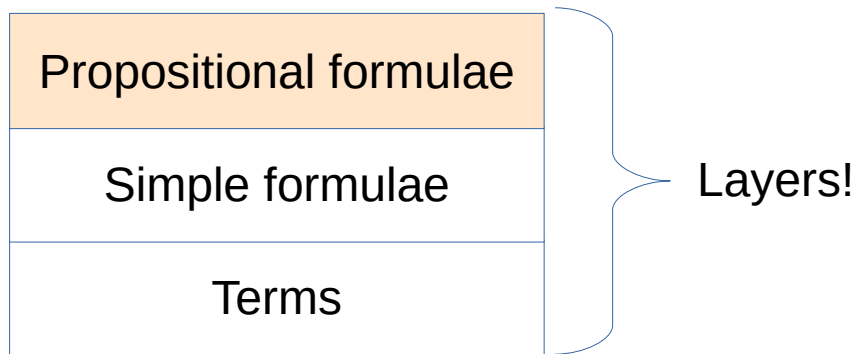
t, u are terms

These can be True
or False

The language of logic



The language of logic

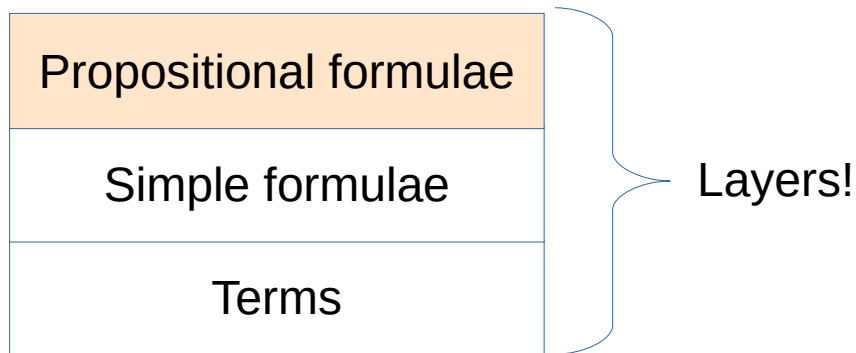


A *propositional formula* is either
(a) a simple formula
(b) a *propositional connective* applied to
the right number of arguments.



$\wedge$ and
 $\vee$ or
 $\neg$ not
 $\rightarrow$ implies
 $\leftrightarrow$ iff

The language of logic



false

$x \in y \wedge x \geq 2$

$n \neq n \leftrightarrow (n = 1 \vee n = 3)$

Q: Is this formula True?

Truth tables

A	B	$A \wedge B$
True	True	True
True	False	False
False	False	False
False	True	False

A	B	$A \rightarrow B$
True	True	True
True	False	False
False	False	True
False	True	True

A	B	$A \vee B$
True	True	True
True	False	True
False	False	False
False	True	True

A	B	$A \leftrightarrow B$
True	True	True
True	False	False
False	False	True
False	True	False

The language of logic: summary

Propositional formulae	Propositional connectives
Simple formulae	Predicate symbols
Terms	Constants, functions, variables

Truth tables define what the propositional connectives mean.

Q: Did the layered, systematic approach help against Russell's paradox?

Calculating with logic

	D The arithmetic of conditions	207
	D.1 Introduction and rationale	207
	D.2 Why is my program correct?	207
	D.3 How do I write my program in the first place?	209
	D.4 Calculating with conditions	210
	D.5 Simple calculations in logic	212
	D.6 Terms	213
	D.7 Simple formulae	214
	D.8 Propositions, and propositional formulae	215
	D.9 Operator precedence	215
Now we're here →	D.10 Calculation with logical formulae	217
	D.11 <i>Exercises</i> on propositions	218
	D.12 Quantifiers	221
	D.13 <i>Exercises</i> on quantifiers	222
	D.14 (General) formulae	223
	E Some helpful logical identities	225
	E.1 Some basic propositional rules	225
	E.2 Some basic predicate rules	228
	E.3 <i>Exercises</i> on rules for logic	232
	E.4 Epilogue on notation and terminology	232

Calculating with logic

Propositional formulae

Are like the conditions in if-then-else, while

Terms

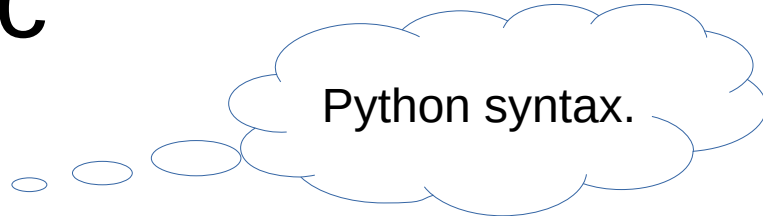
are like the RHS of assignment statements

We can *calculate* with logic as a thinking tool for programming,

...just as we can use mathematical calculation as a thinking tool for physics.

Calculating with logic

```
if l <= m < h:  
    ...  
else:  
    ... #what's true here?
```



Python syntax.

(let's calculate)

Calculating with logic

```
if l <= m < h:  
    thing1  
elif m < l:  
    thing2  
else:  
    thing3
```

```
if l <= m < h:  
    thing1  
elif m >= h:  
    thing3  
else:  
    thing2
```

Are these programs the same?

(let's calculate)

Notes (Week 1 Monday)

These are the notes I wrote during the lecture.

Q: why not RFC:s?

A: it may be ambiguous
it's written in text, you could
misinterpret it

Informally,
a set is a collection of elements

$x \in S$	"x in S"	x is a member (element) of the set S
-----------	----------	---

$x \notin S$ "x not in S" abbreviates $\neg(x \in S)$

- ¬ negation

$x \in A \cap B$ "x in (A and B)" intersection

If A and B are sets,

$A \cap B$ is a set that contains all elements common to A and B .

`{1,2,3}` explicitly enumerated elements of a set

```
"the set containing only the elements 1, 2 and 3"
```

$$\{1, 2, 3\} \cap \{2, 3, 4\} = \{2, 3\}$$

Q: Subset is not like the others

A: It's asymmetrical

$$A \cup B = B \cup A \quad (\text{commutativity})$$

$$A \cap B = B \cap A \quad (\text{commutativity})$$

$$A \subseteq B \quad \neq \quad B \subseteq A \quad (\text{not symmetry})$$

(only holds if A,B are equal)

A: The operator doesn't return a set

The "types" are different:

$$U : \text{set} \times \text{set} \rightarrow \text{set}$$

$$n : \text{set} \times \text{set} \rightarrow \text{set}$$

$\subseteq : \text{set} \times \text{set} \rightarrow \text{bool}$

Union and intersection are *combinators* of sets:

if I have two sets, I can make a new set

Subset inclusion is a *predicate* about sets

makes a truth claim about two sets, but
isn't itself a set.

Q: Brackets? Order of operations:

wrt $\subseteq$ and $\cup, \cap$

$A \subseteq A \cup B$ parses as $A \subseteq (A \cup B)$

the "types" don't make sense otherwise

$(A \subseteq A) \cup B$ "doesn't typecheck"
^^^^ ^
bool set

Theorem union_subset:

$A \subseteq A \cup B$

Proof

By definition of $\subseteq$, it suffices to prove

$x \in A$ implies $x \in A \cup B$

By definition of union, it suffices to prove

$x \in A$ implies $(x \in A \text{ or } x \in B)$

This holds by disjunction introduction.

QED

Disjunction introduction?

A proof rule:

If A holds then $A \vee B$ holds

Set enumeration:

$\{1, 2, 3\}$

Set comprehension:

$\{x \mid P\}$ or $\{x : P\}$ or $\{x . P\}$

Here, P is a predicate (truth claim about x)

$\{x \mid P\}$ "the set of all x such that P holds"

Q: is zero a natural number?

A: yes

$\{x \mid x \in \mathbb{N} \text{ and } x < 5\} = \{0,1,2,3,4\}$

$x = \{1,2\}$

$\{y \mid y \subseteq \{1,2\}\} = \{\{\},$
 $\{1\},$
 $\{2\},$
 $\{1,2\}\}$

powerset

$\{y \mid y \in x\}$ "the set of all elements of x "

We're taking the view that if two sets have the same elements, they're equal. (principle of extensionality)

$x \in x$ "x contains itself as an element"

only possible if x has "infinite depth"

Most mathematicians start courses by saying that the foundations we use are

ZFC (Zermelo–Fraenkel set theory with the axiom of Choice)

ZFC has an axiom called the axiom of foundation, which says that infinite depth sets can't exist

Suppose $y \in y$

By definition, $y \in \{x \mid x \notin x\}$

Which implies that the predicate in the comprehension holds about y itself, i.e.

$y \notin y$

Q:Why is this not just a minor flaw?

A:If there's a contradiction somewhere, it's everywhere

Most logics allow *proof by contradiction*

In order to prove P , it suffices that from $\neg P$, False follows.

If our system allows Russell's paradox, we can prove False from nothing.

Therefore, we can prove everything.

We want to build a logical system that is *consistent*

Consistent: some things are unprovable

Here, we distinguished between constant symbols
and function symbols.

We don't have to:

constant symbols are essentially function symbols
with arity 0

We haven't specified precisely what functions
and constant symbols exist, nor will we.

These are not part of the logic as such,
they are the things the logic is talking about.
Instead, the logic is parameterised on these.

$\forall n. n! = n \leftrightarrow (n=1 \vee n=3)$ "for all n, ..."

If I ask whether this is true

$n! = n \leftrightarrow (n=1 \vee n=3)$

The answer is: it depends on the value of n.

Formulas on their are neither true nor false (in general)

Instead, their truth value depends on a **valuation**
it depends on what values we want to give to the variables

$\{x \mid x \notin x\}$ a set comprehension
is a set defined in terms of a predicate
a term defined in terms of a predicate

$l \leq m < h$

At the else statement, if we reach it,
therefore the following must hold:

$\neg(l \leq m < h)$
 $\equiv$ (by arithmetic)
 $\neg(l \leq m \wedge m < h)$
 $\equiv$ (by de Morgan's laws $\neg(A \wedge B) \equiv \neg A \vee \neg B$)
 $\neg(l \leq m) \vee \neg(m < h)$
 $\equiv$ (by arithmetic)
 $l > m \vee m \geq h$

Does this equivalence hold?

$\neg(l \leq m < h) \wedge m < l$
 $\equiv$
 $\neg(l \leq m < h) \wedge \neg(m \geq h)$

COMP2111 Week 1
Term 1, 2023
Calculating with Logic II

Monday Recap

- 1 Logic is *really simple*: it's like elementary set theory, but with formulae instead of pictures.

Monday Recap

- 1 Logic is *really simple*: it's like elementary set theory, but with formulae instead of pictures.
- 2 Um, actually.. set theory is not that simple.
Can you draw a picture of the empty set?
What is $\{x|x \notin x\}$?
- 3 ...so maybe logic is not as simple as it looks either.

Monday recap

Indeed, logic is not as simple as it looks.

Why do we care?

Monday recap

Indeed, logic is not as simple as it looks.

Why do we care?

Because we need a logic that works, to make sure that our programs work.

Today

D12–D13, then D.1–D.3 from the IFM book.

- State
- Bound and free variables
- Substitution

All these relate to *quantifiers*.

$\forall$ for all

$\exists$ exists

State

States interpret terms (and by extension, formulae)

$n \mapsto 1$	integer
$b \mapsto \text{true}$	boolean
$x \mapsto 3.5$	real
$x \mapsto \{1, 2, 3\}$	set (of integers)
$x \mapsto [1, 2, 3]$	sequence (of integers)

A state is a mapping from *variables* to *values*.

Free and bound variables

```
{  int y = 2;  
   printf("%d,%d.", x, y);  
}
```

Free and bound variables

```
{  int y = 2;  
    printf("%d, %d.", x, y);  
}
```

x, *y* are **variables**.

Free and bound variables

```
{  int y = 2;  
    printf("%d, %d.", x, y);  
}
```

x, y are **variables**.

x is a **free variable**.

Free and bound variables

```
{ int y = 2;  
  printf("%d,%d.", x, y);  
}
```

x, y are **variables**.

x is a **free variable**.

y is a **bound variable**, whose scope is delimited by $\{ \}$

Free and bound variables

```
{  int y = 2;  
    printf("%d,%d.", x, y);  
}
```

← *binding* occurrence of *y*

← *bound* occurrence of *y*

x, y are **variables**.

x is a **free variable**.

y is a **bound variable**, whose scope is delimited by { }

Free and bound variables

```
{ int y = 2;  
  printf("%d,%d.", x, y);  
}
```

← *binding* occurrence of *y*

← *bound* occurrence of *y*

x, *y* are **variables**.

x is a **free variable**.

y is a **bound variable**, whose scope is delimited by { }

The program depends on *x*.

We could rename *y* to anything (except *x*) without changing the program.

Free and bound variables

$$\int_0^1 x^y dy$$

x is a **free variable**.

y is a **bound variable** with scope x^y .

The ~~program~~ integral depends on x .

We could rename y to anything (except x) without changing the integral.

Free and bound variables

$$(\forall y \cdot y^2 \geq x)$$

x is a **free variable**.

y is a **bound variable** with scope delimited by $()$.

The ~~program~~ formula depends on x .

We could rename y to anything (except x) without changing the formula.

Substitution

$$\text{E.60} \quad (\forall x \cdot A) \Rightarrow A[x := E]$$

$$\text{E.61} \quad A[x := E] \Rightarrow (\exists x \cdot A)$$

$A[x := E]$ replaces all *free* occurrences of x in A with E .

Watch out for variable capture!

Quantifier examples

(live)

Checking programs with logic

$$\text{E.60} \quad (\forall x \cdot A) \Rightarrow A[x := E]$$

$$\text{E.61} \quad A[x := E] \Rightarrow (\exists x \cdot A)$$

Idea

// $A[x := E]$

$x = E$

// A

If this is true initially...

...and we execute this line of code...

...then this is true afterwards.

Notice that execution flows forwards, but the substitution flows backwards!

Checking programs with logic

$$\text{E.60} \quad (\forall x \cdot A) \Rightarrow A[x := E]$$

$$\text{E.61} \quad A[x := E] \Rightarrow (\exists x \cdot A)$$

Idea

//x = 5

x = x + 2

//x = 7

If this is true initially...

...and we execute this line of code...

...then this is true afterwards.

Notice that execution flows forwards, but the substitution flows backwards!

Checking programs with logic

Example (Swapping two variables)

// ?

What must be true here...

$t = x$

$x = y$

$y = t$

// $x = Y \wedge y = X$

...to make this true here?

Use the substitution principle from the previous slide repeatedly to check whether this swapping procedure works!

Checking programs with logic

Similar reasoning principles apply to other programming constructs!

Idea

```
//A
if B then
  //A ∧ B
  ⟨code⟩
  //C
else
  //A ∧ ¬B
  ⟨code⟩
  //C
//C
```

Checking programs with logic

Here's while:

Idea

```
//A
while B
  //A ∧ B
  ⟨code⟩
  //A
//A ∧ ¬B
```

A here is a *loop invariant*.

Checking programs with logic

Example: exponentiation
(live)

That was *Hoare logic*. You can use it to synthesise programs from specifications, or to check that your programs are correct.

Main idea: annotate your code with comments that say *what's true here*. Follow the comments backwards using substitution to check them.

Hoare Logic combines *first-order predicate logic* and *program semantics*. We will study their foundations—that is, why they work—later in the course.

Notes (Week 1 Wednesday)

These are the notes I wrote during the lecture.

Write down a formula that characterises what must be true for the programs to differ.

The programs differ iff one program does thing2 while the other does thing3.

Left program is doing thing2

$$\neg(l \leq m < h) \wedge m < l$$

Right program is doing thing3

$$\neg(l \leq m < h) \wedge m \geq h$$

Q: can both these expressions be true at the same time?

$$\begin{aligned} & \neg(l \leq m < h) \wedge m < l \wedge \neg(l \leq m < h) \wedge m \geq h \\ \equiv & \text{(associativity, commutativity)} \\ & \neg(l \leq m < h) \wedge \neg(l \leq m < h) \wedge m < l \wedge m \geq h \\ \equiv & \text{(idempotence of } \wedge \text{ (} P \wedge P \equiv P \text{))} \\ & \neg(l \leq m < h) \wedge m < l \wedge m \geq h \\ \equiv & \text{(fact of arithmetic)} \\ & \neg(l \leq m \wedge m < h) \wedge m < l \wedge m \geq h \\ \equiv & \text{(de Morgan: } \neg(P \wedge Q) \equiv \neg P \vee \neg Q \text{)} \\ & (\neg(l \leq m) \vee \neg(m < h)) \wedge m < l \wedge m \geq h \\ \equiv & \text{(arithmetic)} \\ & (l > m \vee m \geq h) \wedge (m < l \wedge m \geq h) \\ \equiv & \text{(distribution } (P \vee Q) \wedge R \equiv (P \wedge R) \vee (Q \wedge R) \text{)} \\ & (l > m \wedge m < l \wedge m \geq h) \vee (m \geq h \wedge m < l \wedge m \geq h) \\ \equiv & \text{(idempotence + arithmetic)} \\ & (l > m \wedge m \geq h) \vee (l > m \wedge m \geq h) \\ \equiv & \text{(idempotence: } P \vee P \equiv P \text{)} \\ & l > m \wedge m \geq h \\ & \text{(transitivity of inequality)} \\ \Rightarrow & l > h \end{aligned}$$

Conclusion: the programs can differ! But only if $l > h$.

$n! = n \leftrightarrow (n = 1) \vee (n = 3)$
^ on its own, neither true nor false.
^ true in a state where n maps to 1
^ false in state where n maps to 2
...

A state is, formally,
a mathematical function from variable names to values.

Q: since state can change,
is time an implicit extra parameter?

A: no.

A logical formula is interpreted in exactly one state.

In programs, the state may change.

In logic, we'd then talk about different states:
the state before and after an operation for example

$A[x:=E]$ A, but with all *free* occurrences of x
replaced by E.

$$\begin{aligned} & \forall (x + 1 = 2 \wedge (\forall x. x+1=2)) \\ & \Rightarrow (E.60) \\ & (x + 1 = 2 \wedge (\forall x. x+1=2)) [x:=1] \\ & = \\ & (1 + 1 = 2 \wedge (\forall x. x+1=2)) \end{aligned}$$

Q: is . multiplication?

A: No. it's just a separator
 $\forall x. P$ "for all x, P holds"
alternative notations:
 $\forall x \ P$
 $\forall x. \ P$
 $\forall x: \ P$

Q: difference between $\Rightarrow$ $\Rightarrow$?

A: Yes.

$\Rightarrow, \rightarrow$	Implication	a combinator on logical formulas
$\Rightarrow$	Entailment	a truth claim about the relation between two formulas

$P \Rightarrow Q$ in all states, if P holds, then Q must also hold.

```
//(y+1) even
x := y + 1
//x even
```

Scenario:

I dimly recall that in order to swap two variables,
I need to use a third variable as a temp, but I
can't quite remember how to get it right.

Convention here:

lower-case variables are program variables

X,Y are placeholders for constants.

think: X is the constant that holds
the original value of x

goal:

```
//y = Y  $\wedge$  x = X
z = y
//z = Y  $\wedge$  x = X
y = x
//z = Y  $\wedge$  y = X
x = z
//x = Y  $\wedge$  y = X
```

What did I do?

- Wrote a swap program, and proved it correct, at the same time
- I started from the result I wanted (postcondition), and filled in the program backwards, working to the initial state.

You can use logic not just check programs,
but to help write them.

Q: what does // mean

A: I'm pretending // is the comment syntax
of some programming language.
I'm annotating a program with logical formulas
saying what's true in the state at that particular
point of execution.

```
// precondition
prog
// postcondition
```

Read: if we execute prog in a state where precondition is true,
postcondition must be true after prog is done.
// both x,y are even, or both are odd

```
//(x+y) is even  $\vee$  (x is even  $\wedge$   $\neg$ x > y)
//((x+y) is even  $\vee$  x is even)  $\wedge$  (x+y is even  $\vee$   $\neg$ x > y)
// ((x+y) is even  $\wedge$  x > y)  $\vee$  x is even)
 $\wedge$ 
((x+y) is even  $\wedge$  x > y)  $\vee$   $\neg$ x>y
//((x+y) is even  $\wedge$  x > y)  $\vee$  (x is even  $\wedge$   $\neg$ (x>y))
if(x>y) then
  //((x+y) is even  $\wedge$  x > y)  $\vee$ 
  x is even  $\wedge$   $\neg$ (x>y)
   $\wedge$  x>y
  //(x+y) is even  $\wedge$  x>y
  //(x+y) is even
  x := x + y
  //x is even
else
  //((x+y) is even  $\wedge$  x > y)  $\vee$ 
  x is even  $\wedge$   $\neg$ (x>y)
   $\wedge$   $\neg$ (x > y)
  //x is even  $\wedge$   $\neg$ (x>y)
```

```

//x is even
y := x - y
//x is even
//x is even

```

Exponentiation

```

// True
// 1*B^E = B^E
x = 1
// x*B^E = B^E
e = E
// x*B^e = B^E
b = B
// x*b^e = B^E
while(e≠0)
  // x*b^e = B^E ∧ e≠0
  // x*b*b^(e-1) = B^E
  x = x*b
  // x*b^(e-1) = B^E
  e = e - 1;
  // x*b^e = B^E
// x*b^e = B^E ∧ e=0
// x*1 = B^E
// x = B^E

```

Q: We haven't actually proven termination though?

A: Nope! We proved that *if* the program terminates, it exponentiated correctly.
Typically, you divide these sorts of arguments in two bits:

- (partial) correctness: if we terminate, we do the right thing
 - invariant (something that doesn't change)
- termination: we terminate
 - variant (something that changes)

Specifically, we need to find some natural number-valued expression about the state that:

- decrease in value with every iteration
- when it's 0, the loop guard must be false.

Better exponentiation

```

x = 1
e = E
b = B
while(e≠0)
  //x*(b)^e = B^E ∧ e ≠ 0
  //x*(b)^e = B^E
  if(e even)
    //x*(b)^e = B^E ∧ e even

```

```
//x*(b^2)^(e/2) = B^E
e = e/2
//x*(b^2)^e = B^E
b = b*b
//x*b^e = B^E
else
  //x*b^e = B^E ∧ e odd
  x = x*b
  // ..
  e = e - 1
  //x*b^e = B^E
//x*b^e = B^E
// x*b^e = B^E ∧ e=0
// x = B^E
```

IFM Book

The first week's lectures will be based in part of Appendices D–E of Carroll Morgan's book (In-)formal Methods: the Lost Art. I recommend them as background reading.

This (draft) book is the textbook of Carroll's course COMP6721, which is highly recommended as a complement to this course.

Tute (Week 1)

Sets and Propositions

Why does the set fact $A \cap B \subseteq A$ look so much like the propositional fact $\square \wedge \square \rightarrow \square$?

We've seen that intersection $\cap$ between sets corresponds to $\wedge$ between propositions: you are an element of the *intersection* of two sets, say the set of women and the set of people taller than 1.75m, just if you are a woman *and* you are taller than 1.75m. Similarly $\cup$ and $\vee$ correspond.

In the same vein,

- What operator between sets corresponds to $\Rightarrow$ between propositions?
- What operator between sets corresponds to $\rightarrow$ between propositions?
- What operator between sets corresponds to $\neg$ on propositions?

$\Rightarrow$ corresponds to $\subseteq$. The former is a binary relation on propositions, in the same way the latter is a relation between sets. $A \rightarrow B$ can be modelled by $A^c \cup B$. The third corresponds to complement (A^c).

Logical Reasoning

Assuming that the one-place predicates even, odd mean "is an even number", "is an odd number" respectively, write general formulae for the following:

1. Every integer is either even or odd.
2. Every odd natural number is one more than some even natural number.
3. There is an even integer that is not one more than any odd natural number.
4. Zero is the least natural number.
5. There is no least integer.
6. Given any positive real number, there is another real number strictly between it and zero.

1. $\forall i \in \mathbb{Z}. \text{even } i \vee \text{odd } i$
2. $\forall m \in \mathbb{N}. \text{odd } m \rightarrow \exists n \in \mathbb{N}. \text{even } n \wedge m = n + 1$
3. $\exists i \in \mathbb{Z}. \text{even } i \wedge \neg \exists n \in \mathbb{N}. \text{odd } n \wedge i = n + 1$
4. $\forall n \in \mathbb{N}. 0 \leq n$
5. $\neg \exists i \in \mathbb{Z}. \forall j \in \mathbb{Z}. i \leq j$
6. $\forall r \in \mathbb{R}. r > 0 \rightarrow \exists s \in \mathbb{R}. 0 < s < r$

Quantifiers

Question 1

Recall that $\exists x. \square$ means "there is *at least* one x such that $\square$ ". Write another formula that means "there is *at most* one x such that $\square$ ".

$$\exists y. \forall x. \square \rightarrow x = y.$$

Question 2

Now write a formula that means "there is *exactly* one x such that $\square$ ".

$$\exists y. \forall x. \square \leftrightarrow x = y.$$

Proofs

Question 1

Prove this, using the logical reasoning rules from Appendix E of [the IFM book](#):

$$(\exists x. (\square \rightarrow \square) \wedge (\neg \square \rightarrow \square)) \equiv (\exists x. \square \wedge \square) \vee (\exists x. \neg \square \wedge \square).$$

$$\begin{aligned} & \exists x. (\square \rightarrow \square) \wedge (\neg \square \rightarrow \square) && \text{E.42} \\ \equiv & \exists x. \square \wedge \square \vee \neg \square \wedge \square && \text{E.72} \\ \equiv & (\exists x. \square \wedge \square) \vee (\exists x. \neg \square \wedge \square) \end{aligned}$$

Question 2

Suppose $\square$ contains no free x . Prove this:

$$\exists x. (\square \rightarrow \square) \wedge (\neg \square \rightarrow \square) \equiv (\square \rightarrow \exists x. \square) \wedge (\neg \square \rightarrow \exists x. \square)$$

Hint: recall the previous question.

$\exists x. (\Box \rightarrow \Box) \wedge (\neg \Box \rightarrow \Box)$	Prev. Question
$\equiv (\exists x. \Box \wedge \Box) \vee (\exists x. \neg \Box \wedge \Box)$	E.86
$\equiv \Box \wedge (\exists x. \Box) \vee \neg \Box \wedge (\exists x. \Box)$	E.42
$\equiv (\Box \rightarrow \exists x. \Box) \wedge (\neg \Box \rightarrow \exists x. \Box)$	

Question 3

Show that

$$\exists x, y. x \neq y \equiv \forall x. \exists y. x \neq y$$

Hint: To show $\Box \equiv \Box$, show $\Box \Rightarrow \Box$ and $\Box \Rightarrow \Box$.

$\exists x, y. x \neq y$	E.80
$\equiv \forall z. \exists x, y. x \neq y$	E.13, E.20
$\equiv \forall z. \exists x, y. (x = z \vee x \neq z) \wedge x \neq y$	E.7
$\equiv \forall z. \exists x, y. x = z \wedge x \neq y \vee x \neq z \wedge x \neq y$	Properties of =
$\Rightarrow \forall z. \exists x, y. y \neq z \vee x \neq z$	E.72
$\equiv \forall z. (\exists x, y. y \neq z) \vee (\exists x, y. x \neq z)$	E.81
$\equiv \forall z. (\exists y. y \neq z) \vee (\exists x. x \neq z)$	E.90, E.91
$\equiv \forall x. (\exists y. y \neq x) \vee (\exists y. y \neq x)$	E.1
$\equiv \forall x. \exists y. x \neq y$	E.74
$\Rightarrow \exists x, y. x \neq y$	

Video game proofs

Head over to [the incredible proof machine](#) and play the game. By trial and error, you'll pick up the basics of how to do *natural deduction* proofs, which is something we'll return to later in the course. You don't need to understand the theory behind what you're doing at this point; playing around with it is enough for now.

COMP2111 Week 2
Term 1, 2023
Discrete Mathematics Recap I

Summary of topics

- Sets
- Formal languages
- Relations
- Functions
- Propositional Logic

Summary of topics

- Sets
- Formal languages
- Relations
- Functions
- Propositional Logic

Sets

A set is defined by the collection of its elements.

Sets are typically described by:

(a) Explicit enumeration of their elements

$$\begin{aligned} S_1 &= \{a, b, c\} = \{a, a, b, b, b, c\} \\ &= \{b, c, a\} = \dots \quad \text{three elements} \end{aligned}$$

$$S_2 = \{a, \{a\}\} \quad \text{two elements}$$

$$S_3 = \{a, b, \{a, b\}\} \quad \text{three elements}$$

$$S_4 = \{\} \quad \text{zero elements}$$

$$S_5 = \{\{\{\}\}\} \quad \text{one element}$$

$$S_6 = \{\{\}, \{\{\}\}\} \quad \text{two elements}$$

(b) Specifying the properties their elements must satisfy; the elements are taken from some 'universal' domain, $\mathcal{U}$. A typical description involves a **logical** property $P(x)$

$$S = \{ x : x \in \mathcal{U} \text{ and } P(x) \} = \{ x \in \mathcal{U} : P(x) \}$$

We distinguish between an element and the set comprising this single element. Thus always $a \neq \{a\}$.

Set $\{\}$ is empty (no elements);

set $\{\{\}\}$ is nonempty — it has one element.

There is only one empty set; only one set consisting of a single a ; only one set of all natural numbers.

(c) Constructions from other sets (already defined)

- Union, intersection, set difference, symmetric difference, complement
- **Power set** $\text{Pow}(X) = \{ A : A \subseteq X \}$
- Cartesian product (below)
- Empty set $\emptyset$
 $\emptyset \subseteq X$ for all sets X .

$S \subseteq T$ S is a **subset** of T ; includes the case of $T \subseteq T$
 $S \subset T$ a **proper** subset: $S \subseteq T$ and $S \neq T$

NB

Element and subset are two different concepts

$$a \in \{a, b\}, \quad a \not\subseteq \{a, b\}; \quad \{a\} \subseteq \{a, b\}, \quad \{a\} \notin \{a, b\}$$

Cardinality

Number of elements in a set X (various notations):

$$|X| = \#(X) = \text{card}(X)$$

Fact

Always $|\text{Pow}(X)| = 2^{|X|}$ (for finite X)

$$\begin{array}{lll} |\emptyset| = 0 & \text{Pow}(\emptyset) = \{\emptyset\} & |\text{Pow}(\emptyset)| = 1 \\ \text{Pow}(\text{Pow}(\emptyset)) = \{\emptyset, \{\emptyset\}\} & & |\text{Pow}(\text{Pow}(\emptyset))| = 2 \quad \dots \end{array}$$

$$|\{a\}| = 1 \quad \text{Pow}(\{a\}) = \{\emptyset, \{a\}\} \quad |\text{Pow}(\{a\})| = 2 \quad \dots$$

Sets of Numbers

Natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$

Positive integers $\{1, 2, \dots\}$

Common notation: $\mathbb{N}^+ = \mathbb{N}_{>0} = \mathbb{Z}_{>0} = \mathbb{N} \setminus \{0\}$

Integers $\mathbb{Z} = \{\dots, -1, 0, 1, 2, \dots\}$

Rational numbers $\mathbb{Q} \subseteq \left\{ \frac{m}{n} : m, n \in \mathbb{Z}, n \neq 0 \right\}$

Real numbers $\mathbb{R}$

Intervals of numbers (applies to any type)

$$[a, b] = \{x | a \leq x \leq b\}; \quad (a, b) = \{x | a < x < b\}$$

$$[a, b] \supseteq [a, b), (a, b] \supseteq (a, b)$$

NB

$(a, a) = (a, a] = [a, a] = \emptyset$; however $[a, a] = \{a\}$.

Intervals of $\mathbb{N}, \mathbb{Z}$ are finite: if $m \leq n$

$$[m, n] = \{m, m+1, \dots, n\} \quad |[m, n]| = n - m + 1$$

Set Operations

Union $A \cup B$; Intersection $A \cap B$

There is a correspondence between set operations and logical operators (to be discussed later)

We say that A, B are **disjoint** if $A \cap B = \emptyset$

NB

- $A \cup B = B$ if and only if $A \subseteq B$
- $A \cap B = B$ if and only if $A \supseteq B$

Other set operations

- $A \setminus B$ — **difference**, set difference, relative complement
It corresponds (logically) to a but not b
- $A \oplus B$ — **symmetric difference**

$$A \oplus B \stackrel{\text{def}}{=} (A \setminus B) \cup (B \setminus A)$$

It corresponds to a and not b or b and not a ; also known as **xor (exclusive or)**

- A^c — set **complement** w.r.t. the 'universe' $\mathcal{U}$
It corresponds to 'not a '

Laws of Set Operations

Commutativity

$$A \cup B = B \cup A$$

$$A \cap B = B \cap A$$

Associativity

$$(A \cup B) \cup C = A \cup (B \cup C)$$

$$(A \cap B) \cap C = A \cap (B \cap C)$$

Distribution

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

Identity

$$A \cup \emptyset = A$$

$$A \cap \mathcal{U} = A$$

Complementation

$$A \cup (A^c) = \mathcal{U}$$

$$A \cap (A^c) = \emptyset$$

Other useful set laws

The following are all derivable from the previous 10 laws.

Idempotence

$$A \cap A = A$$

$$A \cup A = A$$

Double complementation

$$(A^c)^c = A$$

Annihilation

$$A \cap \emptyset = \emptyset$$

$$A \cup \mathcal{U} = \mathcal{U}$$

de Morgan's Laws

$$(A \cap B)^c = A^c \cup B^c$$

$$(A \cup B)^c = A^c \cap B^c$$

Example (Idempotence of $\cup$)

$$A = A \cup \emptyset$$

(Identity)

Example (Idempotence of $\cup$)

$$\begin{aligned} A &= A \cup \emptyset && \text{(Identity)} \\ &= A \cup (A \cap A^c) && \text{(Complementation)} \end{aligned}$$

Example (Idempotence of $\cup$)

$$\begin{aligned} A &= A \cup \emptyset && \text{(Identity)} \\ &= A \cup (A \cap A^c) && \text{(Complementation)} \\ &= (A \cup A) \cap (A \cup A^c) && \text{(Distributivity)} \end{aligned}$$

Example (Idempotence of $\cup$)

$$\begin{aligned} A &= A \cup \emptyset && \text{(Identity)} \\ &= A \cup (A \cap A^c) && \text{(Complementation)} \\ &= (A \cup A) \cap (A \cup A^c) && \text{(Distributivity)} \\ &= (A \cup A) \cap \mathcal{U} && \text{(Complementation)} \end{aligned}$$

Example (Idempotence of $\cup$)

$$\begin{aligned} A &= A \cup \emptyset && \text{(Identity)} \\ &= A \cup (A \cap A^c) && \text{(Complementation)} \\ &= (A \cup A) \cap (A \cup A^c) && \text{(Distributivity)} \\ &= (A \cup A) \cap \mathcal{U} && \text{(Complementation)} \\ &= (A \cup A) && \text{(Identity)} \end{aligned}$$

A useful result

Definition

If A is a set defined using $\cap$, $\cup$, $\emptyset$ and $\mathcal{U}$, then $\text{dual}(A)$ is the expression obtained by replacing $\cap$ with $\cup$ (and vice-versa) and $\emptyset$ with $\mathcal{U}$ (and vice-versa).

Theorem (Principle of Duality)

If you can prove $A_1 = A_2$ using the Laws of Set Operations then you can prove $\text{dual}(A_1) = \text{dual}(A_2)$

Example

Absorption law: $A \cup (A \cap B) = A$

Dual: $A \cap (A \cup B) = A$

Application (Idempotence of $\cap$)

Recall Idempotence of $\cup$:

$$\begin{aligned} A &= A \cup \emptyset && \text{(Identity)} \\ &= A \cup (A \cap A^c) && \text{(Complementation)} \\ &= (A \cup A) \cap (A \cup A^c) && \text{(Distributivity)} \\ &= (A \cup A) \cap \mathcal{U} && \text{(Complementation)} \\ &= (A \cup A) && \text{(Identity)} \end{aligned}$$

Application (Idempotence of $\cap$)

Invoke the dual laws!

$$\begin{aligned} A &= A \cap \mathcal{U} && \text{(Identity)} \\ &= A \cap (A \cup A^c) && \text{(Complementation)} \\ &= (A \cap A) \cup (A \cap A^c) && \text{(Distributivity)} \\ &= (A \cap A) \cup \emptyset && \text{(Complementation)} \\ &= (A \cap A) && \text{(Identity)} \end{aligned}$$

Cartesian Product

$S \times T \stackrel{\text{def}}{=} \{ (s, t) : s \in S, t \in T \}$ where (s, t) is an **ordered** pair

$\times_{i=1}^n S_i \stackrel{\text{def}}{=} \{ (s_1, \dots, s_n) : s_k \in S_k, \text{ for } 1 \leq k \leq n \}$

$S^2 = S \times S, \quad S^3 = S \times S \times S, \dots, \quad S^n = \times_{i=1}^n S, \dots$

$\emptyset \times S = \emptyset$, for every S

$|S \times T| = |S| \cdot |T|, \quad |\times_{i=1}^n S_i| = \prod_{i=1}^n |S_i|$

Summary of topics

- Sets
- Formal languages
- Relations
- Functions
- Propositional Logic

Formal Languages

Σ — **alphabet**, a finite, nonempty set

Examples (of various alphabets and their intended uses)

$\Sigma = \{a, b, \dots, z\}$ for single words (in lower case)

$\Sigma = \{\sqcup, -, a, b, \dots, z\}$ for composite terms

$\Sigma = \{0, 1\}$ for binary integers

$\Sigma = \{0, 1, \dots, 9\}$ for decimal integers

The above cases all have a natural ordering; a formal language does not need this.

Definition

word — any finite string of symbols from Σ

empty word — λ (sometimes ϵ)

Example

$w = aba$, $w = 01101 \dots 1$, etc.

$\text{length}(w)$ — # of symbols in w

$\text{length}(aaa) = 3$, $\text{length}(\lambda) = 0$

The only operation on words (discussed here) is **concatenation**, written as juxtaposition vw , wvw , abw , wbv , $\dots$

NB

$\lambda w = w = w\lambda$

$\text{length}(vw) = \text{length}(v) + \text{length}(w)$

Notation: Σ^k — set of all words of length k

We often identify $\Sigma^0 = \{\lambda\}$, $\Sigma^1 = \Sigma$

Σ^* — set of all words (of all [finite] lengths)

Σ^+ — set of all nonempty words (of any positive length)

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots; \quad \Sigma^{\leq n} = \bigcup_{i=0}^n \Sigma^i$$

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \dots = \Sigma^* \setminus \{\lambda\}$$

A **language** is a subset of Σ^* . Typically, only the subsets that can be formed (or described) according to certain rules are of interest. Such a collection of ‘descriptive/formative’ rules is called a **grammar**.

Examples: Programming languages, Database query languages

Example (Decimal numbers)

The “language” of all numbers written in decimal to at most two decimal places can be described as follows:

- $\Sigma = \{-, ., 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- Consider all words $w \in \Sigma^*$ which satisfy the following:
 - w contains at most one instance of $-$, and if it contains an instance then it is the first symbol.
 - w contains at most one instance of $.$, and if it contains an instance then it is preceded by a symbol in $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, and followed by either one or two symbols in that set.
 - w contains at least one symbol from $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

NB

According to these rules 123, 123.0 and 123.00 are all (distinct) words in this language.

Example (HTML documents)

Take $\Sigma = \{ \text{“<html>”, “</html>”, “<head>”, “</head>”, “<body>”, ...} \}$.

The (language of) **valid HTML documents** is loosely described as follows:

- Starts with “<html>”
- Next symbol is “<head>”
- Followed by zero or more symbols from the set of HeadItems (defined elsewhere)
- Followed by “</head>”
- Followed by “<body>”
- Followed by zero or more symbols from the set of BodyItems (defined elsewhere)
- Followed by “</body>”
- Followed by “</html>”

Set Operations for Languages

Languages are sets, so the standard set operations ($\cap$, $\cup$, $\setminus$, $\oplus$, etc) can be used to build new languages.

Two set operations that apply to languages uniquely:

- Concatenation (written as juxtaposition):
 $XY = \{xy : x \in X \text{ and } y \in Y\}$
- Kleene star: X^* is the set of words that are made up by concatenating 0 or more words in X

Set Operations for Languages

Example

Let $A = \{aa, bb\}$ and $B = \{\lambda, c\}$ be languages over $\Sigma = \{a, b, c\}$.

- $A \cup B = \{\lambda, c, aa, bb\}$
- $AB = \{aa, bb, aac, bbc\}$
- $AA = \{aaaa, aabb, bbaa, bbbb\}$

Set Operations for Languages

Example

Let $A = \{aa, bb\}$ and $B = \{\lambda, c\}$ be languages over $\Sigma = \{a, b, c\}$.

- $A \cup B = \{\lambda, c, aa, bb\}$
- $AB = \{aa, bb, aac, bbc\}$
- $AA = \{aaaa, aabb, bbaa, bbbb\}$
- $A^* = \{\lambda, aa, bb, aaaa, aabb, bbaa, bbbb, aaaaaa, \dots\}$
- $B^* = \{\lambda, c, cc, ccc, cccc, \dots\}$

Set Operations for Languages

Example

Let $A = \{aa, bb\}$ and $B = \{\lambda, c\}$ be languages over $\Sigma = \{a, b, c\}$.

- $A \cup B = \{\lambda, c, aa, bb\}$
- $AB = \{aa, bb, aac, bbc\}$
- $AA = \{aaaa, aabb, bbaa, bbbb\}$
- $A^* = \{\lambda, aa, bb, aaaa, aabb, bbaa, bbbb, aaaaaa, \dots\}$
- $B^* = \{\lambda, c, cc, ccc, cccc, \dots\}$
- $\{\lambda\}^* =$

Set Operations for Languages

Example

Let $A = \{aa, bb\}$ and $B = \{\lambda, c\}$ be languages over $\Sigma = \{a, b, c\}$.

- $A \cup B = \{\lambda, c, aa, bb\}$
- $AB = \{aa, bb, aac, bbc\}$
- $AA = \{aaaa, aabb, bbaa, bbbb\}$
- $A^* = \{\lambda, aa, bb, aaaa, aabb, bbaa, bbbb, aaaaaa, \dots\}$
- $B^* = \{\lambda, c, cc, ccc, cccc, \dots\}$
- $\{\lambda\}^* = \{\lambda\}$

Set Operations for Languages

Example

Let $A = \{aa, bb\}$ and $B = \{\lambda, c\}$ be languages over $\Sigma = \{a, b, c\}$.

- $A \cup B = \{\lambda, c, aa, bb\}$
- $AB = \{aa, bb, aac, bbc\}$
- $AA = \{aaaa, aabb, bbaa, bbbb\}$
- $A^* = \{\lambda, aa, bb, aaaa, aabb, bbaa, bbbb, aaaaaa, \dots\}$
- $B^* = \{\lambda, c, cc, ccc, cccc, \dots\}$
- $\{\lambda\}^* = \{\lambda\}$
- $\emptyset^* =$

Set Operations for Languages

Example

Let $A = \{aa, bb\}$ and $B = \{\lambda, c\}$ be languages over $\Sigma = \{a, b, c\}$.

- $A \cup B = \{\lambda, c, aa, bb\}$
- $AB = \{aa, bb, aac, bbc\}$
- $AA = \{aaaa, aabb, bbaa, bbbb\}$
- $A^* = \{\lambda, aa, bb, aaaa, aabb, bbaa, bbbb, aaaaaa, \dots\}$
- $B^* = \{\lambda, c, cc, ccc, cccc, \dots\}$
- $\{\lambda\}^* = \{\lambda\}$
- $\emptyset^* = \{\lambda\}$

Summary of topics

- Sets
- Formal languages
- **Relations**
- Functions
- Propositional Logic

Relations and Functions

Relations capture the idea that objects are *related* (well, duh)

- $\leq$ (“less than”)
- “is a Facebook friend of”
- “has a different hair colour than”

Functions capture the idea of transforming *inputs* into *outputs*.

In general, functions and relations formalise the concept of interaction among objects from various domains; however, there must be a specified domain for each type of object.

Applications

Relations and functions are ubiquitous in Computer Science

- Databases are collections of relations
- Common data structures (e.g. graphs) are relations
- Any ordering is a relation
- Functions, procedures and programs are relations between their inputs and outputs

Relations are therefore used in most problem specifications and to describe formal properties of programs.

For this reason, studying relations and their properties helps with formalisation, implementation and verification of programs.

Relations

An **n-ary relation** is a subset of the Cartesian product of n sets.

$$R \subseteq S_1 \times S_2 \times \dots \times S_n$$

$$x \in R \rightarrow x = (x_1, x_2, \dots, x_n) \text{ where each } x_i \in S_i$$

If $n = 2$ we have a **binary** relation $R \subseteq S \times T$.

(mostly we consider binary relations)

equivalent notations: $(x_1, x_2, \dots, x_n) \in R \iff R(x_1, x_2, \dots, x_n)$

for binary relations: $(x, y) \in R \iff R(x, y) \iff xRy$.

Examples

- Equality: $=$
- Inequality: $\leq, \geq, <, >, \neq$
- Divides relation: $|$ (recall $m|n$ if $n = km$ for some $k \in \mathbb{Z}$)
- Element of: $\in$
- Subset, superset: $\subseteq, \subset, \supseteq, \supset$
- Size functions (sort of): $|\cdot|, \text{length}(\cdot)$

Database Examples

Example (Course enrolments)

S = set of CSE students

C = set of CSE courses

E = enrolments = $\{ (s, c) : s \text{ takes } c \}$

$$E \subseteq S \times C$$

In practice, almost always there are various 'onto' (nonemptiness) and 1-1 (uniqueness) constraints on database relations.

Example (Class schedule)

C = CSE courses

T = starting time (hour & day)

R = lecture rooms

S = schedule =

$$\{ (c, t, r) : c \text{ is at } t \text{ in } r \} \subseteq C \times T \times R$$

Example (sport stats)

$$R \subseteq \text{competitions} \times \text{results} \times \text{years} \times \text{athletes}$$

n -ary Relations

Relations can be defined linking $k \geq 1$ domains $D_1, \dots, D_k$ simultaneously.

In database situations one also allows for *unary* ($n = 1$) relations. Most common are **binary** relations

$$R \subseteq S \times T; \quad R = \{(s, t) : \text{“some property that links } s, t\text{”}\}$$

For related s, t we can write $(s, t) \in R$ or sRt ; for unrelated items either $(s, t) \notin R$ or $s \not R t$.

R can be defined by

- explicit enumeration of interrelated k -tuples (ordered pairs in case of binary relations);
- properties that identify relevant tuples within the entire $D_1 \times D_2 \times \dots \times D_k$;
- construction from other relations.

Relation R as Correspondence From S to T

Given $R \subseteq S \times T$, $A \subseteq S$, and $B \subseteq T$.

- $R(A) \stackrel{\text{def}}{=} \{t \in T : (s, t) \in R \text{ for some } s \in A\}$
- Converse relation $R^{\leftarrow} \subseteq T \times S$:

$$R^{\leftarrow} \stackrel{\text{def}}{=} \{(t, s) \in T \times S : (s, t) \in R\}$$

- $R^{\leftarrow}(B) = \{s \in S : (s, t) \in R \text{ for some } t \in B\}$

Observe that $(R^{\leftarrow})^{\leftarrow} = R$.

Binary Relations

A binary relation, say $R \subseteq S \times T$, can be presented as a matrix with rows enumerated by (the elements of) S and the columns by T ; eg. for $S = \{s_1, s_2, s_3\}$ and $T = \{t_1, t_2, t_3, t_4\}$ we may have

$$\begin{bmatrix} \bullet & \circ & \bullet & \bullet \\ \circ & \bullet & \bullet & \bullet \\ \bullet & \bullet & \circ & \circ \end{bmatrix}$$

Relations on a Single Domain

Particularly important are binary relationships between the elements of the same set. We say that ' R is a relation on S ' if

$$R \subseteq S \times S$$

Such relations can be visualized as a directed graph:

- Vertices: Elements of S
- Edges: Elements of R

Example

$$S = \{1, 2, 3\}$$

$$R = \{(1, 2), (2, 3), (3, 2)\}$$

As a matrix:

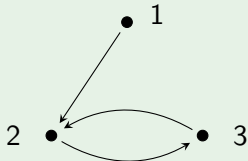
$$\begin{pmatrix} \circ & \bullet & \circ \\ \circ & \circ & \bullet \\ \circ & \bullet & \circ \end{pmatrix}$$

Example

$$S = \{1, 2, 3\}$$

$$R = \{(1, 2), (2, 3), (3, 2)\}$$

As a graph:



Special (Trivial) Relations

(all w.r.t. set S)

Identity (diagonal, equality)

$$E = \{ (x, x) : x \in S \}$$

Empty $\emptyset$

Universal $\mathcal{U} = S \times S$

Important Properties of Binary Relations $R \subseteq S \times S$

- (R) reflexive $(x, x) \in R$ $\forall x \in S$
- (AR) antireflexive $(x, x) \notin R$ $\forall x \in S$
- (S) symmetric $(x, y) \in R \rightarrow (y, x) \in R$ $\forall x, y \in S$
- (AS) antisymmetric $(x, y), (y, x) \in R \rightarrow x = y$ $\forall x, y \in S$
- (T) transitive $(x, y), (y, z) \in R \rightarrow (x, z) \in R$ $\forall x, y, z \in S$

NB

An object, notion etc. is considered to satisfy a property if none of its instances violates any defining statement of that property.

Examples

(R) reflexive $(x, x) \in R$ for all $x \in S$ $\begin{bmatrix} \bullet & \bullet & \circ \\ \circ & \bullet & \circ \\ \bullet & \circ & \bullet \end{bmatrix}$

(AR) antireflexive $(x, x) \notin R$ $\begin{bmatrix} \circ & \bullet & \bullet \\ \circ & \circ & \circ \\ \bullet & \circ & \circ \end{bmatrix}$

(S) symmetric $(x, y) \in R \rightarrow (y, x) \in R$ $\begin{bmatrix} \bullet & \circ & \bullet \\ \circ & \circ & \bullet \\ \bullet & \bullet & \circ \end{bmatrix}$

(AS) antisymmetric $(x, y), (y, x) \in R \rightarrow x = y$
 $\begin{bmatrix} \bullet & \bullet & \circ \\ \circ & \circ & \bullet \\ \bullet & \circ & \circ \end{bmatrix}$

(T) transitive $(x, y), (y, z) \in R \rightarrow (x, z) \in R$
 $\begin{bmatrix} \circ & \circ & \bullet \\ \bullet & \bullet & \bullet \\ \circ & \circ & \circ \end{bmatrix}$

Common relations and their properties

	(R)	(AR)	(S)	(AS)	(T)
$=$					
$\leq$					
$<$					
$\emptyset$					
$\mathcal{U}$					
$ $					

Common relations and their properties

	(R)	(AR)	(S)	(AS)	(T)
$=$	✓		✓	✓	✓
$\leq$					
$<$					
$\emptyset$					
$\mathcal{U}$					
$ $					

Common relations and their properties

	(R)	(AR)	(S)	(AS)	(T)
$=$	✓		✓	✓	✓
$\leq$	✓			✓	✓
$<$					
$\emptyset$					
$\mathcal{U}$					
$ $					

Common relations and their properties

	(R)	(AR)	(S)	(AS)	(T)
$=$	✓		✓	✓	✓
$\leq$	✓			✓	✓
$<$		✓		✓	✓
$\emptyset$					
$\mathcal{U}$					
$ $					

Common relations and their properties

	(R)	(AR)	(S)	(AS)	(T)
$=$	✓		✓	✓	✓
$\leq$	✓			✓	✓
$<$		✓		✓	✓
$\emptyset$		✓	✓	✓	✓
$\mathcal{U}$					
$ $					

Common relations and their properties

	(R)	(AR)	(S)	(AS)	(T)
$=$	✓		✓	✓	✓
$\leq$	✓			✓	✓
$<$		✓		✓	✓
$\emptyset$		✓	✓	✓	✓
$\mathcal{U}$	✓		✓		✓
$ $					

Common relations and their properties

	(R)	(AR)	(S)	(AS)	(T)
$=$	✓		✓	✓	✓
$\leq$	✓			✓	✓
$<$		✓		✓	✓
$\emptyset$		✓	✓	✓	✓
$\mathcal{U}$	✓		✓		✓
$ $	✓			✓	✓

Interaction of Properties

A relation *can* be both symmetric and antisymmetric. Namely, when R consists only of some pairs $(x, x), x \in S$.

A relation *cannot* be simultaneously reflexive and antireflexive (unless $S = \emptyset$).

NB

$\left. \begin{array}{l} \text{nonreflexive} \\ \text{nonsymmetric} \end{array} \right\}$ is not the same as $\left\{ \begin{array}{l} \text{antireflexive/irreflexive} \\ \text{antisymmetric} \end{array} \right.$

Equivalence Relations and Partitions

Relation R is called an *equivalence* relation if it satisfies (R), (S), (T). Every equivalence R defines *equivalence classes* on its domain S .

The equivalence class $[s]$ (w.r.t. R) of an element $s \in S$ is

$$[s] = \{ t \in S : tRs \}$$

The collection of all equivalence classes $[S]_R = \{ [s] : s \in S \}$ is a partition of S :

$$S = \bigcup_{s \in S} [s]$$

Thus, the equivalence classes are disjoint and jointly cover the entire domain. It means that every element belongs to one (and only one) equivalence class.

For $s, t \in S$ either $[s] = [t]$, when $s R t$, or $[s] \cap [t] = \emptyset$, when $s \not R t$. We write $s \sim_R t$ when s, t are in the same equivalence class. In the opposite direction, a partition of a set defines an equivalence relation on that set. If $S = S_1 \dot{\cup} \dots \dot{\cup} S_k$, then we specify $s \sim t$ exactly when s and t belong to the same S_i .

Partial Order

A **partial order** $\preceq$ on S satisfies (R), (AS), (T).

We call $(S, \preceq)$ a **poset** — partially ordered set

Examples

Posets:

- $(\mathbb{Z}, \leq)$
- $(\text{Pow}(X), \subseteq)$ for some set X
- $(\mathbb{N}, |)$

Not posets:

- $(\mathbb{Z}, <)$
- $(\mathbb{Z}, |)$

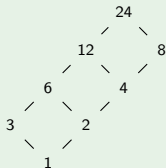
Hasse diagram

Every finite poset $(S, \preceq)$ can be represented with a **Hasse diagram**:

- Nodes are elements of S
- An edge is drawn *upward* from x to y if $x \prec y$ and there is no z such that $x \prec z \prec y$

Example

Hasse diagram for positive divisors of 24 ordered by $|$:



Ordering Concepts

Definition

Let $(S, \preceq)$ be a poset.

- **Minimal** element: x such that there is no y with $y \preceq x$
- **Maximal** element: x such that there is no y with $x \preceq y$
- **Minimum (least)** element: x such that $x \preceq y$ for all $y \in S$
- **Maximum (greatest)** element: x such that $y \preceq x$ for all $y \in S$

NB

- *There may be multiple minimal/maximal elements.*
- *Minimum/maximum elements are the unique minimal/maximal elements if they exist.*
- *Minimal/maximal elements always exist in finite posets, but not necessarily in infinite posets.*

Examples

Examples

- $\text{Pow}(\{a, b, c\})$ with the order $\subseteq$
 $\emptyset$ is minimum; $\{a, b, c\}$ is maximum
- $\text{Pow}(\{a, b, c\}) \setminus \{\{a, b, c\}\}$ (proper subsets of $\{a, b, c\}$)
Each two-element subset $\{a, b\}, \{a, c\}, \{b, c\}$ is maximal.
 - But there is no maximum

Summary of topics

- Sets
- Formal languages
- Relations
- **Functions** (tomorrow)
- Propositional Logic

Notes (Week 2 Monday)

These are the notes I wrote during the lecture.

Sets are uniquely defined by their *membership*

$x \in S$ pronounced "x in S" means that x is an element of S (member of)

$a \in \{a,b,c\}$ $b \in \{a,b,c\}$ $c \in \{a,b,c\}$

$d \notin \{a,b,c\}$ (not a member of)

Two sets S and T are equal iff

for all x,
 $x \in S$ if and only if $x \in T$ (extensionality)

As a consequence:

$\{1\}$ $\{1,1,1,1,1,1,1,1,1,1,1,1,1,1,1\}$
are the same sets, because they have the same members.
i.e. multiplicity doesn't matter

$S = \{1,2,\{3,4\}\}$ how many elements? A: 3

$3 \notin S$
 $\{3,4\} \in S$ $3 \in \{3,4\}$

set membership is not transitive: we count only the
"top-level" elements

Last week, we looked unrestricted comprehension:

$\{x: P(x)\}$ the set of all x such that P(x) holds

now our comprehensions look like this:

$\{x: x \in \mathcal{U} \text{ and } P(x)\}$
 $\mathcal{U}$ (that's just U with a fancy font "calligraphic U")
is pronounced "universe".
the universe is, intuitively, the set of everything
currently under consideration.

Why this extra complexity? A: Russell's paradox

Using unrestricted comprehension, we were able to write

$Y = \{x : x \notin x\}$, which is paradoxical:

$Y \in Y$ iff $Y \notin Y$

Here, the issue is that unrestricted comprehension
lets us conjure sets out of thin air.

$S = \{x: x \in \mathcal{U} \text{ and } P(x)\}$
^ forces the elements of S to be carved out of some
pre-existing set $\mathcal{U}$, meaning that we can no longer
use comprehension to make "new" sets, only carve

out specific subsets of old sets.

$\{\}$: the most boring set
 $x \notin \{\}$ for all x

$\{\{\}$ "the set containing the empty set"
– it has an element, and the element is empty
 $\{\}$
– has no element

There is only one empty set.

Proof:

Suppose there are two empty sets E_1 and E_2 ,
and they are different.
By extensionality, they must thus disagree about
membership of an element. If so, one of them is not empty.

$\{x : x \in \{1,2,3,4\} \text{ and } x \text{ is even}\} = \{2,4\}$

Sometimes, we will still be sloppy and elide the $x \in \mathcal{U}$ clause.
But it's still there in invisible ink.

$A \subseteq B$ "A is a subset of B"
it means that every element of A is an element of B

$\{1,2\} \subseteq \{1,2,3\}$

Power set of X

$\text{Pow}(X)$ $\mathcal{P}(X)$ the set of all subsets of X
 $\mathcal{P}(\{\{\}\}) = \{\{\}, \{\{\}\}\}$
 $\{\} \subseteq \{\{\}\}$ $\{\} \subseteq A$ for all A
 $\{\{\}\} \subseteq \{\{\}\}$ $A \subseteq A$ for all A

Empty set is sometimes written: $\emptyset$

Q: Is it a problem that we define the powerset without
having having a universe?

A: No.

Usually, we'd assume that our universe is closed under
certain standard operators (powerset, products)

We've seen (in the slides) that membership and subset are distinct

$1 \in \{1,2\}$ $1 \notin \{1,2\}$

Q: can you think an example where

$x \in S$ $x \subseteq S$
A: $\{\} \in \{\{\}\}$ $\{\} \subseteq \{\{\}\}$

$|X|$ "the cardinality of X"
 $\text{card}(X)$ the number of elements in X

$|\{1,2,3\}| = 3$
 $|\{1,1,1\}| = 1$

This is straightforward for finite sets, but gets weird for infinite
sets.

$|\mathbb{N}| = ??$ there's no natural number we can put on the RHS

If we cared about the size of non-finite sets, we'd now introduce cardinal numbers, but we won't.

$$|\mathbb{N}| < |\mathbb{R}|$$

$$\{0,1\} \cup \{1,2,3\} = \{0,1,2,3\}$$

$$\{0,1\} \cap \{1,2,3\} = \{1\}$$

$A \setminus B$ sometimes $A - B$ "the difference of A and B"
everything that is in A, but not in B

$$\{0,1\} - \{1,2,3\} = \{0\}$$

$A \oplus B$ "the symmetric difference of A and B"
everything that is in exactly one of A and B

$$\{0,1\} \oplus \{1,2,3\} = \{0,2,3\}$$

$A^c = \mathcal{U} - A$ "everything that is outside A"

but supposing the universe is $\mathbb{N}$

(the even non-negative numbers) c = (the odd non-negative numbers)

Q: can we use this notation for complementation

$$\bar{A}$$

A: Yes. We'll use this notation some in the course,
but usually not for set complementation but for
boolean negation

$$\bar{A} = \neg A$$

Q: In proofs, do we have to go from LHS to RHS?

A: No.

Let's prove idempotence $A \cap A = A$

$$A \cap A = (\text{identity})$$

$$(A \cap A) \cup \emptyset = (\text{complementation})$$

$$(A \cap A) \cup (A \cap A^c) = (\text{distributivity})$$

$$A \cap (A \cup A^c) = (\text{compl})$$

$$A \cap \mathcal{U} = (\text{identity})$$

$$A$$

The dual $A \cap A$ is $A \cup A$

The dual of A is A

The principle of duality is:

If $A = B$ is provable from the
laws of set operations

then $\text{dual}(A) = \text{dual}(B)$ also holds.

By invoking the principle of duality,
we get from our proof above that $A \cup A = A$

If you take a proof of $A = A$,
and dualise all formulas in the proof,
you get a proof of $\text{dual}(A) = \text{dual}(A)$

$A \cup A = (\text{identity})$
 $(A \cup A) \cap \mathcal{U} = (\text{complementation})$
 $(A \cup A) \cap (A \cup A^c) = (\text{distributivity})$
 $A \cup (A \cap A^c) = (\text{compl})$
 $A \cup \emptyset = (\text{identity})$
 A

$S \times T$ "the (Cartesian) product of S and T "
 is the set of all pairs (x,y) such that
 $x \in S$ and $y \in T$
 ordered pairs: $(x,y) \neq (y,x)$ (unless $x=y$)

classic example: complex numbers
 $\mathbb{R} \times \mathbb{R}$ coordinates

$(x,y) \in \emptyset \times S$
 it must be the case that
 $x \in \emptyset$ and $y \in S$

$\emptyset \times S = \emptyset$
 reminiscent of $0 \cdot x = 0$

Formal languages are sets of strings

Strings are sequences of symbols,
 symbols are taken from an *alphabet*

Σ denotes the alphabet under consideration

empty word: λ or ε is: a string consisting of
 no alphabet symbols (the empty string)

$\{\}$ the empty language (a language with no words)
 $\neq \{\lambda\}$ the language containing the empty string

vw if v and w denote words, is the
 concatenation of v and w ,
 all the symbols of v followed by all the
 symbols of w

$v \cdot w$

$aa \cdot bb = aabb$

$vwx = (vw)x = v(wx)$ (concatenation is associative)

Σ^0 is all words of length 0 formed by symbols in Σ

$\Sigma^0 = \{\lambda\}$

$\Sigma^1 = \Sigma$

$\Sigma = \{0,1\}$

$\Sigma^2 = \{00,01,10,11\}$

...

Σ^* (sigma-star)
 the set of all (finite) words over Σ

Σ^+ (sigma-plus)

the set of all finite, non-empty words over Σ

A **language** is a set of strings over some Σ .

A **language** is a subset of Σ^*

The set of all grammatically correct English sentences is a language.

The set of all syntactically correct C programs.

0-1-1-2.2....

...0.

-23523.1

The last clause prevents:

-

λ

Alphabets don't have to be over single characters;
they can be over bigger things

In parsing, we'd call this sort of composite symbol
a **token**

Generally, the first thing a compiler does when given a blob of text is to convert it to a blob of tokens.
When this fails, you typically get a lexical error in your face.

baguette $\in$ french $\cap$ english

COMP2111 Week 2
Term 1, 2023
Discrete Mathematics Recap II

Summary of topics

- Sets
- Formal languages
- Relations
- **Functions**
- Propositional Logic

Functions

A **function**, $f : S \rightarrow T$, is a binary relation $f \subseteq S \times T$ such that for all $s \in S$ there is *exactly one* $t \in T$ such that $(s, t) \in f$.

We write $f(s)$ for the unique element related to s .

A **partial function** $f : S \rightharpoonup T$ is a binary relation $f \subseteq S \times T$ such that for all $s \in S$ there is *at most one* $t \in T$ such that $(s, t) \in f$. That is, it is a function $f : S' \rightarrow T$ for $S' \subseteq S$

Functions

A function $f : S \longrightarrow T$ is a pairing of the sets: it means that f assigns to every element $s \in S$ a single element $t \in T$. To emphasise where a specific element is sent, we can write $f : x \mapsto y$, which means the same as $f(x) = y$

		Symbol	
S	domain of f	$\text{dom}(f)$	(inputs)
T	co-domain of f	$\text{codom}(f)$	(<i>possible</i> outputs)
$f(S)$	image of f	$\text{im}(f)$	(<i>actual</i> outputs)
$= \{ f(x) : x \in \text{dom}(f) \}$			

Important!

The domain and co-domain are critical aspects of a function's definition.

$$f : \mathbb{N} \rightarrow \mathbb{Z} \quad \text{given by} \quad f : x \mapsto x^2$$

and

$$g : \mathbb{N} \rightarrow \mathbb{N} \quad \text{given by} \quad g : x \mapsto x^2$$

are different functions even though they have the same behaviour!

Composition of Functions

Composition of functions is described as

$$g \circ f : x \mapsto g(f(x)), \quad \text{requiring } \text{im}(f) \subseteq \text{dom}(g)$$

Composition is associative

$$h \circ (g \circ f) = (h \circ g) \circ f, \quad \text{can write } h \circ g \circ f$$

Composition of Functions

If a function maps a set into itself, i.e. when $\text{dom}(f) = \text{codom}(f)$ (and thus $\text{im}(f) \subseteq \text{dom}(f)$), the function can be composed with itself — **iterated**

$$f \circ f, f \circ f \circ f, \dots, \quad \text{also written } f^2, f^3, \dots$$

Identity function on S

$$\text{Id}_S(x) = x, x \in S; \text{dom}(i) = \text{codom}(i) = \text{im}(i) = S$$

For $g : S \longrightarrow T$ $g \circ \text{Id}_S = g, \text{Id}_T \circ g = g$

Extension: Composition of Binary Relations

If $R_1 \subseteq S \times T$ and $R_2 \subseteq T \times U$ then the composition of R_1 and R_2 is the relation:

$$R_1; R_2 := \{(a, c) : \text{there is a } b \in T \text{ such that} \\ (a, b) \in R_1 \text{ and } (b, c) \in R_2\}.$$

Note that if $f : S \rightarrow T$ and $g : T \rightarrow U$ are functions then $f; g = g \circ f$.

Properties of Functions

A function is called **surjective** or **onto** if every element of the codomain is mapped to by at least one x in the domain, i.e.

$$\text{im}(f) = \text{codom}(f)$$

Examples (of surjective functions)

- $f : \mathbb{N} \longrightarrow \mathbb{N}$ with $f(x) = x$
- Floor, ceiling

Examples (of non-surjective functions)

- $f : \mathbb{N} \longrightarrow \mathbb{N}$ with $f(x) = x^2$
- $f : \{a, \dots, z\}^* \longrightarrow \{a, \dots, z\}^*$ with $f(w) = awe$

Injective Functions

A function is called **injective** or **1-1 (one-to-one)** if different inputs give different outputs, i.e.

$$f(x) = f(y) \rightarrow x = y$$

Examples (of functions that are injective)

- $f : \mathbb{N} \rightarrow \mathbb{N}$ with $f(x) \mapsto x$
- set complement (for a fixed universe)

Examples (of functions that are not injective)

- absolute value, floor, ceiling
- length of a word

Function is **bijjective** if it is both surjective and injective.

Converse of a function

Question

$f^{\leftarrow}$ is a relation; when is it a function?

Question

$f^{\leftarrow}$ is a relation; when is it a function?

Answer

When f is a bijection.

Inverse Functions

Inverse function — $f^{-1} : T \rightarrow S$;
for a given $f : S \rightarrow T$ exists exactly
when f is bijective.

Image of a subdomain A under a function

$$f(A) = \{ f(s) : s \in A \} = \{ t \in T : t = f(s) \text{ for some } s \in A \}$$

Inverse image — $f^{\leftarrow}(B) = \{ s \in S : f(s) \in B \} \subseteq S$;
it is defined for every f (recall: converse of a relation)

If f^{-1} exists then $f^{\leftarrow}(B) = f^{-1}(B)$

$$f(\emptyset) = \emptyset, f^{\leftarrow}(\emptyset) = \emptyset$$

Summary of topics

- Sets
- Formal languages
- Relations
- Functions
- Propositional Logic

Propositions

A sentence of a natural language (like English, Cantonese, Warlpiri) is *declarative*, or a *proposition*, if it can be meaningfully be said to be either true or false.

Examples

- Richard Nixon was president of Ecuador.
- A square root of 16 is 4.
- Euclid's program gets stuck in an infinite loop if you input 0.
- Whatever list of numbers you give as input to this program, it outputs the same list but in increasing order.
- $x^n + y^n = z^n$ has no nontrivial integer solutions for $n > 2$.
- 3 divides 24.

The following are *not* declarative sentences:

- Wingardium leviosa.
- For Pete's sake, take out the garbage!
- Are elephants made of cardboard?
- Please waive the prerequisites for this subject for me.

Declarative sentences in natural languages can be *compound* sentences, built out of other sentences.

Propositional Logic is a formal representation of some constructions for which the truth value of the compound sentence can be determined from the truth value of its components.

- My pants are on fire *and* my hat is soaked.
- Nixon won the debate *or* Nixon applied a coat of Lazy Shave.
- *It is not the case that* this program always halts.

Not all constructions of natural language are truth-functional:

- *Carroll believes that* Santa Claus is real.
- *Johannes knows* Santa Claus is real.
- This program always halts *because* it contains no loops.
- The disk crashed *after* I saved my file.

NB

Various **modal logics** extend classical propositional logic to cover these.

The Three Basic Connectives of Propositional Logic

symbol	text
$\wedge$	"and", "but", ":", ":", "
$\vee$	"or", "either ...or ..."
$\neg$	"not", "it is not the case that"

Truth tables:

A	B	$A \wedge B$
F	F	F
F	T	F
T	F	F
T	T	T

A	B	$A \vee B$
F	F	F
F	T	T
T	F	T
T	T	T

A	$\neg A$
F	T
T	F

Applications I: Program Logic

Example

```
if x > 0 or (x <= 0 and y > 100):
```

Let $p \stackrel{\text{def}}{=} (x > 0)$ and $q \stackrel{\text{def}}{=} (y > 100)$

$p \vee (\neg p \wedge q)$

p	q	$\neg p$	$\neg p \wedge q$	$p \vee (\neg p \wedge q)$
F	F	T	F	F
F	T	T	T	T
T	F	F	F	T
T	T	F	F	T

This is equivalent to $p \vee q$. Hence the code can be simplified to

```
if x > 0 or y > 100:
```

Now consider the following constructions:

- if A then B
- A only if B
- B if A
- A implies B
- it follows from A that B
- whenever A, B
- A is a sufficient condition for B
- B is a necessary condition for A

Each has the property that if the whole statement is true, and A is true, then B is true.

We can *approximate* the English meaning of these by “not (A and not B)”, written $A \rightarrow B$ which has the following truth table:

A	B	$A \rightarrow B$
F	F	T
F	T	T
T	F	F
T	T	T

While only an approximation to the English, 100+ years of experience have shown this to be adequate for capturing *mathematical reasoning*.

(Moral: mathematical reasoning does not need all the features of English.)

“If it rains, I bring an umbrella”

Conversational implication (in informal English):



Material implication (in formal logic):



“If it rains, I bring an umbrella”

Conversational implication (in informal English):



Material implication (in formal logic):



Conversational implication also has a whiff of *causality*.

“If I don’t bring an umbrella, it doesn’t rain”

In formal logic, this new sentence is equivalent to “If it rains...”.

In English, it suggests that my behaviour changes the weather, not that the weather changes my behaviour.

Examples

p = “you get an HD on your final exam”

q = “you do every exercise in the book”

r = “you get an HD in the course”

Translate into logical notation:

(a) You get an HD in the course although you do not do every exercise in the book.

(c) To get an HD in the course, you must get an HD on the exam.

(d) You get an HD on your exam, but you don't do every exercise in this book; nevertheless, you get an HD in this course.

Examples

p = “you get an HD on your final exam”

q = “you do every exercise in the book”

r = “you get an HD in the course”

Translate into logical notation:

(a) You get an HD in the course although you do not do every exercise in the book.

Examples

p = “you get an HD on your final exam”

q = “you do every exercise in the book”

r = “you get an HD in the course”

Translate into logical notation:

(a) You get an HD in the course although you do not do every exercise in the book. $r \wedge \neg q$

(c) To get an HD in the course, you must get an HD on the exam.

Examples

p = “you get an HD on your final exam”

q = “you do every exercise in the book”

r = “you get an HD in the course”

Translate into logical notation:

(a) You get an HD in the course although you do not do every exercise in the book. $r \wedge \neg q$

(c) To get an HD in the course, you must get an HD on the exam.
 $r \rightarrow p$

(d) You get an HD on your exam, but you don't do every exercise in this book; nevertheless, you get an HD in this course.

Examples

p = “you get an HD on your final exam”

q = “you do every exercise in the book”

r = “you get an HD in the course”

Translate into logical notation:

(a) You get an HD in the course although you do not do every exercise in the book. $r \wedge \neg q$

(c) To get an HD in the course, you must get an HD on the exam.
 $r \rightarrow p$

(d) You get an HD on your exam, but you don't do every exercise in this book; nevertheless, you get an HD in this course. $p \wedge \neg q \wedge r$

Unless

A unless B can be approximated as $\neg B \rightarrow A$

E.g.

I go swimming unless it rains = If it is not raining then I go swimming.

Correctness of the translation is perhaps easier to see in:

I don't go swimming unless the sun shines = If the sun does not shine then I don't go swimming.

Note that “I go swimming unless it rains, but sometimes I swim even though it is raining” makes sense, so the translation of “A unless B” should not imply $B \rightarrow \neg A$.

If and only if

A if, and only if, B is written $A \leftrightarrow B$

I will have an entree if and only if I won't have desert.

= If I have desert I will not have an entree and vice versa.

It has the following truth table:

A	B	$A \leftrightarrow B$
F	F	T
F	T	F
T	F	F
T	T	T

Same as $(A \rightarrow B) \wedge (B \rightarrow A)$

A **Propositional formula** is made up of **propositional variables** and **logical connectives** ($\wedge, \vee, \neg, \rightarrow, \leftrightarrow$).

A **truth assignment** (aka **valuation**, aka **state**) assigns T or F to each propositional variable, and, using the logical connectives, gives a truth value to all propositional formulas.

Logical Equivalence

Two formulas ϕ, ψ are **logically equivalent**, denoted $\phi \equiv \psi$ if they have the same truth value for all truth valuations.

Application: If ϕ and ψ are two formulae such that $\phi \equiv \psi$, then the digital circuits corresponding to ϕ and ψ compute the same function. Thus, proving equivalence of formulas can be used to *optimise* circuits.

Some Well-Known Equivalences

Excluded Middle $p \vee \neg p \equiv \top$

Contradiction $p \wedge \neg p \equiv \perp$

Identity $p \vee \perp \equiv p$

$$p \wedge \top \equiv p$$
$$p \vee \top \equiv \top$$
$$p \wedge \perp \equiv \perp$$

Idempotence $p \vee p \equiv p$

$$p \wedge p \equiv p$$

Double Negation $\neg\neg p \equiv p$

Commutativity $p \vee q \equiv q \vee p$

$$p \wedge q \equiv q \wedge p$$

Associativity

$$(p \vee q) \vee r \equiv p \vee (q \vee r)$$

$$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$$

Distribution

$$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$$

$$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$$

De Morgan's laws

$$\neg(p \wedge q) \equiv \neg p \vee \neg q$$

$$\neg(p \vee q) \equiv \neg p \wedge \neg q$$

Implication

$$p \rightarrow q \equiv \neg p \vee q$$

$$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$$

Satisfiability of Formulas

A formula is **satisfiable**, if it evaluates to T for *some* assignment of truth values to its basic propositions.

Example

A	B	$\neg(A \rightarrow B)$
F	F	F
F	T	F
T	F	T
T	T	F

Applications II: Constraint Satisfaction Problems

These are problems such as timetabling, activity planning, etc. Many can be understood as showing that a formula is satisfiable.

Example

You are planning a party, but your friends are a bit touchy about who will be there.

- ❶ If John comes, he will get very hostile if Sarah is there.
- ❷ Sarah will only come if Kim will be there also.
- ❸ Kim says she will not come unless John does.

Who can you invite without making someone unhappy?

Translation to logic: let J, S, K represent “John (Sarah, Kim) comes to the party”. Then the constraints are:

① $J \rightarrow \neg S$

② $S \rightarrow K$

③ $K \rightarrow J$

Thus, for a successful party to be possible, we want the formula $\phi = (J \rightarrow \neg S) \wedge (S \rightarrow K) \wedge (K \rightarrow J)$ to be satisfiable.

Truth values for J, S, K making this true are called *satisfying assignments*, or *models*.

We figure out where the conjuncts are false, below. (so blank = T)

J	K	S	$J \rightarrow \neg S$	$S \rightarrow K$	$K \rightarrow J$	ϕ
F	F	F				
F	F	T		F		F
F	T	F			F	F
F	T	T			F	F
T	F	F				
T	F	T	F	F		F
T	T	F				
T	T	T	F			F

Conclusion: a party satisfying the constraints can be held. Invite nobody, or invite John only, or invite Kim and John.

Exercise

Which of the following formulae are *always* true?

(a) $(p \wedge (p \rightarrow q)) \rightarrow q$

(b) $((p \vee q) \wedge \neg p) \rightarrow \neg q$

(e) $((p \rightarrow q) \vee (q \rightarrow r)) \rightarrow (p \rightarrow r)$

(f) $(p \wedge q) \rightarrow q$

Exercise

Which of the following formulae are *always* true?

(a) $(p \wedge (p \rightarrow q)) \rightarrow q$ — always true

(b) $((p \vee q) \wedge \neg p) \rightarrow \neg q$ — not always true

(e) $((p \rightarrow q) \vee (q \rightarrow r)) \rightarrow (p \rightarrow r)$ — not always true

(f) $(p \wedge q) \rightarrow q$ — always true

Validity, Entailment, Arguments

An *argument* consists of a set of declarative sentences called *premises* and a declarative sentence called the *conclusion*.

Example

Premises:	Frank took the Ford or the Toyota. If Frank took the Ford he would be late. Frank is not late.
Conclusion:	Frank took the Toyota

An argument is *valid* if the conclusions are true *whenever* all the premises are true. Thus: if we believe the premises, we should also believe the conclusion.

(Note: we don't care what happens when one of the premises is false.)

Other ways of saying the same thing:

- The conclusion *logically follows* from the premises.
- The conclusion is a *logical consequence* of the premises.
- The premises **entail** the conclusion.

The argument above is valid. The following is invalid:

Example

Premises: Frank took the Ford or the Toyota.
 If Frank took the Ford he would be late.
 Frank is late.

Conclusion: Frank took the Ford.

Validity of arguments in propositional logic is captured as follows:

Let $\phi_1, \dots, \phi_n$ and ϕ be formulae of propositional logic. Draw a truth table with columns for each of $\phi_1, \dots, \phi_n$ and ϕ .

The argument with premises $\phi_1, \dots, \phi_n$ and conclusion ϕ is valid, denoted

$$\phi_1, \dots, \phi_n \models \phi$$

if for every row of the truth table where $\phi_1, \dots, \phi_n$ are all true, ϕ is true also.

NB: $\models$ is the same thing as $\Rightarrow$ from Week 1.

We mark only true locations (blank = F)

Frd	$Tyta$	$Late$	$Frd \vee Tyta$	$Frd \rightarrow Late$	$\neg Late$	$Tyta$
F	F	F		T	T	
F	F	T		T		
F	T	F	T	T	T	T
F	T	T	T	T		T
T	F	F	T		T	
T	F	T	T	T		
T	T	F	T		T	T
T	T	T	T	T		T

This shows $Frd \vee Tyta, Frd \rightarrow Late, \neg Late \models Tyta$

The following row shows $Frd \vee Tyta$, $Frd \rightarrow Late$, $Late \not\models Frd$

Frd	$Tyta$	$Late$	$Frd \vee Tyta$	$Frd \rightarrow Late$	$Late$	Frd
F	T	T	T	T	T	F

Applications III: Reasoning About Requirements/Specifications

Suppose a set of English language requirements R for a software/hardware system can be formalised by a set of formulae $\{\phi_1, \dots, \phi_n\}$.

Suppose C is a statement formalised by a formula ψ . Then

- 1 The requirements cannot be implemented if $\phi_1 \wedge \dots \wedge \phi_n$ is not satisfiable.
- 2 If $\phi_1, \dots, \phi_n \models \psi$ then every correct implementation of the requirements R will be such that C is always true in the resulting system.
- 3 If $\phi_1, \dots, \phi_{n-1} \models \phi_n$, then the condition ϕ_n of the specification is redundant and need not be stated in the specification.

Example

Requirements R: An alarm system for a house is to operate as follows. The alarm should not sound unless the system has been armed or there is a fire. If the system has been armed and a door is disturbed, the alarm should ring. Irrespective of whether the system has been armed, the alarm should go off when there is a fire.

Conclusion C: If the alarm is ringing and there is no fire, then the system must have been armed.

Questions

- 1 Will every system correctly implementing requirements R satisfy C?
- 2 Is the final sentence of the requirements redundant?

Expressing the requirements as formulas of propositional logic,
with

- S = the alarm sounds = the alarm rings
- A = the system is armed
- D = a door is disturbed
- F = there is a fire

we get

Requirements:

- 1 $S \rightarrow (A \vee F)$
- 2 $(A \wedge D) \rightarrow S$
- 3 $F \rightarrow S$

Conclusion: $(S \wedge \neg F) \rightarrow A$

Our two questions then correspond to

- 1 Does $S \rightarrow (A \vee F), (A \wedge D) \rightarrow S, F \rightarrow S \models (S \wedge \neg F) \rightarrow A$?
- 2 Does $S \rightarrow (A \vee F), (A \wedge D) \rightarrow S \models F \rightarrow S$?

Validity of Formulas

A formula ϕ is **valid**, or a **tautology**, denoted $\models \phi$, if it evaluates to T for *all* assignments of truth values to its basic propositions.

Example

A	B	$(A \rightarrow B) \rightarrow (\neg B \rightarrow \neg A)$
F	F	T
F	T	T
T	F	T
T	T	T

Validity, Equivalence and Entailment

Theorem

The following are equivalent:

- $\phi_1, \dots, \phi_n \models \psi$
- $\models (\phi_1 \wedge \dots \wedge \phi_n) \rightarrow \psi$
- $\models \phi_1 \rightarrow (\phi_2 \rightarrow \dots (\phi_n \rightarrow \psi) \dots)$

Theorem

$\phi \equiv \psi$ if and only if $\models \phi \leftrightarrow \psi$

Beyond propositions

Entailment captures a form of logical reasoning, but it cannot handle relatively simple logical arguments like the following:

- 1 Socrates is a man
- 2 All men are mortal
- 3 Therefore Socrates is mortal

We need to add **expressiveness** to propositional logic so that we can capture notions such as the *relation* between man and men in the first two statements; and the *quantified* statement “all men”.

NB

Adding expressiveness comes at a cost: it is now more difficult to determine truth values.

Predicates

Predicates are functions that take inputs from a set and return either true or false – i.e. they are relations. Predicates enable us to establish relationships between different propositions, such as the man/men connection between the first and second propositions on the previous slide, allowing more expressiveness than propositional logic can give.

Quantifiers

Quantifiers allow us to make quantified statements over predicates, e.g.

“If there exists a satisfying assignment ...”

or

“Every natural number greater than 2 ...”

The two standard quantifiers are

- $\forall$: “for all”, “for any”, “every”
- $\exists$: “there exists”, “there is”, “for some”, “at least one”

Example

Goldbach conjecture

$$\forall n \in 2\mathbb{N} (n > 2 \rightarrow \exists p, q \in \mathbb{N} (p, q \in \text{PRIMES} \wedge n = p + q))$$

Predicate logic

Predicate (or first-order) logic extends propositional logic by adding predicates and quantifiers.

More on predicate logic in Weeks 1 and 5.

Summary of topics

- Sets
- Formal languages
- Relations
- Functions
- Propositional Logic

Notes (Week 2 Wednesday)

These are the notes I wrote during the lecture.

Formal languages

Formal language are
sets of words

Words are strings (sequences of symbols)
over some Σ

Σ is a finite set of symbols

Σ^* the set of all words over Σ

A Σ -language over Σ is a subset of Σ^*

Example:

$\{\text{hello, world}\} \cdot \{\text{salut, monde}\}$
 $= \{\text{hellosalut, hellomonde, worldsalut, worldmonde}\}$

If A is a language, A^* consists of:
any finite sequence of concatenated
words from A

$\text{hello} \in \{\text{hello, world}\}^*$
 $\text{hellohello} \in \{\text{hello, world}\}^*$
 $\text{hellohelloworldhello} \in \{\text{hello, world}\}^*$
 $\lambda \in \{\text{hello, world}\}^*$

$\text{DIG} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$D = \{\lambda, -\} \cdot \text{DIG} \cdot \text{DIG}^* \cdot (\{\lambda\} \cup \{.\} \cdot (\text{DIG} \cup \text{DIG}^2))$

$\{\}^* = \{\lambda\}$

$\{\lambda\}^* = \{\lambda\}$

A relation is a subset of a Cartesian product.

$\leq \subseteq \mathbb{N} \times \mathbb{N}$

$(x, y) \in \leq$ iff x is less than or equal to y

$$x \leq y$$

$$\leq = \{(x,y) : (x,y) \in \mathbb{N} \times \mathbb{N} \text{ and} \\ \text{there exists } z \in \mathbb{N} \text{ and} \\ x+z=y\}$$

$$\begin{aligned} (0,3) &\in \leq \\ &\equiv (\text{by definition}) \\ &\text{there exists } z \in \mathbb{N} \text{ and } 0+z=3 \\ &\equiv (\text{too obvious}) \\ &\text{True} \end{aligned}$$

Notation: For *binary* relations, we often write them in infix.

$$(x,y) \in R \quad x R y \quad R(x,y)$$

$R(A)$, if R is a binary relation $R \subseteq S \times T$,
and $A \subseteq S$
gives you every T that is related to some S .

$$R = \{(mango, tasty), (banana, yucky), \\ (mango, sweet)\}$$

$$\begin{aligned} R(\{mango\}) &= \{tasty, sweet\} \\ R(\{banana\}) &= \{yucky\} \end{aligned}$$

$$\begin{aligned} R(\{mango, banana\}) &= \{tasty, yucky, sweet\} \\ R(\{\}) &= \{\} \end{aligned}$$

$R^{\leftarrow}$ converse of a relation

$$R^{\leftarrow} = \{(tasty, mango), (yucky, banana), \\ (sweet, mango)\}$$

$$\leq^{\leftarrow} = \geq$$

If $x = y$ and $y = x$ then $x = y$

Def: If $x R y$ and $y R x$ then $x = y$

A not-symmetric relation,
and an antisymmetric relation,
is not the same thing.

	R	AR	S	AS	T
=	✓		✓	✓	✓
≤	✓			✓	✓
<		✓		✓	✓
∅	?	✓	✓	✓	✓
<i>u</i>	✓	?	✓	?	✓

$$\emptyset \subseteq \mathbb{N} \times \mathbb{N}$$

$$\emptyset \subseteq \emptyset \times \emptyset$$

Reflexive means

$$\forall x \in S. (x,x) \in R$$

If I'm a frog then I'm deep-fried

If a relation $\sim$ is reflexive, symmetric and transitive, we call it an equivalence relations.

$\equiv$ (logical equivalence) is an equivalence

$$R = \{(x,y) \mid x,y \in \mathbb{N} \text{ and } x \bmod 3 = y \bmod 3\}$$

R happens to be reflexive, symmetric and transitive

$$[1] = \{1,4,7,\dots\}$$

Note: $[1] = R(\{1\})$

One nice property of equivalence classes:

$$x R y$$

$$[x] = [y]$$

For elements x,y that are not equivalent

$$[x] \neq [y]$$

$[x]$ and $[y]$ are disjoint

Partial orders:

$$(R), (AS), (T)$$

Strict partial orders:

$$(AR), (AS), (T)$$

$<$ "precedes"

$\leq$ "precedes or equals"

$>$ "succeeds"

In the slides, $\leq$ is just a variable name, and denotes some arbitrary partial order

$$2|-2 \quad \text{and} \quad -2|2 \quad \text{but} \quad 2 \neq -2$$

Note: definitions in slides are for

strict partial orders.

These are for partial orders.

Minimal element:

an x such that for every $y \leq x$, $x = y$

Minimum element:

an x such that $x \leq y$ for y

First, an obvious(?) point:

minimal and minimum elements do not always exist.

Consider $(\mathbb{Z}, \leq)$

Functions map each input to exactly one output.

Functions are relations $f \subseteq S \times T$ between inputs and outputs.

$f(x) = y$ means the same as $(x, y) \in f$

not a function:

$R = \{(1, 2), (1, 1)\}$

is a function:

$g = \{(1, 1), (2, 1)\}$

we might write g as:

$g : \{1, 2\} \rightarrow \{1, 2\}$

$g(x) = 1$

domain $\{1, 2\}$ (inputs)

codomain $\{1, 2\}$ (outputs)

image $\{1\}$

the image is always a subset of the codomain, but might not cover it

The input-output relation that characterises a function f is sometimes called the *graph* of f .

$g : \mathbb{R} \rightarrow \mathbb{R}$

$g(x) = 5/x$

^that's a partial function, because it's undefined at 0

$f(x) = x + 1$

$(f \circ f)x = f^2(x) = f(f(x)) = x + 2$

"the identity function on S "

$\text{Id}_S : S \rightarrow S$

$$\text{Id}_S(x) = x$$

the identity function is the same thing as
the identity relation

If R_1 and R_2 are relations,

$R_1; R_2$ "the composition of R_1 and R_2 "
relates (x,y) if y can be reached from
 x with one hop through R_1 and one hop through
 R_2

$$R_1; R_2 = \{(x,y) \mid \text{exists } z. x R_1 z \text{ and } z R_2 y\}$$

$$R = \{(x,y) \mid x+1 = y\}$$

$x R y$ means y is one larger x

$x (R;R) y$ there must be some z
such that z is one larger than x
and y is one larger than x

in other words:

y is two larger than x

Tute (Week 2)

Cardinalities

Question 1

Find the cardinalities of the following sets

1. $\left\{ \frac{1}{n} : n \in \mathbb{N} \text{ and } n \in [1, 4] \right\}$
2. $\left\{ n^2 - n : n \in \mathbb{N} \text{ and } n \in [0, 4] \right\}$
3. $\left\{ \frac{1}{n^2} : n \in \mathbb{N}_{>0} \text{ and } 2|n \text{ and } n < 11 \right\}$
4. $\{2 + (-1)^n : n \in \mathbb{N}\}$

1. This is just $\left\{ \frac{1}{1}, \frac{1}{2}, \frac{1}{3}, \frac{1}{4} \right\}$, so the cardinality is 4.
2. This is just $\{0, 0, 2, 6, 12\}$, so the cardinality is 4.
3. This is just $\left\{ \frac{1}{4}, \frac{1}{16}, \frac{1}{36}, \frac{1}{64}, \frac{1}{100} \right\}$ so the cardinality is 5.
4. This is just $\{3, 1, 3, 1, \dots\}$ so the cardinality is 2.

Question 2

If $|A| = m$, $|B| = n$ and $|A \cap B| = k$, what is:

1. $|A \cup B|$
2. $|A \setminus B|$
3. $|A \oplus B|$

1. $|A \cup B| = |A| + |B| - |A \cap B| = m + n - k$
2. $|A \setminus B| = |A| - |A \cap B| = m - k$
3. $|A \oplus B| = |A \cup B| - |A \cap B| = m + n - 2k$

Question 3

How many elements in the following sets:

1. $A = \{w \in \Sigma^* : \text{length}(w) \leq 3\}$ where $\Sigma = \{a, b, c\}$
2. $B = \{w \in \Sigma'^* : \text{length}(w) \in [2, 4]\}$ where $\Sigma' = \{a, b\}$
3. $A \oplus B$

4. $AB := \{wv : w \in A \text{ and } v \in B\}$ *Hint: it's less than $40 \times 28 = 1120$*

The general approach here is to tally up the words of each possible length separately.

1. A has $3^0 + 3^1 + 3^2 + 3^3 = 1 + 3 + 9 + 27 = 40$ elements.

2. B has $4 + 8 + 16 = 28$ elements

3. Since B has no words of length 0 and 1, we only need to tally up the words from A. For lengths 2 and 3 we have that the B-words are a subset of the A-words, so to get the words that only occur in A we subtract the number of A-words from the number of B-words. Finally, since A has no words of length 4, we only need to count the B-words. Thus:

$$\begin{aligned} \text{card}(A \oplus B) &= \\ 1 + 3 + (9 - 4) + (27 - 8) + 16 &= \\ 44 \end{aligned}$$

4. It is certainly fewer than $40 \times 28 = 1120$ because $wv = w'v'$ when, e.g. $w = \lambda, v = aaa, w' = a, v' = aa$. We need to count all the words over $\{a, b, c\}$, with length between 2 and 7, where c's can only occur up to the position $\min\{3, \text{length}(w) - 2\}$

i. Length 2 (no c's): $2 \times 2 = 4$ words

ii. Length 3 (c only possible in first position): $3 \times 2 \times 2 = 12$

iii. Length 4 (c only possible in first two positions): $3 \times 3 \times 2 \times 2 = 36$

iv. Length 5 (c only possible in first three positions): $3^3 \times 2^2 = 108$

v. Length 6 (c only possible in first three positions): $3^3 \times 2^3 = 216$

vi. Length 7 (c only possible in first three positions): $3^3 \times 2^4 = 432$

Total: 808.

Set Reasoning

Question 1

When is $(A \setminus B) \setminus C = A \setminus (B \setminus C)$?

As can be checked with a Venn diagram:

$$A \setminus (B \setminus C) = ((A \setminus B) \setminus C) \cup (A \cap C)$$

so when $A \cap C = \emptyset$ we have equality.

Question 2

Show, using the Laws of Set Operations (and Idempotence) the following equalities hold for all sets A and B (subsets of $\square$):

1. Annihilation: $A \cap \emptyset = \emptyset$ and $A \cup \square = \square$
2. Double complementation: $(A^c)^c = A$ *Hint*: First show that if $A \cap B = \emptyset$ and $A \cup B = \square$ then $B = A^c$
3. De Morgan's Laws: $(A \cap B)^c = A^c \cup B^c$ and $(A \cup B)^c = A^c \cap B^c$ *Hint*: the lemma we used to prove double complementation is useful here too. *Hint*: by the principle of duality it suffices to prove one of the laws.

Note: using the Principle of Duality, we only need to show one equality for each part.

Annihilation (Part 1):

$$\begin{aligned}
 A \cap \emptyset &= A \cap (A \cap A^c) && \text{(Complementation)} \\
 &= (A \cap A) \cap A^c && \text{(Associativity)} \\
 &= A \cap A^c && \text{(Idempotence)} \\
 &= \emptyset && \text{(Complementation)}
 \end{aligned}$$

Double Complementation (Part 2):

Following the hint, if $A \cap B = \emptyset$ and $A \cup B = \square$:

$$\begin{aligned}
 A^c &= A^c \cap \square && \text{(Identity)} \\
 &= A^c \cap (A \cup B) && \text{(Assumption)} \\
 &= (A^c \cap A) \cup (A^c \cap B) && \text{(Distributivity)} \\
 &= (A \cap A^c) \cup (A^c \cap B) && \text{(Commutativity)*} \\
 &= \emptyset \cup (A^c \cap B) && \text{(Complementation)} \\
 &= (A^c \cap B) \cup \emptyset && \text{(Commutativity)*} \\
 &= A^c \cap B && \text{(Identity)} \\
 &= B \cap A^c && \text{(Commutativity)} \\
 &= (B \cap A^c) \cup \emptyset && \text{(Identity)} \\
 &= (B \cap A^c) \cup (B \cap A) && \text{(Assumption)} \\
 &= B \cap (A^c \cup A) && \text{(Distributivity)} \\
 &= B \cap (A \cup A^c) && \text{(Commutativity)*} \\
 &= B \cap \square && \text{(Complementation)} \\
 &= B && \text{(Identity)}
 \end{aligned}$$

Now we have, using Commutativity and Complementation:

$$\begin{aligned}
 A^c \cap A &= A \cap A^c = \emptyset \\
 A^c \cup A &= A \cup A^c = \square
 \end{aligned}$$

So $A = (A^c)^c$.

De Morgan's Laws (Part 3):

We have:

$$\begin{aligned}
(A \cap B) \cap (A^c \cup B^c) &= A \cap (B \cap (A^c \cup B^c)) && \text{(Associativity)} \\
&= A \cap ((B \cap A^c) \cup (B \cap B^c)) && \text{(Distributivity)} \\
&= A \cap ((B \cap A^c) \cup \emptyset) && \text{(Complementation)} \\
&= A \cap (B \cap A^c) && \text{(Identity)} \\
&= (A \cap B) \cap A^c && \text{(Associativity)} \\
&= (B \cap A) \cap A^c && \text{(Complementation)} \\
&= B \cap (A \cap A^c) && \text{(Associativity)} \\
&= B \cap \emptyset && \text{(Complementation)} \\
&= \emptyset && \text{(Annihilation: see (a))}
\end{aligned}$$

By the Principle of Duality we also have

$$(X \cup Y) \cup (X^c \cap Y^c) = \square$$

So,

$$\begin{aligned}
(A \cap B) \cup (A^c \cup B^c) &= (A^c \cup B^c) \cup (A \cap B) && \text{(Commutativity)} \\
&= (A^c \cup B^c) \cup ((A^c)^c \cap (B^c)^c) && \text{(Double complement)} \\
&= \square && \text{(from above)}
\end{aligned}$$

It follows from the previous part (Uniqueness of complement) that $(A \cap B)^c = (A^c \cup B^c)$.

Question 3

Prove, using the Laws of Set Operations (and derived laws), that:

$$(A \cap B^c) \cup (B \cap A^c) = (A \cup B) \cap (A \cap B)^c$$

$$\begin{aligned}
&(A \cap B^c) \cup (B \cap A^c) \\
&= ((A \cap B^c) \cup B) \cap ((A \cap B^c) \cup A^c) && \text{(Distributivity)} \\
&= (B \cup (A \cap B^c)) \cap (A^c \cup (A \cap B^c)) && \text{(Commutativity)} \\
&= ((B \cup A) \cap (B \cup B^c)) \cap ((A^c \cup A) \cap (A^c \cup B^c)) && \text{(Distributivity)} \\
&= ((B \cup A) \cap (B \cup B^c)) \cap ((A^c \cup (A^c)^c) \cap (A^c \cup B^c)) && \text{(Dbl. compl.)} \\
&= ((B \cup A) \cap \square) \cap (\square \cap (A^c \cup B^c)) && \text{(Complement)} \\
&= ((A \cup B) \cap \square) \cap ((A^c \cup B^c) \cap \square) && \text{(Commutativity)} \\
&= (A \cup B) \cap (A^c \cup B^c) && \text{(Identity)} \\
&= (A \cup B) \cap (A \cap B)^c && \text{(De Morgan's law)}
\end{aligned}$$

Relations and Orders

Question 1

Let Σ be a finite alphabet.

We define a relation $R \subseteq \Sigma^* \times \Sigma^*$ as follows:

$$(w, v) \in R \quad \text{if, and only if,} \quad \exists x \in \Sigma^*. (wx = v)$$

Show that R is a partial order.

To show that R is a partial order, we need to show that it is reflexive, antisymmetric, and transitive.

1. **Reflexive:** For all $w \in \Sigma^*$ we observe that $w = w\lambda$. Hence, $(w, w) \in R$, and so R is reflexive.
2. **Antisymmetric:** Suppose $(w, v) \in R$ and $(v, w) \in R$. Then there exists $x, y \in \Sigma^*$ such that $v = wx$ and $w = vy$. So $w = wxy$, and so $\text{length}(xy) = 0$ and $xy = \lambda$. Therefore $x = y = \lambda$ so $w = v\lambda = v$. So R is antisymmetric.
3. **Transitive:** Suppose $(w, v), (v, x) \in R$. Then there exists y, z such that $wy = v$ and $vz = x$. But then $w(yz) = (wy)z = vz = x$ so $(w, x) \in R$. Hence R is transitive.

COMP2111 lecture notes: sets, languages, relations, and functions.

Johannes Åman Pohjola
University of New South Wales

February 24, 2023

Contents

1	Introduction	2
2	Sets	2
2.1	Definition by enumeration	2
2.2	Set equality	2
2.3	Set specification	3
2.4	Set operators	4
2.5	Derived operators	4
2.6	Laws of Set Operations	4
2.7	Duality	5
2.8	Powerset	6
2.9	Cardinality	6
3	Formal languages	6
3.1	Basic notions	7
3.2	Length and concatenation	7
3.3	Sets of words	7
3.4	Operations on languages	8
4	Relations	9
4.1	Cartesian products	9
4.2	Relations	10
4.3	Image and converse	10
4.4	Binary relations on a single domain	10
4.5	Properties of binary relations	10
4.6	Equivalence Relations and Partitions	11
4.7	Partial orders	12
4.8	Relational composition	12
5	Functions	13
5.1	Mathematical functions	13
5.2	Procedures as relations	15

1 Introduction

We promised that this course would be largely self-contained: we would teach, not assume, the mathematical and logical foundations we'll need. In this document, we begin by reviewing some of the basic building blocks we'll use throughout the course: sets, languages, relations, and functions.

2 Sets

A *set* is a collection of *elements* (aka *members*). What counts as an “element” is not fixed, but can vary from application area to application area. In mathematics, we are often interested in sets where the elements are numbers. But set theory works the same for other notions of elements: we could have sets of strings, sets of programs, sets of cute baby animals, sets of literary critics, or whatever we like. We can even have sets whose elements are themselves sets.

On close inspection, the notion of set turns out to be rather slippery. (You may recall Russell's paradox from Week 1). Nailing down this slippery notion is important for studying the foundations of mathematics, and particularly so if one wants to use set theory to study itself. In this course, however, we won't do that: we are not interested in set theory for its own sake. Rather, we want to use it to model computer programs and systems. Therefore, we will not attempt to define set theory from first principles. Rather, we limit ourselves to sketching a common vocabulary we can use to define and describe sets. Nonetheless, we will attempt to develop this vocabulary in a way that allows us stay clear of paradoxes such as Russell's.

2.1 Definition by enumeration

The simplest way of defining a set is by explicitly enumerating its elements. For example, here's a set with three elements, 1, 4 and 5:

$$\{1, 4, 5\}$$

The infix symbol $\in$ is used to assert that elements are contained in sets as follows:

$$\begin{array}{ll} 1 \in \{1, 4, 5\} & \text{“1 is an element of } \{1, 4, 5\}\text{”} \\ 4 \in \{1, 4, 5\} & \text{“4 is an element of } \{1, 4, 5\}\text{”} \\ 17 \notin \{1, 4, 5\} & \text{“17 is *not* an element of } \{1, 4, 5\}\text{”} \end{array}$$

We snuck in an extra symbol $\notin$ for asserting that an element is *not* contained in a set. This is mainly for convenience: we can consider $a \notin S$ as syntactic sugar for $\neg a \in S$.

An important special case is the empty set, which is defined by an empty enumeration:

$$\{\}$$

It is the most boring set: it has no elements. The empty set is sometimes written $\emptyset$.

2.2 Set equality

When we define a set by enumeration, the order we write the elements in is irrelevant. For example, these sets are considered equal.

$$\{1, 4, 5\} = \{5, 4, 1\}$$

The multiplicity of the elements is also irrelevant: writing an element twice changes nothing. For example, these sets are considered equal:

$$\{1, 4, 5\} = \{1, 1, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5\}$$

The only thing that is relevant is which elements a set contains. In other words, a set A is uniquely defined by how the membership relation $\in$ operates on A :

Definition 1 (Set equality). Two sets A and B are considered equal, written $A = B$, if for all x it holds that

$$x \in A \text{ if and only if } x \in B$$

There is also a standard notion of inequality on sets, called *subset* and written $\subseteq$.

Definition 2 (Subset). A set A is a subset of a set B , written $A \subseteq B$ if for all x it holds that

$$x \in A \text{ implies } x \in B$$

For example, $\{1, 2\} \subseteq \{1, 2, 3\}$.

2.3 Set specification

Sometimes, we're too lazy to write an explicit enumeration. Other times, we just can't. For example, the set of all natural numbers (written $\mathbb{N}$) is infinite, and we only have finite ink. Another common way of defining a set is by *specification* (aka *set comprehension*), where we implicitly define a set by specifying a property P that all its elements satisfy. They take the following general form:

$$\{ x : P(x) \} \quad \text{“the set of all } x \text{ such that } P(x) \text{ holds”}$$

Here $P(x)$ stands for some logical formula where x is a free variable. The idea is that $a \in \{ x : P(x) \}$ holds precisely when $P(a)$ holds.

For example, the set $\{2, 3, 7, 11, 13, 17, 19\}$ could also be written as

$$\{ n : n \in \mathbb{N} \text{ and } n \text{ is prime and } n < 20 \} \quad \text{“the set of all primes below 20”}$$

On avoiding paradoxes Unrestricted set comprehensions would represent a huge gaping hole in the foundations of mathematics, as first pointed out by Bertrand Russell. The issue is that we can write the following set comprehension:

$$\{ x : x \notin x \} \quad \text{“the set of all sets that do not contain themselves”}$$

For short, let $Y = \{ x : x \notin x \}$. Does this innocent-looking but strangely self-referential set contain itself? Or more formally, is it true that $Y \in Y$? It turns out that if $Y \in Y$ then $Y \notin Y$, and vice versa. (Take a moment to convince yourself of this).

In other words, we have a contradiction. Contradiction is the death of logic: if your logic (or set theory, or systems modelling language) contains a contradiction, then we lose the ability to distinguish between truth and falsehood.

This suggests that set comprehensions need to be tamed somehow. What we will do is insist that all set comprehensions are of the following form:

$$S = \{ x : x \in \mathcal{U} \text{ and } P(x) \} = \{ x \in \mathcal{U} : P(x) \}$$

$\mathcal{U}$ here denotes a “universe”, which informally is the set of everything that is currently under consideration. For example, if we're using set theory to reason about natural numbers, the universe might be $\mathbb{N}$. Other applications demand other universes.

We will sometimes leave the universe implicit if it is irrelevant or obvious from the context. But even when it's only present in invisible ink, it serves an important purpose: it makes it so that comprehension cannot be used to conjure sets out of thin air. Instead, they must be *carved out* from some set $\mathcal{U}$ that we have already obtained by other means.

2.4 Set operators

In the same way that arithmetic comes with operators (such as $+$ and $-$) that allow us to combine numbers into new numbers, set theory has some standard operators on sets.

Union The *union* ($\cup$) of two sets includes everything that is in *either* set. Formally, $x \in A \cup B$ iff $x \in A$ or $x \in B$.

For example, $\{1, 2\} \cup \{2, 3\} = \{1, 2, 3\}$.

Intersection The *intersection* ($\cap$) of two sets includes everything that is in *both* sets. Formally, $x \in A \cap B$ iff $x \in A$ and $x \in B$.

We say that A, B are *disjoint* if $A \cap B = \emptyset$.

For example, $\{1, 2\} \cap \{2, 3\} = \{2\}$.

Complement The *complement* ($\cdot^C$) of a set includes everything in the universe that is *outside* the set. Formally, $x \in A^C$ iff $x \notin A$ and $x \in \mathcal{U}$.

For example, if $\mathcal{U} = \mathbb{N}$ then $\{1, 2\}^C = \{0, 3, 4, 5, 6, 7, 8, 9, 10, \dots\}$.

2.5 Derived operators

Here are two common derived operators:

Set difference The *difference* ($\setminus$) of two sets A and B is obtained by removing all elements of B from A . Formally, $x \in A \setminus B$ iff $x \in A$ and $x \notin B$.

For example, $\{1, 2\} \setminus \{2, 3\} = \{1\}$.

Symmetric difference The *symmetric difference* of two sets includes everything that occurs in *exactly one* of the two sets. Formally, $x \in A \oplus B$ iff either (a) $x \in A$ and $x \notin B$, or (b) $x \notin A$ and $x \in B$.

For example, $\{1, 2\} \oplus \{2, 3\} = \{1, 3\}$.

...derived how? We consider these derived because we can define them in terms of the existing operators as follows:

$$\begin{aligned} A \setminus B &\stackrel{\text{def}}{=} A \cap B^C \\ A \oplus B &\stackrel{\text{def}}{=} (A \setminus B) \cup (B \setminus A) \end{aligned}$$

2.6 Laws of Set Operations

The three basic set operations—union, intersection, and complement—are the building blocks of an algebra of sets, in the same way that $+$, $*$ are the building block of an algebra of numbers. The basic laws of this algebra are shown in Table 1.

The commutativity, associativity and distribution laws should all be familiar from the laws of arithmetic: imagine reading $+$ and $*$ instead of $\cup$ and $\cap$. Similarly, the identity law $A \cup \emptyset = A$ is analogous to $x + 0 = x$. The second identity law, $A \cap \mathcal{U} = A$, has no counterpart in arithmetic. In words, since A is part of the universe, the intersection of A and the universe is just A . The two complementation laws describe the relationship between a set and its complement: the union of a set and its complement is the universe, and the intersection of a set and its complement is empty.

All the 10 basic laws are provable from the definitions of set equality and the relevant operators. We will not prove them here, but the reader may wish to try one or two as an exercise.

Commutativity	$A \cup B = B \cup A$
	$A \cap B = B \cap A$
Associativity	$(A \cup B) \cup C = A \cup (B \cup C)$
	$(A \cap B) \cap C = A \cap (B \cap C)$
Distribution	$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
Identity	$A \cup \emptyset = A$
	$A \cap \mathcal{U} = A$
Complementation	$A \cup (A^c) = \mathcal{U}$
	$A \cap (A^c) = \emptyset$

Table 1: Laws of Set Operations

Idempotence	$A \cap A = A$
	$A \cup A = A$
Double complementation	$(A^c)^c = A$
Annihilation	$A \cap \emptyset = \emptyset$
	$A \cup \mathcal{U} = \mathcal{U}$
de Morgan's Laws	$(A \cap B)^c = A^c \cup B^c$
	$(A \cup B)^c = A^c \cap B^c$

Table 2: Derived Laws of Set Operations

Some useful derived laws are shown in Table 2. *Derivable* here means provable using equational reasoning with just the laws from Table 1. For example, we can prove idempotence like this:

$$\begin{aligned}
A &= A \cup \emptyset && \text{(Identity)} \\
&= A \cup (A \cap A^c) && \text{(Complementation)} \\
&= (A \cup A) \cap (A \cup A^c) && \text{(Distributivity)} \\
&= (A \cup A) \cap \mathcal{U} && \text{(Complementation)} \\
&= (A \cup A) && \text{(Identity)}
\end{aligned}$$

Note that we never had to worry about how $=, \cup, \cap, \emptyset$ and friends are *defined* in the above proof: we only needed to know what laws they satisfy.

2.7 Duality

The attentive reader may have noticed that the laws in Table 1 come in pairs, called *duals*. For example, two laws share the name Distribution. In each pair, the laws are identical except that every $\cup$ and $\cap$ has been flipped upside down, and the $\emptyset$ s and the $\mathcal{U}$ s have been flipped. Isn't that curious?

This is not a coincidence! In fact, it suggests a striking general principle known as the *principle of duality*. Let's first define what a dual is:

Definition 3. If A is a set defined using $\cap, \cup, \emptyset$ and $\mathcal{U}$, then $\text{dual}(A)$ is the expression obtained by replacing $\cap$ with $\cup$ (and vice-versa) and $\emptyset$ with $\mathcal{U}$ (and vice-versa).

For example, $\text{dual}(A \cap (B \cup \emptyset)) = A \cup (B \cap \mathcal{U})$

Theorem 1 (Principle of Duality). *If you can prove $A_1 = A_2$ using the Laws of Set Operations then you can prove $\text{dual}(A_1) = \text{dual}(A_2)$*

This theorem holds because any such proof of $A_1 = A_2$ can be converted into a proof of $\text{dual}(A_1) = \text{dual}(A_2)$ by replacing each invocation of a law with an invocation of the dual law. For example, here is a proof of the other idempotence law:

$$\begin{aligned}
 A &= A \cap \mathcal{U} && \text{(Identity)} \\
 &= A \cap (A \cup A^c) && \text{(Complementation)} \\
 &= (A \cap A) \cup (A \cap A^c) && \text{(Distributivity)} \\
 &= (A \cap A) \cup \emptyset && \text{(Complementation)} \\
 &= (A \cap A) && \text{(Identity)}
 \end{aligned}$$

Compare this proof with the one from Section 2.6 to see the duality at work. Textually it is the exact same proof, except all symbols have been replaced with their dual symbols. This second derivation was only for illustrative purposes: we could save ourselves the effort by invoking the principle of duality.

2.8 Powerset

A very common construction in set theory is the *power set*. The power set of X is written $\text{Pow}(X)$, and is the set of all subsets of X . For example, if $X = \{1, 2\}$ then $\text{Pow}(X) = \{\{\}, \{1\}, \{2\}, \{1, 2\}\}$. Note that $\{\} \in \text{Pow}(X)$ holds for any X , because the empty set is a subset of every set.

2.9 Cardinality

It is often relevant to count the number of elements in a set. The number of elements in X is called the *cardinality* of X , and written $|X|$ or (equivalently) $\#(X)$ or $\text{card}(X)$.

For example, $|\{1, 2, 3\}| = 3$ and $|\emptyset| = 0$.

The cardinality of a powerset $\text{Pow}(X)$ can be computed from the cardinality of X as follows:

Theorem 2. *When X is finite, $|\text{Pow}(X)| = 2^{|X|}$*

Try a few simple examples to convince yourself of this.

When X is finite, we have that $|X| \in \mathbb{N}$. The cardinality of infinite sets is a meaningful concept in set theory, but such cardinalities will obviously not be natural numbers: they will be some representation of infinity. Where things get strange is that we distinguish between different “sizes” of infinity; for example, $|\mathbb{N}| < |\mathbb{R}|$. Don’t worry too much about that—it’s beyond the scope of this course. We will mainly be concerned with the cardinality of finite sets.

3 Formal languages

As you probably know already, a *string* in programming is just a sequence of characters. A *formal language* is a set of strings—that’s essentially all there is to it.

For example, here are two strings:

```
#include <stdio.h>
int main() {
    printf("Hello_world!");
    return 0;
}
```

```
#include <stdio.h>>>>>
int main() {
    printf("Hello_world!");
    return 0;
}
```

The string on the left is a member of the set of syntactically valid C programs; the string on the right is not. Formal languages is the mathematical formalism for defining sets of strings (such as the syntactically valid C programs) precisely and unambiguously.

Formal languages have many applications to system modelling and design beyond syntax, however. For example, they are useful for describing the behaviour interactive systems. We will return to this theme later in the course.

3.1 Basic notions

A formal language needs a finite, non-empty set Σ called the *alphabet*. A *word* is a finite sequence of elements from the alphabet Σ . We will use the variable names w, v to range over words.

For example, for the two code blocks above, the alphabet under consideration would be the set of ASCII characters (or perhaps UTF-8 characters). If we're considering machine code on the level of ones and zeroes, we might choose $\Sigma = \{0, 1\}$.

In programming, strings are traditionally written inside string quotes, like "Hello". In formal languages, we dispose of the quotes and just write *Hello*. The empty word is written λ or ϵ .¹

3.2 Length and concatenation

In this section, we'll use the running example alphabet $\Sigma = \{a, b\}$.

The length of a word w is written $\text{length}(w)$. It has the obvious definition. For example, $\text{length}(aaa) = 3$ and $\text{length}(\lambda) = 0$.

Two words can be combined by forming their *concatenation*. This is analogous to string concatenation in programming. The traditional notation is to simply juxtapose the two words: the concatenation of w and v is written wv .

For example, if $w = abab$ and $v = bbb$, $wv = ababbbb$.

3.3 Sets of words

We write Σ^k for the set of all words of length k . Hence, for example, $aaa \in \{a, b\}^3$ but $abab \notin \{a, b\}^3$.

Two relevant special cases occur: Σ^0 (the set of all words of length 0) is just $\{\lambda\}$,² and Σ^1 (the set of all words of length 1) is just Σ .

Definition 4. The set of all (finite) words, written Σ^* , is defined as follows:

$$w \in \Sigma^* \quad \text{iff} \quad w \in \Sigma^k \text{ for some } k.$$

We will also write Σ^+ for the set of all non-empty (finite) words. Note that $\Sigma^+ = \Sigma - \{\lambda\}$.

Example 1 (Decimal numbers). We will use the tools at our disposal to define the set of well-formed decimal numbers to at most two places. That is, our language should include strings such as 3.14, -3.14 and 3, but not 3.1415 or 3.1.4.1.5 or λ or .1.

For our alphabet, the obvious choice is $\Sigma = \{-, ., 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

Let D denote the set of decimal numbers. We define $w \in D$ to hold if and only if $w \in \Sigma^*$ and all the following hold

- w contains at most one instance of $-$, and if it contains an instance then it is the first symbol.

¹This may cause confusion if your alphabet includes Greek letters.

²Note that $\{\}$ and $\{\lambda\}$ are different languages! The former contains no words. The latter contains one word, namely the empty word.

- w contains at most one instance of $.$, and if it contains an instance then it is preceded by a symbol in $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, and followed by either one or two symbols in that set.
- w contains at least one symbol from $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Here is a more CS-flavoured example:

Example 2 (HTML documents). Take $\Sigma = \{<html>, </html>, <head>, </head>, <body>, \dots\}$. The (language of) valid HTML documents is loosely described as follows:

1. Starts with `<html>`
2. Next symbol is `<head>`
3. Followed by zero or more symbols from the set of *HeadItems* (defined elsewhere)
4. Followed by `</head>`
5. Followed by `<body>`
6. Followed by zero or more symbols from the set of *BodyItems* (defined elsewhere)
7. Followed by `</body>`
8. Followed by `</html>`

Languages, such as HTML and C, typically have such formal descriptions of their syntax defined in standards documents, in a mix of natural language and formal notation.

This example makes an important technical point: alphabet symbols need not correspond to textual symbols, but can be more (or less) abstract. In compiler implementation, multi-character symbols such as those in Example 2 are often called *tokens*. The first step of a compiler (called *lexing*) is precisely to translate the input (a raw text string) to a word over a token alphabet. If this process fails, the compiler will usually abort with a syntax error.

3.4 Operations on languages

Languages are a special case of sets, so all the set operations apply to languages.

For example, suppose *French* and *English* denotes the set of sentences in French and English, respectively. The language $\text{French} \cup \text{English}$ would include the sentences “Je ne suis pas fou.” and “I am not crazy.”. It would not, however, include “I ne is pas crazy”—this sentence is a member of neither French nor English. $\text{French} \cap \text{English}$ would include none of the above, but may contain some French phrases that have been loaned into English, such as “carte blanche”.

There are two important set operations that are specific to languages.

Concatenation The concatenation of two languages X and Y is defined as

$$XY = \{xy : x \in X \text{ and } y \in Y\}$$

The idea is that a word of XY is the concatenation of a word from X and a word from Y . Thus, a sentence from FrenchEnglish might be “Je ne suis pas fou.I am not crazy.”.

Kleene star Kleene star is used for self-concatenation. X^* is the set of words that are made up by concatenating 0 or more words in X . Thus, if *French* contains French sentences then French^* would correspond to French paragraphs. Note that we've seen the Kleene star before, when we defined the set of words over an alphabet, Σ^* .

Here are a few basic examples of how the set operations for languages work:

Example 3. Let $A = \{aa, bb\}$ and $B = \{\lambda, c\}$ be languages over $\Sigma = \{a, b, c\}$.

- $A \cup B = \{\lambda, c, aa, bb\}$
- $AB = \{aa, bb, aac, bbc\}$
- $AA = \{aaaa, aabb, bbaa, bbbb\}$
- $A^* = \{\lambda, aa, bb, aaaa, aabb, bbaa, bbbb, aaaaaa, \dots\}$
- $B^* = \{\lambda, c, cc, ccc, cccc, \dots\}$
- $\{\lambda\}^* = \{\lambda\}$
- $\emptyset^* = \{\lambda\}$

4 Relations

You've seen relations before. For example, $<$ ("less than") is a relation on numbers. $\subseteq$ is a relation on sets.

Relations are ubiquitous in computer science, and therefore studying relations and their properties helps with formalisation, implementation and verification of programs and systems. For example:

- Databases are collections of relations
- Common data structures (e.g. graphs) are relations
- Any ordering is a relation
- Functions, procedures and programs are relations between their inputs and outputs

Similarly to formal languages, relations can be viewed as just another special case of sets. To see how, we need to develop a few preliminary notions.

4.1 Cartesian products

Given two sets S and T , we may form their (binary) *Cartesian product* $S \times T$. The Cartesian product is a set of *pairs* of the form (s, t) such that $s \in S$ and $t \in T$. With pairs, unlike sets, the order matters: (s, t) and (t, s) are different (if $s \neq t$).

A familiar example is (x, y) -coordinates, which are elements of $\mathbb{R} \times \mathbb{R}$.

The above defines the special case of pairs. There are also triples (x, y, z) , five-tuples (a, b, c, d, e) , and in general n -tuples for any natural number n . This suggests the following more general construction for n -ary Cartesian products:

$$\times_{i=1}^n S_i \stackrel{\text{def}}{=} \{ (s_1, \dots, s_n) : s_k \in S_k, \text{ for all } k \text{ such that } 1 \leq k \leq n \}$$

4.2 Relations

An n -ary relation is a subset of a Cartesian product of n sets. In the context of relations, each of these n sets is called a *domain*.

For example, consider the relation $<$ on the natural numbers. We can think of this relation as a set, which satisfies $< \subseteq \mathbb{N} \times \mathbb{N}$. In this reading, when we write $x < y$ in ordinary mathematical prose, that is shorthand for $(x, y) \in <$. In this view, one definition of $<$ would be

$$\{ (n, m) : n, m \in \mathbb{N} \text{ and there exists } k \in \mathbb{N} \text{ such that } n + k + 1 = m \}$$

When R is a binary relation, we will write $(x, y) \in R$ and $x R y$ and $R(x, y)$ interchangeably.

4.3 Image and converse

We can think of binary relations $R \subseteq S \times T$ as defining a notion of correspondence between S and T . This leads to the notion of *image*:

Definition 5 (Image of a relation). Let $R \subseteq S \times T$ be a binary relation and let $A \subseteq S$. The image of A through R is defined as

$$R(A) \stackrel{\text{def}}{=} \{t \in T : (s, t) \in R \text{ for some } s \in A\}$$

In words, the image of a set A through a relation R is the set of everything that is related to A . For example, the image of $\{2\}$ through the $<$ relation on natural numbers is the set of all $x \in \mathbb{N}$ such that $x < 2$, namely $\{0, 1\}$.

Every binary relation has a *converse*, which is obtained by taking the relation backwards. For example, the converse of $<$ is $>$.

Definition 6 (Converse of a relation). Let $R \subseteq S \times T$ be a binary relation. The converse of R , written $R^{\leftarrow}$, is defined as follows:

$$R^{\leftarrow} \stackrel{\text{def}}{=} \{(t, s) \in T \times S : (s, t) \in R\}$$

Observe that $(R^{\leftarrow})^{\leftarrow} = R$.

4.4 Binary relations on a single domain

Binary relations on a single domain form a common, and important, special case. Many familiar relations ($=$, $<$, $\subseteq$) fall in this bucket. We say that ' R is a relation on S ' if $R \subseteq S \times S$.

Some other common "boilerplate" relations that fall into this category are the following:

The identity relation $E = \{ (x, x) : x \in S \}$

The empty relation $\emptyset$

The universal relation $\mathcal{U} = S \times S$

The identity relation relates everything to itself; the empty relation relates nothing; and the universal relation relates everything. These relations are rather trivial, but they are nonetheless important for building a theory of relations, for the same reasons that 0 and 1 are important for building a theory of numbers.

4.5 Properties of binary relations

For binary relations on a single domain, there are five standard properties that you should know about. They are summarised in Table 3 and elaborated on below.

(R)	reflexive	$(x, x) \in R$	$\forall x \in S$
(AR)	antireflexive	$(x, x) \notin R$	$\forall x \in S$
(S)	symmetric	$(x, y) \in R \rightarrow (y, x) \in R$	$\forall x, y \in S$
(AS)	antisymmetric	$(x, y), (y, x) \in R \rightarrow x = y$	$\forall x, y \in S$
(T)	transitive	$(x, y), (y, z) \in R \rightarrow (x, z) \in R$	$\forall x, y, z \in S$

Table 3: Properties of binary relations.

Reflexivity A *reflexive* relation is one that relates every element of the domain to itself. Examples include $=$ and $\leq$.

A non-example follows here. Let $R \subseteq \mathbb{N} \times \mathbb{N}$ be defined by $R = \{(1, 1), (2, 2)\}$. This relation is *nonreflexive*; for example $(3, 3) \notin R$. This illustrates that the domain is an important component of a relation's definition. If the domain had been just $\{1, 2\}$, the relation would have been reflexive.

Antireflexivity An *antireflexive* relation is one that maps no element of the domain to itself. An example includes $<$. Note the distinction between nonreflexive and antireflexive relations: the relation R above is nonreflexive, but *not* antireflexive.

Symmetry A *symmetric* relation is a two-way relation. In terms of family relations, “is a sibling of” is symmetric (if I have a sibling, they are my siblings too) but “is an ancestor of” is not symmetric (I am not the ancestor of my ancestors). In more mathematical examples, $=$ is symmetric, but $<$ and $\leq$ are not.

A symmetric relation R satisfies the equation $R^{\leftarrow} = R$.

Antisymmetry An *antisymmetric* relation cannot have two different elements that are related both ways. $<$ and $\leq$, and $=$ are all antisymmetric. Of these, $=$ may be most surprising because it is also symmetric. The key idea is that antisymmetry only prohibits two-way relations between *different* elements. We have that if $x = y$ then $y = x$, but if so x and y are not different.

Example of relations that are not antisymmetric are the universal relation, and the “is a sibling of” relation.

Transitivity For a *transitive* relation, taking two steps through the relation doesn't add any new relationships that weren't already available by taking a single step. For example, $\leq$ is transitive: whenever we have $x \leq y$ and $y \leq z$, then clearly $x \leq z$.

A non-transitive relation is the “is one larger than” relation on $\mathbb{N}$, defined as $R = \{(x, y) : y = x + 1\}$. Here we have $1 R 2$ and $2 R 3$, but $1 \not R 3$.

4.6 Equivalence Relations and Partitions

An important special case is relations that are both reflexive, symmetric and transitive. We call such relations *equivalence relations*.

Intuitively, an equivalence relation captures the idea that related elements can be considered interchangeable for some purposes. For example, the equivalence relation

$$R = \{(n, m) : n \bmod 7 = m \bmod 7\} \subseteq \mathbb{N} \times \mathbb{N}$$

would be appropriate in a context where we are doing modular arithmetic, and are only interested in numbers modulo seven: therefore, in context 1 and 8 are interchangeable to us.

To mathematically characterise which objects are interchangeable under some equivalence relation R , we have the notion of *equivalence class*.

Definition 7 (Equivalence class). *If $R \subseteq S \times S$ is an equivalence relations, then the equivalence class $[s]$ is defined as follows for all $s \in S$:*

$$[s] = \{ t \in S : tRs \}$$

The collection of all equivalence classes $[S]_R = \{ [s] : s \in S \}$ forms a partition of the domain:

$$S = \bigcup_{s \in S} [s]$$

Note that equivalence classes are disjoint: if $(s, t) \notin R$ then $[s] \cap [t] = \emptyset$

4.7 Partial orders

Another important special case of binary relations is partial orders.

A binary relation R is a (*non-strict*) *partial order* if it is reflexive, transitive and anti-symmetric. A (*strict*) *partial order* is anti-reflexive, transitive and anti-symmetric.

$=$ is not a partial order (because it is symmetric), $<$ is a strict partial order, and $\leq$ is a non-strict partial order.

We call these *partial orders* because they allow for some elements to be uncomparable. For example, consider the “is a prefix of” relation $R \subseteq \Sigma^* \times \Sigma^*$ defined as

$$R = \{ (v, w) : \text{there exists } u \in \Sigma^* \text{ such that } vu = w \}$$

R is a (non-strict) partial order such that some words are completely unrelated by R . We have both $ab \not R ba$ and $ba \not R ab$.

A partial order where distinct domain elements are related one way or the other (as is the case with $<$ and $\leq$ on numbers) we call a *total order*.

When $R \subseteq S \times S$ is a partial order, we refer to (S, R) as a poset (short for “partially ordered set”).

Minimal and minimum elements We say that $x \in S$ A *minimal* element of a poset $(S, \preceq)$ if there is no $y \in S$ such that $y \preceq x$. Intuitively, this means that there is no element in S that is smaller than x . For example, the minimal element of $(\mathbb{N}, <)$ is 0. The poset $(\mathbb{Z}, <)$ has no minimal element because there is no smallest integer.

Because posets may have uncomparable elements, the minimal element is not unique in general. The poset $(\{a, b, c\}, \{(a, c), (b, c)\})$ has two minimal elements, a and b .

A *minimum* element, by contrast is an $x \in S$ such that $x \preceq y$ for all $y \in S$. Intuitively, everything else in the domain is below the minimum element, and if a minimum element exists, it is also minimal.

There are also the dual notions of *maximal* and *maximum* elements. A maximal element is an x such that there is no y with $x \preceq y$, and a maximum element is an x such that $y \preceq x$ for all $y \in S$.

4.8 Relational composition

It is often useful to compose binary relations with each other. Given two binary relations $R_1 \subseteq S \times T$ and $R_2 \subseteq T \times U$ with matching domains—note the two occurrences of T — we can construct their *composition*, denoted $R_1; R_2$.

$$R_1; R_2 \stackrel{\text{def}}{=} \{ (s, u) \in S \times U : \exists t. (s, t) \in R_1 \text{ and } (t, u) \in R_2 \}$$

Intuitively, $R_1; R_2$ means “first R_1 then R_2 ”: it relates everything that can be reached by one hop through R_1 , and then one hop through R_2 .

For example, for the standard $<$ relation on the natural numbers, we can construct the self-composition $<;<$. If $<$ is less than, $<;<$ is “at least two less than”.

Recalling the boilerplate relations from Section 4.4, we can now sketch out the contours of an algebra of relations. The following laws are all provable from the definitions we have given:

$$\begin{array}{llllll} R;(S;T) = (R;S);T & R;E = E;R = R & R;\emptyset = \emptyset;R = \emptyset & (R;S)^{\leftarrow} = S^{\leftarrow};T^{\leftarrow} & E^{\leftarrow} = E \\ \emptyset^{\leftarrow} = \emptyset & \mathcal{U}^{\leftarrow} = \mathcal{U} & (R^{\leftarrow})^{\leftarrow} = R \end{array}$$

5 Functions

Functions are ubiquitous in mathematics and computer science, but it’s important to keep in mind that a mathematical function, and a function in a computer program, are not the same thing.

Mathematical functions, such as $f(x) = x^2 - 3$, map inputs to outputs. Let’s say the f from the previous sentence is a function from $\mathbb{Z}$ to $\mathbb{Z}$. This means that for every $x \in \mathbb{Z}$, $f(x)$ is a value from $\mathbb{Z}$.

Now consider:

```
int f(int x) {
    return(x * x - 3);
}
```

Clearly the C function f implements the mathematical function f .³ But it’s not quite correct to say that $f(x)$ is an **int**. Rather $f(x)$ is an expression that, *when evaluated*, will eventually return an **int**.

Here are some examples of C functions that make the distinction even clearer:

```
double g() {
    return(Math.random());
}

int h(int x) {
    printf("Checkmate, _maths!!");
    return x;
}
```

For mathematical functions, we always get the same value back: $f(x) = f(x)$. But $g() == g()$ will not necessarily be true. h behaves like the identify function, except it also produces side effects.

Despite this distinction, the word function tends to be used for both notions. When we want to emphasise the distinction, functions in the programming sense are sometimes called *subroutines* or *procedures*.

5.1 Mathematical functions

Formally, a *function*, $f : S \rightarrow T$, is a binary relation $f \subseteq S \times T$ such that for all $s \in S$ there is exactly one $t \in T$ such that $(s, t) \in f$. We write $f(s)$ for the unique element related to s .

Every function has a *domain*, *co-domain* and *image*. For a function $f : S \rightarrow T$, they are defined as follows:

		Symbol	
S	<i>domain</i> of f	$\text{dom}(f)$	(inputs)
T	<i>co-domain</i> of f	$\text{codom}(f)$	(possible outputs)
$f(S)$	<i>image</i> of f	$\text{im}(f)$	(actual outputs)
	$= \{ f(x) : x \in \text{dom}(f) \}$		

³Here we ignore the complication that there are infinitely many integers, but only finitely many of them fit in an **int**.

For example, consider the function $abs : \mathbb{Z} \rightarrow \mathbb{Z}$ that computes the absolute value of its input. It has domain $\mathbb{Z}$, co-domain $\mathbb{Z}$ and image $\mathbb{N}$.

Functions are most often presented equationally:

$$abs(x) = sgn(x) * x$$

sgn here denotes the sign function. To emphasise the point that functions are just relations with special properties, here are some equivalent alternate presentations:

$$abs = \{ (x, sgn(x) * x) \in \mathbb{Z} \times \mathbb{Z} : T \} \quad abs = \{ (x, -x) \in \mathbb{Z} \times \mathbb{Z} : x < 0 \} \cup \{ (x, x) \in \mathbb{Z} \times \mathbb{Z} : x \geq 0 \}$$

Composition and identity Function composition, written $f \circ g$, denotes a function that given an input first applies g , then applies f to the intermediate result:

$$(f \circ g)(x) \stackrel{\text{def}}{=} f(g(x))$$

This composition is defined whenever $\text{im}(f) \subseteq \text{dom}(g)$.

Since functions are just relations, the perceptive reader may have noticed that relational composition ; also applies to functions. In fact, when f and g are functions we have

$$f;g = f \circ g$$

The identity function $\text{Id}_S : S \rightarrow S$ is the most boring function: it just returns its input unchanged. It may not seem like much, but it has nice algebraic properties.

$$f \circ (g \circ h) = (f \circ g) \circ h \quad f \circ \text{Id}_S = f \quad \text{Id}_S \circ f = f$$

Here we subscript the identity function with its intended domain, emphasising that the identity functions for two distinct domains are formally not the same function. We will elide the domain when it is irrelevant, obvious or just in the way.

Injectivity and surjectivity The most important properties of functions are injectivity and surjectivity.

An *injective* function maps different inputs to different outputs:

$$f(x) = f(y) \rightarrow x = y$$

If a function is injective, no information is lost as a result of applying f . For example, any encryption function worth its salt had better be injective—otherwise, the same ciphertext could be obtained from different cleartexts, leaving the recipient to guess the intended message.

A *surjective* function is a function whose possible outputs cover the entire co-domain:

$$\text{im}(f) = \text{codom}(f)$$

Functions that are both injective and surjective are called *bijective*. Bijective functions have the property that they are invertible. Specifically, if $f : S \rightarrow T$ is bijective then $f^{\leftarrow}$ is a function such that $f^{\leftarrow}(f(s)) = s$ for all s .

When f is bijective, we will therefore write $f^{\leftarrow}$ as f^{-1} (pronounced “the inverse of f ”).

Partial functions A *partial function* $f : S \rightarrow T$ is a binary relation $f \subseteq S \times T$ such that for all $s \in S$ there is at most one $t \in T$ such that $(s, t) \in f$. That is, it is a function $f : S' \rightarrow T$ for some $S' \subseteq S$.

The canonical example of a partial function is division on the real numbers: its domain is $\mathbb{R} \times \mathbb{R}$, yet it is undefined when the denominator is 0.

5.2 Procedures as relations

While functions in the programming sense are different from functions in the mathematical sense, they share an important thing in common: they can both be modelled as relations between input and output. For example, consider the following function:

```
int f(int x) {  
    while(x != 0) {  
        x--;  
    }  
    return 0;  
}
```

This function will return 0, except when x is negative. In the latter case, it loops forever. To capture this, we can model f as the following relation between inputs and outputs as follows:

$$\{(0,0), (1,0), (2,0), \dots\}$$

The fact that this relation has no input-output pairs for negative inputs models the fact that the function returns no value for negative x . Here's another example, where we assume that `roll_d6()` returns a random integer between 1 and 6:

```
int nd6(int n) {  
    int sum=0;  
    while(n != 0) {  
        sum = sum + roll_d6();  
        n--;  
    }  
    return sum;  
}
```

The intuition is that $\text{nd6}(n)$ returns the result of rolling n 6-sided dice. An input-output relation R representing this function could be defined as follows:

$$R = \{ (n, m) \in \mathbb{Z} \times \mathbb{Z} : 0 \leq n \leq m \leq 6n \}$$

For example, the fact that rolling 2d6 can yield e.g. 3 or 11 is modelled by the fact that both $2 R 3$ and $2 R 11$.

This way of modelling functions as relations deliberately abstracts from many aspects of the function's behaviour. It ignores function's runtime complexity, and the possibility of integer overflow. It only considers what results are *possible* given an input, and completely ignores their probability.

However, this abstraction is useful: for most purposes knowing what's possible suffices to prove that a program meets its specification.

This doesn't account for the possibility that programs have side-effects. For example, if a function changes the value of a global variable before returning, this is not modelled by its input-output relations. For such programs, it is better to model programs as relations between initial and final states. More on that later in the course.

Acknowledgements

These notes borrow some from material developed by previous COMP2111 staff, including but not limited to Paul Hunter, Carroll Morgan, Liam O'Connor and Christine Rizkallah.

COMP2111 Week 3
Term 1, 2023
Recursion and induction

Summary of topics

- Recursion
- Recursive Data Types
- Induction
- Structural Induction

Summary of topics

- Recursion
- Recursive Data Types
- Induction
- Structural Induction

Recursion

Fundamental concept in Computer Science

- Recursion in algorithms: Solving problems by reducing to smaller cases
 - Factorial
 - Towers of Hanoi
 - Mergesort, Quicksort

Recursion

Fundamental concept in Computer Science

- Recursion in algorithms: Solving problems by reducing to smaller cases
 - Factorial
 - Towers of Hanoi
 - Mergesort, Quicksort
- Recursion in data structures: Finite definitions of **arbitrarily large** objects
 - Natural numbers
 - Words
 - Linked lists
 - Formulas
 - Binary trees

Recursion

Fundamental concept in Computer Science

- Recursion in algorithms: Solving problems by reducing to smaller cases
 - Factorial
 - Towers of Hanoi
 - Mergesort, Quicksort
- Recursion in data structures: Finite definitions of **arbitrarily large** objects
 - Natural numbers
 - Words
 - Linked lists
 - Formulas
 - Binary trees
- Analysis of recursion: Proving properties
 - Recursive sequences (e.g. Fibonacci sequence)
 - Structural induction

Recursion

A recursive definition has base cases (B) and recursive cases (R):

$$(B) \quad 0! = 1$$

$$(R) \quad (n + 1)! = (n + 1) \cdot n!$$

`fact(n):`

`(B)       if(n = 0): 1`

`(R)       else: n * fact(n - 1)`

Factorial is defined in terms of *smaller* instances of factorial.

Recursion

A recursive definition has base cases (B) and recursive cases (R):

$$(B) \quad 0! = 1$$

$$(R) \quad (n + 1)! = (n + 1) \cdot n!$$

`fact(n):`

$$(B) \quad \text{if}(n = 0): 1$$

$$(R) \quad \text{else: } n * \text{fact}(n - 1)$$

Factorial is defined in terms of *smaller* instances of factorial.

Question

Why do we need base cases in programming?

Why do we need them in maths?

Example: Towers of Hanoi



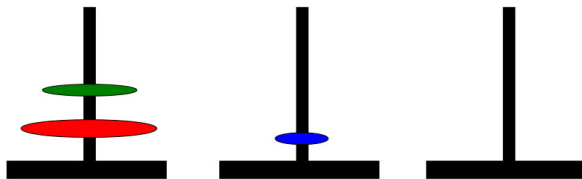
- 1 There are 3 towers (pegs).
- 2 n disks of decreasing size are placed on the first tower.
- 3 Every move, you take the top disk from one peg and put it on top of another peg.
- 4 You win when all disks are on the middle tower.
- 5 Larger disks cannot be placed on of smaller disks.

The last tower can be used to temporarily hold disks.

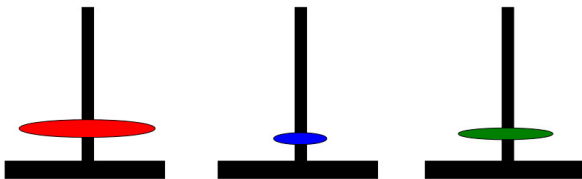
Example: Towers of Hanoi



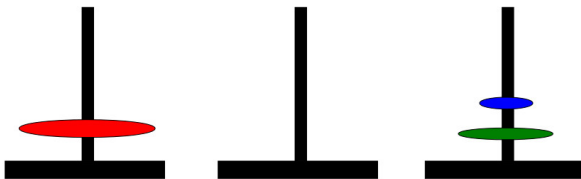
Example: Towers of Hanoi



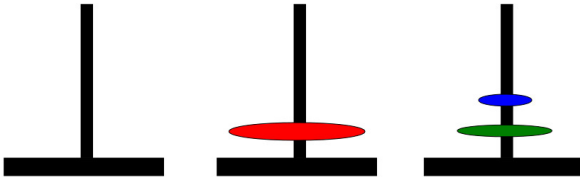
Example: Towers of Hanoi



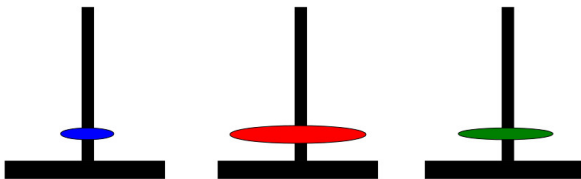
Example: Towers of Hanoi



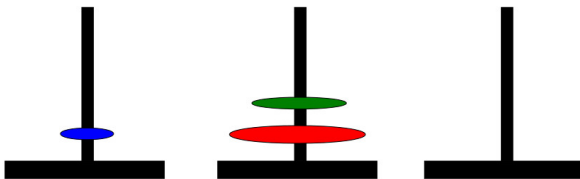
Example: Towers of Hanoi



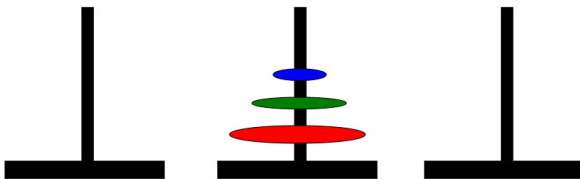
Example: Towers of Hanoi



Example: Towers of Hanoi



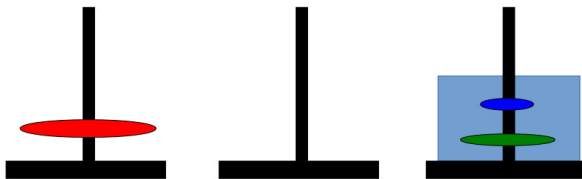
Example: Towers of Hanoi



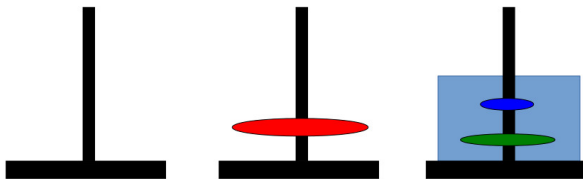
Example: Towers of Hanoi



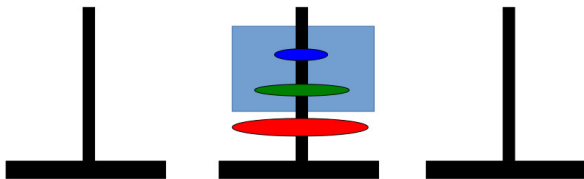
Example: Towers of Hanoi



Example: Towers of Hanoi



Example: Towers of Hanoi



Summary of topics

- Recursion
- Recursive Data Types
- Induction
- Structural Induction

Example: Natural numbers

A natural number is either 0 (B) or one more than a natural number (R).

Formally:

$$(B) \quad 0 \in \mathbb{N}$$

$$(R) \quad \text{If } n \in \mathbb{N} \text{ then } (n + 1) \in \mathbb{N}$$

This is an **inductive** definition of $\mathbb{N}$ (aka a **recursive** definition): $\mathbb{N}$ contains everything that can be constructed by finitely many applications of (B) and (R), and nothing else.

Example: Fibonacci numbers

The Fibonacci sequence starts $0, 1, 1, 2, 3, \dots$ where, after $0, 1$, each term is the sum of the previous two terms.

Formally, the set of Fibonacci numbers: $\mathbb{F} = \{F_n : n \in \mathbb{N}\}$, where the n -th Fibonacci number F_n is defined as:

$$\text{(B)} \quad F_0 = 0,$$

$$\text{(B)} \quad F_1 = 1,$$

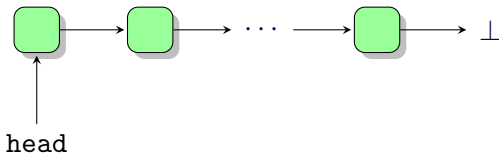
$$\text{(I)} \quad F_n = F_{n-1} + F_{n-2}$$

NB

Could also define the Fibonacci sequence as a function
 $\text{FIB} : \mathbb{N} \rightarrow \mathbb{F}$. *Choice of perspective depends on what structure you view as your base object (ground type).*

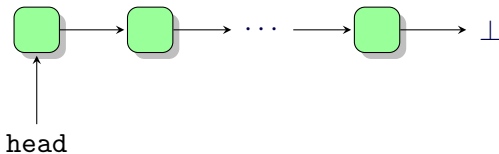
Example: Linked lists

Recall: A linked list is zero or more linked list nodes:



Example: Linked lists

Recall: A linked list is zero or more linked list nodes:



In C:

```
struct node{  
    int data;  
    struct node *next;  
}
```

Example: Linked lists

We can view the linked list **structure** abstractly. A linked list is either:

- (B) an empty list, or
- (R) an ordered pair (Data, List).

Example: Words over Σ

A word over an alphabet Σ is either λ (B) or a symbol from Σ followed by a word (R).

Formal definition of Σ^* :

- (B) $\lambda \in \Sigma^*$
- (R) If $w \in \Sigma^*$ then $aw \in \Sigma^*$ for all $a \in \Sigma$

NB

*This matches the recursive definition of a **Linked List** data type.*

Example: Propositional formulas

A well-formed formula (wff) over a set of propositional variables, PROP is defined as:

- (B) $\top$ is a wff
- (B) $\perp$ is a wff
- (B) p is a wff for all $p \in \text{PROP}$
- (R) If φ is a wff then $\neg\varphi$ is a wff
- (R) If φ and ψ are wffs then:
 - $(\varphi \wedge \psi)$,
 - $(\varphi \vee \psi)$,
 - $(\varphi \rightarrow \psi)$, and
 - $(\varphi \leftrightarrow \psi)$ are wffs.

Programming over recursive datatypes

Recursive datatypes make recursive programming/functions easy.

Example

The factorial function:

```
fact( $n$ ):  
( $B$ )    if( $n = 0$ ): 1  
( $R$ )    else:  $n * \text{fact}(n - 1)$ 
```

Programming over recursive datatypes

Recursive datatypes make recursive programming/functions easy.

Example

Summing the first n natural numbers:

```
sum( $n$ ):  
( $B$ )    if( $n = 0$ ): 0  
( $R$ )    else:  $n + \text{sum}(n - 1)$ 
```

Programming over recursive datatypes

Recursive datatypes make recursive programming/functions easy.

Example

Concatenation of words (defining wv):

$$\begin{array}{ll} & \text{For all } w, v \in \Sigma^* \text{ and } a \in \Sigma : \\ (B) & \lambda v = v \\ (R) & (aw)v = a(wv) \end{array}$$

Programming over recursive datatypes

Recursive datatypes make recursive programming/functions easy.

Example

Length of words:

$$(B) \quad \text{length}(\lambda) = 0$$

$$(R) \quad \text{length}(aw) = 1 + \text{length}(w)$$

Programming over recursive datatypes

Recursive datatypes make recursive programming/functions easy.

Example

“Evaluation” of a propositional formula

Summary of topics

- Recursion
- Recursive Data Types
- Induction
- Structural Induction

Recursive datatypes

Describe arbitrarily large objects in a finite way

Recursive functions

Define behaviour for these objects in a finite way

Induction

Reason about these objects in a finite way

Inductive Reasoning

Suppose we would like to reach a conclusion of the form

$P(x)$ for all x (of some type)

Inductive reasoning (as understood in philosophy) proceeds from examples.

E.g. From “This swan is white, that swan is white, in fact every swan I have seen so far is white”

Conclude: “Every Swan is white”

Inductive Reasoning

Suppose we would like to reach a conclusion of the form

$P(x)$ for all x (of some type)

Inductive reasoning (as understood in philosophy) proceeds from examples.

E.g. From “This swan is white, that swan is white, in fact every swan I have seen so far is white”

Conclude: “Every Swan is white”

NB

This may be a good way to discover hypotheses.

But it is not a valid principle of reasoning!

Mathematical induction is a variant that is valid.

Mathematical Induction

Mathematical Induction is based not just on a set of examples, but also a rule for deriving new cases of $P(x)$ from cases where P is known to hold.

General structure of reasoning by mathematical induction:

Base Case (B): $P(a_1), P(a_2), \dots, P(a_n)$ for some small set of examples $a_1 \dots a_n$ (often $n = 1$)

Inductive Step (I): A general rule showing that if $P(x)$ holds for some cases $x = x_1, \dots, x_k$ then $P(y)$ holds for some new case y , constructed in some way from $x_1, \dots, x_k$.

Conclusion: By starting with $a_1 \dots a_n$ and repeatedly applying (I), we can construct all values in the domain.

Basic induction

Basic induction is this principle applied to the natural numbers.

Goal: Show $P(n)$ holds for all $n \in \mathbb{N}$.

Approach: Show that:

Base case (B): $P(0)$ holds; and

Inductive case (I): If $P(k)$ holds then $P(k + 1)$ holds.

Example

Recall the recursive program:

Example

Summing the first n natural numbers:

```
sum( $n$ ):  
  if( $n = 0$ ): 0  
  else:  $n + \text{sum}(n - 1)$ 
```

Another attempt:

Example

```
sum2( $n$ ):  
  return  $n * (n + 1) / 2$ 
```

Induction proof **guarantees** that these programs will behave the same.

Example

Let $P(n)$ be the proposition that:

$$P(n) : \sum_{i=0}^n i = \frac{n(n+1)}{2}.$$

We will show that $P(n)$ holds for all $n \in \mathbb{N}$ by induction on n .

Example

Let $P(n)$ be the proposition that:

$$P(n) : \sum_{i=0}^n i = \frac{n(n+1)}{2}.$$

We will show that $P(n)$ holds for all $n \in \mathbb{N}$ by induction on n .

Proof.

(B) $P(0)$, i.e.

$$\sum_{i=0}^0 i = \frac{0(0+1)}{2}$$

Example

Let $P(n)$ be the proposition that:

$$P(n) : \sum_{i=0}^n i = \frac{n(n+1)}{2}.$$

We will show that $P(n)$ holds for all $n \in \mathbb{N}$ by induction on n .

Proof.

(B) $P(0)$, i.e.

$$\sum_{i=0}^0 i = \frac{0(0+1)}{2}$$

(I) $\forall k \geq 0 (P(k) \rightarrow P(k+1))$, i.e.

$$\sum_{i=0}^k i = \frac{k(k+1)}{2} \rightarrow \sum_{i=0}^{k+1} i = \frac{(k+1)(k+2)}{2}$$

(proof?)



Example (cont'd)

Proof.

Inductive step (I):

$$\begin{aligned}\sum_{i=0}^{k+1} i &= \left(\sum_{i=0}^k i \right) + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) \quad (\text{by the inductive hypothesis}) \\ &= \frac{k(k+1) + 2(k+1)}{2} \\ &= \frac{(k+1)(k+2)}{2}\end{aligned}$$



Variations

- ① Induction from m upwards
- ② Induction steps > 1
- ③ Strong induction
- ④ Backward induction
- ⑤ Structural induction

Induction From m Upwards

If

(B) $P(m)$

(I) $\forall k \geq m (P(k) \rightarrow P(k + 1))$

then

(C) $\forall n \geq m (P(n))$

Example

Theorem. For all $n \geq 1$, the number $8^n - 2^n$ is divisible by 6.

(B) $8^1 - 2^1$ is divisible by 6

(I) if $8^k - 2^k$ is divisible by 6, then so is $8^{k+1} - 2^{k+1}$, for all $k \geq 1$

Prove (I) using the “trick” to rewrite 8^{k+1} as $8 \cdot (8^k - 2^k + 2^k)$
which allows you to apply the IH on $8^k - 2^k$

Induction Steps $\ell > 1$

If

(B) $P(m)$

(I) $P(k) \rightarrow P(k + \ell)$ for all $k \geq m$

then

(C) $P(n)$ for every ℓ 'th $n \geq m$

Example

Every 4th Fibonacci number is divisible by 3.

(B) $F_4 = 3$ is divisible by 3

(I) if $3 \mid F_k$, then $3 \mid F_{k+4}$, for all $k \geq 4$

Prove (I) by rewriting F_{k+4} in such a way that you can apply the IH on F_k

Strong Induction

This is a version in which the inductive hypothesis is stronger.
Rather than using the fact that $P(k)$ holds for a single value, we use *all* values up to k .

If

(B) $P(m)$

(I) $[P(m) \wedge P(m+1) \wedge \dots \wedge P(k)] \rightarrow P(k+1)$ for all $k \geq m$

then

(C) $P(n)$, for all $n \geq m$

Example

Claim: All integers ≥ 2 can be written as a product of primes.

(B) 2 is a product of primes

(I) If all x with $2 \leq x \leq k$ can be written as a product of primes, then $k + 1$ can be written as a product of primes, for all $k \geq 2$

Proof for (I)?

Negative Integers, Backward Induction

NB

Induction can be conducted over any subset of $\mathbb{Z}$ with least element. Thus m can be negative; eg. base case $m = -10^6$.

NB

One can apply induction in the 'opposite' direction $p(m) \rightarrow p(m-1)$. It means considering the integers with the opposite ordering where the next number after n is $n-1$. Such induction would be used to prove some $p(n)$ for all $n \leq m$.

NB

Sometimes one needs to reason about all integers $\mathbb{Z}$. This requires two separate simple induction proofs: one for $\mathbb{N}$, another for $-\mathbb{N}$. They both would start from some initial values, which could be the same, e.g. zero. Then the first proof would proceed through positive integers; the second proof through negative integers.

Summary of topics

- Recursion
- Recursive Data Types
- Induction
- Structural Induction

Structural Induction

Basic induction allows us to prove properties for **all natural numbers**. The induction scheme (layout) uses the recursive definition of $\mathbb{N}$.

$$\text{(B)} \quad 0 \in \mathbb{N}$$

$$\text{(R)} \quad \text{If } n \in \mathbb{N} \text{ then} \\ (n + 1) \in \mathbb{N}$$

$$\text{(B)} \quad P(0)$$

$$\text{(I)} \quad P(k) \rightarrow \\ P(k + 1).$$

Structural Induction

Basic induction allows us to prove properties for **all natural numbers**. The induction scheme (layout) uses the recursive definition of $\mathbb{N}$.

$$\text{(B)} \quad 0 \in \mathbb{N}$$

$$\text{(R)} \quad \text{If } n \in \mathbb{N} \text{ then} \\ (n + 1) \in \mathbb{N}$$

$$\text{(B)} \quad P(0)$$

$$\text{(I)} \quad P(k) \rightarrow \\ P(k + 1).$$

NB

Every clause in the induction principle is there because of a similar-looking clause in the (recursive) definition!

The same connection between recursive definition and induction principle applies not just to $\mathbb{N}$, but to any **well-founded strict** poset.

The basic approach is always the same. To prove $\forall x. P(x)$, we show:

- (B) P holds for all minimal objects
- (I) If P holds for all predecessors of x , then $P(x)$.

Strict poset

A **strict** poset is a pair $(S, \prec)$ consisting of a set S and a relation $\prec \subseteq S \times S$ such that $\prec$ is anti-reflexive, anti-symmetric and transitive.

Example

$(\mathbb{N}, <)$ is a strict poset.

Example

$(\mathbb{N}, \leq)$ is a non-strict poset. (why?)

A (**non-strict**) partial order is reflexive, anti-symmetric and transitive.

Well-founded?

A strict poset $(S, <)$ is **well-founded** if there are no infinitely descending chains:

$$\cdots < r_{k+2} < r_{k+1} < r_k$$

Example

$(\mathbb{N}, <)$ is well-founded: every chain starting from a number n ends in 0 after finitely many steps.

Example

$(\mathbb{Z}, <)$ is **not** well-founded (why?)

Example

$(\mathbb{R}^+, <)$ is **not** well-founded (why?)

Example: Induction on Σ^*

Recall definition of Σ^* :

$$\lambda \in \Sigma^*$$

If $w \in \Sigma^*$ then $aw \in \Sigma^*$ for all $a \in \Sigma$

Structural induction on Σ^* :

Goal: Show $P(w)$ holds for all $w \in \Sigma^*$.

Approach: Show that:

Base case (B): $P(\lambda)$ holds; and

Inductive case (I): If $P(w)$ holds then $P(aw)$ holds for all $a \in \Sigma$.

Example: Induction on Σ^*

Recall:

Formal definition of Σ^* :

$$\lambda \in \Sigma^*$$

If $w \in \Sigma^*$ then $aw \in \Sigma^*$ for all $a \in \Sigma$

Formal definition of concatenation:

(concat.B) $\lambda v = v$

(concat.I) $(aw)v = a(wv)$

Formal definition of length:

(length.B) $\text{length}(\lambda) = 0$

(length.I) $\text{length}(aw) = 1 + \text{length}(w)$

Example: Induction on Σ^*

Recall:

Formal definition of Σ^* :

$$\lambda \in \Sigma^*$$

If $w \in \Sigma^*$ then $aw \in \Sigma^*$ for all $a \in \Sigma$

Formal definition of concatenation:

(concat.B) $\lambda v = v$

(concat.I) $(aw)v = a(wv)$

Formal definition of length:

(length.B) $\text{length}(\lambda) = 0$

(length.I) $\text{length}(aw) = 1 + \text{length}(w)$

Prove:

$$\text{length}(wv) = \text{length}(w) + \text{length}(v)$$

Example: Induction on Σ^*

Let $P(w)$ be the proposition that, for all $v \in \Sigma^*$:

$$\text{length}(wv) = \text{length}(w) + \text{length}(v).$$

We will show that $P(w)$ holds for all $w \in \Sigma^*$ by **structural induction on w** .

Proof:

Example: Induction on Σ^*

Let $P(w)$ be the proposition that, for all $v \in \Sigma^*$:

$$\text{length}(wv) = \text{length}(w) + \text{length}(v).$$

We will show that $P(w)$ holds for all $w \in \Sigma^*$ by **structural induction on w** .

Proof:

Base case ($w = \lambda$):

$$\text{length}(\lambda v) =$$

Example: Induction on Σ^*

Let $P(w)$ be the proposition that, for all $v \in \Sigma^*$:

$$\text{length}(wv) = \text{length}(w) + \text{length}(v).$$

We will show that $P(w)$ holds for all $w \in \Sigma^*$ by **structural induction on w** .

Proof:

Base case ($w = \lambda$):

$$\text{length}(\lambda v) = \text{length}(v) \quad (\text{concat.B})$$

Example: Induction on Σ^*

Let $P(w)$ be the proposition that, for all $v \in \Sigma^*$:

$$\text{length}(wv) = \text{length}(w) + \text{length}(v).$$

We will show that $P(w)$ holds for all $w \in \Sigma^*$ by **structural induction on w** .

Proof:

Base case ($w = \lambda$):

$$\begin{aligned} \text{length}(\lambda v) &= \text{length}(v) && (\text{concat.B}) \\ &= 0 + \text{length}(v) \end{aligned}$$

Example: Induction on Σ^*

Let $P(w)$ be the proposition that, for all $v \in \Sigma^*$:

$$\text{length}(wv) = \text{length}(w) + \text{length}(v).$$

We will show that $P(w)$ holds for all $w \in \Sigma^*$ by **structural induction on w** .

Proof:

Base case ($w = \lambda$):

$$\begin{aligned} \text{length}(\lambda v) &= \text{length}(v) && \text{(concat.B)} \\ &= 0 + \text{length}(v) \\ &= \text{length}(w) + \text{length}(v) && \text{(length.B)} \end{aligned}$$

Example: Induction on Σ^*

Proof cont'd:

Inductive case ($w = aw'$): Assume that $P(w')$ holds. That is, for all $v \in \Sigma^*$:

$$(IH): \quad \text{length}(w'v) = \text{length}(w') + \text{length}(v).$$

Then, for all $a \in \Sigma$, we have:

$$\text{length}((aw')v) =$$

Example: Induction on Σ^*

Proof cont'd:

Inductive case ($w = aw'$): Assume that $P(w')$ holds. That is, for all $v \in \Sigma^*$:

$$(IH): \quad \text{length}(w'v) = \text{length}(w') + \text{length}(v).$$

Then, for all $a \in \Sigma$, we have:

$$\text{length}((aw')v) = \text{length}(a(w'v)) \quad (\text{concat.1})$$

Example: Induction on Σ^*

Proof cont'd:

Inductive case ($w = aw'$): Assume that $P(w')$ holds. That is, for all $v \in \Sigma^*$:

$$(IH): \quad \text{length}(w'v) = \text{length}(w') + \text{length}(v).$$

Then, for all $a \in \Sigma$, we have:

$$\begin{aligned} \text{length}((aw')v) &= \text{length}(a(w'v)) && \text{(concat.l)} \\ &= 1 + \text{length}(w'v) && \text{(length.l)} \end{aligned}$$

Example: Induction on Σ^*

Proof cont'd:

Inductive case ($w = aw'$): Assume that $P(w')$ holds. That is, for all $v \in \Sigma^*$:

$$(IH): \quad \text{length}(w'v) = \text{length}(w') + \text{length}(v).$$

Then, for all $a \in \Sigma$, we have:

$$\begin{aligned} \text{length}((aw')v) &= \text{length}(a(w'v)) && \text{(concat.I)} \\ &= 1 + \text{length}(w'v) && \text{(length.I)} \\ &= 1 + \text{length}(w') + \text{length}(v) && (IH) \end{aligned}$$

Example: Induction on Σ^*

Proof cont'd:

Inductive case ($w = aw'$): Assume that $P(w')$ holds. That is, for all $v \in \Sigma^*$:

$$(IH): \quad \text{length}(w'v) = \text{length}(w') + \text{length}(v).$$

Then, for all $a \in \Sigma$, we have:

$$\begin{aligned} \text{length}((aw')v) &= \text{length}(a(w'v)) && (\text{concat.l}) \\ &= 1 + \text{length}(w'v) && (\text{length.l}) \\ &= 1 + \text{length}(w') + \text{length}(v) && (IH) \\ &= \text{length}(aw') + \text{length}(v) && (\text{length.l}) \end{aligned}$$

Example: Induction on Σ^*

Proof cont'd:

Inductive case ($w = aw'$): Assume that $P(w')$ holds. That is, for all $v \in \Sigma^*$:

$$(IH): \quad \text{length}(w'v) = \text{length}(w') + \text{length}(v).$$

Then, for all $a \in \Sigma$, we have:

$$\begin{aligned} \text{length}((aw')v) &= \text{length}(a(w'v)) && (\text{concat.I}) \\ &= 1 + \text{length}(w'v) && (\text{length.I}) \\ &= 1 + \text{length}(w') + \text{length}(v) && (IH) \\ &= \text{length}(aw') + \text{length}(v) && (\text{length.I}) \end{aligned}$$

So $P(aw')$ holds.

Example: Induction on Σ^*

Proof cont'd:

Inductive case ($w = aw'$): Assume that $P(w')$ holds. That is, for all $v \in \Sigma^*$:

$$(IH): \quad \text{length}(w'v) = \text{length}(w') + \text{length}(v).$$

Then, for all $a \in \Sigma$, we have:

$$\begin{aligned} \text{length}((aw')v) &= \text{length}(a(w'v)) && \text{(concat.I)} \\ &= 1 + \text{length}(w'v) && \text{(length.I)} \\ &= 1 + \text{length}(w') + \text{length}(v) && (IH) \\ &= \text{length}(aw') + \text{length}(v) && \text{(length.I)} \end{aligned}$$

So $P(aw')$ holds.

We have $P(\lambda)$ and for all $w' \in \Sigma^*$ and $a \in \Sigma$: $P(w') \rightarrow P(aw')$.
Hence $P(w)$ holds for all $w \in \Sigma^*$.

Example 2: Induction on Σ^*

Define $\text{reverse} : \Sigma^* \rightarrow \Sigma^*$:

(rev.B) $\text{reverse}(\lambda) = \lambda$,

(rev.I) $\text{reverse}(a \cdot w) = \text{reverse}(w) \cdot a$

Example 2: Induction on Σ^*

Theorem

For all $w, v \in \Sigma^$, $\text{reverse}(wv) = \text{reverse}(v) \cdot \text{reverse}(w)$.*

Example 2: Induction on Σ^*

Theorem

For all $w, v \in \Sigma^$, $\text{reverse}(wv) = \text{reverse}(v) \cdot \text{reverse}(w)$.*

Proof: By induction on w ...

Example 2: Induction on Σ^*

Theorem

For all $w, v \in \Sigma^*$, $\text{reverse}(wv) = \text{reverse}(v) \cdot \text{reverse}(w)$.

Proof: By induction on w ...

(B)	$\text{reverse}(\lambda v)$	$= \text{reverse}(v)$	(concat.B)
		$= \text{reverse}(v)\lambda$	(*)
		$= \text{reverse}(v)\text{reverse}(\lambda)$	(reverse.B)
(I)	$\text{reverse}((aw')v)$	$= \text{reverse}(a(w'v))$	(concat.I)
		$= \text{reverse}(w'v) \cdot a$	(reverse.I)
		$= \text{reverse}(v)\text{reverse}(w') \cdot a$	(IH)
		$= \text{reverse}(v)\text{reverse}(aw')$	(reverse.I)

Mutual Recursion

Mutual recursion is when two or more functions are defined in terms of each other:

```
odd(n):  
(B)    if(n = 0): false  
(R)    else: even(n - 1)
```

```
even(n):  
(B)    if(n = 0): true  
(R)    else: odd(n - 1)
```

Mutual Recursion

Example

Alternative definition of Fibonacci numbers:

$$(B) \quad f(1) = 1$$

$$(B) \quad g(1) = 1$$

$$(R) \quad f(n) = f(n-1) + g(n-1)$$

$$(R) \quad g(n) = f(n-1)$$

Mutual Recursion

Example

Alternative definition of Fibonacci numbers:

$$(B) \quad f(1) = 1$$

$$(B) \quad g(1) = 1$$

$$(R) \quad f(n) = f(n-1) + g(n-1)$$

$$(R) \quad g(n) = f(n-1)$$

In matrix form:

$$\begin{pmatrix} f(n) \\ g(n) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} f(n-1) \\ g(n-1) \end{pmatrix}$$

Corollary:

$$\begin{pmatrix} f(n) \\ g(n) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} f(0) \\ g(0) \end{pmatrix}$$

Summary of topics

- Recursion
- Recursive Data Types
- Induction
- Structural Induction

Tute (Week 3)

Recursive definitions

...of data

The recursive definitions we studied in Week 3 included recursive definitions of *data*, such as lists, trees and even natural numbers. In the case of lists, we ended up defining *finite* lists, of which however there are infinitely many (just as there are infinitely many natural numbers, although each one is finite).

Do you think it would be possible to write a program, one that actually runs, that contained a data type with *infinite* values? (Infinite lists would be a good example; another would be a program that looked like it calculated the entire (infinite) decimal expansion of π .)

This can actually be found in the wild, in programming languages featuring *lazy evaluation*, where all computation is deferred until the point where its results would be needed.

The most prominent example of such a language is Haskell. (For the purposes of this course, you're not expected to know or learn anything about Haskell). It's possible that we'll later discuss lazy evaluation, and how you prove things about infinite structures.

...of languages

We also defined propositions recursively, informally so in Week 1 and with a bit more rigour in Week 3. Do those definitions allow propositions (so called *well formed formulae*, or *wff's*) to be infinitely long? If they don't, would it matter if they did?

No, they do not. The fact that the wff's are finite (in length, or structurally) is why structural induction works over them.

Proofs by induction

Question 1

Comment on this inductive proof that all horses have the same colour.

Base cases: The empty set of horses does not contain two horses of differing colour.

Singleton sets $\{h\}$ of horses do not contain two horses of differing colour.

Inductive step: Consider any set H of horses: by appeal to the inductive hypothesis, they all have the same colour $\square$. We now add a different horse h to the set. Do all horses in the now-bigger set $H \cup \{h\}$ still have the same colour $\square$?

Remove some (other) horse h' from the set $H \cup \{h\}$: by the inductive hypotheses all horses in the smaller set $(H - \{h'\}) \cup \{h\}$ have the same colour. In particular, this means that the new horse h has the same colour $\square$ as the ones that are still there. Thus the new 1-horse-bigger set $H \cup \{h\}$ also contains only horses of colour $\square$.

- What exactly are we doing induction on?
- How come we succeeded in proving something so absurd? Are the foundations of mathematics unraveling beneath our feet? Or is there a mistake in the proof?

The induction is (implicitly) on the cardinality of the size of a set of horses H .

There is a mistake in the proof. In particular, the inductive step neglects to consider the possibility that $|H| = 1$. If so, the proof fails because there is no overlap between $(H - \{h'\}) \cup \{h\}$ and H .

Question 2

Peano arithmetic is a way of defining the natural numbers from first principles. It starts from the following axioms:

1. There is a number Z .
2. For every number n there is a successor $S\ n$ of n .
3. $S\ n \neq Z$ for all n .
4. If $S\ n = S\ m$ then $n = m$.
5. If some set of numbers A (i) contains Z , and (ii) contains $S\ n$ if it contains n , then A contains all numbers.

Here Z is the number we call 0 in day-to-day mathematics, $S\ Z$ is 1, and so on.

Giuseppe Peano's (inductive) definition of addition in this system was

$$\begin{aligned} p + Z &= p \\ p + S\ q &= S\ (p + q) \end{aligned}$$

Can you give a Peano-style definition of multiplication?

$$\begin{aligned}
 p \times Z &= Z \\
 p \times S q &= p + (p \times q)
 \end{aligned}$$

Question 3

Use Peano's Axioms and the definition you gave for multiplication just above to prove associativity of multiplication.

You may assume that addition distributes over multiplication:

$$p \times q + p \times r = p \times (q + r)$$

We prove $(p \times q) \times r = p \times (q \times r)$ by induction on r .

Base case:

$$\begin{aligned}
 & (p \times q) \times Z \\
 = & Z && \text{definition} \\
 = & p \times Z && \text{definition} \\
 = & p \times (q \times Z) && \text{definition}
 \end{aligned}$$

Inductive case: Our inductive hypothesis is that

$$(p \times q) \times r = p \times (q \times r)$$

We proceed as follows:

$$\begin{aligned}
 & (p \times q) \times (S r) \\
 = & p \times q + (p \times q) \times r && \text{definition} \\
 = & p \times q + p \times (q \times r) && \text{inductive hypothesis} \\
 = & p \times (q + q \times r) && \text{distributivity} \\
 = & p \times (q \times (S r)) && \text{definition}
 \end{aligned}$$

That takes care of that. But why did we get to do basic induction here, when the Peano axioms don't mention it?

The answer is that we can use axiom (5). The key observation is that the following logical equivalence holds:

$$\forall n \in \mathbb{N}. P(n) \quad \equiv \quad \mathbb{N} \subseteq \{n \in \mathbb{N} : P(n)\}$$

Notice that the LHS is the kind of statement we can prove using basic induction. The RHS is the kind of statement we may prove using axiom (5)! To see this, just let

$A = \{n \in \mathbb{N} : P(n)\}$, and note that we use $\mathbb{N}$ here to refer to what we called "all numbers" in the Peano axioms.

To conclude $\mathbb{N} \subseteq \{n \in \mathbb{N} : P(n)\}$, axiom (5) then requires us to prove

1. $Z \in \{n \in \mathbb{N} : P(n)\}$
2. If $n \in \{n \in \mathbb{N} : P(n)\}$ then $(S\ n) \in \{n \in \mathbb{N} : P(n)\}$

A little simplification will show that these proof obligations are exactly the same as the (B) and (I) cases of structural induction.

For this particular proof, we chose our P as follows:

$$P(n) \quad = \quad ((p \times q) \times n = p \times (q \times n))$$

Question 4

Use the definition of *reverse* from the lectures (but written in a functional style)

$$\begin{aligned} \text{rev } [] &= [] \\ \text{rev } (x:xs) &= \text{rev } xs \# [x] \end{aligned}$$

to show that $\text{rev}(\text{rev } xs) = xs$.

Hint: You might need a lemma.

We need a lemma that allows us to distribute *rev* over concatenation.

1. We prove that $\forall ys. \text{rev}(xs \# ys) = \text{rev } ys \# \text{rev } xs$ by induction on xs .

Base case: with justifications elided:

$$\text{rev}([] \# ys) = \text{rev } ys = \text{rev } ys \# []$$

Inductive case

$$\begin{aligned} &\text{rev}(x:xs \# ys) \\ = &\text{rev}(x:xs \# ys) && \text{identity} \\ = &\text{rev}(x:(xs \# ys)) && \text{definition of } \# \\ = &\text{rev}(xs \# ys) \# [x] && \text{definition of rev} \\ = &(\text{rev } ys \# \text{rev } xs) \# [x] && \text{induction hypothesis} \\ = &\text{rev } ys \# (\text{rev } xs \# [x]) && \text{associativity of } \# \\ = &\text{rev } ys \# \text{rev}(x:xs) && \text{definition of rev} \end{aligned}$$

2. We prove that $\text{rev}(\text{rev } xs) = xs$ by structural induction on xs .

Base case:

$$\begin{aligned}
 & \text{rev}(\text{rev } []) \\
 = & \text{rev}[] && \text{definition} \\
 = & [] && \text{definition}
 \end{aligned}$$

Inductive case

$$\begin{aligned}
 & \text{rev}(\text{rev}(x:xs)) \\
 = & \text{rev}(\text{rev } xs \# [x]) && \text{definition} \\
 = & \text{rev}[x] \# \text{rev}(\text{rev } xs) && \text{lemma 1} \\
 = & \text{rev}[x] \# xs && \text{induction hypothesis} \\
 = & x:xs && \text{definition(s)}
 \end{aligned}$$

In the above proofs, we assumed that the associativity and right identity of $\#$ were proven elsewhere. They would require their own inductive proofs.

Notes (Week 3 Monday)

These are the notes I wrote during the lecture.

Choose your own adventure!

1. wrap up last week
(functions, logic)
2. wrap up functions, ignore logic
3. move on to w3 topic
(induction, recursion)

Quickly, functions.

Functions are a special case of
binary relations.

Binary relations
are a special case of sets.

Specifically, a binary relation R
is a set of the form
 $R \subseteq S \times T$
or in other words, it's a set
of pairs.

E.g., $\leq$ is the set of all pairs
of numbers (x,y) such that y
is no less than x .

Functions f from S to T can be seen
as relations
 $f \subseteq S \times T$

$(x,y) \in f$ means that $f(x) = y$

In order for a relation to count
as a function, for every $x \in S$,
there exists *exactly one* $y \in T$
such that $(x,y) \in f$

In words: every input must produce
exactly one output.

Injective functions

A function is *injective* if

whenever $x_1 \neq x_2$,

$$f(x_1) \neq f(x_2)$$

An injective function maps distinct elements to distinct elements.

Let's say these are both $\mathbb{N} \rightarrow \mathbb{N}$

$$f(x) = x + 1 \quad \text{injective}$$

^ the *range* of this function is the positive $\mathbb{N}$ s

$$f(x) = 0$$

^ the *range* of this function is $\{0\}$

Surjective functions

$$f \subseteq S \times T$$

S the set of inputs (domain)

T the set of outputs (codomain)

range the outputs that actually happen.

A function is *surjective* if the codomain equals the range.

Or: if every output is possible.

$$f(x) = \begin{cases} x - 1 & \text{if } x > 0 \\ 124894 & \text{if } x = 0 \end{cases}$$

$$f(124895) = 124894$$

$$f(0) = 124894$$

^ this happens to be surjective, but not injective.

Bijjective = injective + surjective

$$f(x) = x$$

If a function is bijective, it has an *inverse*.

Intuitively, a way to "undo" the function.

If $f : S \rightarrow T$ is bijective,

$f^{-1} : T \rightarrow S$ (its inverse) exists, and has the property

$$f^{-1}(f(x)) = x$$

To construct the inverse function,
you take the converse of the relation
that characterises the function.

If $R \subseteq S \times T$ is a relation,
the **converse** of R ,
written $R^{\leftarrow} \subseteq T \times S$ defined as

$$\{(t,s) \mid (s,t) \in R\}$$

In general, the converse of a function
is not a function.

But for bijective functions,
the converse **is** a function.

Q: What happens if I leave out the
base case in the program?

A: Stack overflow
eventually, we run out of memory
to represent the stack frames

A: Go on forever

A: Integer overflow

Q: What goes wrong in maths?

A: You might be describing a property
that many functions, instead of
a specific one.

A: for induction purposes
(in order for inductive proofs
to be valid)

$$f(x) = 1 + f(x)$$

If I allow the above definition,
cancel $f(x)$ on both sides,
 $0 = 1$

In programming, the world doesn't
end if I forget a base case.
In mathematics, allowing such
definitions can end the world.

$$(1) \ 0 \in S$$

$$(2) \text{ If } n \in S \text{ then } n+1 \in S$$

There are many sets that satisfy
the properties (1) and (2) above.

$\mathbb{N}$ satisfies them.

$\mathbb{Z}$ also does

$\mathbb{R}$ also does

....

Q: What is the definition
of a definition?

A: A definition is a property
such that only one object
has that property.

Example:

Let π be the number s.t.

$c = \pi * d$ for every circle

Non-example:

Let x be the integer that is
greater than 5.

When we say

S is inductively defined by:

(1) $0 \in S$

(2) If $n \in S$ then $n+1 \in S$

What we really mean is:

S is the **smallest** set that
satisfies (1) and (2).

By smallest I mean wrt subset
inclusion.

S is the (unique) set that
satisfies (1) and (2), and,
for every **other** set T that
satisfies (1) and (2),
 $S \subseteq T$.

How do we know that this S exists?

A: take the intersection of every
set that satisfies (1) and (2)

Q: Aren't we just defining sets
in terms of stuff we already had?
Are we going on circles?

We do generally presuppose the
existence of **something**, but not
necessarily numbers.

Suppose all I have is sets.

How do I make numbers?

$0 = \{\}$

$1 = \{\{\}\}$

$2 = \{\{\}, \{\{\}\}\}$

A number n is the union of all numbers less than n .

Q: Is it **always** the case that the intersection construct above satisfies the clauses and/or is unique

The clauses in an inductive definition we require:

- the conclusion of every clause has the form $x \in T$, where T is the set we're defining
- if you mention T in the premises, it must be a **positive** mention

$x \in T \Rightarrow x+1 \in T$ $\leftarrow$ positive premise on T

$x \notin T \Rightarrow x+1 \in T$ $\leftarrow$ negative premise on T

As long as I obey the above two constraints, then the least set satisfying the clauses always exists.

As a trivial example:

$x \notin T \Rightarrow x \in T$ what does this "definition" mean?

A: no set satisfies this clause.

Q: So why does the intersection of the sets that satisfy positive clauses always satisfy the clauses?

A: Positive clauses have a property called **monotonicity**

A function on sets is **monotonic** if

whenever $S \subseteq T$ then $f(S) \subseteq f(T)$

Imagine f is the "apply the clauses once" function, and S is the intersection.

"What if you forget the base case"

For mathematical functions: you introduce contradictions

In programming: you may introduce bugs

For inductive sets: you just end up defining the empty set.

$n \in \mathbb{N} \Rightarrow n+1 \in \mathbb{N}$ -- the empty set satisfies this

In general, inductive (or recursive) definitions of sets (or functions) can have

- many base cases
- many inductive cases

The C struct

```
struct node {
    int data;
    struct node* next;
}
```

looks like a recursive definition, but it isn't quite one.

Elements of a recursive definition must be iteratively constructed from smaller elements, but pointers can point anywhere.

```
// that's an infinite linked list
// whose every element is 2141
node foo;
foo.data = 2141;
foo.next = &foo;
```

Recursively defined objects (in maths) cannot have this kind of circularity.

$$\Sigma = \{h, i\}$$

Prove that $hi \in \Sigma^*$

1. By (B) $\lambda \in \Sigma^*$
2. By (R) and (1), $i\lambda = i \in \Sigma^*$
3. By (R) and (2), $h \cdot i = hi \in \Sigma^*$

In this course,

we often write $\top$ "top" to mean "true"
and $\perp$ "bottom" to mean "false"

Because we think of the wffs as an inductive definition, some things are ruled out by construction.

We can't write an infinite conjunction

$$\phi_0 \wedge \phi_1 \wedge \dots$$

Formulas can't have infinite nesting depth

```
hi·hello
= (R)
h·(i·hello)
  (R)
=  $\lambda \cdot (h \cdot (i \cdot \text{hello}))$ 
```

```

    (by def of single-character concatenation)
=  $\lambda \cdot (h \cdot ihello)$ 
  (by def of single-character concatenation)
=  $\lambda \cdot hihello$ 
  (B)
= hihello

```

Remember that formulas don't have truth values,
but in particular states (aka valuations) they
have truth values.

A valuation gives values to all variables.

"PROP" is the set of variable names for boolean values.

A valuation is a function from PROP to truth values
 $\{\text{True}, \text{False}\} = \mathbb{B}$

$\text{eval} : (\text{wff}, \text{PROP} \rightarrow \mathbb{B}) \rightarrow \mathbb{B}$

$\text{eval}(\top, v) = \text{True}$

$\text{eval}(\perp, v) = \text{False}$

$\text{eval}(p, v) = v(p)$

$\text{eval}(\neg\phi, v) =$
 if $\text{eval}(\phi, v) = \text{True}$ then
 False
 else
 True

...

Notes (Week 3 Wednesday)

These are the notes I wrote during the lecture.

The number of 2-tuples (pairs)

$$(x,y) \in S \times T$$

$$|S| * |T|$$

Alternative question:

how many $\emptyset$ -tuples are there?

$$() \in \prod_{i \in \{ \}} \text{foo}$$

A: 1

$$\prod_{i \in \{1,2,3\}} i = 1*2*3$$

$$\prod_{i \in (S \cup T)} i \quad (\text{supposing } S, T \text{ disjoint})$$

$$\prod_{i \in (S \cup T)} i = \prod_{i \in S} i * \prod_{i \in T} i$$

^ in order for this law to work when $T=\emptyset$,
we need the product of no numbers to be 1.

$$|V| = 5$$

$$|V^{\emptyset}| + |V| + |V^2| + |V^3|$$

$$1 + 5 + 5*5 + 5^3 = 156$$

Step1: translate $A*VA^*$ into English

V means "a vowel"

A means "a letter"

A^* means " $\emptyset$ or more letters"

Therefore, $A*VA^*$ means:

- $\emptyset$ more letters (any letter)
- followed by one vowel
- followed by $\emptyset$ or more letters

$C*VC^*$ this becomes:

- $\emptyset$ more consonants
- followed by one vowel
- followed by $\emptyset$ or more consonants

$V(CV)^*$ the words that:

- start with a vowel,
- followed by $\emptyset$ more consonant-vowel pairs

$V+(CV+)^*$

- $V+$: one or vowels
- $CV+$ a consonant, followed by one or more vowels

Words comprised of:

- one or more vowels, followed by
- zero or more groups of one consonant followed by at least one vowel

$$\text{sum}(n) = \text{sum2}(n) \text{ for } n \in \mathbb{N}$$

By basic induction, it suffices to prove:

$$(B) \text{sum}(0) = \text{sum2}(0)$$

$$\begin{aligned} \text{sum}(0) &= 0 \\ &= 0*(0+1)/2 = \text{sum2}(0) \end{aligned}$$

(I) If we assume

$$\text{sum}(n) = \text{sum2}(n) \text{ (IH)}$$

"induction hypothesis"

then it follows that

$$\text{sum}(n+1) = \text{sum2}(n+1)$$

$$\begin{aligned} \text{sum}(n+1) &= \\ (n+1) + \text{sum}(n) &= \text{(by IH)} \\ (n+1) + \text{sum2}(n) &= \\ (n+1) + n*(n+1)/2 &= \\ \text{(a bunch of algebra)} &= \\ \text{sum2}(n+1) \end{aligned}$$

Assuming we've accepted that basic induction is valid, you can prove that induction from m upwards is valid using basic induction.

Assuming

- (i) $P(m)$,
- and (ii) $\forall x \geq m. P(x) \rightarrow P(x+1)$

I would like to prove: $\forall n \geq m. (P(n))$

$$\text{Observation: } \forall n \geq m. (P(n)) \equiv \forall k. (P(m+k))$$

Then, I can prove $\forall k. (P(m+k))$

by basic induction on k .

For that proof by basic induction, I get two proof obligations:

(B) $P(m+0) \leftarrow$ immediate from (i)
 (I) $P(m+k) \rightarrow P(m+k+1) \leftarrow$ follows from (ii),
 by reasoning similar
 to the "observation"

$\text{fib}(0) = 0$
 $\text{fib}(1) = 1$
 $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

Every 4th Fibonacci number is divisible by 3,
 by 4-step induction.

(B) $\text{fib}(4) = \text{fib}(3) + \text{fib}(2)$
 $= \text{fib}(1) + 2*\text{fib}(2)$
 $= 3*\text{fib}(1) + 2*\text{fib}(0)$
 $= 3$

(I) Assume $3 \mid \text{fib}(n)$

Prove $3 \mid \text{fib}(n+4)$

$\text{fib}(n+4) = \text{fib}(n+3) + \text{fib}(n+2)$
 $= 2*\text{fib}(n+2) + \text{fib}(n+1)$
 $= 3*\text{fib}(n+1) + 2*\text{fib}(n)$

$\wedge \text{fib}(n+4)$ is the sum of two numbers that
 are divisible by 3, and therefore,
 it's divisible by 3

Strong induction (alternative presentation)

To prove $\forall n \in \mathbb{N}. P(n)$ holds, it suffices to prove

(B+I) For all n , assuming
 $\forall m < n. P(m)$
 then $P(n)$ holds

$\wedge$ that's a base case when $n = 0$,
 it's an inductive case otherwise

An inductive case is any case with at least
 one inductive hypothesis.

The more general principle is called
 Noetherian induction (Emmy Noether)

Well-founded strict poset

Strict poset $\rightarrow$ strict partially ordered set

A strict poset $(<, S)$ such that $< \subseteq S \times T$
 such that:

$<$ is antireflexive

antisymmetric
and transitive

Well-founded: has no infinitely descending chains.
Chain is a sequence of elements of S linked by $<$
Descending chain has a largest element.

$(<, \mathbb{N})$ is well-founded

I'm stuck :($0 < 1 < 4 < 5 < 7$

$(<, \mathbb{Z})$ is **not** well-founded

$\dots < -4 < -3 < 0 < 1 < 4 < 5 < 7$

$(\leq, \mathbb{N})$ is this well-founded?

$\dots \leq 0 \leq 0 \leq 0$

Q: Does that mean if something is well-founded
it has to be strict?

A: yes

Alternative presentation of well-founded induction:

Suppose $(<, S)$ is a well-founded strict poset.

In order to prove $\forall s \in S. P(s)$, it suffices to prove:

For all s , if
 $\forall t < s. P(t)$
then $P(s)$ holds

Every induction principle (yes, every)
are special cases of Noetherian (aka well-founded)
induction.

Q: why is well-foundedness important?

A: if a poset is well-founded, intuitively,
the base case is always reachable.
or, in programming terms, your thing terminates

Q: What's the connection between induction on
 Σ^* and well-founded induction?

A: In the same way that induction on natural
numbers is well-founded induction
with $(<, \mathbb{N})$,

induction on Σ^* is well-founded induction
with

$(<, \Sigma^*)$

$<$ is "suffix inclusion"

$w < w'$ iff w is a suffix of w'
a word is a suffix of another if it
occurs at the end of it

"lo" is a suffix of "hello"

Prove this by induction on w :

$$\text{length}(wv) = \text{length}(w) + \text{length}(v)$$

(B) $\text{length}(\lambda v) = \text{length}(\lambda) + \text{length}(v)$

$$\begin{aligned} &\text{length}(\lambda v) \\ &= (\text{by def of concatenation}) \\ &\text{length}(v) \\ &= 0 + \text{length}(v) \\ &= \text{length}(\lambda) + \text{length}(v) \end{aligned}$$

(I) assuming

(IH) $\text{length}(wv) = \text{length}(w) + \text{length}(v)$

prove

$$\text{length}(awv) = \text{length}(aw) + \text{length}(v)$$

$$\begin{aligned} &\text{length}(awv) \\ &= (\text{by def of length}) \\ &1 + \text{length}(wv) \\ &= (\text{by IH}) \\ &1 + \text{length}(w) + \text{length}(v) \\ &= (\text{by def of length backwards}) \\ &\text{length}(aw) + \text{length}(v) \end{aligned}$$

$$\text{odd}(0) = \text{false}$$

$$\text{even}(0) = \text{true}$$

$$\text{odd}(n) = \text{even}(n-1)$$

$$\text{even}(n) = \text{odd}(n-1)$$

^ we could collapse this into a single function
as follows:

$$\text{oddeven}(0, b) = b$$

$$\text{oddeven}(n, b) = \text{oddeven}(n-1, \neg b)$$

These should satisfy:

$$\text{oddeven}(n, \text{false}) = \text{odd}(n)$$

$$\text{oddeven}(n, \text{true}) = \text{even}(n)$$

We've seen several inductive proof techniques for $\mathbb{N}$ s.
They're all equally strong.

You can prove the exact same propositions using
basic, or complete, or whatever induction,

but sometimes, one of them is more convenient.

Every well-founded poset has an induction principle.

We've seen how you can use this to derive (structural) induction on words, and induction on $\mathbb{N}$ s.

(B) E denotes the empty tree (leaf node)

(I) $N(l, v, r)$ denotes a non-leaf node with
left subtree l , right subtree r and
value v .

^ that's an inductive definition of a set

Therefore, we can prove facts about trees using
tree induction!

To prove $P(t)$,
it suffices to prove

(B) $P(E)$

(I) If $P(l)$ and $P(r)$ then $P(N(l, v, r))$

COMP2111 Week 4

Term 1, 2023

Propositional Logic

Summary of topics

- Well-formed formulas (SYNTAX)
- Boolean Algebras
- Valuations (SEMANTICS)
- CNF/DNF
- Proof
- Natural deduction

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- CNF/DNF
- Proof
- Natural deduction

Well-formed formulas

Let $\text{PROP} = \{p, q, r, \dots\}$ be a set of propositional letters.
Consider the alphabet

$$\Sigma = \text{PROP} \cup \{\top, \perp, \neg, \wedge, \vee, \rightarrow, \leftrightarrow, (,)\}.$$

The **well-formed formulas** (wffs) over PROP is the smallest set of words over Σ such that:

- $\top, \perp$ and all elements of PROP are wffs
- If φ is a wff then $\neg\varphi$ is a wff
- If φ and ψ are wffs then $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, $(\varphi \rightarrow \psi)$, and $(\varphi \leftrightarrow \psi)$ are wffs.

Examples

The following are well-formed formulas:

- $(p \wedge \neg \top)$
- $\neg(p \wedge \neg \top)$
- $\neg\neg(p \wedge \neg \top)$

The following are **not** well-formed formulas:

- $p \wedge \wedge$
- $p \wedge \neg \top$
- $(p \wedge q \wedge r)$
- $\neg(\neg p)$

Conventions

To aid readability some conventions and binding rules can and will be used.

- Parentheses omitted if there is no ambiguity (e.g. $p \wedge q$)
- $\neg$ binds more tightly than $\wedge$ and $\vee$, which bind more tightly than $\rightarrow$ and $\leftrightarrow$ (e.g. $p \wedge q \rightarrow r$ instead of $((p \wedge q) \rightarrow r)$)

Conventions

To aid readability some conventions and binding rules can and will be used.

- Parentheses omitted if there is no ambiguity (e.g. $p \wedge q$)
- $\neg$ binds more tightly than $\wedge$ and $\vee$, which bind more tightly than $\rightarrow$ and $\leftrightarrow$ (e.g. $p \wedge q \rightarrow r$ instead of $((p \wedge q) \rightarrow r)$)

Other conventions (rarely used/assumed in this course):

- $'$ or $\bar{}$ for $\neg$
- $+$ for $\vee$
- $\cdot$ or juxtaposition for $\wedge$
- $\wedge$ binds more tightly than $\vee$
- $\wedge$ and $\vee$ associate to the left: $p \vee q \vee r$ instead of $((p \vee q) \vee r)$
- $\rightarrow$ and $\leftrightarrow$ associate to the right: $p \rightarrow q \rightarrow r$ instead of $(p \rightarrow (q \rightarrow r))$

Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$

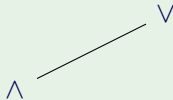
$\vee$

Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$

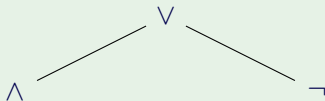


Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$

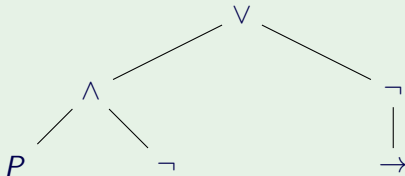


Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$

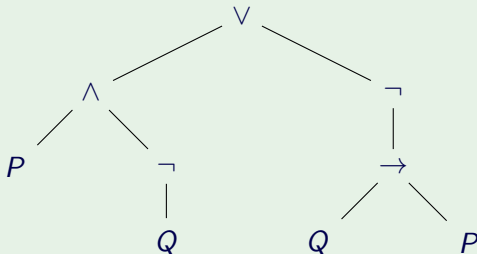


Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$



Parse trees formally

Formally, we can define a parse tree as follows:

A parse tree is either:

- (B) A node containing $\top$;
- (B) A node containing $\perp$;
- (B) A node containing a propositional variable;
- (R) A node containing $\neg$ with a single parse tree child;
- (R) A node containing $\wedge$ with two parse tree children;
- (R) A node containing $\vee$ with two parse tree children;
- (R) A node containing $\rightarrow$ with two parse tree children; or
- (R) A node containing $\leftrightarrow$ with two parse tree children.

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- CNF/DNF
- Proof
- Natural deduction

Laws (Week 2 flashback)

Look again at the laws of set operations and boolean equivalences:

$$x \cup y = y \cup x$$

$$x \cap y = y \cap x$$

$$(x \cup y) \cup z = x \cup (y \cup z)$$

$$(x \cap y) \cap z = x \cap (y \cap z)$$

$$x \cup (y \cap z) = (x \cup y) \cap (x \cup z)$$

$$x \cap (y \cup z) = (x \cap y) \cup (x \cap z)$$

$$x \cup \emptyset = x, \quad x \cap \mathbb{U} = x$$

$$x \cup x^c = \mathbb{U}, \quad x \cap x^c = \emptyset$$

$$x \vee y \equiv y \vee x$$

$$x \wedge y \equiv y \wedge x$$

$$(x \vee y) \vee z \equiv x \vee (y \vee z)$$

$$(x \wedge y) \wedge z \equiv x \wedge (y \wedge z)$$

$$x \vee (y \wedge z) \equiv (x \vee y) \wedge (x \vee z)$$

$$x \wedge (y \vee z) \equiv (x \wedge y) \vee (x \wedge z)$$

$$x \vee \perp \equiv x, \quad x \wedge \top \equiv x$$

$$x \vee \neg x \equiv \top, \quad x \wedge \neg x \equiv \perp$$

Laws (Week 2 flashback)

Look again at the laws of set operations and boolean equivalences:

$$x \cup y = y \cup x$$

$$x \cap y = y \cap x$$

$$(x \cup y) \cup z = x \cup (y \cup z)$$

$$(x \cap y) \cap z = x \cap (y \cap z)$$

$$x \cup (y \cap z) = (x \cup y) \cap (x \cup z)$$

$$x \cap (y \cup z) = (x \cap y) \cup (x \cap z)$$

$$x \cup \emptyset = x, \quad x \cap \mathbb{U} = x$$

$$x \cup x^c = \mathbb{U}, \quad x \cap x^c = \emptyset$$

$$x \vee y \equiv y \vee x$$

$$x \wedge y \equiv y \wedge x$$

$$(x \vee y) \vee z \equiv x \vee (y \vee z)$$

$$(x \wedge y) \wedge z \equiv x \wedge (y \wedge z)$$

$$x \vee (y \wedge z) \equiv (x \vee y) \wedge (x \vee z)$$

$$x \wedge (y \vee z) \equiv (x \wedge y) \vee (x \wedge z)$$

$$x \vee \perp \equiv x, \quad x \wedge \top \equiv x$$

$$x \vee \neg x \equiv \top, \quad x \wedge \neg x \equiv \perp$$

These are the same laws written with different symbols!

Definition: Boolean Algebra

A *Boolean algebra* is a structure $(T, \vee, \wedge, ', 0, 1)$ where

- $0, 1 \in T$
- $\vee : T \times T \rightarrow T$ (called **join**)
- $\wedge : T \times T \rightarrow T$ (called **meet**)
- $' : T \rightarrow T$ (called **complementation**)

and the following laws hold for all $x, y, z \in T$:

commutativity: $x \vee y = y \vee x$

$$x \wedge y = y \wedge x$$

associativity: $(x \vee y) \vee z = x \vee (y \vee z)$

$$(x \wedge y) \wedge z = x \wedge (y \wedge z)$$

distributivity: $x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$

$$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$$

identity: $x \vee 0 = x, \quad x \wedge 1 = x$

complementation: $x \vee x' = 1, \quad x \wedge x' = 0$

Examples of Boolean Algebras

The set of subsets (power set) of a set $\mathcal{U}$:

- $T : \text{Pow}(\mathcal{U})$
- $\wedge : \cap$
- $\vee : \cup$
- $' : ^c$
- $0 : \emptyset$
- $1 : \mathcal{U}$

Examples of Boolean Algebras

The two element Boolean Algebra :

$$\mathbb{B} = (\{\text{true}, \text{false}\}, \&\&, ||, !, \text{false}, \text{true})$$

where $!$, $\&\&$, $||$ are defined as:

- $!\text{true} = \text{false}; !\text{false} = \text{true},$
- $\text{true} \&\& \text{true} = \text{true}; \dots$
- $\text{true} || \text{true} = \text{true}; \dots$

NB

We will often use $\mathbb{B}$ for the two element set $\{\text{true}, \text{false}\}$. For simplicity this may also be abbreviated as $\{T, F\}$ or $\{1, 0\}$.

Examples of Boolean Algebras

Cartesian products of $\mathbb{B}$, that is n -tuples of 0's and 1's with Boolean operations, e.g. $\mathbb{B}^4$:

$$\text{join: } (1, 0, 0, 1) \vee (1, 1, 0, 0) = (1, 1, 0, 1)$$

$$\text{meet: } (1, 0, 0, 1) \wedge (1, 1, 0, 0) = (1, 0, 0, 0)$$

$$\text{complement: } (1, 0, 0, 1)' = (0, 1, 1, 0)$$

$$0: (0, 0, 0, 0)$$

$$1: (1, 1, 1, 1).$$

NB

These are the bitwise operations on 4-bit machine words.

Examples of Boolean Algebras

Functions from any set S to $\mathbb{B}$; their set is denoted $\text{Map}(S, \mathbb{B})$

If $f, g : S \longrightarrow \mathbb{B}$ then

- $(f \vee g) : S \longrightarrow \mathbb{B}$ is defined by $s \mapsto f(s) \parallel g(s)$
- $(f \wedge g) : S \longrightarrow \mathbb{B}$ is defined by $s \mapsto f(s) \&\& g(s)$
- $f' : S \longrightarrow \mathbb{B}$ is defined by $s \mapsto !f(s)$
- $0 : S \longrightarrow \mathbb{B}$ is the function $f(s) = \text{false}$
- $1 : S \longrightarrow \mathbb{B}$ is the function $f(s) = \text{true}$

All those examples were just the power set example in disguise.

- The two-element algebra with $\{\text{true}, \text{false}\}$ is just the power set algebra with $\mathcal{U} = \{\emptyset\}$, but with different notation.
- A function $f : S \rightarrow \mathbb{B}$ coincides with the membership relation $(\in)$ of the set $\{x : x \in S \text{ and } f(x) = \text{true}\}$
- ...

All those examples were just the power set example in disguise.

- The two-element algebra with $\{\text{true}, \text{false}\}$ is just the power set algebra with $\mathcal{U} = \{\emptyset\}$, but with different notation.
- A function $f : S \rightarrow \mathbb{B}$ coincides with the membership relation $(\in)$ of the set $\{x : x \in S \text{ and } f(x) = \text{true}\}$
- ...

In fact, every boolean algebra is the power set example in disguise!

Every finite Boolean algebra satisfies: $|T| = 2^k$ for some k .

All algebras with the same number of elements are **isomorphic**, i.e. “structurally similar”, written $\simeq$. Therefore, studying one such algebra describes properties of all.

The algebras mentioned above are all of this form

- n -tuples $\simeq \mathbb{B}^n$
- $\text{Pow}(S) \simeq \mathbb{B}^{|S|}$
- $\text{Map}(S, \mathbb{B}) \simeq \mathbb{B}^{|S|}$

Duality revisited

If E is an expression made up with $\wedge, \vee, ', 0, 1$ and variables; then $\text{dual}(E)$ is the expression obtained by replacing $\wedge$ with $\vee$ and vice-versa; and 0 with 1 and vice-versa.

Theorem (Principle of Duality)

If $E_1 = E_2$ holds in all Boolean Algebras^a, then $\text{dual}(E_1) = \text{dual}(E_2)$ holds in all Boolean Algebras.

^ai.e. is provable using the Boolean Algebra Laws

In Week 2, we convinced ourselves of this with some handwaving.

Duality revisited

If $(T, \vee, \wedge, ', 0, 1)$ is a Boolean algebra, then the **dual algebra** $(T, \wedge, \vee, ', 1, 0)$ is also a Boolean Algebra. For example:

- $T : \text{Pow}(X)$
- $\wedge : \cup$
- $\vee : \cap$
- $' : ^c$
- $0 : X$
- $1 : \emptyset$

The principle of duality follows immediately from this observation.

Dualising, formally

A Boolean Algebra **expression** is defined inductively as follows:

- $0, 1$ are expressions
- A variable, $x, y, \dots$, is an expression.
- If E is an expression then E' is an expression.
- If E_1 and E_2 are expressions, then $(E_1 \wedge E_2)$ and $(E_1 \vee E_2)$ are expressions.

Use EXP for the set of such expressions.

Dualising, formally

We define $\text{dual} : \text{EXP} \rightarrow \text{EXP}$ recursively as follows:

- $\text{dual}(0) = 1, \text{dual}(1) = 0$
- $\text{dual}(x) = x$ for all variables x
- $\text{dual}(E') = \text{dual}(E)'$ for all expressions E
- $\text{dual}((E_1 \wedge E_2)) = (\text{dual}(E_1) \vee \text{dual}(E_2))$ for all expressions E_1 and E_2
- $\text{dual}((E_1 \vee E_2)) = (\text{dual}(E_1) \wedge \text{dual}(E_2))$ for all expressions E_1 and E_2

Example dual

$$\text{dual}((x \vee (x \wedge y))) = (\text{dual}(x) \wedge \text{dual}((x \wedge y)))$$

Example dual

$$\begin{aligned}\text{dual}((x \vee (x \wedge y))) &= (\text{dual}(x) \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge \text{dual}((x \wedge y)))\end{aligned}$$

Example dual

$$\begin{aligned}\text{dual}((x \vee (x \wedge y))) &= (\text{dual}(x) \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge (\text{dual}(x) \vee \text{dual}(y)))\end{aligned}$$

Example dual

$$\begin{aligned}\text{dual}((x \vee (x \wedge y))) &= (\text{dual}(x) \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge (\text{dual}(x) \vee \text{dual}(y))) \\ &= (x \wedge (x \vee y))\end{aligned}$$

Boolean algebras: punchline

- Boolean algebras allow us to talk about set operations, propositions and bitwise operations on machine words at the same time.
- Properties proved in boolean algebra hold for all these instances: no more double labour.
- Calculation with set operations and calculation with propositional logic is, ultimately, the same thing. Therefore, you can freely change between the two perspectives in specifications and proofs.

Summary of topics

- Well-formed formulas
- Boolean Algebras
- **Valuations**
- CNF/DNF
- Proof
- Natural deduction

Valuations

A **truth assignment** (or **valuation**, or **model**) is a function

$$v : \text{PROP} \rightarrow \mathbb{B}$$

In week 2, we said:

*Two formulas ϕ, ψ are **logically equivalent**, denoted $\phi \equiv \psi$ if they have the same truth value for all truth valuations.*

But we never really defined valuation of formulas—instead, we handwaved it by saying “draw a truth table”.

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = \neg \llbracket \varphi \rrbracket_v$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = \neg \llbracket \varphi \rrbracket_v$
- $\llbracket (\varphi \wedge \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \&\& \ \llbracket \psi \rrbracket_v$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = \neg \llbracket \varphi \rrbracket_v$
- $\llbracket (\varphi \wedge \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \&\& \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \vee \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \|\ \llbracket \psi \rrbracket_v$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = !\llbracket \varphi \rrbracket_v$
- $\llbracket (\varphi \wedge \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \&\& \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \vee \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ || \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \rightarrow \psi) \rrbracket_v = !\llbracket \varphi \rrbracket_v \ || \ \llbracket \psi \rrbracket_v$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = !\llbracket \varphi \rrbracket_v$
- $\llbracket (\varphi \wedge \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \&\& \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \vee \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ || \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \rightarrow \psi) \rrbracket_v = !\llbracket \varphi \rrbracket_v \ || \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \leftrightarrow \psi) \rrbracket_v = (!\llbracket \varphi \rrbracket_v \ || \ \llbracket \psi \rrbracket_v) \ \&\& \ (!\llbracket \psi \rrbracket_v \ || \ \llbracket \varphi \rrbracket_v)$

Satisfiability, Validity and Equivalence

A formula φ is

- **satisfiable** if $\llbracket \varphi \rrbracket_v = \text{true}$ for some model v (v **satisfies** φ)
- **valid** or a **tautology** if $\llbracket \varphi \rrbracket_v = \text{true}$ for all models v
- **unsatisfiable** or a **contradiction** if $\llbracket \varphi \rrbracket_v = \text{false}$ for all models v

Logical equivalence

Two formulas, φ and ψ , are **logically equivalent**, $\varphi \equiv \psi$, if $\llbracket \varphi \rrbracket_v = \llbracket \psi \rrbracket_v$ for all models v .

Theorem

$\equiv$ *is an equivalence relation.*

Logical equivalence

Two formulas, φ and ψ , are **logically equivalent**, $\varphi \equiv \psi$, if $\llbracket \varphi \rrbracket_v = \llbracket \psi \rrbracket_v$ for all models v .

Theorem

$\equiv$ is an equivalence relation.

Example

- Commutativity: $(p \vee q) \equiv (q \vee p)$
- Double negation: $\neg\neg p \equiv p$
- Contrapositive: $(p \rightarrow q) \equiv (\neg q \rightarrow \neg p)$
- De Morgan's: $(p \vee q)' \equiv p' \wedge q'$

Logical equivalence

Two formulas, φ and ψ , are **logically equivalent**, $\varphi \equiv \psi$, if $\llbracket \varphi \rrbracket_v = \llbracket \psi \rrbracket_v$ for all models v .

Theorem

$\equiv$ is an equivalence relation.

Example

- Commutativity: $(p \vee q) \equiv (q \vee p)$
- Double negation: $\neg\neg p \equiv p$
- Contrapositive: $(p \rightarrow q) \equiv (\neg q \rightarrow \neg p)$
- De Morgan's: $(p \vee q)' \equiv p' \wedge q'$

Theorem

$\varphi \equiv \psi$ if, and only if, $(\varphi \leftrightarrow \psi)$ is a tautology.

Theories and entailment

A set of (well-formed) formulas is a **theory**

A model v *satisfies* a theory T if $\llbracket \varphi \rrbracket_v = \text{true}$ for all $\varphi \in T$

A theory T **entails** a formula φ , $T \models \varphi$, if $\llbracket \varphi \rrbracket_v = \text{true}$ for all models v which satisfy T

Example

- $T_1 = \{p\}$, $T_2 = \emptyset$, $T_3 = \{\perp\}$
- $v : p \mapsto \text{true}$ satisfies T_1 and T_2 but not T_3
- $T_1 \models (p \vee p)$ and $T_3 \models (p \vee p)$ but T_2 does not entail $(p \vee p)$

Theories and entailment

A set of (well-formed) formulas is a **theory**

A model v *satisfies* a theory T if $\llbracket \varphi \rrbracket_v = \text{true}$ for all $\varphi \in T$

A theory T **entails** a formula φ , $T \models \varphi$, if $\llbracket \varphi \rrbracket_v = \text{true}$ for all models v which satisfy T

Theorem

The following are equivalent:

- $\varphi_1, \varphi_2, \dots, \varphi_n \models \psi$
- $\emptyset \models ((\varphi_1 \wedge \varphi_2) \wedge \dots \varphi_n) \rightarrow \psi$
- $((\varphi_1 \wedge \varphi_2) \wedge \dots \varphi_n) \rightarrow \psi$ *is a tautology*
- $\emptyset \models \varphi_1 \rightarrow (\varphi_2 \rightarrow (\dots \rightarrow \varphi_n) \rightarrow \psi)) \dots$

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- **CNF/DNF** (tomorrow)
- Proof
- Natural deduction

Notes (Week 4 Monday)

These are the notes I wrote during the lecture.

If $n + 1 = m + 1$ then $n = m$

If a set A

(i) $0 \in A$

(ii) If $n \in A$, then $S(n) \in A$

then $\forall n. n \in A$

If we replace the set with a predicate,
and successor arithmetic with "normal"
arithmetic, we get

If a set P

(i) $P(0)$

(ii) If $P(n)$, then $P(n+1)$

then $\forall n. P(n)$

Often, you can switch at will between
sets and predicates.

A set is uniquely defined by its
membership predicate.

A predicate can be viewed as the set
of all things that satisfy it.

If $\text{my_fun}(x)$ returns x , (IH)

then $\text{my_fun}(x+1)$ returns $x+1$

by definition of the function,

$\text{my_fun}(x+1)$ returns $x+1$

iff

$\text{my_fun}(x+2)$ returns $x+2$

$(\mathbb{N}, >)$

.. $7 > 6 > 5$

$(\mathbb{R}, <)$

$$\dots < 1/8 < 1/4 < 1/2 < 1$$
 $(\mathbb{Z}, <)$
 $(\mathbb{N}, <)$

$$\dots < 30 < 31$$

^ the length of this chain
is at most 31

 $(\mathcal{P}(\mathbb{N}), \subseteq)$
 $\{1, 2\} \subseteq \{1, 2\}$
 $z(\text{hello}) = 00000$

```
open bossLib;
```

```
Definition z_def:
```

```
  z [] = [] ∧
  z [a] = [0] ∧
  z (a::b::w) =
  0::z(0::w)
```

```
End
```

```
Theorem z_zeroises:
```

```
  ∀w. z(w) =
    REPLICATE (LENGTH w) 0
```

```
Proof
```

```
  Induct_on 'w'
  >- rw[z_def]
  >- (Cases_on 'w'
    >- (gvs[z_def] \EVAL_TAC)
    rw[z_def] >>
    Cases_on 't' >>
    gvs[z_def])
```

```
QED
```

$x + 1 > 5 \wedge p = 3$ $\leftarrow$ this week, we don't have that

$p \wedge q$ $\leftarrow$ no term language, instead, we just have
connectives ($\wedge, \vee$) and boolean variables
(more proper: propositional letters)

According to the first slide,

```
p ∧ ¬q  <- that's not a wff :(
(p ∧ ¬q) <- now it's a wff
```

$p \wedge q \wedge r$ might be written $p \cdot q \cdot r$ or pqr

$p \rightarrow q \rightarrow r$

$(p \rightarrow q) \rightarrow r$

is *not* equivalent to

$p \rightarrow (q \rightarrow r) \equiv (p \wedge q) \rightarrow r$

$(T, \wedge, \vee, ', 0, 1)$ <-- when you see this in the Boolean Algebra slide,

$\wedge, \vee, ', 0, 1$ are not fixed. rather, think of them as variables names denoting functions over T .
And by choosing a specific T , and specific functions, we make a particular instance of
(the abstract concept of) a boolean algebra.

$\&\&$	vs	$\wedge$ (conjunction)
$\wedge$		$\wedge$
computes a truth value		given two <i>formulas</i> , constructs a new formula

$\mathcal{U} = \{\emptyset\}$

$T = \text{Pow}(\mathcal{U}) = \{\emptyset, \{\emptyset\}\}$

Think of $\emptyset$ as false,
and $\{\emptyset\}$ as true

In W2, we stated something like:

If you prove an equality on sets

$S = T$

with the laws of set operations,
you get
 $\text{dual}(S) = \text{dual}(T)$
for free

One-element boolean algebra

$(\{x\}, (s, t) \mapsto x, (s, t) \mapsto x, s \mapsto x, x, x)$

^ this boolean algebra satisfies

$$0 = 1$$

$$x \wedge x = x$$

$$x \vee x = x$$

$p \wedge q \rightarrow \neg q$ <-- this formula is neither true or false
 it's just a piece of syntax:
 it's a word over a particular alphabet,
 or a parse tree

We can *assign* truth values to the formula if we decide what the values of p and q are.

A *valuation* (aka states, models) is a function of type

$\text{Prop} \rightarrow \mathbb{B}$ function from propositional letters
 into the two-element boolean algebra

$p \mid q \mid p \wedge q \mid \neg q \mid p \wedge q \rightarrow \neg q$

$\top \mid \top \mid \top \mid \perp \mid$ $\perp$

$\top \mid \perp \mid \perp \mid \top \mid$ $\top$

.....

^ if we can be bothered to complete the table,
 we get the value of the formula under every possible valuation. A valuation just corresponds to a particular row in the truth table

Truth values are unsatisfactory for two reasons:

- We'd like to be more formal than "draw a picture"
- They don't scale that well.

For a formula with n distinct prop. letters, you'll have 2^n rows in your truth table.

[[]] these are called "semantic brackets"
 it's all Dana Scott's fault

If I have a thing (a piece of syntax), then
 [[thing]] (pronounced "the denotation of thing"),
 is some mathematical object that gives meaning

to thing.

$\llbracket \phi \rrbracket v$ "the truth value of the formula ϕ under the valuation v "

Logical law:

$$\phi \rightarrow \psi \equiv \neg \phi \vee \psi$$

Suppose we have a valuation v such that

$$p \mapsto 1$$

$$q \mapsto 0$$

$$\llbracket p \wedge q \rightarrow \neg q \rrbracket v =$$

$$!\llbracket p \wedge q \rrbracket v \mid \mid \llbracket \neg q \rrbracket v =$$

$$!(\llbracket p \rrbracket v \ \&\& \ \llbracket q \rrbracket v) \mid \mid \ !\llbracket q \rrbracket v =$$

$$!(1 \ \&\& \ 0) \mid \mid \ !0 =$$

$$!(0) \mid \mid \ 1 =$$

$$1 \mid \mid \ 1 =$$

$$1$$

$p \wedge q \rightarrow \neg q$ is satisfiable

$p \rightarrow p$ is a *tautology*

$\perp$ is unsatisfiable

$p \wedge \neg p$ is unsatisfiable

(Trivial) observation: every tautology is satisfiable.

Q: is biimplication the negation of xor?

A: yes

$$\llbracket p \leftrightarrow q \rrbracket v = \llbracket p \rightarrow q \rrbracket v \ \&\& \ \llbracket q \rightarrow p \rrbracket v$$

COMP2111 Week 4
Term 1, 2023
Propositional Logic II

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- **CNF/DNF**
- Proof
- Natural deduction

Terminology and Rules

We'll look at two normal forms of propositional formulae:

- Conjunctive normal form (CNF)

"a conjunction of disjunctions of possibly negated atoms"

Example: $a \wedge (b \vee \neg c) \wedge (a \vee c \vee d)$

Terminology and Rules

We'll look at two normal forms of propositional formulae:

- Conjunctive normal form (CNF)

"a conjunction of disjunctions of possibly negated atoms"

Example: $a \wedge (b \vee \neg c) \wedge (a \vee c \vee d)$

- Disjunctive normal form (DNF)

"a disjunction of conjunctions of possibly negated atoms"

Example: $(a \wedge b) \vee (\neg a \wedge \neg b)$

Terminology and Rules

For readability, we write $\bar{\varphi}$ for $\neg\varphi$. A **literal** is an expression p or $\bar{p}$, where p is a propositional atom.

A formula is in CNF if it has the form

$$C_0 \wedge C_1 \wedge C_2 \wedge \cdots \wedge C_n$$

where each **clause** C_i is a disjunction of literals e.g. $p \vee q \vee \bar{r}$.

A propositional formula is in DNF if it has the form

$$D_0 \vee D_1 \vee D_2 \vee \cdots \vee D_n$$

where each clause D_i is a conjunction of literals e.g. $p \wedge q \wedge \bar{r}$.

Motivation

- Checking satisfiability in DNF is straightforward.
- Checking validity in CNF is straightforward.
- Every propositional formula can be rewritten into DNF/CNF form (but it's computationally expensive)

Motivation

- Checking satisfiability in DNF is straightforward.
Is there a clause that doesn't have two contradictory literals?
I.e., doesn't have both p and $\bar{p}$ for some p ? If yes, it's satisfiable.
- Checking validity in CNF is straightforward.
- Every propositional formula can be rewritten into DNF/CNF form (but it's computationally expensive)

Motivation

- Checking satisfiability in DNF is straightforward.
Is there a clause that doesn't have two contradictory literals?
I.e., doesn't have both p and $\bar{p}$ for some p ? If yes, it's satisfiable.
- Checking validity in CNF is straightforward.
Does every clause have contradictory literals? If yes, it's valid.
- Every propositional formula can be rewritten into DNF/CNF form (but it's computationally expensive)

Theorem

For every propositional formula ϕ , there is an equivalent formula in CNF and an equivalent formula in DNF.

We will show two methods of obtaining the CNF/DNF:

- An algorithm for rewriting formulas.
- Reading them off the truth table.

Rewriting to CNF/DNF

- 1 Eliminate all $\rightarrow$ and $\leftrightarrow$, using

$$A \rightarrow B \equiv \bar{A} \vee B \quad A \leftrightarrow B \equiv (A \wedge B) \vee (\bar{A} \wedge \bar{B})$$

- 2 Push negations inwards, using **De Morgan** and **double negation**.
- 3 Push disjunctions inwards (for CNF), or conjunctions inwards (for DNF), using **distributivity**.

Step 2: Push Negations Down

Using **De Morgan's** laws and the **double negation** rule

$$\overline{x \vee y} \equiv \bar{x} \wedge \bar{y}$$

$$\overline{x \wedge y} \equiv \bar{x} \vee \bar{y}$$

$$\overline{\bar{x}} \equiv x$$

we push negations down towards the atoms, until we obtain a formula that is formed from literals using only $\wedge$ and $\vee$.

Step 3: Use Distribution to Convert to CNF

Using the distributivity rules (until we can't):

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

$$(A \wedge B) \vee C \equiv (A \vee C) \wedge (B \vee C)$$

we obtain a CNF formula. For a DNF formula, use the dual rules.

Step 2:

Example

$$\begin{aligned}\overline{p(\overline{rs} \vee q)} &\equiv \overline{\overline{p}} \vee \overline{\overline{rs} \vee q} \\ &\equiv p \vee \overline{\overline{rs}} \wedge \overline{q} \\ &\equiv p \vee rs\overline{q}\end{aligned}$$

Step 3:

Example

$$\begin{aligned}p \vee rs\overline{q} &\equiv (p \vee r)(p \vee s\overline{q}) \\ &\equiv (p \vee r)(p \vee s)(p \vee \overline{q}) \quad \text{CNF}\end{aligned}$$

Canonical Form DNF, informally

Extract a canonical DNF E^{dnf} from a truth table by making one disjunct for every row where E holds:

x	y	E
F	F	T
F	T	F
T	F	T
T	T	T

Canonical Form DNF, informally

Extract a canonical DNF E^{dnf} from a truth table by making one disjunct for every row where E holds:

x	y	E	
F	F	T	$\bar{x} \wedge \bar{y}$
F	T	F	
T	F	T	
T	T	T	

Canonical Form DNF, informally

Extract a canonical DNF E^{dnf} from a truth table by making one disjunct for every row where E holds:

x	y	E	
F	F	T	$\bar{x} \wedge \bar{y}$
F	T	F	
T	F	T	$x \wedge \bar{y}$
T	T	T	

Canonical Form DNF, informally

Extract a canonical DNF E^{dnf} from a truth table by making one disjunct for every row where E holds:

x	y	E	
F	F	T	$\bar{x} \wedge \bar{y}$
F	T	F	
T	F	T	$x \wedge \bar{y}$
T	T	T	$x \wedge y$

Canonical Form DNF, informally

Extract a canonical DNF E^{dnf} from a truth table by making one disjunct for every row where E holds:

x	y	E	
F	F	T	$\bar{x} \wedge \bar{y}$
F	T	F	
T	F	T	$x \wedge \bar{y}$
T	T	T	$x \wedge y$

thus $E^{dnf} = (\bar{x} \wedge \bar{y}) \vee (x \wedge \bar{y}) \vee (x \wedge y)$

Note that this can be simplified to $\bar{y} \vee (x \wedge y)$ or $(\bar{x} \wedge \bar{y}) \vee x$

Canonical Form DNF, formally

Given a Boolean expression E , we can construct an equivalent DNF E^{dnf} from the lines of the truth table where E is true:

Given an assignment v from $\{x_1 \dots x_i\}$ to $\mathbb{B}$, define the literal

$$\ell_i = \begin{cases} x_i & \text{if } v(x_i) = \text{true} \\ \overline{x_i} & \text{if } v(x_i) = \text{false} \end{cases}$$

and a product $t_v = \ell_1 \wedge \ell_2 \wedge \dots \wedge \ell_n$.

Example

If $v(x_1) = \text{true}$ and $v(x_2) = \text{false}$ then $t_v = x_1 \wedge \overline{x_2}$

The **canonical DNF** of E is

$$E^{dnf} = \bigvee_{\llbracket E \rrbracket_v = \text{true}} t_v$$

Canonical Form CNF, informally

Extract a canonical CNF by making one conjunct for every row where E does not hold, *negating the atoms*:

x	y	E
F	F	F
F	T	F
T	F	T
T	T	F

Canonical Form CNF, informally

Extract a canonical CNF by making one conjunct for every row where E does not hold, *negating the atoms*:

x	y	E	
F	F	F	$x \vee y$
F	T	F	
T	F	T	
T	T	F	

Canonical Form CNF, informally

Extract a canonical CNF by making one conjunct for every row where E does not hold, *negating the atoms*:

x	y	E	
F	F	F	$x \vee y$
F	T	F	$x \vee \bar{y}$
T	F	T	
T	T	F	

Canonical Form CNF, informally

Extract a canonical CNF by making one conjunct for every row where E does not hold, *negating the atoms*:

x	y	E	
F	F	F	$x \vee y$
F	T	F	$x \vee \bar{y}$
T	F	T	
T	T	F	$\bar{x} \vee \bar{y}$

Canonical Form CNF, informally

Extract a canonical CNF by making one conjunct for every row where E does not hold, *negating the atoms*:

x	y	E	
F	F	F	$x \vee y$
F	T	F	$x \vee \bar{y}$
T	F	T	
T	T	F	$\bar{x} \vee \bar{y}$

thus $E^{cnf} = (x \vee y) \wedge (x \vee \bar{y}) \wedge (\bar{x} \vee \bar{y})$

Canonical CNF

After pushing negations down, the negation of a DNF is a CNF (and vice versa).

$\Rightarrow$ Given an expression E , we can obtain an equivalent CNF by finding a DNF for $\neg E$ and then applying De Morgan's laws.

$\Leftrightarrow$ Look at rows in the truth table of E that contain **false** and *negate* the literals.

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- CNF/DNF
- **Proof**
- Natural deduction

Motivation

(Informal) proofs are hard to check. Why?

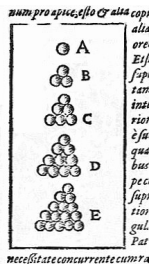
Most mathematicians don't explain all the details when they write proofs; the idea is that an expert reader can fill in the gaps.

“Without loss of generality, we may assume . . .”, “It is easily seen that . . .”

Checking such proofs take significant time and expertise.

Motivation

In 1998,



Tom Hales proved the Kepler conjecture in 300 pages of mathematical prose and 40k lines of (unverified) code. He submitted his work to a journal for peer review.

Motivation

Seven years later, the proof is accepted. A journal editor describes the process:

The referees put a level of energy into this that is, in my experience, unprecedented. They ran a seminar on it for a long time. A number of people were involved, and they worked hard. They checked many local statements in the proof, and each time they found that what you claimed was in fact correct. Some of these local checks were highly non-obvious at first, and required weeks to see that they worked out . . . They have not been able to certify the correctness of the proof, and will not be able to certify it in the future, because they have run out of energy to devote to the problem.

Motivation

Formal proofs are *easy* to check.

“Easy” as in: completely mechanical, requires no domain expertise, can be outsourced to a computer program—just check that each step follows the rules.

“The rules”: a precise, unambiguous description of which proof steps are allowed. Different formalisms may have different rules; we’ll study one (natural deduction).

Given a theory T and a formula φ , how do we show $T \models \varphi$?

- Consider all valuations v (SEMANTIC approach)
- Use a sequence of **inference rules** to show that φ is a logical consequence of T (SYNTACTIC approach)

New notation

We will distinguish between two concepts:

Notation	Name	Meaning
$T \models \varphi$	entailment	whenever T is true, φ is true.
$T \vdash \varphi$	provability	we can prove φ from T using the inference rules.

Why does this distinction matter?

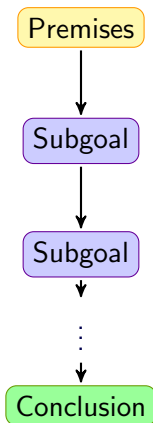
New notation

Why distinguish truth from provability?

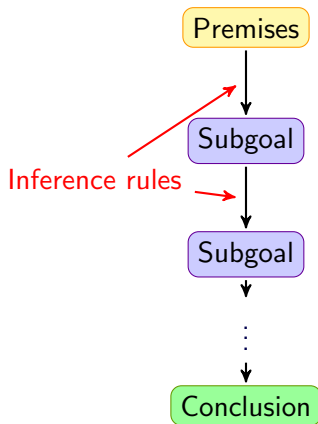
If our inference rules are poorly chosen, they could “prove” things that are not true.

Conversely, something might be true, but our inference rules might not be powerful enough to prove it.

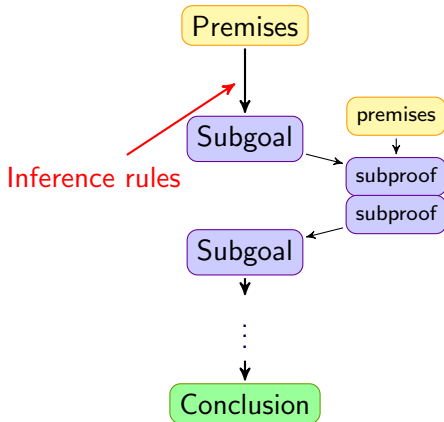
Proof structure



Proof structure



Proof structure



Inference rules

In its simplest form, an **inference rule** is a statement of the form:

If I have a proof of this then I have a proof of that

Inference rules

In its simplest form, an **inference rule** is a statement of the form:

If I have a proof of this then I have a proof of that

A more complicated form:

If I have a proof of this (under those assumptions)
then I have a proof of that (under these* assumptions)

NB

The sets of assumptions need not be the same!

Inference rules

In its simplest form, an **inference rule** is a statement of the form:

If I have a proof of this then I have a proof of that

Yet more complicated form:

If I have a proof of this (under those assumptions)
and I have a proof of this (under those assumptions)
and ...
then I have a proof of that (under these assumptions)

NB

The sets of assumptions need not be the same!

Inference rules: notation

So an inference rule is a statement of the form:

If $T_1 \vdash \varphi_1$ and $T_2 \vdash \varphi_2$ and ... and $T_n \vdash \varphi_n$ then $T \vdash \psi$

Inference rules: notation

So an inference rule is a statement of the form:

If $T_1 \vdash \varphi_1$ and $T_2 \vdash \varphi_2$ and ... and $T_n \vdash \varphi_n$ then $T \vdash \psi$

Alternative notation:

$$\frac{T_1 \vdash \varphi_1 \quad T_2 \vdash \varphi_2 \quad \dots \quad T_n \vdash \varphi_n}{T \vdash \psi}$$

Inference rules: notation

So an inference rule is a statement of the form:

If $T_1 \vdash \varphi_1$ and $T_2 \vdash \varphi_2$ and ... and $T_n \vdash \varphi_n$ then $T \vdash \psi$

Alternative notation:

$$\frac{\varphi_1 \quad \varphi_2 \quad \cdots \quad \varphi_n}{\psi}$$

(If $T_1 = T_2 = \dots = T$)

Inference rules: notation

So an inference rule is a statement of the form:

If $T_1 \vdash \varphi_1$ and $T_2 \vdash \varphi_2$ and ... and $T_n \vdash \varphi_n$ then $T \vdash \psi$

Alternative notation:

$$\frac{\begin{array}{cccc} & [\alpha] & & \\ & \vdots & & \\ \varphi_1 & \varphi_2 & \cdots & \varphi_n \end{array}}{\psi}$$

(If $T_2 = T \cup \{\alpha\}$)

Inference rules: examples

$\wedge$ -elimination:

$$\frac{A \wedge B}{A} (\wedge\text{-E1})$$

Inference rules: examples

$\wedge$ -elimination:

$$\frac{A \wedge B}{A} (\wedge\text{-E1})$$

$\rightarrow$ -elimination:
(Modus Ponens)

$$\frac{A \rightarrow B \quad A}{B} (\rightarrow\text{-E})$$

Inference rules: examples

$\wedge$ -elimination:

$$\frac{A \wedge B}{A} (\wedge\text{-E1})$$

$\rightarrow$ -elimination:
(Modus Ponens)

$$\frac{A \rightarrow B \quad A}{B} (\rightarrow\text{-E})$$

$\rightarrow$ -introduction:

$$\frac{\begin{array}{c} [A] \\ \vdots \\ B \end{array}}{A \rightarrow B} (\rightarrow\text{-I})$$

Proof layout

We will look at three alternative ways to present proofs:

Tabular Easy to typeset, annoying to read.

Fitch-style Easy to read, annoying to typeset.

Proof trees Nice for short proofs, very unwieldy for long proofs.

Tabular

Line	Premises	Formula	Rule	References

Tabular

Example:

Prove: $A \rightarrow B, B \rightarrow C \vdash A \rightarrow C$

Line	Premises	Formula	Rule	References
1		$A \rightarrow B$	Premise	
2		$B \rightarrow C$	Premise	

Tabular

Example:

Prove: $A \rightarrow B, B \rightarrow C \vdash A \rightarrow C$

Line	Premises	Formula	Rule	References
1		$A \rightarrow B$	Premise	
2		$B \rightarrow C$	Premise	
	1, 2	$A \rightarrow C$		

Tabular

Example:

Prove: $A \rightarrow B, B \rightarrow C \vdash A \rightarrow C$

Line	Premises	Formula	Rule	References
1		$A \rightarrow B$	Premise	
2		$B \rightarrow C$	Premise	
3		A	Premise	

Tabular

Example:

Prove: $A \rightarrow B, B \rightarrow C \vdash A \rightarrow C$

Line	Premises	Formula	Rule	References
1		$A \rightarrow B$	Premise	
2		$B \rightarrow C$	Premise	
3		A	Premise	
4	1, 3	B	$\rightarrow$ -E	1, 3

Tabular

Example:

Prove: $A \rightarrow B, B \rightarrow C \vdash A \rightarrow C$

Line	Premises	Formula	Rule	References
1		$A \rightarrow B$	Premise	
2		$B \rightarrow C$	Premise	
3		A	Premise	
4	1, 3	B	$\rightarrow$ -E	1, 3
5	1, 2, 3	C	$\rightarrow$ -E	2, 4

Tabular

Example:

Prove: $A \rightarrow B, B \rightarrow C \vdash A \rightarrow C$

Line	Premises	Formula	Rule	References
1		$A \rightarrow B$	Premise	
2		$B \rightarrow C$	Premise	
3		A	Premise	
4	1, 3	B	$\rightarrow$ -E	1, 3
5	1, 2, 3	C	$\rightarrow$ -E	2, 4
6	1, 2	$A \rightarrow C$	$\rightarrow$ -I	5

Premises

You may assume anything, but it may not be helpful!

You must **discharge** any assumptions you make along the way.

Premises

You may assume anything, but it may not be helpful!

You must **discharge** any assumptions you make along the way.

For example, let's "prove" $A \rightarrow B \vdash B$:

Line	Premises	Formula	Rule	References
1		$A \rightarrow B$	Premise	
2		A	Premise	
3	1, 2	B	$\rightarrow$ -E	1, 2

Now we have no way of discharging A . Thus we ended up proving this instead:

$$A \rightarrow B, A \vdash B$$

Proof tree layout

Upside-down tree (root at bottom):

- Natural structure arising from rule syntax
- Premises at leaves
- Conclusion at root

$$\frac{\frac{B \rightarrow C}{C} (\rightarrow\text{-I})}{\frac{B \rightarrow C \quad \frac{A \rightarrow B \quad [A]}{B} (\rightarrow\text{-E})}{A \rightarrow C} (\rightarrow\text{-E})}$$

Other proof layouts: Fitch-style

- Style used in online tool
- Premises above the notch, subgoals below.
- Subproofs are indented

1.	$A \rightarrow B$	
2.	$B \rightarrow C$	
3.	A	
4.	B	$\rightarrow$ -E: 1,3
5.	C	$\rightarrow$ -E: 2,4
6.	$A \rightarrow C$	$\rightarrow$ -I: 3-5

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- CNF/DNF
- Proof
- Natural deduction

Online resources

Open logic project (<https://proofs.openlogicproject.org/>)

- Freely available textbooks
- Online proof checker
- Sample exercises

Natural deduction

Proof system intended to mirror “natural reasoning”:

- No axioms (no inherent truths)
- 15 inference rules: primarily grouped into pairs (and trios) of rules tasked with **introducing** and **eliminating** boolean operators from the chain of reasoning.

Operator	Introduction	Elimination
$\wedge$	$\wedge$ -I	$\wedge$ -E1 $\wedge$ -E2
$\vee$	$\vee$ -I1 $\vee$ -I2	$\vee$ -E
$\rightarrow$	$\rightarrow$ -I	$\rightarrow$ -E
$\leftrightarrow$	$\leftrightarrow$ -I	$\leftrightarrow$ -E1 $\leftrightarrow$ -E2
$\neg$	$\neg$ -I	$\neg$ -E IP
$\perp$	$\neg$ -E	X

$\wedge$ Introduction and Elimination

$\wedge$ -introduction:

$$\frac{A \quad B}{A \wedge B} (\wedge\text{-I})$$

$\wedge$ Introduction and Elimination

$\wedge$ -introduction:
$$\frac{A \quad B}{A \wedge B} (\wedge\text{-I})$$

$\wedge$ -elimination (1):
$$\frac{A \wedge B}{A} (\wedge\text{-E1})$$

$\wedge$ -elimination (2):
$$\frac{A \wedge B}{B} (\wedge\text{-E2})$$

$\vee$ Introduction and Elimination

$\vee$ -introduction (1): $\frac{A}{A \vee B} (\vee\text{-I1})$

$\vee$ -introduction (2): $\frac{B}{A \vee B} (\vee\text{-I1})$

∨ Introduction and Elimination

∨-introduction (1):
$$\frac{A}{A \vee B} \text{ (}\vee\text{-I1)}$$

∨-introduction (2):
$$\frac{B}{A \vee B} \text{ (}\vee\text{-I2)}$$

∨-elimination:
$$\frac{A \vee B \quad \begin{array}{c} [A] \\ \vdots \\ C \end{array} \quad \begin{array}{c} [B] \\ \vdots \\ C \end{array}}{C} \text{ (}\vee\text{-E)}$$

→ Introduction and Elimination

→-introduction:

$$\frac{\begin{array}{c} [A] \\ \vdots \\ B \end{array}}{A \rightarrow B} (\rightarrow\text{-I})$$

→-elimination:
(Modus Ponens)

$$\frac{A \rightarrow B \quad A}{B} (\rightarrow\text{-E})$$

$\leftrightarrow$ Introduction and Elimination

$\leftrightarrow$ -introduction:

$$\frac{\begin{array}{c} [A] \\ \vdots \\ B \end{array} \quad \begin{array}{c} [B] \\ \vdots \\ A \end{array}}{A \leftrightarrow B} (\leftrightarrow\text{-I})$$

$\leftrightarrow$ -elimination (1):

$$\frac{A \leftrightarrow B \quad A}{B} (\leftrightarrow\text{-E1})$$

$\leftrightarrow$ -elimination (2):

$$\frac{A \leftrightarrow B \quad B}{A} (\leftrightarrow\text{-E1})$$

¬ Introduction and Elimination and Indirect Proof

¬-introduction:

$$\frac{\begin{array}{c} [A] \\ \vdots \\ \perp \end{array}}{\neg A} (\neg\text{-I})$$

¬-elimination:
($\perp$ -introduction)

$$\frac{A \quad \neg A}{\perp} (\neg\text{-E})$$

¬ Introduction and Elimination and Indirect Proof

¬-introduction:

$$\frac{\begin{array}{c} [A] \\ \vdots \\ \perp \end{array}}{\neg A} (\neg\text{-I})$$

¬-elimination:
($\perp$ -introduction)

$$\frac{A \quad \neg A}{\perp} (\neg\text{-E})$$

Indirect proof:

$$\frac{\begin{array}{c} [\neg A] \\ \vdots \\ \perp \end{array}}{A} (\text{IP})$$

Explosion

Explosion:
($\perp$ -elimination)

$$\frac{\perp}{A} (X)$$

Derived rules

Several useful rules are available in the proof checker. Feel free to use them in assignments and exam, unless otherwise stated.

Double negation elimination:
$$\frac{\neg\neg A}{A} \text{ (DNE)}$$

Reiteration:
$$\frac{A}{A} \text{ (R)}$$

Law of excluded middle:
$$\frac{\begin{array}{c} [A] \\ \vdots \\ B \end{array} \quad \begin{array}{c} [\neg A] \\ \vdots \\ B \end{array}}{B} \text{ (LEM)}$$

Derived rules

Disjunctive syllogism:
$$\frac{A \vee B \quad \neg A}{B} \text{ (DS)}$$

Modus Tollens:
$$\frac{A \rightarrow B \quad \neg B}{\neg A} \text{ (MT)}$$

De Morgans Laws (e.g.):
$$\frac{\neg(A \vee B)}{\neg A \wedge \neg B} \text{ (DM)}$$

Soundness and completeness

Two key properties of a “good” system:

Soundness: The system only proves valid statements: If $T \vdash \varphi$ then $T \models \varphi$.

Completeness: The system can prove any valid statement: If $T \models \varphi$ then $T \vdash \varphi$.

NB

An unsound system is kinda useless.

An incomplete system can still be useful. For most logics, no sound and complete system exists (c.f. Gödel's incompleteness theorem).

Soundness and completeness

Theorem

Natural deduction is sound and complete. That is,

$$T \vdash \varphi \text{ if and only if } T \models \varphi$$

Soundness and completeness

Theorem

Natural deduction is sound and complete. That is,

$$T \vdash \varphi \quad \text{if and only if} \quad T \models \varphi$$

Corollary

The following are equivalent:

- $\varphi_1, \varphi_2, \dots, \varphi_n \models \varphi$
- $\varphi_1 \wedge \dots \wedge \varphi_n \rightarrow \varphi$ is a tautology
- $\varphi_1 \rightarrow (\varphi_2 \rightarrow (\dots \rightarrow (\varphi_n \rightarrow \varphi)) \dots)$ is a tautology
- $\varphi_1, \varphi_2, \dots, \varphi_n \vdash \varphi$
- $\varphi_1, \varphi_2, \dots, \varphi_{n-1} \vdash \varphi_n \rightarrow \varphi$
- (and so on)

This guy again



Tom Hales, evidently unsatisfied that his peer reviewers were only 99% convinced that he'd proven the Kepler conjecture, decided to construct a *formal* proof.

In 2014, after a decade-long slog, his team of around 20 mathematicians, computer scientists and engineers produced a fully formal proof, with every last inference machine-checked to follow the inference rules of HOL.

The Kepler conjecture was settled.

Credits



This picture ©2012 Slawekb, [CC BY-SA 3.0](#)

The journal editor's quote can be found here:

- Hales, Thomas C. *Formal Proof*. Page 1370–1380 of *Notices of the AMS*, volume 55, number 11. American Mathematical Society 2008.

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- CNF/DNF
- Proof
- Natural deduction
- Bonus examples

Proof examples

What follows is a bunch of example derivations in natural deduction, with all three styles well represented.

I do not plan to go over these in the lecture (favouring instead live demos).

Proof example

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

Line	Premises	Formula	Rule	References
------	----------	---------	------	------------

Proof example

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

Line	Premises	Formula	Rule	References
1		$(A \wedge B) \wedge C$	Premise	

Proof example

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

Line	Premises	Formula	Rule	References
1		$(A \wedge B) \wedge C$	Premise	
2	1	$A \wedge B$	$\wedge$ -E1	1

Proof example

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

Line	Premises	Formula	Rule	References
1		$(A \wedge B) \wedge C$	Premise	
2	1	$A \wedge B$	$\wedge$ -E1	1
3	1	A	$\wedge$ -E1	2

Proof example

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

Line	Premises	Formula	Rule	References
1		$(A \wedge B) \wedge C$	Premise	
2	1	$A \wedge B$	$\wedge$ -E1	1
3	1	A	$\wedge$ -E1	2
4	1	B	$\wedge$ -E2	2

Proof example

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

Line	Premises	Formula	Rule	References
1		$(A \wedge B) \wedge C$	Premise	
2	1	$A \wedge B$	$\wedge$ -E1	1
3	1	A	$\wedge$ -E1	2
4	1	B	$\wedge$ -E2	2
5	1	C	$\wedge$ -E2	1

Proof example

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

Line	Premises	Formula	Rule	References
1		$(A \wedge B) \wedge C$	Premise	
2	1	$A \wedge B$	$\wedge$ -E1	1
3	1	A	$\wedge$ -E1	2
4	1	B	$\wedge$ -E2	2
5	1	C	$\wedge$ -E2	1
6	1	$B \wedge C$	$\wedge$ -I	4, 5

Proof example

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

Line	Premises	Formula	Rule	References
1		$(A \wedge B) \wedge C$	Premise	
2	1	$A \wedge B$	$\wedge$ -E1	1
3	1	A	$\wedge$ -E1	2
4	1	B	$\wedge$ -E2	2
5	1	C	$\wedge$ -E2	1
6	1	$B \wedge C$	$\wedge$ -I	4, 5
7	1	$A \wedge (B \wedge C)$	$\wedge$ -I	3, 6

Proof example (Fitch)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

1. $(A \wedge B) \wedge C$

Proof example (Fitch)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

1. $(A \wedge B) \wedge C$

2. $A \wedge B$ $\wedge$ -E1: 1

Proof example (Fitch)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

- | | |
|----------------------------|-----------------|
| 1. $(A \wedge B) \wedge C$ | |
| 2. $A \wedge B$ | $\wedge$ -E1: 1 |
| 3. A | $\wedge$ -E1: 2 |

Proof example (Fitch)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

- | | |
|----------------------------|-----------------|
| 1. $(A \wedge B) \wedge C$ | |
| — | |
| 2. $A \wedge B$ | $\wedge$ -E1: 1 |
| 3. A | $\wedge$ -E1: 2 |
| 4. B | $\wedge$ -E2: 2 |

Proof example (Fitch)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

- | | |
|----------------------------|-----------------|
| 1. $(A \wedge B) \wedge C$ | |
| <hr/> | |
| 2. $A \wedge B$ | $\wedge$ -E1: 1 |
| 3. A | $\wedge$ -E1: 2 |
| 4. B | $\wedge$ -E2: 2 |
| 5. C | $\wedge$ -E2: 1 |

Proof example (Fitch)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

1.	$(A \wedge B) \wedge C$	
2.	$A \wedge B$	$\wedge$ -E1: 1
3.	A	$\wedge$ -E1: 2
4.	B	$\wedge$ -E2: 2
5.	C	$\wedge$ -E2: 1
6.	$B \wedge C$	$\wedge$ -I: 4,5

Proof example (Fitch)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

1.	$(A \wedge B) \wedge C$	
2.	$A \wedge B$	$\wedge$ -E1: 1
3.	A	$\wedge$ -E1: 2
4.	B	$\wedge$ -E2: 2
5.	C	$\wedge$ -E2: 1
6.	$B \wedge C$	$\wedge$ -I: 4,5
7.	$A \wedge (B \wedge C)$	$\wedge$ -I: 3,6

Proof example (Tree)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

$$\begin{array}{ccccc} & & (A \wedge B) \wedge C & & \\ & & / \quad \backslash & & \\ (A \wedge B) \wedge C & & A \wedge B & & (A \wedge B) \wedge C \\ / \quad \backslash & & / \quad \backslash & & / \quad \backslash \\ A \wedge B & & B & & C \\ / & & & & \\ A & & & & B \wedge C \\ & & & & / \quad \backslash \\ & & & & A & C \\ & & & & / \quad \backslash \\ & & & & A & (B \wedge C) \end{array}$$

Proof example (Tree)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

$$\begin{array}{c} \frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1}) \\ A \end{array} \quad \begin{array}{c} (A \wedge B) \wedge C \\ A \wedge B \\ B \end{array} \quad \begin{array}{c} (A \wedge B) \wedge C \\ C \\ B \wedge C \end{array} \\ A \wedge (B \wedge C)$$

Proof example (Tree)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

$$\begin{array}{c} \frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1}) \\ \frac{A \wedge B}{A} (\wedge\text{-E1}) \\ \hline (A \wedge B) \wedge C \\ A \wedge B \quad B \quad (A \wedge B) \wedge C \\ B \quad C \\ B \wedge C \\ A \wedge (B \wedge C) \end{array}$$

Proof example (Tree)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

$$\begin{array}{c} \frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1}) \\ \frac{A \wedge B}{A} (\wedge\text{-E1}) \end{array} \quad \frac{\frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1})}{B} (\wedge\text{-E1}) \quad \frac{(A \wedge B) \wedge C}{C} (\wedge\text{-E1})$$
$$\begin{array}{c} A \wedge (B \wedge C) \end{array}$$

Proof example (Tree)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

$$\begin{array}{c} \frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1}) \\ \frac{A \wedge B}{A} (\wedge\text{-E1}) \end{array} \quad \frac{\frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1})}{B} (\wedge\text{-E2}) \quad \frac{(A \wedge B) \wedge C}{C} (\wedge\text{-E2})$$
$$\frac{A \wedge (B \wedge C)}{A \wedge (B \wedge C)}$$

Proof example (Tree)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

$$\begin{array}{c} \frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1}) \quad \frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1}) \quad \frac{(A \wedge B) \wedge C}{B} (\wedge\text{-E2}) \quad \frac{(A \wedge B) \wedge C}{C} (\wedge\text{-E2}) \\ \frac{A \wedge B}{A} (\wedge\text{-E1}) \quad \frac{B}{B \wedge C} (\wedge\text{-E2}) \\ A \wedge (B \wedge C) \end{array}$$

Proof example (Tree)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

$$\frac{\frac{\frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1})}{A} (\wedge\text{-E1})}{\frac{\frac{\frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1})}{A} (\wedge\text{-E1}) \quad \frac{\frac{\frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1})}{A \wedge B} (\wedge\text{-E2})}{B} (\wedge\text{-E2}) \quad \frac{\frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E2})}{C} (\wedge\text{-E2})}{B \wedge C} (\wedge\text{-I})} A \wedge (B \wedge C)$$

Proof example (Tree)

Prove: $(A \wedge B) \wedge C \vdash A \wedge (B \wedge C)$

$$\frac{\frac{\frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1})}{A} (\wedge\text{-E1}) \quad \frac{\frac{\frac{(A \wedge B) \wedge C}{A \wedge B} (\wedge\text{-E1})}{B} (\wedge\text{-E2}) \quad \frac{\frac{(A \wedge B) \wedge C}{C} (\wedge\text{-E2})}{B \wedge C} (\wedge\text{-I})}{A \wedge (B \wedge C)} (\wedge\text{-I})$$

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
------	----------	---------	------	------------

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2
4	2	$A \vee C$	$\vee$ -I1	2

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2
4	2	$A \vee C$	$\vee$ -I1	2
5	2	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	3, 4

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2
4	2	$A \vee C$	$\vee$ -I1	2
5	2	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	3,4
6		$(B \wedge C)$	Premise	

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2
4	2	$A \vee C$	$\vee$ -I1	2
5	2	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	3, 4
6		$(B \wedge C)$	Premise	
7	6	B	$\wedge$ -E1	6

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2
4	2	$A \vee C$	$\vee$ -I1	2
5	2	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	3,4
6		$(B \wedge C)$	Premise	
7	6	B	$\wedge$ -E1	6
8	6	$A \vee B$	$\vee$ -I2	7

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2
4	2	$A \vee C$	$\vee$ -I1	2
5	2	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	3,4
6		$(B \wedge C)$	Premise	
7	6	B	$\wedge$ -E1	6
8	6	$A \vee B$	$\vee$ -I2	7
9	6	C	$\wedge$ -E2	6

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2
4	2	$A \vee C$	$\vee$ -I1	2
5	2	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	3,4
6		$(B \wedge C)$	Premise	
7	6	B	$\wedge$ -E1	6
8	6	$A \vee B$	$\vee$ -I2	7
9	6	C	$\wedge$ -E2	6
10	6	$A \vee C$	$\vee$ -I2	9

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2
4	2	$A \vee C$	$\vee$ -I1	2
5	2	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	3,4
6		$(B \wedge C)$	Premise	
7	6	B	$\wedge$ -E1	6
8	6	$A \vee B$	$\vee$ -I2	7
9	6	C	$\wedge$ -E2	6
10	6	$A \vee C$	$\vee$ -I2	9
11	6	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	8,10

Proof example

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

Line	Premises	Formula	Rule	References
1		$A \vee (B \wedge C)$	Premise	
2		A	Premise	
3	2	$A \vee B$	$\vee$ -I1	2
4	2	$A \vee C$	$\vee$ -I1	2
5	2	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	3, 4
6		$(B \wedge C)$	Premise	
7	6	B	$\wedge$ -E1	6
8	6	$A \vee B$	$\vee$ -I2	7
9	6	C	$\wedge$ -E2	6
10	6	$A \vee C$	$\vee$ -I2	9
11	6	$(A \vee B) \wedge (A \vee C)$	$\wedge$ -I	8, 10
12	1	$(A \vee B) \wedge (A \vee C)$	$\vee$ -E	5, 11

Proof example (Fitch)

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

1. $A \vee (B \wedge C)$

2. A

3. $A \vee B$

$\vee$ -I1: 2

4. $A \vee C$

$\vee$ -I1: 2

5. $(A \vee B) \wedge (A \vee C)$

$\wedge$ -I: 3,4

Proof example (Fitch)

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

1. $A \vee (B \wedge C)$

2. A

3. $A \vee B$

$\vee$ -I1: 2

4. $A \vee C$

$\vee$ -I1: 2

5. $(A \vee B) \wedge (A \vee C)$

$\wedge$ -I: 3,4

6. $B \wedge C$

7. B

$\wedge$ -E1: 6

8. $A \vee B$

$\vee$ -I2: 7

9. C

$\wedge$ -E2: 6

10. $A \vee C$

$\vee$ -I2: 9

11. $(A \vee B) \wedge (A \vee C)$

$\wedge$ -I: 8,10

Proof example (Fitch)

Prove: $A \vee (B \wedge C) \vdash (A \vee B) \wedge (A \vee C)$

1. $A \vee (B \wedge C)$

2. A

3. $A \vee B$

$\vee$ -I1: 2

4. $A \vee C$

$\vee$ -I1: 2

5. $(A \vee B) \wedge (A \vee C)$

$\wedge$ -I: 3,4

6. $B \wedge C$

7. B

$\wedge$ -E1: 6

8. $A \vee B$

$\vee$ -I2: 7

9. C

$\wedge$ -E2: 6

10. $A \vee C$

$\vee$ -I2: 9

11. $(A \vee B) \wedge (A \vee C)$

$\wedge$ -I: 8,10

12. $(A \vee B) \wedge (A \vee C)$

$\vee$ -E: 1,3–5,6–11

Proof example: double negation

Prove: $A \vdash \neg\neg A$

Line	Premises	Formula	Rule	References
1		A	Premise	

Proof example: double negation

Prove: $A \vdash \neg\neg A$

Line	Premises	Formula	Rule	References
1		A	Premise	
2		$\neg A$	Premise	

Proof example: double negation

Prove: $A \vdash \neg\neg A$

Line	Premises	Formula	Rule	References
1		A	Premise	
2		$\neg A$	Premise	
3	1, 2	$\perp$	$\neg$ -E	1, 2

Proof example: double negation

Prove: $A \vdash \neg\neg A$

Line	Premises	Formula	Rule	References
1		A	Premise	
2		$\neg A$	Premise	
3	1, 2	$\perp$	$\neg$ -E	1, 2
4	1	$\neg\neg A$	$\neg$ -I	3

Proof example: double negation

Prove: $\neg\neg A \vdash A$

Line	Premises	Formula	Rule	References
1		$\neg\neg A$	Premise	

Proof example: double negation

Prove: $\neg\neg A \vdash A$

Line	Premises	Formula	Rule	References
1		$\neg\neg A$	Premise	
2		$\neg A$	Premise	

Proof example: double negation

Prove: $\neg\neg A \vdash A$

Line	Premises	Formula	Rule	References
1		$\neg\neg A$	Premise	
2		$\neg A$	Premise	
3	1, 2	$\perp$	$\neg$ -E	1, 2

Proof example: double negation

Prove: $\neg\neg A \vdash A$

Line	Premises	Formula	Rule	References
1		$\neg\neg A$	Premise	
2		$\neg A$	Premise	
3	1, 2	$\perp$	$\neg$ -E	1, 2
4			?	

Proof example: double negation

Prove: $\neg\neg A \vdash A$

Line	Premises	Formula	Rule	References
1		$\neg\neg A$	Premise	
2		$\neg A$	Premise	
3	1, 2	$\perp$	$\neg$ -E	1, 2
4	1	A	IP	3

Notes (Week 4 Wednesday)

These are the notes I wrote during the lecture.

Well-formed formulas:

the syntactically correct propositional
logic formulas

Valuations, boolean algebras

The boolean algebra of truth values

$\{\text{true}, \text{false}\} \in \mathbb{B}$

Given a **valuation** v :

a function that assigns truth values
to variables (aka propositional atoms)

We can define a function $\llbracket \cdot \rrbracket v$

which given a valuation maps formulas
to truth values

$\llbracket p \wedge q \rrbracket (p \mapsto \text{true}, q \mapsto \text{false}) = \text{false}$

This gives us enough tools to give

fully formal meaning to $\equiv$ (logical equivalence)

$\phi \equiv \psi$ iff for every v : $\llbracket \phi \rrbracket v = \llbracket \psi \rrbracket v$

Equivalently: the truth tables of ϕ and ψ
are the same

$\Rightarrow$ logical entailment

$\phi \Rightarrow \psi$ iff for every v

whenever $\llbracket \phi \rrbracket v = \text{true}$

then $\llbracket \psi \rrbracket v = \text{true}$

From now on, we'll switch notation for entailment
and its type

$\Rightarrow : \text{wff} \times \text{wff} \rightarrow \mathbb{B}$

$\models$ "entails" or "models"

$T \models \phi$ ϕ is still a wff

T is a **theory**

a theory is a set of wffs

$T \models \phi$ iff
 for every valuation v such that
 if for every $\psi \in T$, $\llbracket \psi \rrbracket v = \text{true}$
 then $\llbracket \phi \rrbracket v = \text{true}$

$$p \rightarrow (q \rightarrow r) \equiv (p \wedge q) \rightarrow r$$

A formula is in CNF if:
 it is a conjunction of
 disjunctions of
 atoms or negated atoms

Some more formulas:

$\perp$ <-- a unary conjunction of a nullary
 disjunction

$\top$ <-- counts as a 0-ary "nullary" conjunction
 the conjunction of 0 formulas

a <-- this also counts!

$a \vee b$ <-- yes this counts!
 we can think of it as a conjunction
 of a single formula ($a \vee b$)

$a \wedge (b \vee (a \wedge b))$
 ^^^^^^ that's a conjunction
 which is neither an
 atom nor a negated atom

Is this formula satisfiable?

$$(p \wedge q \wedge r \wedge f) \vee (q \wedge f \wedge g \wedge \neg f) \\ \vee (\neg r \wedge g \wedge b \wedge c \wedge \neg g \wedge s)$$

When the formula is in DNF
 to satisfy the formula, we only need
 to satisfy one disjunct
 A conjunction of literals is satisfiable
 iff it has no contradictory literals

CNF formula.

With CNF, we can easily check validity.

$$(p \vee q \vee r \vee f) \wedge (q \vee f \vee g \vee \neg f) \\ \wedge (\neg r \vee g \vee b \vee c \vee \neg g \vee s)$$

Does **every** conjunct have contradictory literals?
 If yes, it's valid. If no, it's not.

$$\neg\neg\phi \equiv \phi$$

$$\neg(\phi \wedge \psi) \equiv \neg\phi \vee \neg\psi$$

natural deduction is a syntactic proof technique
 because we never:

- construct truth tables
- consider valuation functions ever
- we never use these weird brackets []

Inference rules are crafting recipes

If I have nine iron ingots, I can make
 iron block

iron	iron	iron	iron	iron	iron	iron	iron	iron
iron block								

^ iron block-introduction

iron block
9 x iron ingots

^ iron block-elimination

Why disassemble the block

A: You might only want an ingot.

e.g. pickaxe for three ingots, two sticks

Every operator comes with:

- introduction rules for proving things
 with that operator outermost
- elimination rules for proving
consequences of statements with that
 operator outermost

We write

$T \vdash \phi$ means " ϕ is provable using ND rules from T "

These rules are sound and complete.

$T \vdash \phi$ iff $T \models \phi$

forall x: Calgary

forall x
CALGARY

**An Introduction to
Formal Logic**



Description

forall x: Calgary is a full-featured textbook on formal logic. It covers key

notions of logic such as consequence and validity of arguments, the syntax of truth-functional propositional logic TFL and truth-table semantics, the syntax of first-order (predicate) logic FOL with identity (first-order interpretations), symbolizing English in TFL and FOL, and Fitch-style natural deduction proof systems for both TFL and FOL. It also deals with some advanced topics such as modal logic, soundness, and functional completeness. Exercises with solutions are available. It is provided in PDF (for screen reading, printing, and a special version for dyslexics), HTML (with additional [accessibility features](#)), and in LaTeX source code.

Instructors wishing to adopt it should consider the open-source [Carnap](#) system, which supports the notation and proof systems of *forall x: Calgary*. Check out the [sample exercises](#) for the book on [carnap.io](#). (There is also an outdated and unsupported proof editor/checker for the proof system used available at [proofs.openlogicproject.org](#).)

The book has been translated into German ([forall x: Dortmund](#)) and Portuguese ([Para Todxs: Natal](#)).

Read online

The book is available in HTML [to read online](#).

The HTML conversion is experimental. If you find issues in it, especially issues that affect screen readers,

please report it by [filing an issue](#) or send an email to [Richard Zach](#). Note that solutions are not yet included in the HTML version.

Download

You can download PDFs of the **most current version under development** here:

- [forallxyyc.pdf](#) (in color, for screen reading)
- [forallxyyc-accessible.pdf](#) (an accessible version for dyslexics)
- [forallxyyc-print.pdf](#) (b/w, for printing on Quarto stock)
- [forallxyyc-letter.pdf](#) (b/w, for printing on regular letter-size paper)
- [forallxyyc-solutions.pdf](#) (solutions booklet)

The a **SCORM** ZIP package of HTML bundled with plain and accessible versions of the PDF is available here:

- [SCORM.forallxyyc.zip](#)

You can use this file to easily provide the entire text inside your LMS (Canvas, Moodle, D2L/Brightspace).

Note that these files change whenever the source files change. So if you use the text in a course, it is best to download the PDFs or the SCORM package and make them available to students directly rather

than to link here, to avoid mismatches between versions.

Major changes between editions are listed in the [changelog](#).

PDFs of the **Fall 2021** edition are archived here:

- [forallxyyc-f21.pdf](#)
- [forallxyyc-accessible-f21.pdf](#)
- [forallxyyc-print-f21.pdf](#)
- [forallxyyc-letter-f21.pdf](#)
- [forallxyyc-solutions-f21.pdf](#)

PDFs of the **Fall 2020** edition are archived here:

- [forallxyyc-f20.pdf](#)
- [forallxyyc-accessible-f20.pdf](#)
- [forallxyyc-print-f20.pdf](#)
- [forallxyyc-letter-f20.pdf](#)
- [forallxyyc-solutions-f20.pdf](#)

Buy a Printed Copy

If you'd like to purchase a nice paperback copy of the Fall 2023 edition, you can do so on Amazon ([US](#) | [CA](#) | [UK](#) | [DE](#) | [AU](#)), or use search in your local Amazon store. Be sure to get the latest version that's available in print (Fall 2023). The version on Amazon usually is not as current as

the PDF. Changes are recorded in the [Changelog](#).

(The process for getting the book printed is described [here](#) and [here](#).)

Make PDFs Yourself

Clone the [GitHub repository](#) locally or download the ZIP file and run [LaTeX](#) on one of

- `forallxyyc.tex` (in color, for screen reading)
- `forallxyyc-accessible.tex` (accessible version)
- `forallxyyc-print.tex` (b/w, for printing on Quarto stock)
- `forallxyyc-letter.tex` (b/w, for printing on regular letter-size paper)

You'll have to run `makeglossaries` to produce the glossary as well, or use `latexmk`.

To make changes to the definitions in the preamble and `foralllyyc.sty` file, put them in a file named `forallxyyc-local.sty`. For instance, to get the connectives to be $\sim$, $\&$, $\supset$, $\equiv$ instead of $\neg$, $\wedge$, $\rightarrow$, $\leftrightarrow$, and atomic formulas Lab instead of $L(a,b)$, copy `forallxyyc-local-sample.sty` to that file.

Credits and License

forall x: Calgary is based on *forall x: Cambridge*, by [Tim Button](#) used under a [CC BY 4.0](#) license, which is based in turn on *forall x*, by [P. D. Magnus](#) used under a [CC BY 4.0](#) license, and was remixed, revised, & expanded by [Aaron Thomas-Bolduc](#) & [Richard Zach](#). It includes additional material from *forall x* by P.~D. Magnus and *Metatheory* by Tim Button, both used under a [CC BY 4.0](#) license, from *forall x: Lorain County Remix*, by [Cathal Woods](#) and J. Robert Loftis, used with permission, and *A Modal Logic Primer* by [Robert Trueman](#), used with permission.



This work is licensed under a [Creative Commons Attribution 4.0 International License](#).

The LaTeX source code for this work is available on GitHub at github.com/rzach/forallx-yyz.

Natural deduction proof editor and checker

This is a demo of a proof checker for Fitch-style natural deduction systems found in many popular introductory logic textbooks. The specific system used here is the one found in *forall x: Calgary*. (Although based on *forall x: an Introduction to Formal Logic*, the proof system in that original version differs from the one used here and in *forall x: Calgary*. However, the system also supports the rules used in the *forall x: Cambridge* remix.)

Create a new problem

Select if TFL or FOL syntax:

☒ TFL ☐ FOL

Premises (separate with “,” or “;”):

Conclusion:

CREATE PROBLEM

Rules

Basic rules

m	A	
	A	$R\ m$
m	A	
n	B	
	$A \wedge B$	$\wedge I\ m, n$
m	$A \wedge B$	
	A	$\wedge E\ m$
m	$A \wedge B$	
	B	$\wedge E\ m$
m	A	
	$A \vee B$	$\vee I\ m$
m	A	
	$B \vee A$	$\vee I\ m$

Sample exercise sets

- [Sample Truth-Functional Logic exercises](#) (Chap. 15, ex. C; Chap. 17, ex. B)
- [Sample First-Order Logic exercises](#) (Chap. 32, ex. E; Chap. 34, ex. A)

Instructions

TFL atomic sentences: A, B, X,
(single uppercase letters) etc.

FOL atomic sentences: Pa, Fcdc, a
(single uppercase letters *other than* = d, etc.
A or E followed by

lowercase letters a–w without parentheses, or identities)	
For negation you may use any of the symbols:	$\neg \sim \neg - -$
For conjunction you may use any of the symbols:	$\wedge \wedge \& . \cdot *$
For disjunction you may use any of the symbols:	$\vee \vee$
For the biconditional you may use any of the symbols:	$\leftrightarrow \equiv <-> <>$ (or in TFL only: =)
For the conditional you may use any of the symbols:	$\rightarrow \Rightarrow \supset \rightarrow >$
For the universal quantifier (FOL only), you may use any of the symbols:	$\forall x (\forall x) Ax$ $(Ax) (x) \wedge x$
For the existential quantifier (FOL only), you may use any of the symbols:	$\exists x (\exists x) Ex$ $(Ex) \vee x$
For a contradiction you may use any of the symbols:	$\perp \text{XX} \#$

The following buttons do the following things:

	= delete this line
	= add a line below this one
	= add a new subproof below this line
	= add a new line below this subproof to the parent subproof
	= add a new subproof below this subproof to the parent subproof

m	$A \vee B$	
i	A	
j	B	
k	B	
l	B	
	B	$\vee E\ m,\ i-j,\ k-l$
i	A	
j	B	
	$A \rightarrow B$	$\rightarrow I\ i-j$
m	$A \rightarrow B$	
n	A	
	B	$\rightarrow E\ m,\ n$
i	A	
j	B	
k	B	
l	A	
	$A \leftrightarrow B$	$\leftrightarrow I\ i-j,\ k-l$
m	$A \leftrightarrow B$	
n	A	
	B	$\leftrightarrow E\ m,\ n$
m	$A \leftrightarrow B$	
n	B	
	A	$\leftrightarrow E\ m,\ n$
i	A	
j	$\perp$	
	$\neg A$	$\neg I\ i-j$
m	$\neg A$	
n	A	
	$\perp$	$\neg E\ m,\ n$

Apart from premises and assumptions, each line has a cell immediately to its right for entering the justification. Click on it to enter the justification as, e.g. "&I 1,2".

Hopefully it is otherwise more or less obvious how to use it.

The PHP, JavaScript, HTML and CSS source for this page is licensed under the [GNU General Purpose License \(GPL\) v3](#). Download it [here](#).

m	$\perp$	
	$\mathcal{A}$	$\text{X } m$
i	$\neg \mathcal{A}$	
j	$\perp$	
	$\mathcal{A}$	$\text{IP } i-j$
m	$\mathcal{A}(\dots c \dots c \dots)$	
	$\forall x \mathcal{A}(\dots x \dots x \dots)$	$\forall \text{I } m$
m	$\forall x \mathcal{A}(\dots x \dots x \dots)$	
	$\mathcal{A}(\dots c \dots c \dots)$	$\forall \text{E } m$
m	$\mathcal{A}(\dots c \dots c \dots)$	
	$\exists x \mathcal{A}(\dots x \dots c \dots)$	$\exists \text{I } m$
m	$\exists x \mathcal{A}(\dots x \dots x \dots)$	
i	$\mathcal{A}(\dots c \dots c \dots)$	
j	$\mathcal{B}$	
	$\mathcal{B}$	$\exists \text{E } m,$
	$c = c$	$=\text{I}$
m	$a = b$	
n	$\mathcal{A}(\dots a \dots a \dots)$	
	$\mathcal{A}(\dots b \dots a \dots)$	$=\text{E } m, n$
m	$a = b$	
n	$\mathcal{A}(\dots b \dots b \dots)$	
	$\mathcal{A}(\dots a \dots b \dots)$	$=\text{E } m, n$

Derived rules

m	$\mathcal{A} \vee \mathcal{B}$	
n	$\neg \mathcal{A}$	
	$\mathcal{B}$	DS m, n
m	$\mathcal{A} \vee \mathcal{B}$	
n	$\neg \mathcal{B}$	
	$\mathcal{A}$	DS m, n
m	$\mathcal{A} \rightarrow \mathcal{B}$	
n	$\neg \mathcal{B}$	
	$\neg \mathcal{A}$	MT m, n
m	$\neg \neg \mathcal{A}$	
	$\mathcal{A}$	DNE m
i	$\mathcal{A}$	
j	$\mathcal{B}$	
k	$\neg \mathcal{A}$	
l	$\mathcal{B}$	
	$\mathcal{B}$	LEM $i-j, k-l$

$$\begin{array}{l|l}
 m & \neg(A \vee B) \\
 & \neg A \wedge \neg B \quad \text{DeM } m
 \end{array}$$

$$\begin{array}{l|l}
 m & \neg A \wedge \neg B \\
 & \neg(A \vee B) \quad \text{DeM } m
 \end{array}$$

$$\begin{array}{l|l}
 m & \neg(A \wedge B) \\
 & \neg A \vee \neg B \quad \text{DeM } m
 \end{array}$$

$$\begin{array}{l|l}
 m & \neg A \vee \neg B \\
 & \neg(A \wedge B) \quad \text{DeM } m
 \end{array}$$

$$\begin{array}{l|l}
 m & \forall x \neg A \\
 & \neg \exists x A \quad \text{CQ } m
 \end{array}$$

$$\begin{array}{l|l}
 m & \neg \exists x A \\
 & \forall x \neg A \quad \text{CQ } m
 \end{array}$$

$$\begin{array}{l|l}
 m & \exists x \neg A \\
 & \neg \forall x A \quad \text{CQ } m
 \end{array}$$

$$\begin{array}{l|l}
 m & \neg \forall x A \\
 & \exists x \neg A \quad \text{CQ } m
 \end{array}$$

Rules for Cambridge

$$\begin{array}{l|l}
 m & A \\
 n & \neg A \\
 & \perp \quad \perp\text{I } m, n
 \end{array}$$

$$\begin{array}{l|l}
 m & \perp \\
 & A \quad \perp\text{E } m
 \end{array}$$

$$\begin{array}{l|l}
 i & \begin{array}{|l} A \end{array} \\
 j & \begin{array}{|l} B \end{array} \\
 k & \begin{array}{|l} \neg A \end{array} \\
 l & \begin{array}{|l} B \end{array} \\
 & B \quad \text{TND } i-j, k-l
 \end{array}$$

Tute (Week 4)

Defining the arithmetic of negative numbers

This question is related to the “*How many elements does the Cartesian product of no sets have?*” from an earlier quiz.

When people were figuring out how to reason with negative numbers, a question arose about multiplying two negative numbers: if $-1 \times 2 = -2$, i.e. multiplication by -1 *decreases* $+2$ down to -2 , shouldn't multiplication of an (already) negative number by -1 “decrease it even more”?

(It was not the proverbial “ancient Greeks” who were worrying about this. It was European mathematicians in the 1700's.)

1. Give an “explain like I'm five” style answer for why $-2 \times -2 = 4$, similar to how a primary school teacher might explain $2 \times 2 = 4$ by saying “if I take two apples twice, then I have four apples”.
2. Give a rigorous argument for why, when z is negative, $z \times -1$ shouldn't be negative. *Hint*: what algebraic laws would fail?

1. First, we might explain $2 \times -2 = -4$ as “If I undo the act of taking two apples, twice, then I've lost four apples”.

Then $-2 \times -2 = 4$ could be stated as “If I undo the act of removing two apples, twice, then I have four apples”.

(Granted, your average kindy student may still find all this a bit abstract.)

2. Suppose $z \times -1 = -w$ for some positive w . We will show that the cancellation law of multiplication fails:

$$a * b = a * c \quad \equiv \quad b = c$$

We have that $w \times -1 = -w$, so

$$z \times -1 = -w \times -1$$

By the law of cancellation, we have $z = -w$, which leads to a contradiction since the LHS is positive and the RHS is negative.

Other similar derivations are possible for other laws. For example, we can show that the distributivity law

$$a \times (b + c) = a \times b + a \times c$$

fails. Consider the following derivation for a positive z :

$$\begin{aligned} 0 &= -1 \times 0 \\ &= -1 \times (z + -z) \\ &= (-z) + (-1 \times -z) \end{aligned}$$

If we require $-1 \times -z$ to be negative, we are forced to conclude that 0 is negative.

Reasoning carefully in everyday life

A speaker at a public seminar wants to make the point that most people know very little about elementary maths in the world around them.

“Look,” he says, “Here’s an example. Most people without a scientific background would not be able to tell me, to the nearest whole number, the length of the circumference of a circle that is 10 metres through the middle.”

(“Yeah, but what’s a *circumference*?!” shouts a voice from the back of the hall.)

“You sir,” he says to a man sitting in the front row. “What do you think?”

“I have absolutely no idea!”

“And you sir,” he says, pointing into the following row. “Not a bloody clue mate.”

“And you, madam?” he says, pointing right up the back.

“It’s just over 31 metres,” she replies. (Does her voice sound familiar?)

“Ah.” He pauses. “And what is your profession?”

“I’m a physicist.”

“See!?” he says. “A scientist! This lady proves my point.”

Does the woman’s answer really help to prove his point? If yes, why? If no, why not?

The speaker is making a claim about *most people without a scientific background*. The physicist has a scientific background, so the speaker makes no claim about her

abilities. Thus, whether she can answer the question or not has no bearing on the speaker's point.

Tautologies

Question 1

Name at least two methods you could use to show that $\phi \rightarrow \psi$ is a tautology.

Here's a few:

1. Make a truth table for $(\phi \rightarrow \psi)$, and check that all rows yield $\top$.
2. Show that $\phi \models \psi$ by making truth tables for ϕ and ψ , checking that they agree in all rows.
3. Give a proof for $\vdash (\phi \rightarrow \psi)$ – i.e. derive $(\phi \rightarrow \psi)$ with no premises
4. Give a proof for $\phi \vdash \psi$
5. Rewrite $\phi \rightarrow \psi$ into CNF, then check that every clause of the CNF formula has contradictory literals.

Question 2

Show that the following are tautologies:

1. $P \rightarrow (Q \rightarrow P)$
2. $(P \rightarrow (Q \rightarrow R)) \rightarrow ((P \rightarrow Q) \rightarrow (P \rightarrow R))$
3. $(\neg P \rightarrow \neg Q) \rightarrow (Q \rightarrow P)$

1. With a truth table following approach (1):

P	Q	$Q \rightarrow P$	$P \rightarrow (Q \rightarrow P)$
$\top$	$\top$	$\top$	$\top$
$\top$	$\perp$	$\top$	$\top$
$\perp$	$\top$	$\perp$	$\top$
$\perp$	$\perp$	$\top$	$\top$

The result follows because the red column is all $\top$.

2. Let's rewrite to CNF:

$$\begin{aligned}
 & (P \rightarrow (Q \rightarrow R)) \rightarrow ((P \rightarrow Q) \rightarrow (P \rightarrow R)) \\
 \equiv & (P \vee P \vee \neg P \vee R) \wedge (Q \vee P \vee \neg P \vee R) \wedge (\neg R \vee P \vee \neg P \vee R) \wedge \\
 & (P \vee \neg Q \vee \neg P \vee R) \wedge (Q \vee \neg Q \vee \neg P \vee R) \wedge (\neg R \vee \neg Q \vee \neg P \vee R)
 \end{aligned}$$

Doing the full CNF conversion manually would be incredibly tedious (I used an automated tool). Nonetheless, once we have it it's easy to check that indeed, every clause has contradictory literals.

3. We use natural deduction:

$$\begin{array}{c}
 [Q] \quad \frac{[\neg P] \quad \neg P \rightarrow \neg Q}{\neg Q} \rightarrow\text{-E} \\
 \hline
 \neg\text{-E} \\
 \hline
 \perp \\
 \hline
 \text{IP} \\
 \hline
 P \\
 \hline
 Q \rightarrow P \quad \rightarrow\text{-I}
 \end{array}$$

Natural deduction

Using natural deduction, give a formal proof of the following:

1. $Q \rightarrow (Q \wedge \neg Q) \vdash \neg Q$
2. $(A \wedge B) \vee C \vdash B \vee C$
3. $\neg A \rightarrow B, A \rightarrow C \vdash B \vee C$
4. $S \leftrightarrow T \vdash S \leftrightarrow (T \vee S)$
5. $C \rightarrow (E \wedge G), \neg C \rightarrow G \vdash G$
6. $(W \vee X) \vee (Y \vee Z), X \rightarrow Y, \neg Z \vdash W \vee Y$

These are selected exercises from the forall x: Calgary textbook. Fitch-style proofs can be found in the [textbook solutions](#) for chapters 16 and 19 in the current version.

For diversity's sake we do a couple in other styles:

$$\begin{array}{c}
 1. \quad \frac{[Q] \quad Q \rightarrow (Q \wedge \neg Q)}{Q \wedge \neg Q} \rightarrow\text{-E} \\
 \hline
 [Q] \quad \frac{Q \wedge \neg Q}{\neg Q} \wedge\text{-I2} \\
 \hline
 \neg\text{-E} \\
 \hline
 \perp \\
 \hline
 \neg Q \quad \text{IP}
 \end{array}$$

	Line	Premises	Formula	Rule	Ref
2.	1		$(A \wedge B) \vee C$	Premise	
	2		$A \wedge B$	Premise	
	3	2	B	$\wedge$ -E	2
	4	2	$B \vee C$	$\vee$ -I	3
	5		C	Premise	
	6	5	$B \vee C$	$\vee$ -I	5
	7	1	$B \vee C$	$\vee$ -E	1,4,6

COMP2111 Week 5
Term 1, 2023
First-Order Predicate Logic

Summary of topics

- Re-introduction to Predicate Logic
- Syntax of Predicate Logic
- Semantics of Predicate Logic
- Natural Deduction for Predicate Logic

Summary of topics

- Re-introduction to Predicate Logic
- Syntax of Predicate Logic
- Semantics of Predicate Logic
- Natural Deduction for Predicate Logic

Motivating example

Consider the statement:

$$\text{For all } x, y \in X : (y = x+1) \rightarrow (x \leq y)$$

Can we encode this statement in propositional logic?

Motivating example

Consider the statement:

$$\text{For all } x, y \in X : (y = x+1) \rightarrow (x \leq y)$$

$X = \{1, 2, 3\}$: 18 propositional variables:

$$\begin{array}{ll} P_{11} &= "1 = 1 + 1" & S_{11} &= "1 \leq 1" \\ P_{12} &= "2 = 1 + 1" & S_{12} &= "1 \leq 2" \\ \vdots & & \vdots & \end{array}$$

Final result: $(P_{11} \rightarrow S_{11}) \wedge (P_{12} \rightarrow S_{12}) \wedge \cdots \wedge (P_{33} \rightarrow S_{33})$

NB

"Normal arithmetic", where P_{11} is false, P_{12} is true, etc is just one of many possibilities.

Motivating example

Consider the statement:

$$\text{For all } x, y \in X : (y = x+1) \rightarrow (x \leq y)$$

$X = \mathbb{N} : \infty$ propositional variables:

$$\begin{array}{ll} P_{00} &= "0 = 0 + 0" & S_{00} &= "0 \leq 0" \\ P_{01} &= "1 = 0 + 1" & S_{01} &= "0 \leq 1" \\ \vdots & & \vdots & \end{array}$$

Final result: $(P_{00} \rightarrow S_{00}) \wedge (P_{01} \rightarrow S_{01}) \wedge \dots$

Motivating example

Consider the statement:

$$\text{For all } x, y \in X : (y = x+1) \rightarrow (x \leq y)$$

$X = \mathbb{N} : \infty$ propositional variables:

$$\begin{array}{ll} P_{00} &= "0 = 0 + 0" & S_{00} &= "0 \leq 0" \\ P_{01} &= "1 = 0 + 1" & S_{01} &= "0 \leq 1" \\ \vdots & & \vdots & \end{array}$$

Final result: $(P_{00} \rightarrow S_{00}) \wedge (P_{01} \rightarrow S_{01}) \wedge \dots$ **Not permitted!**

Motivating example

Consider the statement:

$$\text{For all } x, y \in X : (y = x+1) \rightarrow (x \leq y)$$

Predicate logic introduces:

- Predicates

Motivating example

Consider the statement:

$$\text{For all } x, y \in X : (y = x + 1) \rightarrow (x \leq y)$$

Predicate logic introduces:

- Predicates
- Functions

Motivating example

Consider the statement:

$$\text{For all } x, y \in X : (y = x + 1) \rightarrow (x \leq y)$$

Predicate logic introduces:

- Predicates
- Functions
- Constants

Motivating example

Consider the statement:

$$\text{For all } x, y \in X : (y = x + 1) \rightarrow (x \leq y)$$

Predicate logic introduces:

- Predicates
- Functions
- Constants
- Variables, and

Motivating example

Consider the statement:

$$\text{For all } x, y \in X : (y = x + 1) \rightarrow (x \leq y)$$

Predicate logic introduces:

- Predicates
- Functions
- Constants
- Variables, and
- Quantifiers

Domain of discourse

Q: Is this a true statement?

$$\forall x y. x = y$$

Domain of discourse

Q: Is this a true statement?

$$\forall x y. x = y$$

A: depends on what the domain of discourse is.

Domain of discourse

Fundamental to interpreting formulas is the **domain of discourse**: the set of “ground objects” that we are referring to.

Domain of discourse

Fundamental to interpreting formulas is the **domain of discourse**: the set of “ground objects” that we are referring to.

- **Predicates**: Relations on the domain

Domain of discourse

Fundamental to interpreting formulas is the **domain of discourse**: the set of “ground objects” that we are referring to.

- **Predicates**: Relations on the domain
- **Functions**: Operators on the domain

Domain of discourse

Fundamental to interpreting formulas is the **domain of discourse**: the set of “ground objects” that we are referring to.

- **Predicates**: Relations on the domain
- **Functions**: Operators on the domain
- **Constants**: “Named” elements of the domain
- **Variables**: “Unnamed” elements of the domain (placeholders for elements)

Domain of discourse

Fundamental to interpreting formulas is the **domain of discourse**: the set of “ground objects” that we are referring to.

- **Predicates**: Relations on the domain
- **Functions**: Operators on the domain
- **Constants**: “Named” elements of the domain
- **Variables**: “Unnamed” elements of the domain (placeholders for elements)
- **Quantifiers**: Range over domain elements

Domain of discourse

Fundamental to interpreting formulas is the **domain of discourse**: the set of “ground objects” that we are referring to.

- **Predicates**: Relations on the domain
- **Functions**: Operators on the domain
- **Constants**: “Named” elements of the domain
- **Variables**: “Unnamed” elements of the domain (placeholders for elements)
- **Quantifiers**: Range over domain elements

Example

Consider: $\forall x C(x)$ where $C(x)$ represents “ x studies COMP2111”
It is true if the domain of discourse is the set of students in this room.

Domain of discourse

Fundamental to interpreting formulas is the **domain of discourse**: the set of “ground objects” that we are referring to.

- **Predicates**: Relations on the domain
- **Functions**: Operators on the domain
- **Constants**: “Named” elements of the domain
- **Variables**: “Unnamed” elements of the domain (placeholders for elements)
- **Quantifiers**: Range over domain elements

Example

Consider: $\forall x C(x)$ where $C(x)$ represents “ x studies COMP2111”
It is false if the domain of discourse is the set of students at UNSW.

Multiple domains of discourse

Multiple domains can be combined into one as follows.

For example: the predicate **studies**(x, y) representing “ x (a student) studies y (a subject)”.

Multiple domains of discourse

Multiple domains can be combined into one as follows.

For example: the predicate **studies**(x, y) representing “ x (a student) studies y (a subject)”.

- Take $\text{STUDENTS} \cup \text{SUBJECTS}$ as the domain.
- Use unary predicates, e.g. **isStudent**(x), to restrict the domain.

Multiple domains of discourse

Multiple domains can be combined into one as follows.

For example: the predicate **studies**(x, y) representing “ x (a student) studies y (a subject)”.

- Take $\text{STUDENTS} \cup \text{SUBJECTS}$ as the domain.
- Use unary predicates, e.g. **isStudent**(x), to restrict the domain.
- To restrict quantifiers (applies to any subset of the domain defined by a unary predicate):
 - $\exists x \in \text{STUDENTS} : \varphi$ is equivalent to: $\exists x(\text{isStudent}(x) \wedge \varphi)$
 - $\forall x \in \text{STUDENTS} : \varphi$ is equivalent to: $\forall x(\text{isStudent}(x) \rightarrow \varphi)$

Domain of discourse

Function outputs, **constants**, and **variables** are interpreted as elements of the domain.

Predicates are truth-functional: they map elements of the domain to true or false.

Quantifiers (and the Boolean connectives) are predicate operators: they transform predicates into other predicates.

Example

Consider the following predicates and constants:

$K(x, y)$: x knows y

$S(x, y)$: x is not the son of y

J : Jon Snow

N : Ned Stark

B : Bran Stark

Domain of discourse: PEOPLE

The following are OK:

- $S(B, J)$: Bran is not the son of Jon
- $K(N, J)$: Ned knows Jon
- $\forall x \neg K(J, x)$: Jon Snow knows nothing.

Example

Consider the following predicates and constants:

$K(x, y)$: x knows y

$S(x, y)$: x is not the son of y

J : Jon Snow

N : Ned Stark

B : Bran Stark

Domain of discourse: PEOPLE

The following are OK:

- $S(B, J)$: Bran is not the son of Jon
- $K(N, J)$: Ned knows Jon
- $\forall x \neg K(J, x)$: Jon Snow knows no-one.

Example

Consider the following predicates and constants:

$K(x, y)$: x knows y

$S(x, y)$: x is not the son of y

J : Jon Snow

N : Ned Stark

B : Bran Stark

Domain of discourse: PEOPLE

The following are OK:

- $S(B, J)$: Bran is not the son of Jon
- $K(N, J)$: Ned knows Jon
- $\forall x \neg K(J, x)$: Jon Snow knows no-one.

This is not:

- $K(B, S(J, N))$: Bran knows that Jon is not the son of Ned

Example

Consider the following predicates and constants:

$K(x, y)$: x knows y

$S(x, y)$: x is not the son of y

$F(x, y)$: the fact that x is not the son of y (functional)

J : Jon Snow

N : Ned Stark

B : Bran Stark

Domain of discourse: $\text{PEOPLE} \cup \text{FACTS}$

The following are OK:

- $S(B, J)$: Bran is not the son of Jon
- $K(N, J)$: Ned knows Jon
- $\forall x \neg K(J, x)$: Jon Snow knows no-one.

This is OK:

- $K(B, F(J, N))$: Bran knows that Jon is not the son of Ned

Summary of topics

- Re-introduction to Predicate Logic
- **Syntax of Predicate Logic**
- Semantics of Predicate Logic
- Natural Deduction for Predicate Logic

Vocabulary

A **vocabulary** indicates what **predicates**, **functions** and **constants** we can use to build up our formulas. Very similar to C header files, or Java interfaces, or database schemas.

A vocabulary V is a set of:

- Predicate symbols $P, Q, \dots$, each with an associated *arity* (number of arguments)
- Function symbols $f, g, \dots$, each with an associated *arity*
- Constant symbols $c, d, \dots$ (also known as 0-arity functions)

Vocabulary

A **vocabulary** indicates what **predicates**, **functions** and **constants** we can use to build up our formulas. Very similar to C header files, or Java interfaces, or database schemas.

A vocabulary V is a set of:

- Predicate symbols $P, Q, \dots$, each with an associated *arity* (number of arguments)
- Function symbols $f, g, \dots$, each with an associated *arity*
- Constant symbols $c, d, \dots$ (also known as 0-arity functions)

Example

$V = \{\leq, +, 1\}$ where $\leq$ is a binary predicate symbol, $+$ is a binary function symbol, and 1 is a constant symbol.

Vocabulary: example (databases)

Example

A database schema identifies the various tables, their attributes, and their attributes' types. For example:

Person	
Name:	String
Surname:	String
Address:	String

Employee	
ID:	int
Surname:	String

Vocabulary: example (databases)

Example

A database schema identifies the various tables, their attributes, and their attributes' types. For example:

Person	
Name:	String
Surname:	String
Address:	String

Employee	
ID:	int
Surname:	String

Tables *relate* a number of attributes

The above schema would be represented by the vocabulary:

$$DB = \{\text{Person}, \text{Employee}\}$$

where **Person** is a ternary predicate symbol and **Employee** is a binary predicate symbol

Vocabulary: example (databases)

Example

A database schema identifies the various tables, their attributes, and their attributes' types. For example:

Person	
Name:	String
Surname:	String
Address:	String

Employee	
ID:	int
Surname:	String

Tables *relate* a number of attributes (over several domains). The above schema would be represented by the vocabulary:

$$DB = \{\text{Person}, \text{Employee}, \text{isString}, \text{isInteger}\}$$

where **Person** is a ternary predicate symbol and **Employee** is a binary predicate symbol and **isString** and **isInteger** are unary predicate symbols.

Terms

A **term** is defined inductively as follows:

- A variable is a term
- A constant symbol is a term
- If f is a function symbol with arity k , and $t_1, \dots, t_k$ are terms, then $f(t_1, t_2, \dots, t_k)$ is a term.

NB

Terms will be interpreted as elements of the domain of discourse.

Terms: examples

Example

Over $V = \{\leq, +, 1\}$, the following are all terms:

- x
- 1
- $+(y, 1)$
- $+(y, +(x, 1))$

Formulas

A **formula of Predicate Logic** is defined inductively as follows:

- If P is a predicate symbol with arity k , and $t_1, \dots, t_k$ are terms, then $P(t_1, t_2, \dots, t_k)$ is a formula
- If t_1 and t_2 are terms then $(t_1 = t_2)$ is a formula
- If φ, ψ are a formulas then the following are formulas:
 - $\neg\varphi$
 - $(\varphi \wedge \psi)$
 - $(\varphi \vee \psi)$
 - $(\varphi \rightarrow \psi)$
 - $(\varphi \leftrightarrow \psi)$
 - $\forall x\varphi$
 - $\exists x\varphi$

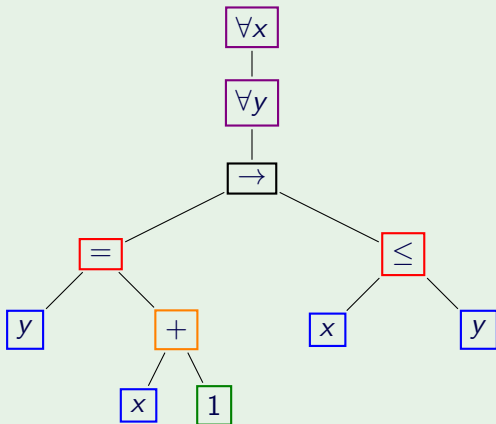
NB

*The base cases are known as **atomic** formulas: they play a similar role in the parse tree as propositional variables.*

Parse trees

Example

$$\forall x \forall y ((y = x + 1) \rightarrow (x \leq y))$$



Formulas: examples

Example

Over $V = \{\leq, +, 1\}$, the following are all formulas:

- $\leq(x, y)$
- $\leq(1, 1)$
- $x = +(y, 1)$
- $\leq(x, y) \rightarrow (x = +(y, 1))$
- $\exists x(1 = +(1, 1))$
- $\forall x \forall y \leq(x, y) \rightarrow (x = +(y, 1))$

Feel free to write predicates and functions in infix for readability.

Formulas: example (databases)

In relational databases, formulas correspond to (select-)queries.

Example

For the vocabulary

$DB = \{\text{Person}, \text{Employee}, \text{isString}, \text{isInteger}\}$:

Select *
from Person



Person(x, y, z)

Formulas: example (databases)

In relational databases, formulas correspond to (select-)queries.

Example

For the vocabulary

$DB = \{\text{Person}, \text{Employee}, \text{isString}, \text{isInteger}, \text{Alice}\}$:

```
Select *  
  from Person  
  where Person.name = "Alice"
```



$\text{Person}(x, y, z) \wedge (x = \text{Alice})$

Formulas: example (databases)

In relational databases, formulas correspond to (select-)queries.

Example

For the vocabulary

$DB = \{\text{Person}, \text{Employee}, \text{isString}, \text{isInteger}, \text{Alice}\}$:

```
Select *  
  from Person inner join Employee  
  on Person.surname = Employee.surname
```



$\text{Person}(x, y, z) \vee \text{Employee}(w, y)$

Free and Bound variables

A variable is **bound** to the closest matching quantifier that lies above it in the parse tree. A variable that is not bound is **free**.

Free and Bound variables

A variable is **bound** to the closest matching quantifier that lies above it in the parse tree. A variable that is not bound is **free**.

Example

In $(\forall x \exists z \exists x P(x, y, z)) \wedge Q(x)$:

Free and Bound variables

A variable is **bound** to the closest matching quantifier that lies above it in the parse tree. A variable that is not bound is **free**.

Example

In $(\forall x \exists z \exists x P(x, y, z)) \wedge Q(x)$:

- z is bound by $\exists z$

Free and Bound variables

A variable is **bound** to the closest matching quantifier that lies above it in the parse tree. A variable that is not bound is **free**.

Example

In $(\forall x \exists z \exists x P(x, y, z)) \wedge Q(x)$:

- z is bound by $\exists z$
- y is free

Free and Bound variables

A variable is **bound** to the closest matching quantifier that lies above it in the parse tree. A variable that is not bound is **free**.

Example

In $(\forall x \exists z \exists x P(x, y, z)) \wedge Q(x)$:

- z is bound by $\exists z$
- y is free
- First x is bound by $\exists x$

Free and Bound variables

A variable is **bound** to the closest matching quantifier that lies above it in the parse tree. A variable that is not bound is **free**.

Example

In $(\forall x \exists z \exists x P(x, y, z)) \wedge Q(x)$:

- z is bound by $\exists z$
- y is free
- First x is bound by $\exists x$
- Second x is free

Free and Bound variables

A variable is **bound** to the closest matching quantifier that lies above it in the parse tree. A variable that is not bound is **free**.

Example

In $(\forall x \exists z \exists x P(x, y, z)) \wedge Q(x)$:

- z is bound by $\exists z$
- y is free
- First x is bound by $\exists x$
- Second x is free

A formula with no free variables is a **sentence**.

Free variables formally

We can define the set of free variables recursively on the structure of a formula:

- $FV(x) = \{x\}$ for all variables x
- $FV(c) = \emptyset$ for all constants c
- $FV(f(t_1, \dots, t_k)) = FV(t_1) \cup \dots \cup FV(t_k)$ for all k -ary functions f

Free variables formally

We can define the set of free variables recursively on the structure of a formula:

- $FV(x) = \{x\}$ for all variables x
- $FV(c) = \emptyset$ for all constants c
- $FV(f(t_1, \dots, t_k)) = FV(t_1) \cup \dots \cup FV(t_k)$ for all k -ary functions f
- $FV(P(t_1, \dots, t_k)) = FV(t_1) \cup \dots \cup FV(t_k)$ for all k -ary predicates P
- $FV(t_1 = t_2) = FV(t_1) \cup FV(t_2)$

Free variables formally

We can define the set of free variables recursively on the structure of a formula:

- $FV(x) = \{x\}$ for all variables x
- $FV(c) = \emptyset$ for all constants c
- $FV(f(t_1, \dots, t_k)) = FV(t_1) \cup \dots \cup FV(t_k)$ for all k -ary functions f
- $FV(P(t_1, \dots, t_k)) = FV(t_1) \cup \dots \cup FV(t_k)$ for all k -ary predicates P
- $FV(t_1 = t_2) = FV(t_1) \cup FV(t_2)$
- $FV(\neg\varphi) = FV(\varphi)$
- $FV(\psi \wedge \varphi) = FV(\psi \vee \varphi) = FV(\psi \rightarrow \varphi) = FV(\psi \leftrightarrow \varphi) = FV(\psi) \cup FV(\varphi)$

Free variables formally

We can define the set of free variables recursively on the structure of a formula:

- $FV(x) = \{x\}$ for all variables x
- $FV(c) = \emptyset$ for all constants c
- $FV(f(t_1, \dots, t_k)) = FV(t_1) \cup \dots \cup FV(t_k)$ for all k -ary functions f
- $FV(P(t_1, \dots, t_k)) = FV(t_1) \cup \dots \cup FV(t_k)$ for all k -ary predicates P
- $FV(t_1 = t_2) = FV(t_1) \cup FV(t_2)$
- $FV(\neg\varphi) = FV(\varphi)$
- $FV(\psi \wedge \varphi) = FV(\psi \vee \varphi) = FV(\psi \rightarrow \varphi) = FV(\psi \leftrightarrow \varphi) = FV(\psi) \cup FV(\varphi)$
- $FV(\forall x\varphi) = FV(\exists x\varphi) = FV(\varphi) \setminus \{x\}$

Substitution

If t is a term, φ a formula, and $x \in FV(\varphi)$, then the **substitution of t for x in φ** (denoted $\varphi[t/x]$) is the formula obtained by replacing every free occurrence of x with t .

Substitution

If t is a term, φ a formula, and $x \in FV(\varphi)$, then the **substitution of t for x in φ** (denoted $\varphi[t/x]$) is the formula obtained by replacing every free occurrence of x with t .

It can be useful to have “access” to the free variables of a formula. So if $x_1, \dots, x_k$ are the free variables of φ , we may denote this as $\varphi(x_1, \dots, x_k)$. Substitution can be easily presented: $\varphi(t)$ for $\varphi(x)[t/x]$.

Note

Variable names matter: $\varphi(x)$ and $\varphi(y)$ are different formulas!

Summary of topics

- Re-introduction to Predicate Logic
- Syntax of Predicate Logic
- Semantics of Predicate Logic
- Natural Deduction for Predicate Logic

Models

Q: Is this a true statement?

There is nothing going on between us.

Models

Q: Is this a true statement?

There is nothing going on between us.

A: It depends upon what the meaning of the word 'is' is.

Models

Q: Is this a true statement?

$$1 + 1 = 2$$

Models

Q: Is this a true statement?

$$1 + 1 = 2$$

A: It depends on what the model of 1, 2 and + is.

Models

$\{\forall, \exists, =, \wedge, \vee, \neg, \rightarrow, \leftrightarrow\}$ have a fixed meaning in first-order logic.

All other symbols are meaningless, unless we specify a *model*.

Models

Predicate formulas are interpreted in **Models**.

Given a vocabulary V a model $\mathcal{M}$ defines:

- A (non-empty) domain $D = \text{dom}(\mathcal{M})$
- For every predicate symbol $P \in V$ with arity k : a k -ary relation $P^{\mathcal{M}}$ on D
- For every function symbol $f \in V$ with arity k : a function $f^{\mathcal{M}} : D^k \rightarrow D$
- For every constant symbol $c \in V$: an element, $c^{\mathcal{M}}$ of D

Models

Predicate formulas are interpreted in **Models**.

Given a vocabulary V a model $\mathcal{M}$ defines:

- A (non-empty) domain $D = \text{dom}(\mathcal{M})$
- For every predicate symbol $P \in V$ with arity k : a k -ary relation $P^{\mathcal{M}}$ on D
- For every function symbol $f \in V$ with arity k : a function $f^{\mathcal{M}} : D^k \rightarrow D$
- For every constant symbol $c \in V$: an element, $c^{\mathcal{M}}$ of D

Example

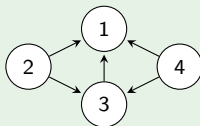
For the vocabulary $V = \{\leq, +, 1\}$: one model could be $\mathbb{N}$ with the standard definitions.

Models: examples

Example

For the vocabulary $V = \{\leq, +, 1\}$ the following are models:

- $\mathbb{N}$ with the standard definitions of $\leq$, $+$, and 1 .
- $\{0, 1, 2, 3, 4\}$ with the standard definition of $\leq$ and 1 , and $m + n$ defined as $m + n \pmod{5}$.
- The directed graph $G = (V, E)$ shown below with $\leq = E$; and $v + w$ defined to be w .



Models: example (databases)

Example

For the vocabulary $DB = \{\text{Person}, \text{Employee}, \text{isString}, \text{isInteger}\}$, the following **database** is a model:

Person		
Name	Surname	Address
Rapunzel	-	Tower
Cinderella	-	c/o Stepmum
Snow	White	Cottage

Employee	
ID	Surname
31415	Psmith
27182	Ukridge
16180	Wooster

`isString` and `isInteger` are defined by what values are permitted in each of the columns (*sanitizing* the input).

Environments

Given a model $\mathcal{M}$, an **environment** (or **lookup table**), η , is a function from the set of variables to $\text{dom}(\mathcal{M})$.

Environments

Given a model $\mathcal{M}$, an **environment** (or **lookup table**), η , is a function from the set of variables to $\text{dom}(\mathcal{M})$.

Given an environment η , we denote by $\eta[x \mapsto c]$ the environment that agrees with η everywhere except possibly at x (where it has value c).

Interpretations

An **interpretation** is a pair $(\mathcal{M}, \eta)$ where $\mathcal{M}$ is a model and η is an environment.

Interpretations

An **interpretation** is a pair $(\mathcal{M}, \eta)$ where $\mathcal{M}$ is a model and η is an environment.

An interpretation $(\mathcal{M}, \eta)$ maps terms to elements of $\text{dom}(\mathcal{M})$ recursively as follows:

- $\llbracket x \rrbracket_{\mathcal{M}}^{\eta} = \eta(x)$
- $\llbracket c \rrbracket_{\mathcal{M}}^{\eta} = c^{\mathcal{M}}$
- $\llbracket f(t_1, \dots, t_k) \rrbracket_{\mathcal{M}}^{\eta} = f^{\mathcal{M}}(\llbracket t_1 \rrbracket_{\mathcal{M}}^{\eta}, \dots, \llbracket t_k \rrbracket_{\mathcal{M}}^{\eta})$

Interpretations

An **interpretation** is a pair $(\mathcal{M}, \eta)$ where $\mathcal{M}$ is a model and η is an environment.

An interpretation $(\mathcal{M}, \eta)$ maps formulas to $\mathbb{B}$ recursively as follows:

- $\llbracket P(t_1, \dots, t_k) \rrbracket_{\mathcal{M}}^{\eta} = \text{true}$ if $P^{\mathcal{M}}(\llbracket t_1 \rrbracket_{\mathcal{M}}^{\eta}, \dots, \llbracket t_k \rrbracket_{\mathcal{M}}^{\eta})$ holds.
- $\llbracket t_1 = t_2 \rrbracket_{\mathcal{M}}^{\eta} = \text{true}$ if $\llbracket t_1 \rrbracket_{\mathcal{M}}^{\eta} = \llbracket t_2 \rrbracket_{\mathcal{M}}^{\eta}$
- $\llbracket \forall x \varphi \rrbracket_{\mathcal{M}}^{\eta} = \text{true}$ if $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta[x \mapsto c]} = \text{true}$ for all $c \in \text{dom}(\mathcal{M})$
- $\llbracket \exists x \varphi \rrbracket_{\mathcal{M}}^{\eta} = \text{true}$ if $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta[x \mapsto c]} = \text{true}$ for some $c \in \text{dom}(\mathcal{M})$
- $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta}$ defined in the same way as Propositional Logic for all other formulas φ .

Interpretations: examples

Example

$$\forall x \forall y ((y = x + 1) \rightarrow (x \leq y))$$

- $\mathbb{N}$ with the standard definitions of $\leq$, $+$, and 1 :

Interpretations: examples

Example

$$\forall x \forall y ((y = x + 1) \rightarrow (x \leq y))$$

- $\mathbb{N}$ with the standard definitions of $\leq$, $+$, and 1 : **true**

Interpretations: examples

Example

$$\forall x \forall y ((y = x + 1) \rightarrow (x \leq y))$$

- $\mathbb{N}$ with the standard definitions of $\leq$, $+$, and 1 : true
- $\{0, 1, 2, 3, 4\}$ with the standard definition of $\leq$ and 1 , and $m + n$ defined as $m + n \pmod{5}$:

Interpretations: examples

Example

$$\forall x \forall y ((y = x + 1) \rightarrow (x \leq y))$$

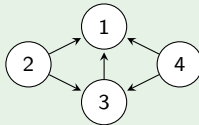
- $\mathbb{N}$ with the standard definitions of $\leq$, $+$, and 1 : true
- $\{0, 1, 2, 3, 4\}$ with the standard definition of $\leq$ and 1 , and $m + n$ defined as $m + n \pmod{5}$: false

Interpretations: examples

Example

$$\forall x \forall y ((y = x + 1) \rightarrow (x \leq y))$$

- $\mathbb{N}$ with the standard definitions of $\leq$, $+$, and 1 : true
- $\{0, 1, 2, 3, 4\}$ with the standard definition of $\leq$ and 1 , and $m + n$ defined as $m + n \pmod{5}$: false
- The directed graph $G = (V, E)$ shown below with $\leq = E$, 1 be the vertex 1 , and $v + w$ defined to be w .

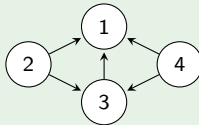


Interpretations: examples

Example

$$\forall x \forall y ((y = x + 1) \rightarrow (x \leq y))$$

- $\mathbb{N}$ with the standard definitions of $\leq$, $+$, and 1 : true
- $\{0, 1, 2, 3, 4\}$ with the standard definition of $\leq$ and 1 , and $m + n$ defined as $m + n \pmod{5}$: false
- The directed graph $G = (V, E)$ shown below with $\leq = E$, 1 be the vertex 1 , and $v + w$ defined to be w .



true

Why separate the environment from the model?

In the definition of $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta}$, η is only used to define values for the free variables. In particular, if φ is a sentence then $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta}$ is independent of η .

Why separate the environment from the model?

In the definition of $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta}$, η is only used to define values for the free variables. In particular, if φ is a sentence then $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta}$ is independent of η .

Define $\llbracket \cdot \rrbracket_{\mathcal{M}}$ by “delaying” the assigning of values to free variables, and propagating them out.

Why separate the environment from the model?

In the definition of $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta}$, η is only used to define values for the free variables. In particular, if φ is a sentence then $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta}$ is independent of η .

Define $\llbracket \cdot \rrbracket_{\mathcal{M}}$ by “delaying” the assigning of values to free variables, and propagating them out. That is, define:

$$\llbracket \varphi(x_1, x_2, \dots, x_n) \rrbracket_{\mathcal{M}} = \llbracket \varphi \rrbracket_{\mathcal{M}}(x_1, x_2, \dots, x_n)$$

where $\llbracket \varphi \rrbracket_{\mathcal{M}} :$

Why separate the environment from the model?

In the definition of $\llbracket \varphi \rrbracket_{\mathcal{M}}^\eta$, η is only used to define values for the free variables. In particular, if φ is a sentence then $\llbracket \varphi \rrbracket_{\mathcal{M}}^\eta$ is independent of η .

Define $\llbracket \cdot \rrbracket_{\mathcal{M}}$ by “delaying” the assigning of values to free variables, and propagating them out. That is, define:

$$\llbracket \varphi(x_1, x_2, \dots, x_n) \rrbracket_{\mathcal{M}} = \llbracket \varphi \rrbracket_{\mathcal{M}}(x_1, x_2, \dots, x_n)$$

where $\llbracket \varphi \rrbracket_{\mathcal{M}} : \text{dom}(\mathcal{M})^n \rightarrow \mathbb{B}$;

Why separate the environment from the model?

In the definition of $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta}$, η is only used to define values for the free variables. In particular, if φ is a sentence then $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta}$ is independent of η .

Define $\llbracket \cdot \rrbracket_{\mathcal{M}}$ by “delaying” the assigning of values to free variables, and propagating them out. That is, define:

$$\llbracket \varphi(x_1, x_2, \dots, x_n) \rrbracket_{\mathcal{M}} = \llbracket \varphi \rrbracket_{\mathcal{M}}(x_1, x_2, \dots, x_n)$$

where $\llbracket \varphi \rrbracket_{\mathcal{M}} : \text{dom}(\mathcal{M})^n \rightarrow \mathbb{B}$; that is, $\llbracket \varphi \rrbracket_{\mathcal{M}}$ is an n -ary relation on $\text{dom}(\mathcal{M})$.

Interpretations: example (databases)

Example

- Vocabulary: database schema
- Formulas: queries (φ)
- Models: databases ($\mathcal{D}$)
- Interpretation:

Interpretations: example (databases)

Example

- Vocabulary: database schema
- Formulas: queries (φ)
- Models: databases ($\mathcal{D}$)
- Interpretation: $\llbracket \varphi \rrbracket_{\mathcal{D}}$ is a relation on $\text{dom}(\mathcal{D})$, i.e. a (derived) table in $\mathcal{D}$

Interpretations: example (databases)

Example

- Vocabulary: database schema
- Formulas: queries (φ)
- Models: databases ($\mathcal{D}$)
- Interpretation: $\llbracket \varphi \rrbracket_{\mathcal{D}}$ is a relation on $\text{dom}(\mathcal{D})$, i.e. a (derived) table in $\mathcal{D}$
- Environment:

Interpretations: example (databases)

Example

- Vocabulary: database schema
- Formulas: queries (φ)
- Models: databases ($\mathcal{D}$)
- Interpretation: $\llbracket \varphi \rrbracket_{\mathcal{D}}$ is a relation on $\text{dom}(\mathcal{D})$, i.e. a (derived) table in $\mathcal{D}$
- Environment: looks up an entry in a (derived) table and returns whether the lookup was successful

Interpretations: example (databases)

Example

- Vocabulary: database schema
- Formulas: queries (φ)
- Models: databases ($\mathcal{D}$)
- Interpretation: $\llbracket \varphi \rrbracket_{\mathcal{D}}$ is a relation on $\text{dom}(\mathcal{D})$, i.e. a (derived) table in $\mathcal{D}$
- Environment: looks up an entry in a (derived) table and returns whether the lookup was successful
- $\llbracket \varphi \rrbracket_{\mathcal{D}}^n$: Success/fail outcome of looking up a specific entry in a query result on $\mathcal{D}$.

Satisfiability, truth, validity

A formula φ of predicate logic is:

- **satisfiable** if there is some model $\mathcal{M}$ and some environment η such that $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta} = \text{true}$.
- **true in a model** $\mathcal{M}$ if for all environments η we have $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta} = \text{true}$
- a **logical validity** if it is true in all models.

NB

For sentences the first two definitions coincide.

Satisfiability, truth, validity

A formula φ of predicate logic is:

- **satisfiable** if there is some model $\mathcal{M}$ and some environment η such that $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta} = \text{true}$.
- **true in a model** $\mathcal{M}$ if for all environments η we have $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta} = \text{true}$
- a **logical validity** if it is true in all models.

NB

For sentences the first two definitions coincide.

Example

The sentence $\forall x \forall y ((y = x + 1) \rightarrow (x \leq y))$ is satisfiable but it is not a logical validity.

Entailment, Logical equivalence

- A theory T **entails** a formula φ , $T \models \varphi$, if φ is satisfied by any interpretation that satisfies all formulas in T .
- φ is **logically equivalent** to ψ , $\varphi \equiv \psi$, if $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta} = \llbracket \psi \rrbracket_{\mathcal{M}}^{\eta}$ for all interpretations $(\mathcal{M}, \eta)$.

Entailment, Logical equivalence

- A theory T **entails** a formula φ , $T \models \varphi$, if φ is satisfied by any interpretation that satisfies all formulas in T .
- φ is **logically equivalent** to ψ , $\varphi \equiv \psi$, if $\llbracket \varphi \rrbracket_{\mathcal{M}}^{\eta} = \llbracket \psi \rrbracket_{\mathcal{M}}^{\eta}$ for all interpretations $(\mathcal{M}, \eta)$.

Theorem

- $\varphi_1, \dots, \varphi_n \models \psi$ if, and only if, $(\varphi_1 \wedge \dots \wedge \varphi_n) \rightarrow \psi$ is a logical validity.
- $\varphi \equiv \psi$ if, and only if, $\varphi \leftrightarrow \psi$ is a logical validity.

Summary of topics

- Re-introduction to Predicate Logic
- Syntax of Predicate Logic
- Semantics of Predicate Logic
- Natural Deduction for Predicate Logic (not today)

Notes (Week 5 Monday)

These are the notes I wrote during the lecture.

Propositional logic has:

boolean variables (propositional atoms)
negation,
conjunction,
disjunction,
implication,
biimplication

^ there's nothing but booleans and combinators
on booleans

In W1, we introduced a logic with three layers:

- terms $+, -, \cup$
- predicates truth claims about terms $\leq, <$
- propositions $\wedge, \vee, \rightarrow$
 $\forall \exists$

$\forall x \in \{1, 2\}. P(x)$
 $\equiv P(1) \wedge P(2)$

^ when the domain is finite,
I can encode universal quantification with
conjunctions

Counterquestion:

what set exactly are x and y from?

Today, quantifiers won't be qualified

$\forall x \in X. P(x)$

we're doing this:

$\forall x (P(x))$

$\forall x y. x = y$

^ holds if the domain of discourse has only one element

otherwise, doesn't hold

There's only ever one domain of discourse.
If you want two, take the union of them.

I want to talk about $\mathbb{N}$ s and $\text{Pow}(\mathbb{R})$ s.
My domain of discourse: $\mathbb{N} \cup \text{Pow}(\mathbb{R})$

$\forall x \in \mathbb{N}. x \geq 0$ $\leftarrow$ I can't write

$\forall x. x \geq 0$ $\leftarrow$ now I'm making a claim also about $\text{Pow}(\mathbb{R})$

$\forall x. \text{is_nat}(x) \rightarrow x \geq 0$
^ added a "is a nat" predicate
to encode quantifier ranges

For exists:

$\exists x \in \mathbb{N}. x < 5$

Encode this as:

$\exists x. \text{is_nat}(x) \wedge x < 5$

Suppose we tried implication instead

$\exists x \in \mathbb{N}. x + 1 = x$

If we used implication instead of conjunction

$\exists x. \text{is_nat}(x) \rightarrow x + 1 = x$

^ this is provable!

Choose $x = \{\pi\}$

$\text{is_nat}(\{\pi\}) \rightarrow \{\pi\} + 1 = \{\pi\}$
 $\equiv \perp \rightarrow \{\pi\} + 1 = \{\pi\}$
 $\equiv \top$

Layers:

terms (constants, functions, variables)
predicates (truth claims on *terms*)
propositions

More appropriate way of saying Jon Snow know nothing:

$\forall x. \text{is_fact}(x) \rightarrow \neg K(J, x)$

In predicate logic, you get to choose some things
about the term structure when you instantiate it.
That's called the vocabulary.

The predicate

`Person("Joe","Biden","The White House")`

^ means that

name: "Joe", surname: "Biden", address: "The White House"
is a row in the Person table

When you specify your vocabulary,
equality always comes "built-in"
Equality is the one predicate that
always has a fixed meaning.

$\forall x P(x) \equiv \forall y P(y)$ if x, y fresh in P
^ doesn't
occur free

$\forall x \exists z \exists x P(x, y, z) \equiv \forall w \exists z \exists x P(x, y, z)$

$\phi = (\forall x \exists z \exists x P(x, y, z)) \wedge Q(x)$

$\phi[1+1/x] = (\forall x \exists z \exists x P(x, y, z)) \wedge Q(1+1)$

$\phi[x/y] \neq (\forall x \exists z \exists x P(x, x, z)) \wedge Q(x) \leftarrow$ that's wrong :(
this illustrates a phenomenon called
variable capture

Solution: rename quantifiers before substituting

$\phi[x/y] = (\forall w \exists z \exists w P(w, x, z)) \wedge Q(x)$

A: It depends upon what the meaning of:
1, 2 and + is.

In propositional logic, this is neither true nor false:

$p \wedge q$

to give it a truth value, we must supply a **valuation**

Vocabulary tells you what symbols you're going to use

The model tells you what they mean

Models give meaning to the **vocabulary**
constants, functions, predicates

But not to variables.

To interpret a formula (to assign truth values to formulas)
we need two things:

interpretation	=	model	+	environment
		^		^
		gives meaning		gives meaning to
		to vocabulary		variables

The reason to treat them separately is:

- the environment may change over the course of a single formula.
 $\forall x. P(x) \wedge \exists y. Q(x,y)$
- when you define e.g. arithmetic (like Giuseppe Peano) you don't define the values of $x,y,z...$ for all time.

η ("eta") stands for an environment

η is a function from variable names
to domain elements

Suppose η is an environment that maps every variable name to 1.

$$\eta(y) = 1$$

Then $\eta[x \mapsto 2]$ is an environment that maps x to 2,
and otherwise behaves exactly as η

$$\begin{aligned}\eta[x \mapsto 2](x) &= 2 \\ \eta[x \mapsto 2](y) &= 1\end{aligned}$$

$\llbracket \phi \rrbracket^{\eta}_{\mathcal{M}}$ the interpretation of formula ϕ in
the environment η and model $\mathcal{M}$.

$\llbracket t \rrbracket^{\eta}_{\mathcal{M}}$ this is a domain element
^ t for term

$\llbracket \phi \rrbracket^{\eta}_{\mathcal{M}}$ this is a truth value

Let's fix η s.t. $\eta(x) = \text{sandwich}$

Domain: $\{\text{fish}, \text{sandwich}\}$

$\mathcal{M}^1 = \text{fish}$

$\mathcal{M}^+ = f$

such that $f(x,y) = x$

```
[[1+x]] = f([[1]], [[x]])  
        = f(fish, sandwich)  
        = fish
```

Isn't it cheating to define $\forall$ in terms of "for all"

I *could* have used a set comprehension instead

```
[[ $\forall x \phi$ ]] $\eta, \mathcal{M}$  = true if  
    {c  $\in$  dom( $\mathcal{M}$ ) | [[ $\phi$ ]] $\eta[x \mapsto c], \mathcal{M}$  = true} = dom( $\mathcal{M}$ )
```

But there is a fundamental problem here.

To give meaning to a formal system

(to define the semantics of it)

formal system: propositional logic, predicate logic,
 ZFC set theory,

I need a bigger formal system.

How do I give meaning to *that* system?

Summary of topics

- Re-introduction to Predicate Logic
- Syntax of Predicate Logic
- Semantics of Predicate Logic
- Natural Deduction for Predicate Logic

Motivation

To show satisfiability, all you need to do is find an interpretation where the formula is true.

Motivation

To show satisfiability, all you need to do is find an interpretation where the formula is true.

Theorem

Show that $1 + 1 = 3$ is satisfiable.

Motivation

To show satisfiability, all you need to do is find an interpretation where the formula is true.

Theorem

Show that $1 + 1 = 3$ is satisfiable.

Proof.

Let $\mathcal{M}$ be

$$\begin{aligned}\text{dom}(\mathcal{M}) &= \{1\} \\ (+)^{\mathcal{M}} &= \{((1, 1), 1)\} \\ 1^{\mathcal{M}} &= 1 \\ 3^{\mathcal{M}} &= 1\end{aligned}$$

Then $\llbracket 1 + 1 = 3 \rrbracket_{\mathcal{M}} = \text{true}$



Motivation

To show non-validity, all you need to do is find an interpretation where the formula is *false*.

Motivation

To show non-validity, all you need to do is find an interpretation where the formula is *false*.

Theorem

$1 + 1 = 3$ is not valid.

Proof.

Let $\mathcal{M}$ be any standard model of arithmetic. We have that

$$\llbracket 1 + 1 = 3 \rrbracket_{\mathcal{M}} = \text{false}$$



Motivation

For satisfiability and non-validity, we only needed to think about one interpretation. For validity, you need to think about *all* interpretations.

Motivation

For satisfiability and non-validity, we only needed to think about one interpretation. For validity, you need to think about *all* interpretations.

Natural deduction allows you to prove validity without thinking about interpretations at all.

Natural deduction for Predicate Logic

In this presentation of natural deduction, we only consider *sentences* (aka formulas with no free variables).

This simplifies the presentation, and the restriction is easy to work around: whenever you'd like a free variable, add a constant symbol to your vocabulary.

We'll use a, b, c for constants, t for terms, and x, y, z for variables.

Natural deduction for Predicate Logic

Inference rules for Propositional Logic + seven rules for quantifiers and equality

Operator	Introduction	Elimination
$\forall$	$\forall$ -I	$\forall$ -E
$\exists$	$\exists$ -I	$\exists$ -E
$=$	$=$ -I	$=$ -E1 $=$ -E2

Contexts

Suppose A is a sentence with one or more term-shaped holes (written $\cdot$). Then $A(t)$ is the result of filling all the holes with the term t .

Example

If A is the context $\cdot = a \wedge b = \cdot$ then

$A(a)$ is $a = a \wedge b = a$, and

$A(f(c))$ is $f(c) = a \wedge b = f(c)$

.

= Introduction and Elimination

=-introduction:

$$\frac{}{t = t} (=I)$$

“Every term equals itself”

= Introduction and Elimination

=-introduction: $\frac{}{t = t} (=I)$

“Every term equals itself”

=-elimination (1): $\frac{t = t' \quad A(t)}{A(t')} (=E1)$

=-elimination (2): $\frac{t = t' \quad A(t')}{A(t)} (=E1)$

“Equal terms have the same properties”

Arbitrary constant symbols

A constant symbol is **arbitrary** if we haven't assumed anything about it—that is, if it does not occur in any undischarged assumption.

Intuitively: an arbitrary constant symbol can be assigned any element of the domain, and the formula will still hold.

$\forall$ Elimination

$\forall$ -elimination:

$$\frac{\forall x A(x)}{A(t)} (\forall\text{-E})$$

“Any property that holds in general holds for any specific instance.”

∀ Introduction

∀-introduction:

$$\frac{A(c) \quad \begin{array}{l} c \text{ not free in } A \\ c \text{ is arbitrary} \end{array}}{\forall x A(x)} \quad (\forall\text{-I})$$

"If we can prove a property about c without assuming anything about c , then the property holds in general."

$\forall$ Introduction

Without the “c is arbitrary” side condition, $\forall$ -I would be completely broken.

Q: Why is the following argument wrong?

Socrates is a man.

All men are mortal.

Therefore, everybody is a man.

$\forall$ Introduction

Without the “c is arbitrary” side condition, $\forall$ -I would be completely broken.

Q: Why is the following argument wrong?

Socrates is a man.

All men are mortal.

Therefore, everybody is a man.

A: Socrates is not arbitrary, so this $\forall$ -I does not apply.

∃ Introduction

∃-introduction:
$$\frac{A(t)}{\exists x A(x)} (\exists\text{-I})$$

"If t satisfies A , then something satisfies A ."

∃ Elimination

∃-elimination:

$$\frac{\begin{array}{c} \exists x A(x) \qquad \begin{array}{c} [A(c)] \\ \vdots \\ B \end{array} \qquad \begin{array}{l} c \text{ not free in } A, B \\ c \text{ is arbitrary} \end{array} \end{array}}{B} \quad (\exists\text{-E})$$

"If a property holds about something, and an arbitrary instance of the property has some consequence, then the consequence holds in general. "

$\exists$ Elimination

Without the “c is not free in” side condition, $\exists$ -E would be completely broken.

Q: Why is the following argument wrong?

Someone is hungry.

If Socrates is hungry, he eats a sandwich.

Therefore, Socrates eats a sandwich.

$\exists$ Elimination

Without the “c is not free in” side condition, $\exists$ -E would be completely broken.

Q: Why is the following argument wrong?

Someone is hungry.

If Socrates is hungry, he eats a sandwich.

Therefore, Socrates eats a sandwich.

A: $\exists$ -E does not apply, since Socrates is free in the conclusion of the argument on line 2.

About the online checker

The version of FOL supported by the online proof checker doesn't have function symbols: there's only constants and predicates.

The rules of natural deduction are the same whether function symbols are allowed or not.

Soundness

Theorem

Natural deduction is sound for Predicate Logic:

$$T \vdash \varphi \text{ implies } T \models \varphi$$

Soundness

Theorem

Natural deduction is sound for Predicate Logic:

$$T \vdash \varphi \text{ implies } T \models \varphi$$

Corollary

Natural deduction is consistent:

$$\emptyset \not\vdash \perp$$

Completeness and Incompleteness

Confusingly, Kurt Gödel proved both of these:

Theorem (Gödel's completeness theorem)

Theorem (Gödel's first incompleteness theorem)

Completeness and Incompleteness

Confusingly, Kurt Gödel proved both of these:

Theorem (Gödel's completeness theorem)

Natural deduction is complete for Predicate Logic:

$$T \models \varphi \text{ implies } T \vdash \varphi$$

Theorem (Gödel's first incompleteness theorem)

Completeness and Incompleteness

Confusingly, Kurt Gödel proved both of these:

Theorem (Gödel's completeness theorem)

Natural deduction is complete for Predicate Logic:

$$T \models \varphi \text{ implies } T \vdash \varphi$$

Theorem (Gödel's first incompleteness theorem)

Roughly: if T is consistent, and expressive enough to do elementary arithmetic, then there are sentences G such that neither

$$T \vdash G \text{ nor } T \vdash \neg G$$

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- CNF/DNF
- Proof
- Natural deduction
- Bonus examples

Proof examples

What follows is a bunch of example derivations in natural deduction.

I do not plan to go over these in the lecture (favouring instead live demos).

Proof example (Fitch)

Prove: $\vdash \forall x \forall y (x = y) \rightarrow (y = x)$



Proof example (Fitch)

Prove: $\vdash \forall x \forall y (x = y) \rightarrow (y = x)$

1. $a = b$

Proof example (Fitch)

Prove: $\vdash \forall x \forall y (x = y) \rightarrow (y = x)$

$\vdash$	
1. $a = b$	
2. $a = a$	$=-I$

Proof example (Fitch)

Prove: $\vdash \forall x \forall y (x = y) \rightarrow (y = x)$

1. $a = b$

2. $a = a$

=-I

3. $b = a$

=-E1: 1,2

Proof example (Fitch)

Prove: $\vdash \forall x \forall y (x = y) \rightarrow (y = x)$

1. $a = b$

2. $a = a$ $=-I$

3. $b = a$ $=-E1: 1,2$

4. $(a = b) \rightarrow (b = a)$ $\rightarrow-I: 1-3$

Proof example (Fitch)

Prove: $\vdash \forall x \forall y (x = y) \rightarrow (y = x)$

1. $a = b$

2. $a = a$

$=-I$

3. $b = a$

$=-E1: 1,2$

4. $(a = b) \rightarrow (b = a)$

$\rightarrow-I: 1-3$

5. $\forall y (a = y) \rightarrow (y = a)$

$\forall-I: 4$

Proof example (Fitch)

Prove: $\vdash \forall x \forall y (x = y) \rightarrow (y = x)$

- | | | |
|--|---|--|
| | | |
| | 1. $a = b$ | |
| | | |
| | | 2. $a = a$ $=-I$ |
| | | 3. $b = a$ $=-E1: 1,2$ |
| | 4. $(a = b) \rightarrow (b = a)$ $\rightarrow-I: 1-3$ | |
| | 5. $\forall y (a = y) \rightarrow (y = a)$ $\forall-I: 4$ | |
| | 6. $\forall x \forall y (x = y) \rightarrow (y = x)$ $\forall-I: 5$ | |

Proof example

Prove: $\forall x \forall y P(x, y) \vdash \forall y \forall x P(x, y)$

Line	Premises	Formula	Rule	References
------	----------	---------	------	------------

Proof example

Prove: $\forall x \forall y P(x, y) \vdash \forall y \forall x P(x, y)$

Line	Premises	Formula	Rule	References
1		$\forall x \forall y P(x, y)$	Premise	

Proof example

Prove: $\forall x \forall y P(x, y) \vdash \forall y \forall x P(x, y)$

Line	Premises	Formula	Rule	References
1		$\forall x \forall y P(x, y)$	Premise	
2	1	$\forall y P(a, y)$	$\forall$ -E	1

Proof example

Prove: $\forall x \forall y P(x, y) \vdash \forall y \forall x P(x, y)$

Line	Premises	Formula	Rule	References
1		$\forall x \forall y P(x, y)$	Premise	
2	1	$\forall y P(a, y)$	$\forall$ -E	1
3	1	$P(a, b)$	$\forall$ -E	2

Proof example

Prove: $\forall x \forall y P(x, y) \vdash \forall y \forall x P(x, y)$

Line	Premises	Formula	Rule	References
1		$\forall x \forall y P(x, y)$	Premise	
2	1	$\forall y P(a, y)$	$\forall$ -E	1
3	1	$P(a, b)$	$\forall$ -E	2
4	1	$\forall x P(x, b)$	$\forall$ -I	3

Proof example

Prove: $\forall x \forall y P(x, y) \vdash \forall y \forall x P(x, y)$

Line	Premises	Formula	Rule	References
1		$\forall x \forall y P(x, y)$	Premise	
2	1	$\forall y P(a, y)$	$\forall$ -E	1
3	1	$P(a, b)$	$\forall$ -E	2
4	1	$\forall x P(x, b)$	$\forall$ -I	3
5	1	$\forall y \forall x P(x, y)$	$\forall$ -I	4

Proof example (Fitch)

Prove: $\exists x \exists y P(x, y) \vdash \exists y \exists x P(x, y)$

1. $\exists x \exists y P(x, y)$



Proof example (Fitch)

Prove: $\exists x \exists y P(x, y) \vdash \exists y \exists x P(x, y)$

1. $\exists x \exists y P(x, y)$

2. $\exists y P(a, y)$

Proof example (Fitch)

Prove: $\exists x \exists y P(x, y) \vdash \exists y \exists x P(x, y)$

1.	$\exists x \exists y P(x, y)$
2.	$\exists y P(a, y)$
3.	$P(a, b)$

Proof example (Fitch)

Prove: $\exists x \exists y P(x, y) \vdash \exists y \exists x P(x, y)$

1.	$\exists x \exists y P(x, y)$	
2.	$\exists y P(a, y)$	
3.	$P(a, b)$	
4.	$\exists x P(x, b)$	$\exists$ -I: 3

Proof example (Fitch)

Prove: $\exists x \exists y P(x, y) \vdash \exists y \exists x P(x, y)$

1. $\exists x \exists y P(x, y)$

2. $\exists y P(a, y)$

3. $P(a, b)$

4. $\exists x P(x, b)$ $\exists$ -I: 3

5. $\exists y \exists x P(x, y)$ $\exists$ -I: 4

Proof example (Fitch)

Prove: $\exists x \exists y P(x, y) \vdash \exists y \exists x P(x, y)$

1. $\exists x \exists y P(x, y)$

2. $\exists y P(a, y)$

3. $P(a, b)$

4. $\exists x P(x, b)$ $\exists$ -I: 3

5. $\exists y \exists x P(x, y)$ $\exists$ -I: 4

6. $\exists y \exists x P(x, y)$ $\exists$ -E: 2, 3–5

Proof example (Fitch)

Prove: $\exists x \exists y P(x, y) \vdash \exists y \exists x P(x, y)$

1.	$\exists x \exists y P(x, y)$	
2.	$\exists y P(a, y)$	
3.	$P(a, b)$	
4.	$\exists x P(x, b)$	$\exists$ -I: 3
5.	$\exists y \exists x P(x, y)$	$\exists$ -I: 4
6.	$\exists y \exists x P(x, y)$	$\exists$ -E: 2, 3–5
7.	$\exists y \exists x P(x, y)$	$\exists$ -E: 1, 2–6

Notes (Week 5 Wednesday)

These are the notes I wrote during the lecture.

An implication $\phi \Rightarrow \psi$ is true iff:

whenever ϕ is true, so is ψ

An entailment $T \models \psi$ is valid iff:

in every interpretation $\mathcal{I}$ where
if for each $\phi \in T$, $\llbracket \phi \rrbracket_{\mathcal{I}} = \text{true}$
then $\llbracket \psi \rrbracket_{\mathcal{I}} = \text{true}$

A = I can see you

B = You can see my mirrors

If you can't see my mirrors, then I can't see you

$\neg B \Rightarrow \neg A$

$A \Rightarrow B$

$\neg A \vee B$

Consider the statement "If you can't see my mirrors, then I can't see you". Which of the following are equivalent statements? Select one or more:

- If I can see you, then you can see my mirrors.
- Either you can see my mirrors, or I can't see you.
- If I can't see you, then you can't see my mirrors.
- If you can see my mirrors, then I can see you.
- You can't see my mirrors, and I can't see you.
- Either you can't see my mirrors, or I can't see you.

Wrap up semantics of predicate logic

Natural deduction for predicate logic

A propositional formula is about a
valuation --- a world where there
are boolean variables that may have
different values

A predicate logic formula is about a
interpretation
interpretation = model + environment

Model says:

- what is the domain of discourse?
what are the set of objects in the world
- what does your vocabulary mean?
maps constants to domain elements,
function symbols to (mathematical) functions
predicate symbols to relations

Environment says:

- what is value of each variable?

The semantics of term:

$\llbracket t \rrbracket_{\mathcal{I}}$ given a term t , maps it to a domain element

$\llbracket \phi \rrbracket_{\mathcal{I}}$ given a formula ϕ , maps it to a truth value

For propositional logic, the semantics
is a computable function.

You can write a computer program that
implements prop. logic semantics,
and compute validity.

For predicate logic, this no longer holds.
 The semantics of predicate logic is a
 mathematical function, but not a
 computable function.
 We can't write a computer program
 to calculate the truth value of a
 pred logic formula.

Instead, the point of predicate logic
 is not to compute,
 but to give a precise description of
 what the logic *means*.
 This is useful is want to convince
 ourselves that we got it right.

This formula is satisfiable:

$\forall x \forall y. x=y$
 ..but not valid.

satisfiable
 true in a model with domain $\{x\}$ for some x
 not valid (false in any model with >1 element)

...we can instead say:

$(\forall x. x = c) \models \forall x \forall y. x=y$
 $\wedge$ here, instead of specifying a model directly,
 we've *axiomatized* the model as a theory.

Now, we essentially prove "in every theory that
 satisfies our axioms, the conclusion holds".

$T \models \forall x \forall y (y = x + 1 \rightarrow y \geq x)$
 ...where T is the set of all Peano axioms

We've seen natural deduction for prop logic.
 For each operator ($\neg, \wedge, \vee, \dots$) we have
 introduction and elimination rules that tell
 us how we can use proofs that we already
 have to get new proofs about that operator.

$\vdash A \quad \vdash B$	$\vdash A \wedge B$	$\vdash A \wedge B$
$\vdash A \wedge B$	$\vdash A$	$\vdash A$
$\wedge I$	$\wedge E$	$\wedge E$

Natural deduction for predicate logic
 is the same, but adds rules for
 $= \quad \forall \quad \exists$

Propositional ND is kind of a toy.

- We don't *really* need it,
 there's very mature algorithms
 for reasoning about propositional
 formulas, none of which use ND

For predicate logic,
 there are no algorithms.
 (If there were, mathematicians would be
 unemployed)

Transitivity:

prove $t = u, u = v \vdash t = v$

| 1. $t = u$
2. $u = v$
3. $t = v$ =E-2 2,1

$A \equiv \cdot = v$
 $A(u) \equiv u = v$
 $A(t) \equiv t = v$

$t = u \quad A(u)$

$$\frac{}{A(t)} =E-2$$

Symmetry:

prove $t = u \vdash u = t$ | 1. $t = u$

| -

| 2. $t = t$ $=I$ | 3. $u = t$ $=E-1 \quad 1,2$ $A \equiv \cdot = t$ $A(t) \equiv t = t$ $A(u) \equiv u = t$

$$\frac{t = u \quad A(t)}{A(u)} =E$$
| 1. $P(a)$

| -

| | $P(b)$

| | -

| | ... (* on this line, neither b nor a is arbitrary *)

| ... (* on this line, b is arbitrary *)

Let's "prove" by natural deduction
using a fake $\forall I$ rule that doesn't
require "c is free in A"

 $\vdash \forall x. x = c$

|

| -

| $c = c$ $=I$ | $\forall x. x = c$ fake $\forall I$ ^ here we used a context $A \equiv \cdot = c$

Why no function symbols?

 $x+y \geq z$

we could instead have a ternary predicate symbol

$P(x,y,z)$ where
the intended meaning of P is $x+y \geq z$

 $\exists y \forall x P(x,y) \models \forall x \exists y P(x,y)$

Quantifier order matters

Tute (Week 5)

Entailment

Question 1

Is the following claim

$$\text{If } \phi \not\models \psi \text{ then } \phi \models \neg\psi$$

1. a true statement about propositional formulas?
2. a true statement about predicate formulas?

1. No it is not true, consider for example $\phi = p$ and $\psi = q$ (where p and q are propositional variables). Then neither $P \models Q$ nor $P \models \neg Q$.
2. Not true either. For example, let $\phi = (x = y)$ and $\psi = (z = w)$.

Question 2

Show that the following is *not* valid:

$$\forall x(P(x) \rightarrow Q(x)) \models \exists xQ(x)$$

We need to find a model where the first formula holds but the second does not. Consider a model $\Box$ where the domain has one element, where $P^\Box = \emptyset$ and $Q^\Box = \emptyset$. Then $\llbracket \forall x(P(x) \rightarrow Q(x)) \rrbracket_\Box$ holds (the implication becomes vacuously true). But $\llbracket \exists xQ(x) \rrbracket_\Box$ does not.

Question 3

Show that the following is valid:

$$\forall xP(x) \models \exists xP(x)$$

We can use natural deduction:

$$\frac{\frac{\forall x P(x)}{P(c)} \forall\text{-E}}{\exists x P(x)} \exists\text{-I}$$

We could also try a direct semantic argument, but this is considerably more involved.

Limitations of first-order logic

Question 1

We can use predicate logic to talk about almost anything, by choosing an appropriate vocabulary. But we can't use it to talk about nothing, because we require a non-empty domain of discourse.

Why do you think this restriction exists?

Natural deduction becomes unsound for an empty domain of discourse. For example, consider $\forall\text{-E}$:

$$\frac{\forall x P(x)}{P(c)}$$

If the domain of discourse is empty, the premise of this rule is vacuously true: everything in the domain of discourse does indeed satisfy P . But the conclusion is false (or ill-formed, if you prefer): $P(c)$ cannot hold because there is no c .

You can use similar arguments to show breakdowns of useful logical laws. For example, the law

$$\exists x. \top \equiv \top$$

would fail in an empty domain of discourse.

Question 2

Recall the Peano axioms from the W3 tute. Can we do Peano arithmetic in first-order predicate logic? First, let our vocabulary include the constant symbol Z and the unary function symbol S .

Let ϕ be a first-order predicate formula with this vocabulary.

1. Suppose it is the case that

$$\models \phi$$

Does it follow that ϕ is true in Peano arithmetic?

2. Suppose it is the case that

$$\not\models \phi$$

Does it follow that ϕ is false in Peano arithmetic?

3. Suppose I want to use FOL + the Peano axioms to prove ϕ . Let's try to make a theory $\square$ that includes the Peano axioms, so that we can prove

$$\square \models \phi$$

Write down a set of formulas that capture all but the last Peano axiom.

4. For the last axiom, it unfortunately turns out that we can't write down a FOL formula that captures induction on arbitrary propositions such as ϕ . Why not? Are there any workarounds?

1. Yes. Peano arithmetic is one possible model of our vocabulary, and $\models \phi$ means that ϕ is true in every possible model (and environment).
2. No. $\not\models \phi$ means that ϕ is false in *at least one* model, but it may still be true in Peano arithmetic. For example, the formula $\neg(S(Z) = Z)$ is false in Peano arithmetic, but true in any model with a one-element domain of discourse.
3. The first two Peano axioms are more syntax than semantics, and hence do not require any FOL axioms. For example, "there is a number Z " follows immediately from the fact that Z is in the vocabulary. For axioms 3 and 4 we can do the following:

$$\{\forall n(\neg(S(n) = Z)), \\ \forall n\forall m(S(n) = S(m) \rightarrow n = m)\}$$

4. Using logical notation instead of set notation, the induction axiom can be stated as:

$$\forall P((P(Z) \wedge \forall n(P(n) \rightarrow P(S(n)))) \rightarrow \forall n(P(n)))$$

Unfortunately, the above is not a wff in first-order predicate logic! That's because first-order logic only allows quantification over (term) variables. P , however, is not a term variable but a predicate symbol.

What we can do is add an infinite family of *instances* of the induction axiom to our theory. The idea is that for every wff $\phi(n)$, we add an axiom

$$(\phi(Z) \wedge \forall n(\phi(n) \rightarrow \phi(S(n)))) \rightarrow \forall n(\phi(n))$$

This approach allows us to do Peano arithmetic for most practical purposes, but will still not uniquely characterise Peano arithmetic: there will be other models that also satisfy these axioms. This is a bit beyond the scope of the course, but if you're interested in why, look up the compactness theorem and non-standard models of arithmetic. One place to start is [here](#).

Predicate Natural Deduction

Give Natural Deduction proofs of the following:

1. $\vdash \forall x F(x) \vee \neg \forall x F(x)$
2. $\vdash \forall x \exists y (Q(y) \rightarrow Q(x))$
3. $(m = n) \vee (n = o), A(n) \vdash A(m) \vee A(o)$
4. $\exists x J(x), \exists x \neg J(x) \vdash \exists x \exists y \neg (x = y)$

1. This proof uses only the rules from the propositional fragment of ND (along with the derived rule LEM, which is not needed but simplifies matters)

$$\frac{\frac{[\forall x F(x)]}{\forall x F(x) \vee \neg \forall x F(x)} \vee\text{-I} \quad \frac{[\neg \forall x F(x)]}{\forall x F(x) \vee \neg \forall x F(x)} \vee\text{-I}}{\forall x F(x) \vee \neg \forall x F(x)} \text{LEM}$$

$$\begin{array}{c} 2. \\ \frac{[Q(c)]}{Q(c) \rightarrow Q(c)} \rightarrow\text{-I} \\ \frac{Q(c) \rightarrow Q(c)}{\exists y (Q(y) \rightarrow Q(c))} \exists\text{-I} \\ \frac{\exists y (Q(y) \rightarrow Q(c))}{\forall x \exists y (Q(y) \rightarrow Q(x))} \forall\text{-I} \end{array}$$

$$\begin{array}{c} 3. \\ \frac{(m = n) \vee (n = o) \quad \frac{[m = n] A(n)}{A(m)} =\text{-E} \quad \frac{[n = o] A(n)}{A(o)} =\text{-E}}{\frac{A(m)}{A(m) \vee A(o)} \vee\text{-I} \quad \frac{A(o)}{A(m) \vee A(o)} \vee\text{-I}} \vee\text{-E} \\ A(m) \vee A(o) \end{array}$$

$$\begin{array}{c} 4. \\ \frac{\exists x J(x) \quad \frac{[\neg J(d)] \quad \frac{[c = d] [J(c)]}{J(d)} =\text{-E}}{\neg J(d)} \neg\text{-E}}{\frac{\perp}{\neg(c = d)} \neg\text{-I}} \neg\text{-E} \\ \frac{\neg(c = d)}{\exists y \neg(c = y)} \exists\text{-I} \\ \frac{\exists x \neg J(x) \quad \exists y \neg(c = y)}{\exists x \exists y \neg(x = y)} \exists\text{-I} \\ \frac{\exists x J(x) \quad \exists x \exists y \neg(x = y)}{\exists x \exists y \neg(x = y)} \exists\text{-E} \\ \exists x \exists y \neg(x = y) \end{array}$$



COMP2111 Week 7

Term 1, 2023

State machines

Summary

- Motivation
- Definitions
- The invariant principle
- Partial correctness and termination
- Input and output
- Finite automata

Motivation

In Assignment 1, we modelled programs as relations between initial and final states of successful executions. So this Tic-Tac-Toe program:

```
void move(int pos, char fill) {  
    if(board[pos] == "E" && (fill == "X" || fill == "O")) {  
        board[pos] := fill;  
    }  
    else abort;  
}
```

can be modelled with this relation:

$$\{(b, b') : \exists n. \forall i. \\ (n = i \rightarrow b_i = E \wedge b'_i \neq E) \wedge \\ (n \neq i \rightarrow b_i = b'_i)\}$$

Motivation

Such relational modelling is useful (spoiler alert: W8-9), but doesn't always capture everything we care about.

Motivation

Such relational modelling is useful (spoiler alert: W8-9), but doesn't always capture everything we care about.

Possibility of failure is sometimes not captured. This:

```
void incr() {  
  
    x := x + 1;  
  
}
```

can be modelled by this relation over $\mathbb{Z} \times \mathbb{Z}$:

$$\{(x, x') : x + 1 = x'\}$$

Motivation

Such relational modelling is useful (spoiler alert: W8-9), but doesn't always capture everything we care about.

Possibility of failure is sometimes not captured. This:

```
void incr() {  
    if(Math.random() < .5) then abort else  
        x := x + 1;  
}
```

can *also* be modelled by this relation over $\mathbb{Z} \times \mathbb{Z}$:

$$\{(x, x') : x + 1 = x'\}$$

Motivation

Such relational modelling is useful (spoiler alert: W8-9), but doesn't always capture everything we care about.

Sometimes the final state isn't what's interesting.

```
void yes() {  
    while(true) print("y\n");  
}
```

This program has no final states, so its relational model doesn't say much:

$\{\}$

Motivation

State machines model step-by-step processes with:

- A set of *states*, possibly including a designated *start state*.
- A *transition relation*, detailing how to move (transition) from one state to another.

Motivation

State machines model step-by-step processes with:

- A set of *states*, possibly including a designated *start state*.
- A *transition relation*, detailing how to move (transition) from one state to another.

Example

The semantics of a program:

- States: functions from variable names to values
- Transitions: execute a line of code.

Motivation

State machines model step-by-step processes with:

- A set of *states*, possibly including a designated *start state*.
- A *transition relation*, detailing how to move (transition) from one state to another.

Example

A game of noughts and crosses

- States: Board positions
- Transitions: Legal moves

Motivation

State machines model step-by-step processes with:

- A set of *states*, possibly including a designated *start state*.
- A *transition relation*, detailing how to move (transition) from one state to another.

Example

Stateful communication protocols: e.g. SMTP

- States: Stages of communication
- Transitions: Determined by commands given (e.g. HELO, DATA, etc)

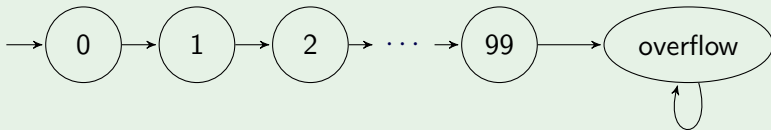
Motivation

State machines model step-by-step processes with:

- A set of *states*, possibly including a designated *start state*.
- A *transition relation*, detailing how to move (transition) from one state to another.

Example

A bounded counter that counts from 0 to 99 and overflows at 100:



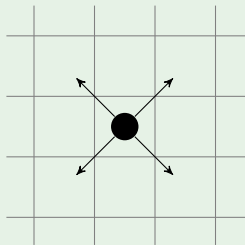
Motivation

State machines model step-by-step processes with:

- A set of *states*, possibly including a designated *start state*.
- A *transition relation*, detailing how to move (transition) from one state to another.

Example

A robot that moves diagonally



States: Locations
Transitions: Moves

Motivation

State machines model step-by-step processes with:

- A set of *states*, possibly including a designated *start state*.
- A *transition relation*, detailing how to move (transition) from one state to another.

Example

Die Hard jug problem: Given jugs of 3L and 5L, measure out exactly 4L.

- States: Defined by amount of water in each jug
- Start state: No water in both jugs
- Transitions: Pouring water (in, out, jug-to-jug)

Summary

- Motivation
- Definitions
- The invariant principle
- Partial correctness and termination
- Input and output
- Finite automata

Definitions

A **transition system** is a pair $(S, \rightarrow)$ where:

- S is a set (of **states**), and
- $\rightarrow \subseteq S \times S$ is a (**transition**) **relation**.

If $(s, s') \in \rightarrow$ we write $s \rightarrow s'$.

Definitions

A **transition system** is a pair $(S, \rightarrow)$ where:

- S is a set (of **states**), and
- $\rightarrow \subseteq S \times S$ is a (**transition**) **relation**.

If $(s, s') \in \rightarrow$ we write $s \rightarrow s'$.

- S may have a designated **start state**, $s_0 \in S$
- S may have designated **final states**, $F \subseteq S$

Definitions

A **transition system** is a pair $(S, \rightarrow)$ where:

- S is a set (of **states**), and
- $\rightarrow \subseteq S \times S$ is a (**transition**) **relation**.

If $(s, s') \in \rightarrow$ we write $s \rightarrow s'$.

- S may have a designated **start state**, $s_0 \in S$
- S may have designated **final states**, $F \subseteq S$
- The transitions may be **labelled** by elements of a set L :
 - $\rightarrow \subseteq S \times L \times S$
 - $(s, a, s') \in \rightarrow$ is written as $s \xrightarrow{a} s'$

Definitions

A **transition system** is a pair $(S, \rightarrow)$ where:

- S is a set (of **states**), and
- $\rightarrow \subseteq S \times S$ is a (**transition**) **relation**.

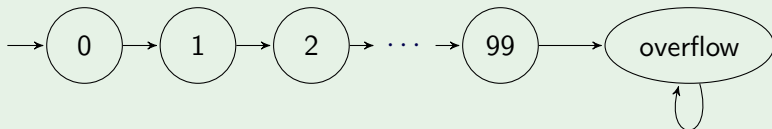
If $(s, s') \in \rightarrow$ we write $s \rightarrow s'$.

- S may have a designated **start state**, $s_0 \in S$
- S may have designated **final states**, $F \subseteq S$
- The transitions may be **labelled** by elements of a set L :
 - $\rightarrow \subseteq S \times L \times S$
 - $(s, a, s') \in \rightarrow$ is written as $s \xrightarrow{a} s'$
- If $\rightarrow$ is a partial function we say that the system is **deterministic**, otherwise it is **non-deterministic**

Example: Bounded counter

Example

A bounded counter that counts from 0 to 99 and overflows at 100:

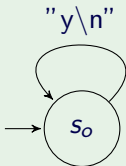


- $S = \{0, 1, \dots, 99, \text{overflow}\}$
 - $\rightarrow = \{(i, i+1) : 0 \leq i < 99\}$
- $\cup \{(99, \text{overflow})\}$
 - $\cup \{(\text{overflow}, \text{overflow})\}$
- $s_0 = 0$
- Deterministic

Example: yes

Example

```
void yes() {  
    while(true) print("y\n");  
}
```



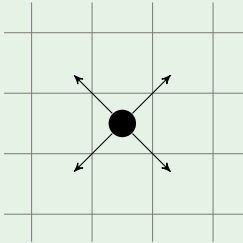
$S = \{s_0\}$

$L = \text{the set of strings}$

$s_0 \xrightarrow{\text{"y\n"}} s_0$

Example: Diagonally moving robot

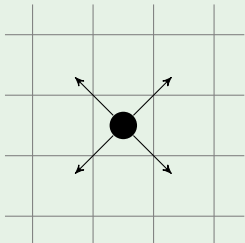
Example



States: Locations
Transitions: Moves

Example: Diagonally moving robot

Example



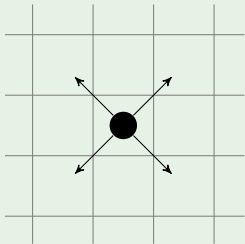
$$S = \mathbb{Z} \times \mathbb{Z}$$

$$(x, y) \rightarrow (x \pm 1, y \pm 1)$$

Non-deterministic

Example: Diagonally moving robot

Example



$$S = \mathbb{Z} \times \mathbb{Z}$$

$$L = \{NW, NE, SW, SE\}$$

$$(x, y) \xrightarrow{NW} (x - 1, y + 1)$$

$$(x, y) \xrightarrow{NE} (x + 1, y + 1)$$

$$(x, y) \xrightarrow{SW} (x - 1, y - 1)$$

$$(x, y) \xrightarrow{SE} (x + 1, y - 1)$$

Deterministic

Example: Die Hard jug problem

Example

Given jugs of 3L and 5L, measure out exactly 4L.

- States: Defined by amount of water in each jug
- Start state: No water in both jugs
- Transitions: Pouring water (in, out, jug-to-jug)

Example: Die Hard jug problem

Example

Given jugs of 3L and 5L, measure out exactly 4L.

- $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- $s_0 = (0, 0)$
- $\rightarrow$ given by

Example: Die Hard jug problem

Example

Given jugs of 3L and 5L, measure out exactly 4L.

- $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- $s_0 = (0, 0)$
- $\rightarrow$ given by
 - $(i, j) \rightarrow (0, j)$ [empty 5L jug]
 - $(i, j) \rightarrow (i, 0)$ [empty 3L jug]

Example: Die Hard jug problem

Example

Given jugs of 3L and 5L, measure out exactly 4L.

- $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- $s_0 = (0, 0)$
- $\rightarrow$ given by

- $(i, j) \rightarrow (5, j)$

[fill 5L jug]

- $(i, j) \rightarrow (i, 3)$

[fill 3L jug]

Example: Die Hard jug problem

Example

Given jugs of 3L and 5L, measure out exactly 4L.

- $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- $s_0 = (0, 0)$
- $\rightarrow$ given by

- $(i, j) \rightarrow (i + j, 0)$ if $i + j \leq 5$ [empty 3L jug into 5L jug]
- $(i, j) \rightarrow (0, i + j)$ if $i + j \leq 3$ [empty 5L jug into 3L jug]

Example: Die Hard jug problem

Example

Given jugs of 3L and 5L, measure out exactly 4L.

- $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- $s_0 = (0, 0)$
- $\rightarrow$ given by

- $(i, j) \rightarrow (5, j - 5 + i)$ if $i + j \geq 5$ [fill 5L jug from 3L jug]
- $(i, j) \rightarrow (i - 3 + j, 3)$ if $i + j \geq 3$ [fill 3L jug from 5L jug]

Example: Die Hard jug problem

Example

Given jugs of 3L and 5L, measure out exactly 4L.

- $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- $s_0 = (0, 0)$
- $\rightarrow$ given by
 - $(i, j) \rightarrow (0, j)$ [empty 5L jug]
 - $(i, j) \rightarrow (i, 0)$ [empty 3L jug]
 - $(i, j) \rightarrow (5, j)$ [fill 5L jug]
 - $(i, j) \rightarrow (i, 3)$ [fill 3L jug]
 - $(i, j) \rightarrow (i + j, 0)$ if $i + j \leq 5$ [empty 3L jug into 5L jug]
 - $(i, j) \rightarrow (0, i + j)$ if $i + j \leq 3$ [empty 5L jug into 3L jug]
 - $(i, j) \rightarrow (5, j - 5 + i)$ if $i + j \geq 5$ [fill 5L jug from 3L jug]
 - $(i, j) \rightarrow (i - 3 + j, 3)$ if $i + j \geq 3$ [fill 3L jug from 5L jug]

Runs and reachability

Given a transition system $(S, \rightarrow)$ and states $s, s' \in S$,

- a **run** (or **trace**) from s is a (possibly infinite) sequence $s_1, s_2, \dots$ such that $s = s_1$ and $s_i \rightarrow s_{i+1}$ for all $i \geq 1$.

Runs and reachability

Given a transition system $(S, \rightarrow)$ and states $s, s' \in S$,

- a **run** (or **trace**) from s is a (possibly infinite) sequence $s_1, s_2, \dots$ such that $s = s_1$ and $s_i \rightarrow s_{i+1}$ for all $i \geq 1$.
- A run is **maximal** if it cannot be extended; i.e., it is either infinite, or ends in a state from which there are no transitions.

Runs and reachability

Given a transition system $(S, \rightarrow)$ and states $s, s' \in S$,

- a **run** (or **trace**) from s is a (possibly infinite) sequence $s_1, s_2, \dots$ such that $s = s_1$ and $s_i \rightarrow s_{i+1}$ for all $i \geq 1$.
- A run is **maximal** if it cannot be extended; i.e., it is either infinite, or ends in a state from which there are no transitions.
- we say s' is **reachable** from s , written $s \rightarrow^* s'$, if (s, s') is in the reflexive and transitive closure of $\rightarrow$.

NB

s' is reachable from s if there is a run from s which contains s' .

Reachability example: Die Hard jug problem

Example

Given jugs of 3L and 5L, measure out exactly 4L.

- States: $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- Transition relation: $(i, j) \rightarrow (0, j)$ etc.

Is $(4, 0)$ reachable from $(0, 0)$?

Reachability example: Die Hard jug problem

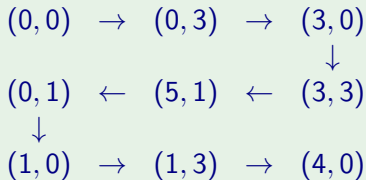
Example

Given jugs of 3L and 5L, measure out exactly 4L.

- States: $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- Transition relation: $(i, j) \rightarrow (0, j)$ etc.

Is $(4, 0)$ reachable from $(0, 0)$?

Yes:



Safety and Liveness

Transition systems can be used to study whether systems satisfy **safety** and **liveness** properties.

Safety: something bad will never happen.

Liveness: something good will happen.

Contrast this with **reachability**:

Reachability: something good can happen.

Safety and Liveness: Examples

Example

Suppose our transition system models a nuclear power plant.

Safety: the reactor never reaches the `meltedown` state.

Liveness: the power plant will keep supplying power.

Safety and Liveness: Examples

Example

```
void yes() {  
    while(true){  
        print("y\n");  
    }  
}
```

Safety: `yes()` never prints anything but "y\n".

Liveness: `yes()` will always print another "y\n".

Safety and Liveness: Examples

Example

```
y := 1;  
z := x;  
while(z ≠ 0){  
    y := y * z;  
    z := z - 1;  
}
```

Safety: If the program ever terminates, then $y = x!$

Liveness: The program will terminate

(How is that a safety property?)

Safety and Liveness

A *property* is a set of infinite runs. (Terminating runs can be made infinite by adding a self-loop to the final state.)

Safety: A *safety property* can be falsified by a **finite prefix** of a behaviour.

Liveness: A *liveness property* can always be satisfied eventually.

Properties Examples

Are they safety or liveness?

- *When I come home, there must be beer in the fridge*

Properties Examples

Are they safety or liveness?

- *When I come home, there must be beer in the fridge* – **Safety**
- *When I come home, I'll drop on the couch and drink a beer*

Properties Examples

Are they safety or liveness?

- *When I come home, there must be beer in the fridge* – **Safety**
- *When I come home, I'll drop on the couch and drink a beer* – **Liveness**
- *I'll be home later* – **Liveness**
- *The program never allocates more than 100MB of memory*

Properties Examples

Are they safety or liveness?

- *When I come home, there must be beer in the fridge* – **Safety**
- *When I come home, I'll drop on the couch and drink a beer* – **Liveness**
- *I'll be home later* – **Liveness**
- *The program never allocates more than 100MB of memory* — **Safety**
- *The program will allocate at least 100MB of memory*

Properties Examples

Are they safety or liveness?

- *When I come home, there must be beer in the fridge* – **Safety**
- *When I come home, I'll drop on the couch and drink a beer* – **Liveness**
- *I'll be home later* – **Liveness**
- *The program never allocates more than 100MB of memory* — **Safety**
- *The program will allocate at least 100MB of memory* – **Liveness**
- *No two processes are simultaneously in their **critical section***

Properties Examples

Are they safety or liveness?

- *When I come home, there must be beer in the fridge* – **Safety**
- *When I come home, I'll drop on the couch and drink a beer* – **Liveness**
- *I'll be home later* – **Liveness**
- *The program never allocates more than 100MB of memory* — **Safety**
- *The program will allocate at least 100MB of memory* – **Liveness**
- *No two processes are simultaneously in their **critical section*** — **Safety**
- *If a process wishes to enter its critical section, it will eventually be allowed to do so*

Properties Examples

Are they safety or liveness?

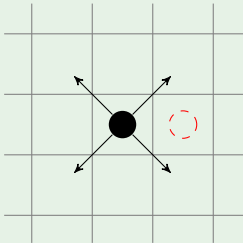
- *When I come home, there must be beer in the fridge* – **Safety**
- *When I come home, I'll drop on the couch and drink a beer* – **Liveness**
- *I'll be home later* – **Liveness**
- *The program never allocates more than 100MB of memory* — **Safety**
- *The program will allocate at least 100MB of memory* – **Liveness**
- *No two processes are simultaneously in their **critical section*** — **Safety**
- *If a process wishes to enter its critical section, it will eventually be allowed to do so* – **Liveness**

Summary

- Motivation
- Definitions
- The invariant principle
- Partial correctness and termination
- Input and output
- Finite automata

Safety example: Diagonally moving robot

Example

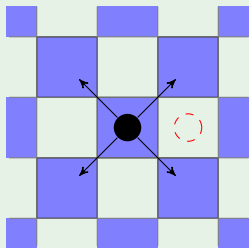


Starting at $(0,0)$

Can the robot get to $(0,1)$?

Safety example: Diagonally moving robot

Example

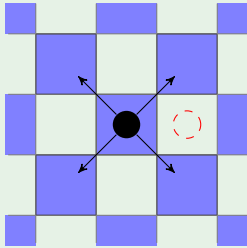


Starting at $(0,0)$

Can the robot get to $(0,1)$?

Safety example: Diagonally moving robot

Example

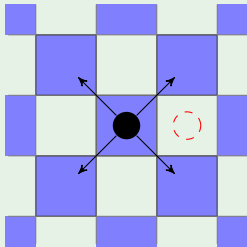


Starting at $(0,0)$

Can the robot get to $(0,1)$? No

Safety example: Diagonally moving robot

Example



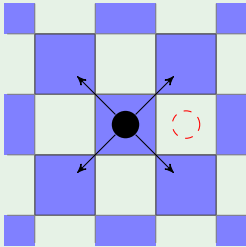
Starting at $(0,0)$

Can the robot get to $(0,1)$? No

$\text{isBlue}((m,n)) := 2|(m+n)$

Safety example: Diagonally moving robot

Example



Starting at $(0,0)$

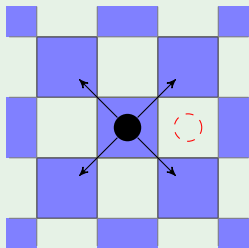
Can the robot get to $(0,1)$? No

$\text{isBlue}((m,n)) := 2|(m+n)$

if $\text{isBlue}(s)$ and $s \rightarrow s'$
then $\text{isBlue}(s')$

Safety example: Diagonally moving robot

Example



Starting at $(0,0)$

Can the robot get to $(0,1)$? No

$\text{isBlue}((m,n)) := 2 \mid (m+n)$

if $\text{isBlue}(s)$ and $s \rightarrow s'$
then $\text{isBlue}(s')$

$\text{isBlue}((0,0))$ and $\neg \text{isBlue}((0,1))$

The invariant principle

A **preserved invariant** of a transition system is a unary predicate φ on states such that if $\varphi(s)$ holds and $s \rightarrow s'$ then $\varphi(s')$ holds.

Invariant principle

If a preserved invariant holds at a state s , then it holds for all states reachable from s .

The invariant principle

A **preserved invariant** of a transition system is a unary predicate φ on states such that if $\varphi(s)$ holds and $s \rightarrow s'$ then $\varphi(s')$ holds.

Invariant principle

If a preserved invariant holds at a state s , then it holds for all states reachable from s .

Proof sketch: Let s' be a state reachable from s . We can show $\varphi(s')$ by induction on the length of the run from s to s' .

Invariant example: Modified Die Hard problem

Example

Given jugs of 3L and 6L, measure out exactly 4L.

- States: $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 6 \text{ and } 0 \leq j \leq 3\}$
- Transition relation: $(i, j) \rightarrow (0, j)$ etc.

Is $(4, 0)$ reachable from $(0, 0)$?

Invariant example: Modified Die Hard problem

Example

Given jugs of 3L and 6L, measure out exactly 4L.

- States: $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 6 \text{ and } 0 \leq j \leq 3\}$
- Transition relation: $(i, j) \rightarrow (0, j)$ etc.

Is $(4, 0)$ reachable from $(0, 0)$?

No. Consider $\varphi((i, j)) = (3|i) \wedge (3|j)$.

Summary

- Motivation
- Definitions
- The invariant principle
- Partial correctness and termination
- Input and output
- Finite automata

Partial correctness

Let $(S, \rightarrow, s_0, F)$ be a transition system with start state s_0 and final states F and a φ be a unary predicate on S . We say the system is **partially correct for φ** if $\varphi(s')$ holds for all states $s' \in F$ that are reachable from s_0 .

NB

Partial correctness is a safety property. It doesn't say whether the transition system will ever reach a final state.

Partial correctness example: Fast exponentiation

Example

Consider the following program in $\mathcal{L}$:

```
x := m;  
y := n;  
r := 1;  
while y > 0 do  
  if 2|y then  
    y := y/2  
  else  
    y := (y - 1)/2;  
    r := r * x  
  fi;  
  x := x * x  
od
```

Partial correctness example: Fast exponentiation

Example

- States: Functions from $\{m, n, x, y, r\}$ to $\mathbb{N}$
- Transitions:

Partial correctness example: Fast exponentiation

Example

- States: $(x, y, r) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$
- Transitions: Effect of each line of code?

Partial correctness example: Fast exponentiation

Example

- States: $(x, y, r) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$
- Transitions: Effect of each iteration of while loop
 - $(x, y, r) \rightarrow (x^2, y/2, r)$ if y is even
 - $(x, y, r) \rightarrow (x^2, (y-1)/2, rx)$ if y is odd

Partial correctness example: Fast exponentiation

Example

- States: $(x, y, r) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$
- Transitions: Effect of each iteration of while loop
 - $(x, y, r) \rightarrow (x^2, y/2, r)$ if y is even
 - $(x, y, r) \rightarrow (x^2, (y-1)/2, rx)$ if y is odd
- Start state: $(m, n, 1)$

Partial correctness example: Fast exponentiation

Example

- States: $(x, y, r) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$
- Transitions: Effect of each iteration of while loop
 - $(x, y, r) \rightarrow (x^2, y/2, r)$ if y is even
 - $(x, y, r) \rightarrow (x^2, (y-1)/2, rx)$ if y is odd
- Start state: $(m, n, 1)$
- Final states: $\{(x, 0, r) : x, r \in \mathbb{N}\}$

Partial correctness example: Fast exponentiation

Example

- States: $(x, y, r) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$
- Transitions: Effect of each iteration of while loop
 - $(x, y, r) \rightarrow (x^2, y/2, r)$ if y is even
 - $(x, y, r) \rightarrow (x^2, (y-1)/2, rx)$ if y is odd
- Start state: $(m, n, 1)$
- Final states: $\{(x, 0, r) : x, r \in \mathbb{N}\}$

Goal: Show partial correctness for $\varphi((x, y, r)) := (r = m^n)$

Partial correctness example: Fast exponentiation

Example

- States: $(x, y, r) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$
- Transitions: Effect of each iteration of while loop
 - $(x, y, r) \rightarrow (x^2, y/2, r)$ if y is even
 - $(x, y, r) \rightarrow (x^2, (y-1)/2, rx)$ if y is odd
- Start state: $(m, n, 1)$
- Final states: $\{(x, 0, r) : x, r \in \mathbb{N}\}$

Goal: Show partial correctness for $\varphi((x, y, r)) := (r = m^n)$

Show $\psi((x, y, r)) := (rx^y = m^n)$ is a preserved invariant...

Partial correctness example: Fast exponentiation

Example

- States: $(x, y, r) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$
- Transitions: Effect of each iteration of while loop
 - $(x, y, r) \rightarrow (x^2, y/2, r)$ if y is even
 - $(x, y, r) \rightarrow (x^2, (y-1)/2, rx)$ if y is odd
- Start state: $(m, n, 1)$
- Final states: $\{(x, 0, r) : x, r \in \mathbb{N}\}$

Goal: Show partial correctness for $\varphi((x, y, r)) := (r = m^n)$

Show $\psi((x, y, r)) := (rx^y = m^n)$ is a preserved invariant...

How can we show total correctness?

Total correctness = safety + liveness

A transition system $(S, \rightarrow)$ **terminates** from a state $s \in S$ if all runs from s have finite length.

A transition system is **totally correct for a unary predicate** φ , if it terminates (from s_0) and φ holds in the last state of every run.

Measure

In a transition system $(S, \rightarrow)$, a **measure** is a function $f : S \rightarrow \mathbb{N}$.

A measure is **strictly decreasing** if $s \rightarrow s'$ implies $f(s') < f(s)$.

Measure

In a transition system $(S, \rightarrow)$, a **measure** is a function $f : S \rightarrow \mathbb{N}$.

A measure is **strictly decreasing** if $s \rightarrow s'$ implies $f(s') < f(s)$.

Theorem

If f is a strictly decreasing measure, then the length of any run from s is at most $f(s)$.

Termination example: Fast exponentiation

Example

- States: $(x, y, r) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$
- Transitions: Effect of each iteration of while loop:
 - $(x, y, r) \rightarrow (x^2, y/2, r)$ if y is even
 - $(x, y, r) \rightarrow (x^2, (y-1)/2, rx)$ if y is odd

Measure:

Termination example: Fast exponentiation

Example

- States: $(x, y, r) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$
- Transitions: Effect of each iteration of while loop:
 - $(x, y, r) \rightarrow (x^2, y/2, r)$ if y is even
 - $(x, y, r) \rightarrow (x^2, (y-1)/2, rx)$ if y is odd

Measure: $f((x, y, r)) = y$

Summary

- Motivation
- Definitions
- The invariant principle
- Partial correctness and termination
- **Input and output**
- Finite automata

Interaction with the environment

We can model the system interacting with an external entity via inputs (Σ) and outputs (Γ) by using **labelled transitions**:
 $\rightarrow \subseteq S \times L \times S$ where $L = \Sigma \times \Gamma$

We'll look at categories of input/output transition systems:

Acceptors: Accept/reject a sequence of inputs

Transducers: Take a sequence of inputs and produce a sequence of outputs

Interaction with the environment

We can model the system interacting with an external entity via inputs (Σ) and outputs (Γ) by using **labelled transitions**:

$\rightarrow \subseteq S \times L \times S$ where $L = \Sigma \times \Gamma$

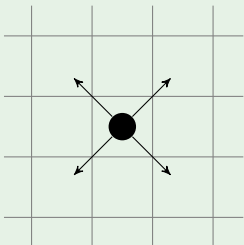
We'll look at categories of input/output transition systems:

Acceptors: Accept/reject a sequence of inputs (Relations)

Transducers: Take a sequence of inputs and produce a sequence of outputs (Functions)

Acceptor example: Diagonally moving robot

Example



$$S = \mathbb{Z} \times \mathbb{Z}$$

$$s_0 = (0, 0)$$

$$(x, y) \xrightarrow{NW} (x - 1, y + 1)$$

$$(x, y) \xrightarrow{NE} (x + 1, y + 1)$$

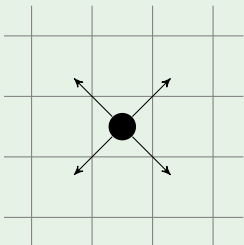
$$(x, y) \xrightarrow{SW} (x - 1, y - 1)$$

$$(x, y) \xrightarrow{SE} (x + 1, y - 1)$$

Accept if $(2, 2)$ reached

Acceptor example: Diagonally moving robot

Example



$$S = \mathbb{Z} \times \mathbb{Z}$$

$$s_0 = (0, 0)$$

$$(x, y) \xrightarrow{NW} (x - 1, y + 1)$$

$$(x, y) \xrightarrow{NE} (x + 1, y + 1)$$

$$(x, y) \xrightarrow{SW} (x - 1, y - 1)$$

$$(x, y) \xrightarrow{SE} (x + 1, y - 1)$$

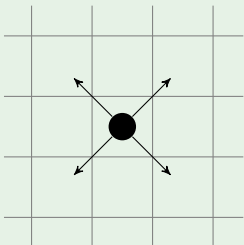
Accept if $(2, 2)$ reached

Accepted sequences:

NE, NE

Acceptor example: Diagonally moving robot

Example



$$S = \mathbb{Z} \times \mathbb{Z}$$

$$s_0 = (0, 0)$$

$$(x, y) \xrightarrow{NW} (x - 1, y + 1)$$

$$(x, y) \xrightarrow{NE} (x + 1, y + 1)$$

$$(x, y) \xrightarrow{SW} (x - 1, y - 1)$$

$$(x, y) \xrightarrow{SE} (x + 1, y - 1)$$

Accept if $(2, 2)$ reached

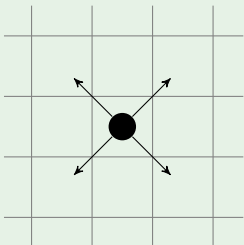
Accepted sequences:

NE, NE

NE, SE, NE, NW

Acceptor example: Diagonally moving robot

Example



$$S = \mathbb{Z} \times \mathbb{Z}$$

$$s_0 = (0, 0)$$

$$(x, y) \xrightarrow{NW} (x - 1, y + 1)$$

$$(x, y) \xrightarrow{NE} (x + 1, y + 1)$$

$$(x, y) \xrightarrow{SW} (x - 1, y - 1)$$

$$(x, y) \xrightarrow{SE} (x + 1, y - 1)$$

Accept if $(2, 2)$ reached

Accepted sequences:

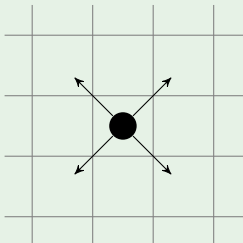
NE, NE

NE, SE, NE, NW

$NE, NE, NE, SW \dots$

Transducer example: Diagonally moving robot

Example



$$S = \mathbb{Z} \times \mathbb{Z}$$

$$s_0 = (0, 0)$$

$$(x, y) \xrightarrow{NW/x} (x - 1, y + 1)$$

$$(x, y) \xrightarrow{NE/x} (x + 1, y + 1)$$

$$(x, y) \xrightarrow{SW/x} (x - 1, y - 1)$$

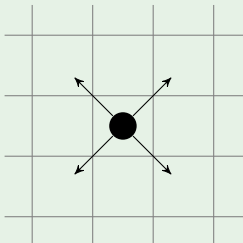
$$(x, y) \xrightarrow{SE/x} (x + 1, y - 1)$$

Input direction

Output x -coordinate

Transducer example: Diagonally moving robot

Example



$$S = \mathbb{Z} \times \mathbb{Z}$$

$$s_0 = (0, 0)$$

$$(x, y) \xrightarrow{NW/x} (x - 1, y + 1)$$

$$(x, y) \xrightarrow{NE/x} (x + 1, y + 1)$$

$$(x, y) \xrightarrow{SW/x} (x - 1, y - 1)$$

$$(x, y) \xrightarrow{SE/x} (x + 1, y - 1)$$

Input direction

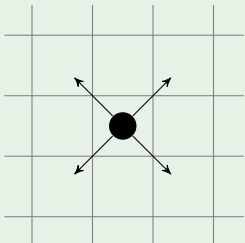
Output x -coordinate

Input: NE, SE, NE, NW

Output: 1, 2, 3, 2

Transducer example: Diagonally moving robot

Example



$$S = \mathbb{Z} \times \mathbb{Z}$$

$$s_0 = (0, 0)$$

$$(x, y) \xrightarrow{NW/y} (x - 1, y + 1)$$

$$(x, y) \xrightarrow{NE/y} (x + 1, y + 1)$$

$$(x, y) \xrightarrow{SW/y} (x - 1, y - 1)$$

$$(x, y) \xrightarrow{SE/y} (x + 1, y - 1)$$

Input direction

Output y -coordinate

Input: NE, SE, NE, NW

Output: $1, 0, 1, 2$

Acceptor example: Die Hard jug problem

Example

- $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- $s_0 = (0, 0)$
- $\rightarrow$ given by
 - $(i, j) \xrightarrow{E5} (0, j)$ [empty 5L jug]
 - $(i, j) \xrightarrow{E3} (i, 0)$ [empty 3L jug]
 - $(i, j) \xrightarrow{F5} (5, j)$ [fill 5L jug]
 - $(i, j) \xrightarrow{F3} (i, 3)$ [fill 3L jug]
 - $(i, j) \xrightarrow{E35} (i + j, 0)$ if $i + j \leq 5$ [empty 3L jug into 5L jug]
 - $(i, j) \xrightarrow{E53} (0, i + j)$ if $i + j \leq 3$ [empty 5L jug into 3L jug]
 - $(i, j) \xrightarrow{F53} (5, j - 5 + i)$ if $i + j \geq 5$ [fill 5L jug from 3L jug]
 - $(i, j) \xrightarrow{F35} (i - 3 + j, 3)$ if $i + j \geq 3$ [fill 3L jug from 5L jug]
- Accept if $(4, 0)$ is reached:

Acceptor example: Die Hard jug problem

Example

- $S = \{(i, j) \in \mathbb{N} \times \mathbb{N} : 0 \leq i \leq 5 \text{ and } 0 \leq j \leq 3\}$
- $s_0 = (0, 0)$
- $\rightarrow$ given by
 - $(i, j) \xrightarrow{E5} (0, j)$ [empty 5L jug]
 - $(i, j) \xrightarrow{E3} (i, 0)$ [empty 3L jug]
 - $(i, j) \xrightarrow{F5} (5, j)$ [fill 5L jug]
 - $(i, j) \xrightarrow{F3} (i, 3)$ [fill 3L jug]
 - $(i, j) \xrightarrow{E35} (i + j, 0)$ if $i + j \leq 5$ [empty 3L jug into 5L jug]
 - $(i, j) \xrightarrow{E53} (0, i + j)$ if $i + j \leq 3$ [empty 5L jug into 3L jug]
 - $(i, j) \xrightarrow{F53} (5, j - 5 + i)$ if $i + j \geq 5$ [fill 5L jug from 3L jug]
 - $(i, j) \xrightarrow{F35} (i - 3 + j, 3)$ if $i + j \geq 3$ [fill 3L jug from 5L jug]
- Accept if $(4, 0)$ is reached: e.g. F3, E35, F3, F53, E5, E35, F3, E35

ϵ -transitions

It can be useful to allow the system to transition without taking input or producing output. We use the special symbol ϵ to denote such transitions.

Formal definitions

An **acceptor** is a $\Sigma \cup \{\epsilon\}$ -labelled transition system

$A = (S, \rightarrow, \Sigma, s_0, F)$ with a start state $s_0 \in S$ and a set of final states $F \subseteq S$.

A **transducer** is a $(\Sigma \cup \{\epsilon\}) \times (\Gamma \cup \{\epsilon\})$ -labelled transition system

$T = (S, \rightarrow, \Sigma, s_0, F)$ with a start state $s_0 \in S$ and a set of final states $F \subseteq S$.

Summary

- Motivation
- Definitions
- The invariant principle
- Partial correctness and termination
- Input and output
- **Finite automata**

Finite state transition systems

State transition systems with a finite set of states are particularly useful in Computer Science.

Acceptors: Finite state automata

Transducers: Mealy machines

Notes (Week 7 Monday)

These are the notes I wrote during the lecture.

A relation $\mathcal{R}$ such that:

$$(s, s') \in \mathcal{R}$$

this means that, if we start from
the initial state s ,
a *possible* final result of
running the program is s'

We're using "state machines"
and "transition systems"
as synonyms

The relations we've been looking
at so far (e.g. in assignment 1)
are "big-step" relations:

$$(s, s') \in \mathcal{R}$$

one jump through the relation
takes us immediately from
initial to final state

In transition systems,
we consider "small-step" relations
one step through the relation
represents one computation step
which isn't necessarily the whole
computation

Bounded counter system:

$$S = \{0, 1, \dots, 99, \text{overflow}\}$$

Transition relation

$$\rightarrow \subseteq S \times S$$

$$\begin{aligned} \rightarrow = & \{(x, x+1) : x \leq 98\} \\ & \cup \{(99, \text{overflow})\} \\ & \cup \{(\text{overflow}, \text{overflow})\} \end{aligned}$$

Very often, when we model programs,

it's enough to track the values
of variables (let's say)

$$s \rightarrow s'$$

Also very often, you distinguish
between (internal) state
and externally observable events.
You'd put the events on the labels,
and the internal stuff in the states.

```
void hi() {
    printf("Hello world!");
}

void hello() {
    int x = 1;
    if(Math.Random() < .5)
        printf("Hello world!");
    else {
        x++;
        printf("Hello world!");
    }
}
```

We might model these as transition systems
where the labels are:

print_hello for the printf event
 τ or no label for unobservable events

The definition of determinism in an
unlabelled transition system is:

for every state s , there is at most one s'
such that $s \rightarrow s'$

For *labelled* transition systems:

for every state s and label α , there is at
most one s' such that

$$s \xrightarrow{\alpha} s'$$

```
(0,0) → (0,3) → (3,0) →
(3,3) → (5,1) → (0,1) → (1,0)
(1,3) → (4,0)
```

The counter transition system

Here are some runs (from $\emptyset$):

$\emptyset, 1, 2, 3, 4, 5$

$\emptyset$

$\emptyset, 1, 2, 3, \dots, 99, \text{overflow}, \text{overflow}, \dots$

^ the last one is the only maximal run

If R is a relation,

then R^* is the

*reflexive and transitive closure of R

Defined inductively by the following rules:

$(x, x) \in R^*$

If $(x, y) \in R$ and $(y, z) \in R^*$

then $(x, z) \in R^*$

If R is a transition relation, then

$x R^* y$ means

"in $\emptyset$ or more steps, we can get from x to y "

Safety and liveness properties are

not moral judgements

Safety "something bad will never happen"

Liveness "something good will happen"

Mostly ignore the words "bad" and "good"

I will come home tonight,

and find a chocolate cake in my fridge

I will come home tonight,

and find something super gross in my fridge

^ both of these are liveness properties

In order to have a useful system,

you want it to satisfy both safety

and liveness properties.

A system that does nothing

satisfies every safety property.

If you only specify a liveness property,

you can't rule out undesired behaviour

Q: How is this a safety property?

If the program terminates, then $y=x!$

A: it can be equivalently stated as follows:

The program will never reach a final state
where $y \neq x$!

Safety:

A safety property can be falsified
by a finite prefix of a behaviour

Liveness:

It is never too late to satisfy the
liveness property.

Any finite prefix of a behaviour
can be extended into one that satisfies
the liveness property.

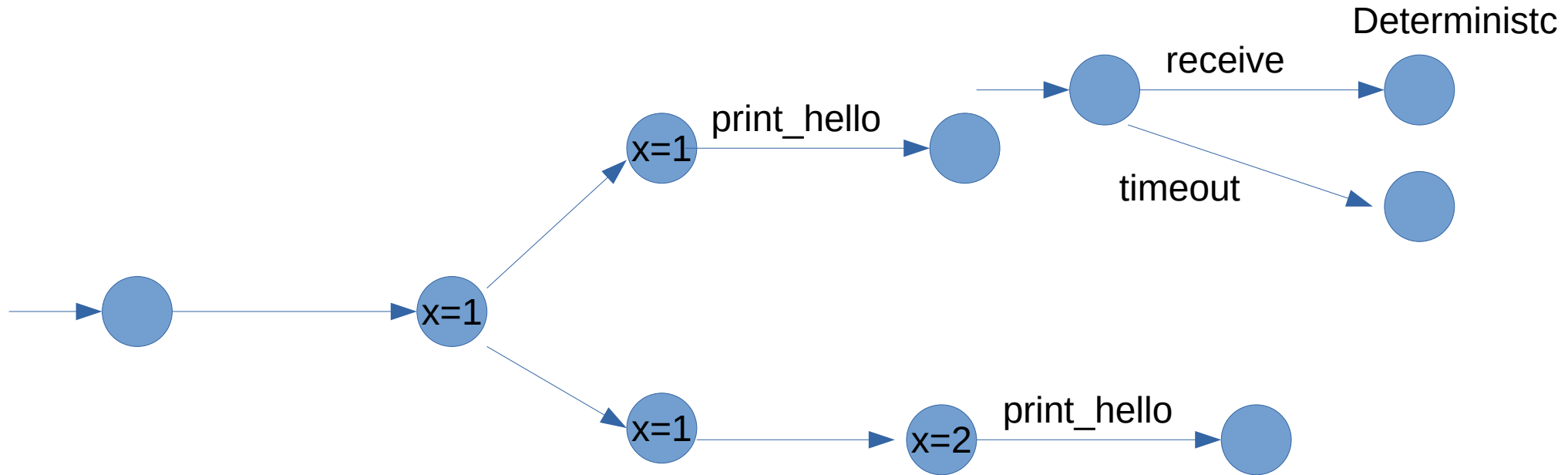
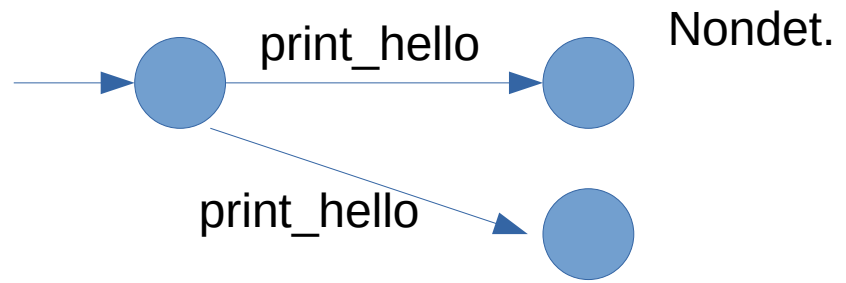
When I come home,
I'll drop on the couch and drink a beer

1. this is a liveness property

When I come home,
I'll **eventually** drop on the couch and drink a beer

2. this is a safety property

When I come home,
I'll **immediately** drop on the couch and drink a beer



COMP2111 Week 7

Term 1, 2023

Finite automata

Summary

- Recap
- Deterministic Finite Automata
- Non-deterministic Finite Automata
- Regular languages
- Regular expressions

Summary

- Recap
- Deterministic Finite Automata
- Non-deterministic Finite Automata
- Regular languages
- Regular expressions

Transition systems

A **transition system** (or **state machine**) is a pair $(S, \rightarrow)$ where S is a set and $\rightarrow \subseteq S \times S$ is a binary relation.

NB

S is not necessarily finite.

Transition systems may have:

- L -labelled transitions: $\rightarrow \subseteq S \times L \times S$
- A start/initial state $s_0 \in S$
- A set of final states $F \subseteq S$ (where runs terminate)

If $\rightarrow$ is a partial function (from $S \times L$ to S), the transition system is **deterministic**. If $\rightarrow$ is a function, the transition system is **total**.

Reachability and Runs

A state s' is **reachable** from a state s if $(s, s') \in \rightarrow^*$ (the reflexive and transitive closure of $\rightarrow$).

A **run** from a state s is a sequence $s_1, s_2, \dots$ such that $s_1 = s$ and $s_i \rightarrow s_{i+1}$ for all i .

NB

In a non-deterministic transition system there may be many (or no) runs from a state. In an unlabelled deterministic transition system there is exactly one maximal run from every state.

Acceptors and Transducers

An **acceptor** is a transition system with:

- (input-)labelled transitions
- a start/initial state
- a set of final states

A **transducer** is a transition system with:

- (input & output-)labelled transitions
- a start/initial state

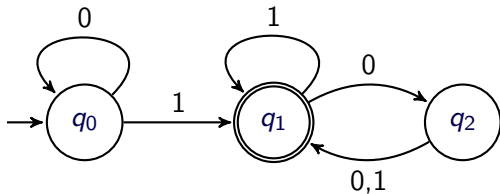
NB

Acceptors accept/reject sequences of inputs. Transducers map sequences of inputs to sequences of outputs.

Summary

- Recap
- **Deterministic Finite Automata**
- Non-deterministic Finite Automata
- Regular languages
- Regular expressions

Deterministic Finite Automata

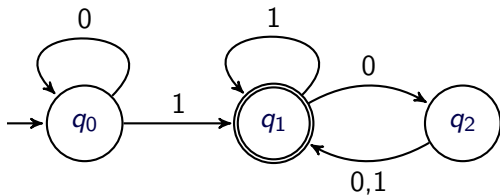


A **deterministic finite automaton (DFA)** is a total, finite state acceptor.

DFAs represent “computation with finite memory”

DFAs are simple, easy to work with and show up all over the place.

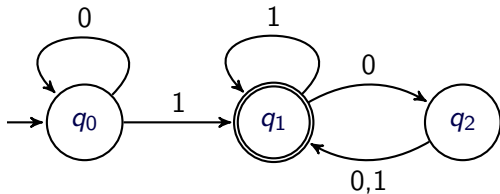
Deterministic Finite Automata



Formally, a **deterministic finite automaton (DFA)** is a tuple $(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states
- Σ is the input alphabet
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of final/accepting states

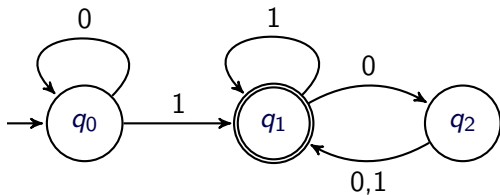
Deterministic Finite Automata



Formally, a **deterministic finite automaton (DFA)** is a tuple $(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states: $Q = \{q_0, q_1, q_2\}$
- Σ is the input alphabet
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of final/accepting states

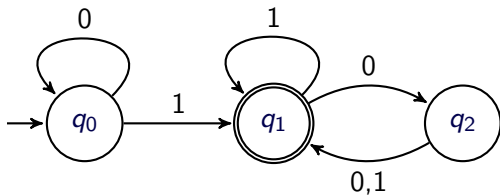
Deterministic Finite Automata



Formally, a **deterministic finite automaton (DFA)** is a tuple $(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states: $Q = \{q_0, q_1, q_2\}$
- Σ is the input alphabet: $\Sigma = \{0, 1\}$
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of final/accepting states

Deterministic Finite Automata



$$\delta(q_0, 0) = q_0$$

$$\delta(q_0, 1) = q_1$$

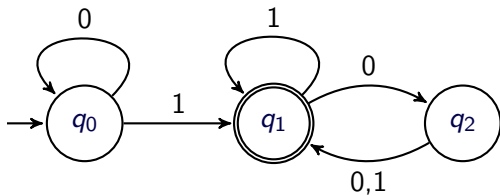
$$\delta(q_1, 0) = q_2$$

$$\delta(q_1, 1) = q_1$$

$$\delta(q_2, 0) = q_1$$

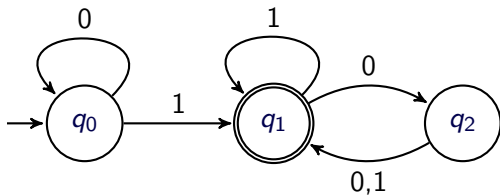
$$\delta(q_2, 1) = q_1$$

Deterministic Finite Automata



δ	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_1	q_1

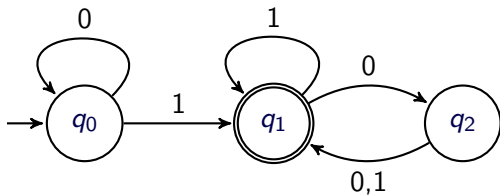
Deterministic Finite Automata



Formally, a **deterministic finite automaton (DFA)** is a tuple $(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states: $Q = \{q_0, q_1, q_2\}$
- Σ is the input alphabet: $\Sigma = \{0, 1\}$
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of final/accepting states: $F = \{q_1\}$

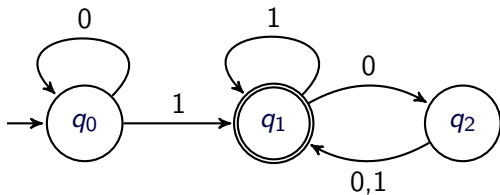
Language of a DFA



A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

Informally: A word defines a run in the DFA and the word is accepted if the run ends in a final state.

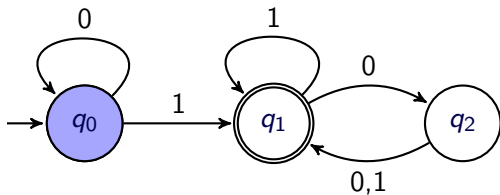
Language of a DFA



w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

Language of a DFA

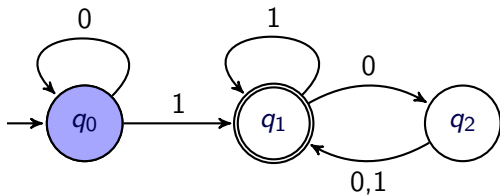


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0

Language of a DFA

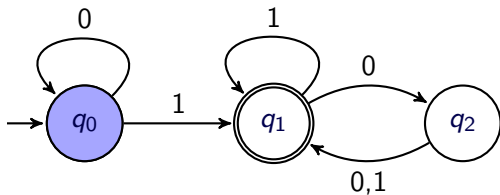


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w

Language of a DFA

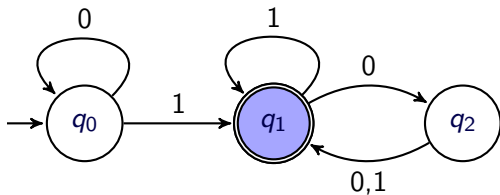


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ

Language of a DFA

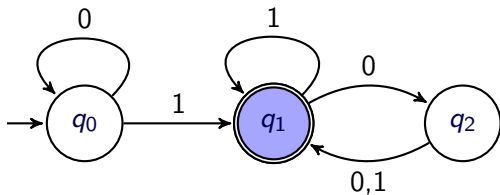


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ

Language of a DFA

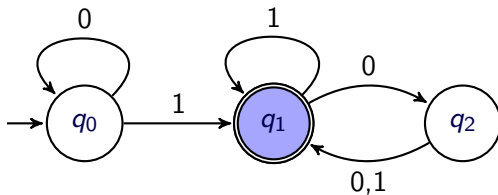


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ
 - Move to the next symbol in w

Language of a DFA

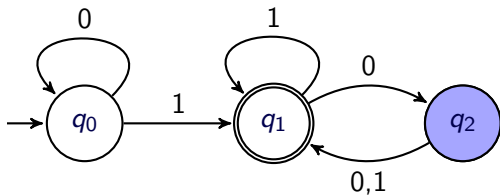


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ
 - Move to the next symbol in w

Language of a DFA

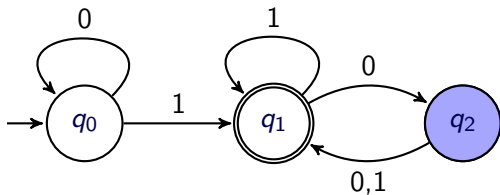


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ
 - Move to the next symbol in w

Language of a DFA

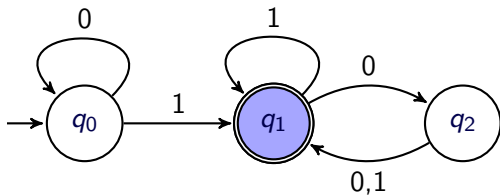


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ
 - Move to the next symbol in w

Language of a DFA

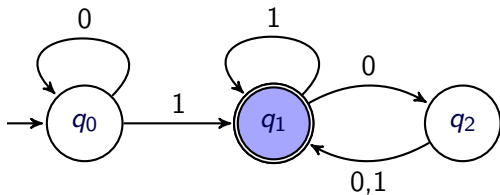


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ
 - Move to the next symbol in w

Language of a DFA

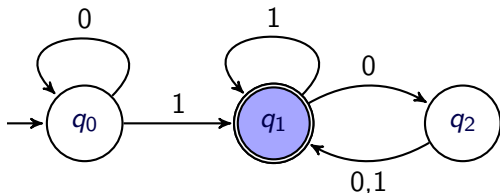


w : 100**1**

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ
 - Move to the next symbol in w

Language of a DFA

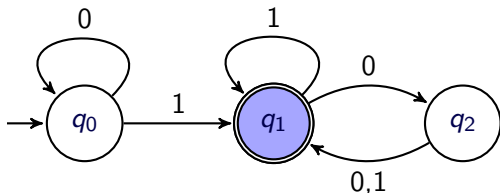


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ
 - Move to the next symbol in w

Language of a DFA

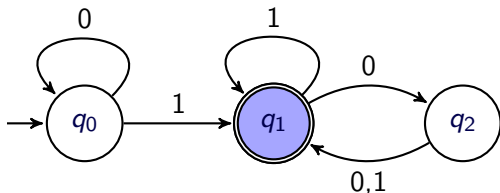


w : 1001

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ
 - Move to the next symbol in w
- Accept if the process ends in a final state, otherwise reject.

Language of a DFA

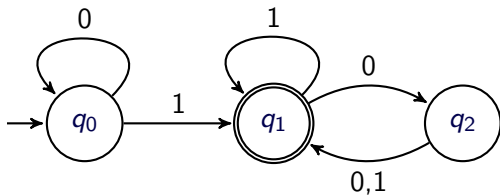


w : 1001 ✓

A DFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

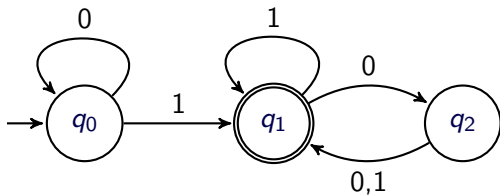
- Start in state q_0
- Take the first symbol of w
- Repeat the following until there are no symbols left:
 - Based on the current state and current input symbol, transition to the appropriate state determined by δ
 - Move to the next symbol in w
- Accept if the process ends in a final state, otherwise reject.

Language of a DFA



For a DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, the **language of $\mathcal{A}$** , $L(\mathcal{A})$, is the set of words from Σ^* which are accepted by $\mathcal{A}$

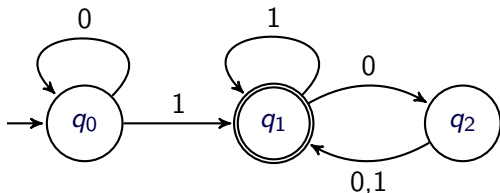
Language of a DFA



$$L(\mathcal{A}) = \{1, 01, 11, 101, \dots\}$$

For a DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, the **language of $\mathcal{A}$** , $L(\mathcal{A})$, is the set of words from Σ^* which are accepted by $\mathcal{A}$

Language of a DFA



$$L(\mathcal{A}) = \{1, 01, 11, 101, \dots\}$$

For a DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, the **language of** $\mathcal{A}$, $L(\mathcal{A})$, is the set of words from Σ^* which are accepted by $\mathcal{A}$

A language $L \subseteq \Sigma^*$ is **regular** if there is some DFA $\mathcal{A}$ such that $L = L(\mathcal{A})$

Language of a DFA: formally

Given a DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ we define $L_{\mathcal{A}} : Q \rightarrow \Sigma^*$ inductively as follows:

- If $q \in F$ then $\lambda \in L_{\mathcal{A}}(q)$
- If $q \xrightarrow{a} q'$ and $w \in L_{\mathcal{A}}(q')$ then $aw \in L_{\mathcal{A}}(q)$

Language of a DFA: formally

Given a DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ we define $L_{\mathcal{A}} : Q \rightarrow \Sigma^*$ inductively as follows:

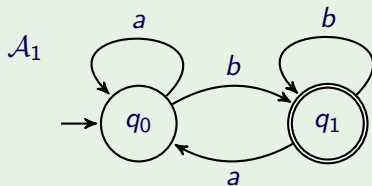
- If $q \in F$ then $\lambda \in L_{\mathcal{A}}(q)$
- If $q \xrightarrow{a} q'$ and $w \in L_{\mathcal{A}}(q')$ then $aw \in L_{\mathcal{A}}(q)$

We then define

$$L(\mathcal{A}) = L_{\mathcal{A}}(q_0)$$

Examples

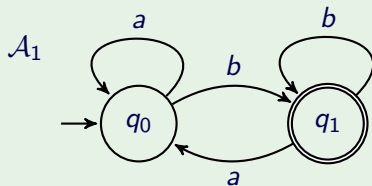
Example



$$L(\mathcal{A}_1) = ?$$

Examples

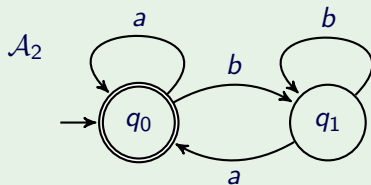
Example



$$L(\mathcal{A}_1) = \{w \in \{a, b\}^* : w \text{ ends with } b\}$$

Examples

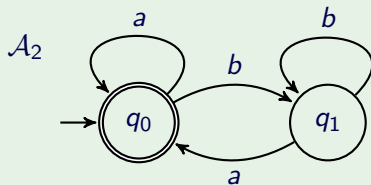
Example



$$L(\mathcal{A}_2) = ?$$

Examples

Example



$$L(\mathcal{A}_2) = \{w \in \{a, b\}^* : w \text{ ends with } a\} \cup \{\lambda\}$$

Examples

Example

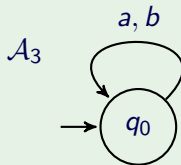
Find $\mathcal{A}_3$ such that $L(\mathcal{A}_3) = \emptyset$

Find $\mathcal{A}_4$ such that $L(\mathcal{A}_4) = \{\lambda\}$

Examples

Example

Find $\mathcal{A}_3$ such that $L(\mathcal{A}_3) = \emptyset$

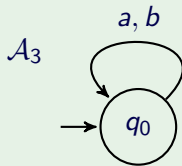


Find $\mathcal{A}_4$ such that $L(\mathcal{A}_4) = \{\lambda\}$

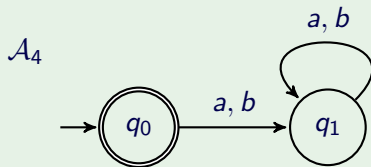
Examples

Example

Find $\mathcal{A}_3$ such that $L(\mathcal{A}_3) = \emptyset$



Find $\mathcal{A}_4$ such that $L(\mathcal{A}_4) = \{\lambda\}$



Examples

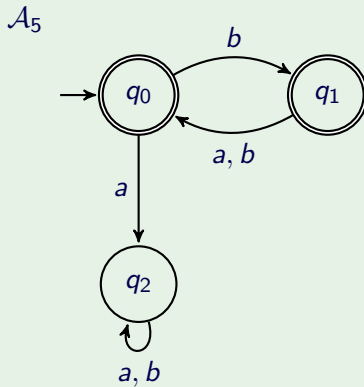
Example

Find $\mathcal{A}_5$ such that $L(\mathcal{A}_5) = \{w \in \{a, b\}^* : \text{every odd symbol is } b\}$

Examples

Example

Find $\mathcal{A}_5$ such that $L(\mathcal{A}_5) = \{w \in \{a, b\}^* : \text{every odd symbol is } b\}$



Examples

Example

Find $\mathcal{A}_6$ such that

$$L(\mathcal{A}_6) = \{w \in \{a, b\}^* : \text{second-last symbol is } b\}$$

Examples

Example

Find $\mathcal{A}_6$ such that

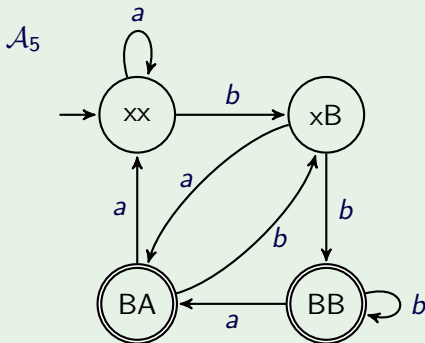
$$L(\mathcal{A}_6) = \{w \in \{a, b\}^* : \text{second-last symbol is } b\}$$

Examples

Example

Find $\mathcal{A}_6$ such that

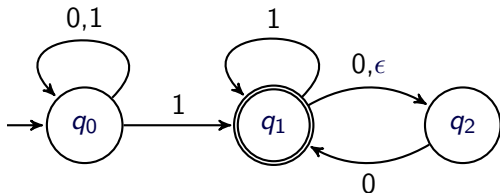
$$L(\mathcal{A}_6) = \{w \in \{a, b\}^* : \text{second-last symbol is } b\}$$



Summary

- Recap
- Deterministic Finite Automata
- **Non-deterministic Finite Automata**
- Regular languages
- Regular expressions

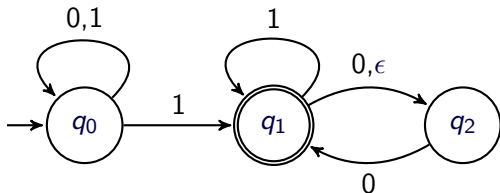
Non-deterministic Finite Automata



A **non-deterministic finite automaton (NFA)** is a non-deterministic, finite state acceptor.

More general than DFAs: A DFA is an NFA

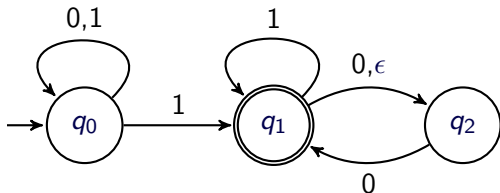
Non-deterministic Finite Automata



Formally, a **non-deterministic finite automaton (NFA)** is a tuple $(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states
- Σ is the input alphabet
- $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ is the transition relation
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of final/accepting states

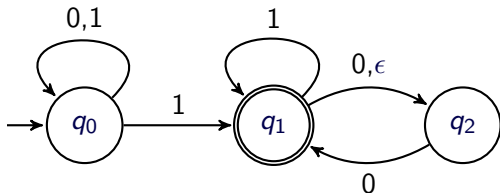
Non-deterministic Finite Automata



Formally, a **non-deterministic finite automaton (NFA)** is a tuple $(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states: $Q = \{q_0, q_1, q_2\}$
- Σ is the input alphabet
- $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ is the transition relation
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of final/accepting states

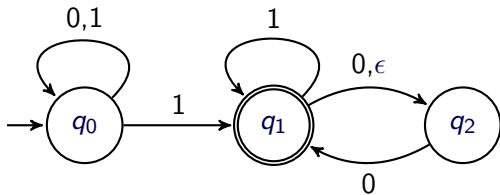
Non-deterministic Finite Automata



Formally, a **non-deterministic finite automaton (NFA)** is a tuple $(Q, \Sigma, \delta, q_0, F)$ where

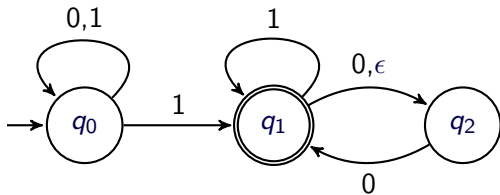
- Q is a finite set of states: $Q = \{q_0, q_1, q_2\}$
- Σ is the input alphabet: $\Sigma = \{0, 1\}$
- $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ is the transition relation
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of final/accepting states

Non-deterministic Finite Automata



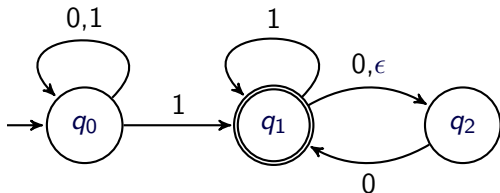
$$\delta = \left\{ \begin{array}{lll} (q_0, 0, q_0), & (q_0, 1, q_0), & (q_0, 1, q_1), \\ (q_1, \epsilon, q_2), & (q_1, 0, q_2), & (q_1, 1, q_1), \\ & (q_2, 0, q_1) \end{array} \right\}$$

Non-deterministic Finite Automata



δ	ϵ	0	1
q_0	$\emptyset$	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	$\{q_2\}$	$\{q_1\}$
q_2	$\emptyset$	$\{q_1\}$	$\emptyset$

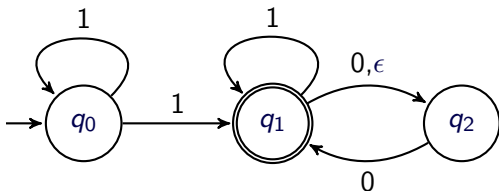
Non-deterministic Finite Automata



Formally, a **non-deterministic finite automaton (NFA)** is a tuple $(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states: $Q = \{q_0, q_1, q_2\}$
- Σ is the input alphabet: $\Sigma = \{0, 1\}$
- $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ is the transition relation
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of final/accepting states: $F = \{q_1\}$

Language of an NFA



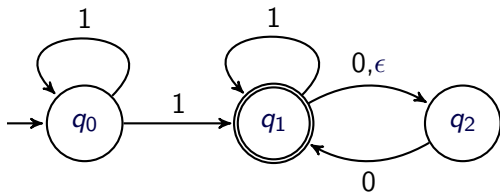
An NFA accepts a sequence of symbols from Σ – i.e. elements of Σ^*

Informally: A word defines several runs in the NFA and the word is accepted if **at least one run** ends in a final state.

Note 1: Runs can end prematurely (these don't count)

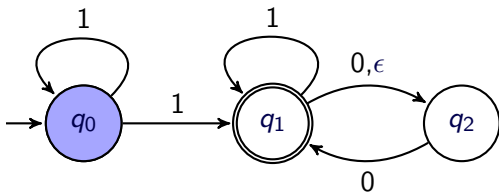
Note 2: An NFA will always “choose wisely”

Language of an NFA



w : 1000

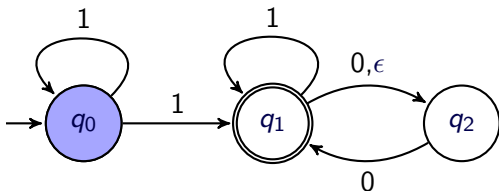
Language of an NFA



w : 1000

- Colour the state q_0

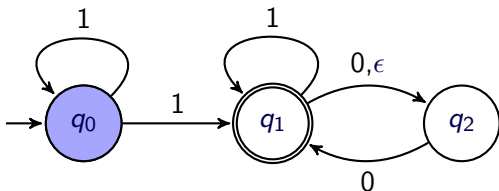
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

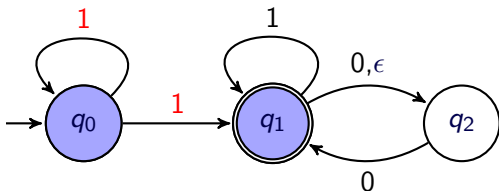
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

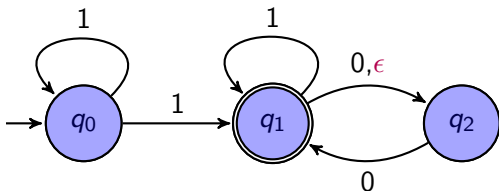
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

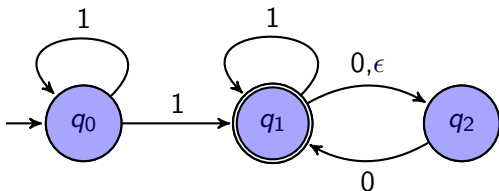
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

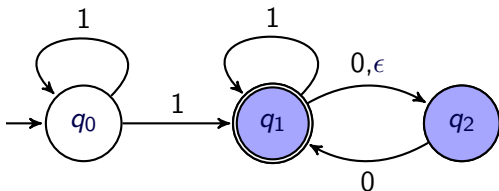
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

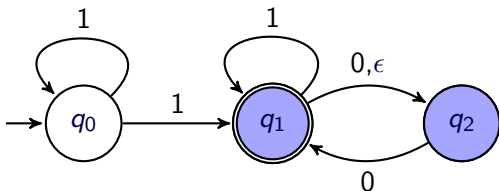
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

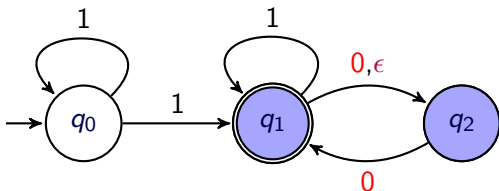
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

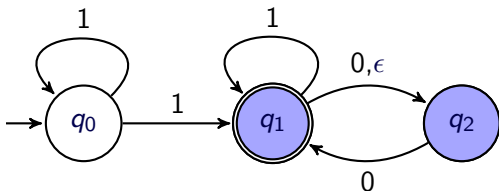
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

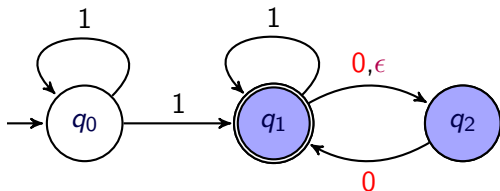
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

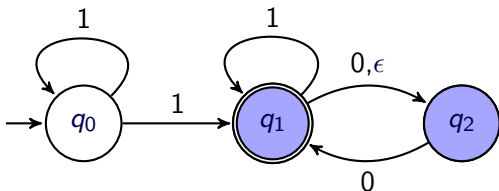
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

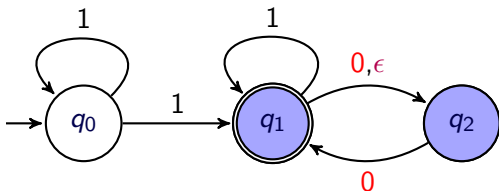
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

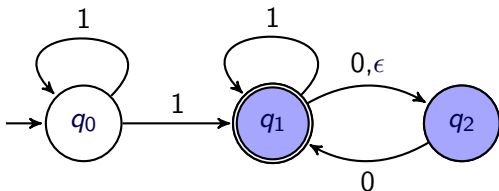
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.

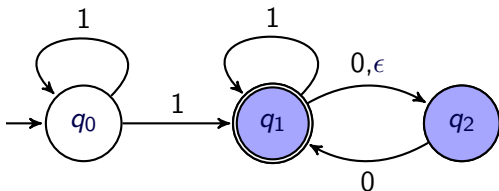
Language of an NFA



w : 1000

- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.
- Accept if there are no symbols left and a final state is coloured; otherwise, reject.

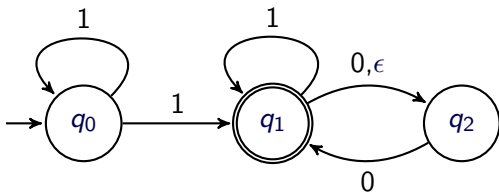
Language of an NFA



w : 1000 ✓

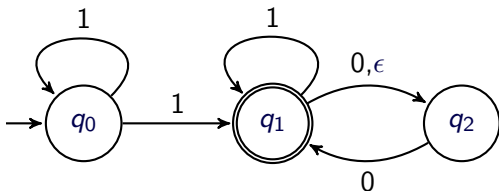
- Colour the state q_0
- Colour states reachable by one or more ϵ transitions from q_0 .
- For each symbol c of w :
 - Colour all states reachable by a c -transition followed by 0 or more ϵ transitions from the coloured states, and uncolour all other states.
- Accept if there are no symbols left and a final state is coloured; otherwise, reject.

Language of an NFA



For an NFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, the **language of $\mathcal{A}$** , $L(\mathcal{A})$, is the set of words from Σ^* which are accepted by $\mathcal{A}$

Language of an NFA



$$L(\mathcal{A}) = \{1, 10, 11, 101, \dots\}$$

For an NFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, the **language of $\mathcal{A}$** , $L(\mathcal{A})$, is the set of words from Σ^* which are accepted by $\mathcal{A}$

Language of an NFA: formally

Given an NFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ we define $L_{\mathcal{A}} : Q \rightarrow \Sigma^*$ inductively as follows:

- If $q \in F$ then $\lambda \in L_{\mathcal{A}}(q)$
- If $q \xrightarrow{a} q'$ and $w \in L_{\mathcal{A}}(q')$ then $aw \in L_{\mathcal{A}}(q)$
- If $q \xrightarrow{\epsilon} q'$ and $w \in L_{\mathcal{A}}(q')$ then $w \in L_{\mathcal{A}}(q)$

Language of an NFA: formally

Given an NFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ we define $L_{\mathcal{A}} : Q \rightarrow \Sigma^*$ inductively as follows:

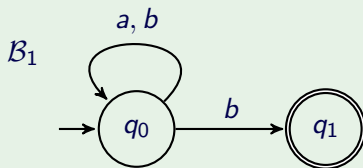
- If $q \in F$ then $\lambda \in L_{\mathcal{A}}(q)$
- If $q \xrightarrow{a} q'$ and $w \in L_{\mathcal{A}}(q')$ then $aw \in L_{\mathcal{A}}(q)$
- If $q \xrightarrow{\epsilon} q'$ and $w \in L_{\mathcal{A}}(q')$ then $w \in L_{\mathcal{A}}(q)$

We then define

$$L(\mathcal{A}) = L_{\mathcal{A}}(q_0)$$

Examples

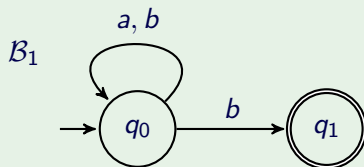
Example



$$L(\mathcal{B}_1) = ?$$

Examples

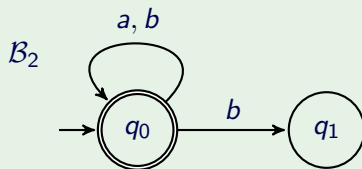
Example



$$L(\mathcal{B}_1) = \{w \in \{a, b\}^* : w \text{ ends with } b\}$$

Examples

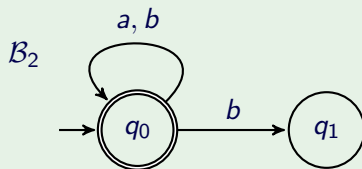
Example



$$L(\mathcal{B}_2) = ?$$

Examples

Example



$$L(\mathcal{B}_2) = \{a, b\}^*$$

Examples

Example

Find $\mathcal{B}_3$ such that $L(\mathcal{B}_3) = \emptyset$

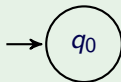
Find $\mathcal{B}_4$ such that $L(\mathcal{B}_4) = \{\lambda\}$

Examples

Example

Find $\mathcal{B}_3$ such that $L(\mathcal{B}_3) = \emptyset$

$\mathcal{B}_3$



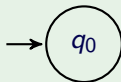
Find $\mathcal{B}_4$ such that $L(\mathcal{B}_4) = \{\lambda\}$

Examples

Example

Find $\mathcal{B}_3$ such that $L(\mathcal{B}_3) = \emptyset$

$\mathcal{B}_3$



Find $\mathcal{B}_4$ such that $L(\mathcal{B}_4) = \{\lambda\}$

$\mathcal{B}_4$



Examples

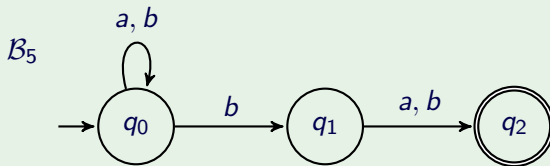
Example

Find $\mathcal{B}_5$ such that $L(\mathcal{B}_5) = \{w \in \{a, b\}^* : \text{second-last symbol is } b\}$

Examples

Example

Find $\mathcal{B}_5$ such that $L(\mathcal{B}_5) = \{w \in \{a, b\}^* : \text{second-last symbol is } b\}$



NFAs vs DFAs

Clearly for any DFA $\mathcal{A}$ there is an NFA $\mathcal{B}$ such that $L(\mathcal{A}) = L(\mathcal{B})$.

NFAs vs DFAs

Clearly for any DFA $\mathcal{A}$ there is an NFA $\mathcal{B}$ such that $L(\mathcal{A}) = L(\mathcal{B})$.

Theorem

For any NFA $\mathcal{B}$ there is a DFA $\mathcal{A}$ such that $L(\mathcal{A}) = L(\mathcal{B})$.

NFAs vs DFAs

Clearly for any DFA $\mathcal{A}$ there is an NFA $\mathcal{B}$ such that $L(\mathcal{A}) = L(\mathcal{B})$.

Theorem

For any NFA $\mathcal{B}$ there is a DFA $\mathcal{A}$ such that $L(\mathcal{A}) = L(\mathcal{B})$.

Proof sketch: (Subset construction)

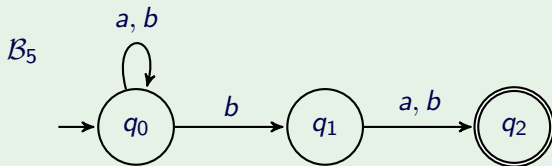
Given $\mathcal{B} = (Q, \Sigma, \delta, q_0, F)$, construct $\mathcal{A} = (Q', \Sigma, \delta', q'_0, F')$ as follows:

- $Q' = \text{Pow}(Q)$
- $\delta'(X, a) = \{q' \in Q : \exists q \in X, q'' \in Q. q \xrightarrow{a} q'' \xrightarrow{\epsilon^*} q'\}$
- $q'_0 = \{q' \in Q : q_0 \xrightarrow{\epsilon^*} q'\}$
- $F' = \{X \in Q' : X \cap F \neq \emptyset\}$

Intuitively: $\mathcal{A}$ keeps track of all the possible states $\mathcal{B}$ could be in after seeing a given sequence of symbols.

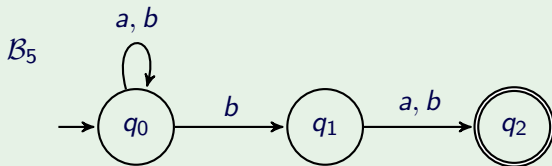
NFA to DFA Example

Example



NFA to DFA Example

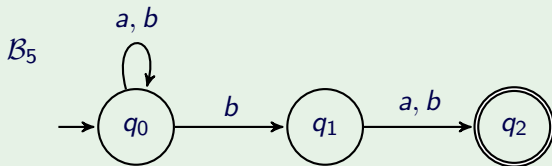
Example



δ'	a	b
$\emptyset$		
$\{q_0\}$		
$\{q_1\}$		
$\{q_2\}$		
$\{q_0, q_1\}$		
$\{q_0, q_2\}$		
$\{q_1, q_2\}$		
$\{q_0, q_1, q_2\}$		

NFA to DFA Example

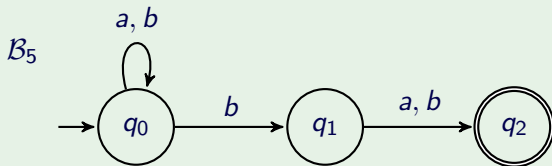
Example



δ'	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$		
$\{q_1\}$		
$\{q_2\}$		
$\{q_0, q_1\}$		
$\{q_0, q_2\}$		
$\{q_1, q_2\}$		
$\{q_0, q_1, q_2\}$		

NFA to DFA Example

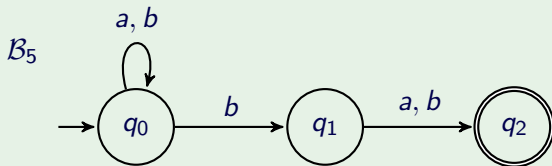
Example



δ'	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$		
$\{q_2\}$		
$\{q_0, q_1\}$		
$\{q_0, q_2\}$		
$\{q_1, q_2\}$		
$\{q_0, q_1, q_2\}$		

NFA to DFA Example

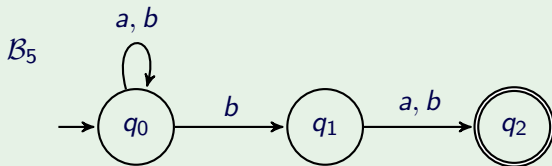
Example



δ'	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$\{q_2\}$		
$\{q_0, q_1\}$		
$\{q_0, q_2\}$		
$\{q_1, q_2\}$		
$\{q_0, q_1, q_2\}$		

NFA to DFA Example

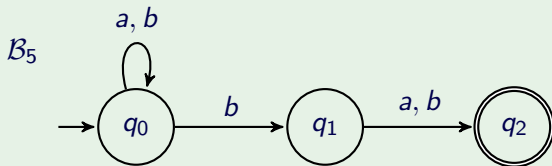
Example



δ'	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$		
$\{q_0, q_2\}$		
$\{q_1, q_2\}$		
$\{q_0, q_1, q_2\}$		

NFA to DFA Example

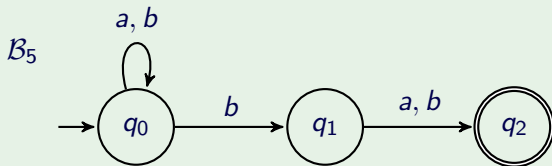
Example



δ'	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_2\}$		
$\{q_1, q_2\}$		
$\{q_0, q_1, q_2\}$		

NFA to DFA Example

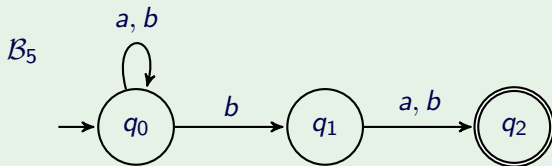
Example



δ'	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1, q_2\}$		
$\{q_0, q_1, q_2\}$		

NFA to DFA Example

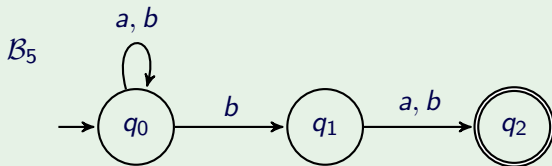
Example



δ'	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1, q_2\}$	$\{q_2\}$	$\{q_2\}$
$\{q_0, q_1, q_2\}$		

NFA to DFA Example

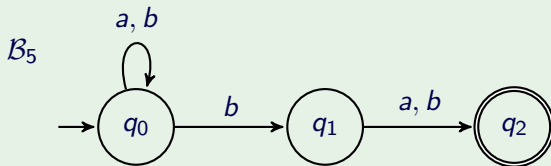
Example



δ'	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1, q_2\}$	$\{q_2\}$	$\{q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$

NFA to DFA Example

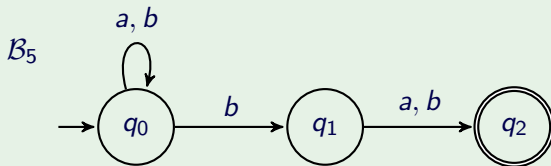
Example



δ'		a	b
$\emptyset$	A	A	A
$\{q_0\}$	B	B	E
$\{q_1\}$	C	D	D
$\{q_2\}$	D	A	A
$\{q_0, q_1\}$	E	F	H
$\{q_0, q_2\}$	F	B	E
$\{q_1, q_2\}$	G	D	D
$\{q_0, q_1, q_2\}$	H	F	H

NFA to DFA Example

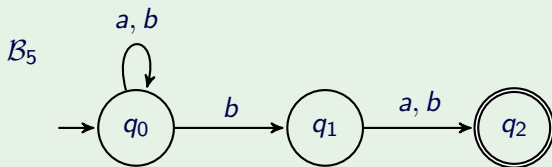
Example



δ'		<i>a</i>	<i>b</i>
$\emptyset$	<i>A</i>	<i>A</i>	<i>A</i>
$\{q_0\}$	<i>B</i>	<i>B</i>	<i>E</i>
$\{q_1\}$	<i>C</i>	<i>D</i>	<i>D</i>
$\{q_2\}$	<i>D</i>	<i>A</i>	<i>A</i>
$\{q_0, q_1\}$	<i>E</i>	<i>F</i>	<i>H</i>
$\{q_0, q_2\}$	<i>F</i>	<i>B</i>	<i>E</i>
$\{q_1, q_2\}$	<i>G</i>	<i>D</i>	<i>D</i>
$\{q_0, q_1, q_2\}$	<i>H</i>	<i>F</i>	<i>H</i>

NFA to DFA Example

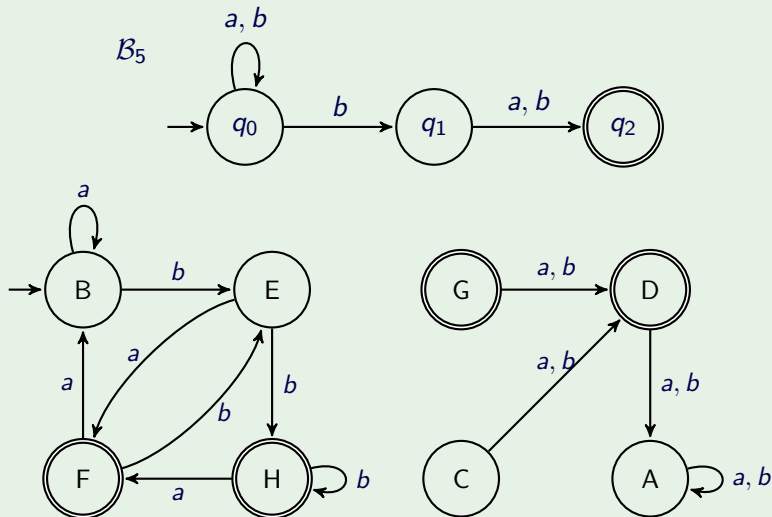
Example



δ'		<i>a</i>	<i>b</i>
$\emptyset$	<i>A</i>	<i>A</i>	<i>A</i>
$\{q_0\}$	<i>B</i>	<i>B</i>	<i>E</i>
$\{q_1\}$	<i>C</i>	<i>D</i>	<i>D</i>
$\{q_2\}$	<i>D</i>	<i>A</i>	<i>A</i>
$\{q_0, q_1\}$	<i>E</i>	<i>F</i>	<i>H</i>
$\{q_0, q_2\}$	<i>F</i>	<i>B</i>	<i>E</i>
$\{q_1, q_2\}$	<i>G</i>	<i>D</i>	<i>D</i>
$\{q_0, q_1, q_2\}$	<i>H</i>	<i>F</i>	<i>H</i>

NFA to DFA Example

Example



NFAs vs DFAs

Theorem

- For any NFA with n states there exists a DFA with at most 2^n states that accepts the same language
- There exist NFAs with n states such that the smallest DFA that accepts the same language has at least 2^n states.

Summary

- Recap
- Deterministic Finite Automata
- Non-deterministic Finite Automata
- Regular languages
- Regular expressions

Regular languages

A language $L \subseteq \Sigma^*$ is **regular** if there is some DFA $\mathcal{A}$ such that $L = L(\mathcal{A})$

Regular languages

A language $L \subseteq \Sigma^*$ is **regular** if there is some DFA $\mathcal{A}$ such that $L = L(\mathcal{A})$

Equivalently, there is some NFA $\mathcal{B}$ such that $L = L(\mathcal{B})$

Non-regular languages

Are there languages which are not regular?

Non-regular languages

Are there languages which are not regular? Yes

“Simple” counting argument: there are uncountably many languages, and only countably many DFAs

Non-regular languages

Are there languages which are not regular? Yes

“Simple” counting argument: there are uncountably many languages, and only countably many DFAs

An example of a non-regular language: $\{0^n1^n : n \in \mathbb{N}\}$

Intuitively: need arbitrary large memory to “remember” the number of 0's

Complementation

Theorem

If L is a regular language then $L^c = \Sigma^ \setminus L$ is a regular language.*

Proof:

- Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a DFA such that $L(\mathcal{A}) = L$
- Consider $\mathcal{A}' = (Q, \Sigma, \delta, q_0, Q \setminus F)$
- For any word $w \in \Sigma^*$, the corresponding run in $\mathcal{A}$ is unique, so:
 - If $w \in L(\mathcal{A})$ then $w \notin L(\mathcal{A}')$, and
 - If $w \notin L(\mathcal{A})$ then $w \in L(\mathcal{A}')$,
- Therefore $L(\mathcal{A}') = \Sigma^* \setminus L(\mathcal{A}) = L^c$

NB

This argument does not apply for NFAs (see $\mathcal{B}_1$ and $\mathcal{B}_2$)

Union

Theorem

If L_1 and L_2 are regular languages, then $L_1 \cup L_2$ is regular.

Proof:

- Let $\mathcal{B}_1$ and $\mathcal{B}_2$ be NFAs such that $L(\mathcal{B}_1) = L_1$ and $L(\mathcal{B}_2) = L_2$
- Construct an NFA $\mathcal{B}$ by having a new start state with ϵ -transitions to the start states of $\mathcal{B}_1$ and $\mathcal{B}_2$
- Consider $w \in L_1 \cup L_2$:
 - If $w \in L_1$ then there is a run in $\mathcal{B}_1$, and hence in $\mathcal{B}$, which ends in a final state
 - If $w \in L_2$ then there is a run in $\mathcal{B}_2$, and hence in $\mathcal{B}$, which ends in a final state
 - In either case $w \in L(\mathcal{B})$
- Conversely, any accepting run in $\mathcal{B}$ will be either an accepting run in $\mathcal{B}_1$ or in $\mathcal{B}_2$; so if $w \in L(\mathcal{B})$ then $w \in L_1 \cup L_2$

Intersection

Theorem

If L_1 and L_2 are regular languages, then $L_1 \cap L_2$ is regular.

Proof:

Intersection

Theorem

If L_1 and L_2 are regular languages, then $L_1 \cap L_2$ is regular.

Proof:

$$L_1 \cap L_2 = (L_1^c \cup L_2^c)^c$$

Concatenation

Recall for languages X and Y : $X \cdot Y = \{xy : x \in X, y \in Y\}$

Theorem

If L_1 and L_2 are regular languages, then $L_1 \cdot L_2$ is regular.

Proof:

- Let $\mathcal{B}_1$ and $\mathcal{B}_2$ be NFAs such that $L(\mathcal{B}_1) = L_1$ and $L(\mathcal{B}_2) = L_2$
- Construct an NFA $\mathcal{B}$ by adding ϵ -transitions from the final states of $\mathcal{B}_1$ to the start state of $\mathcal{B}_2$. Let the start state of $\mathcal{B}$ be the start state of $\mathcal{B}_1$; and let the final states of $\mathcal{B}$ be the final states of $\mathcal{B}_2$.
- Any word in $L_1 \cdot L_2$ can be written as wv with $w \in L_1$ and $v \in L_2$. w has an accepting run in $\mathcal{B}_1$ and v has an accepting run in $\mathcal{B}_2$, so wv has an accepting run in $\mathcal{B}$.
- Conversely, any word w with an accepting run in $\mathcal{B}$ can be broken up into an accepting run in $\mathcal{B}_1$ followed by an accepting run in $\mathcal{B}_2$. Thus w can be broken up into two words $w = xy$ where $x \in L_1$ and $y \in L_2$.

Kleene star

Recall for a language X :

$X^* = \{w : w \text{ is the concatenation of 0 or more words in } X\}$

Theorem

If L is regular languages, then L^ is regular.*

Proof:

- Let $\mathcal{B}$ be an NFA such that $L(\mathcal{B}) = L$
- Construct an NFA $\mathcal{B}'$ by:
 - creating a new start state which is accepting;
 - adding an ϵ -transition from the new start state to the start state of $\mathcal{B}$
 - adding ϵ -transitions from the final states of $\mathcal{B}$ to the new start state.
- Similar arguments as before show that $L(\mathcal{B}') = L(\mathcal{B})^*$

Regular operations

Concatenation, union, and Kleene star are collectively known as the **regular operations**.

Summary

- Recap
- Deterministic Finite Automata
- Non-deterministic Finite Automata
- Regular languages
- Regular expressions

Regular expressions

Regular expressions are a way of describing “finite automaton” patterns:

- Second-last letter is *b*
- Every odd symbol is *b*

Many applications in CS:

- Lexical analysis in compiler construction
- Search facilities provided by text editors and databases; utilities such as `grep` and `awk`
- Pattern matching on strings

Regular expressions

Given a finite set Σ , a **regular expression (regexp)** over Σ is defined recursively as follows:

- $\emptyset$ is a regular expression
- ϵ is a regular expression
- a is a regular expression for all $a \in \Sigma$
- If E_1 and E_2 are regular expressions, then $E_1 E_2$ is a regular expression
- If E_1 and E_2 are regular expressions, then $E_1 + E_2$ is a regular expression
- If E is a regular expression, then E^* is a regular expression

We use parentheses to disambiguate regexps, though $*$ binds tighter than concatenation, which binds tighter than $+$.

Examples

Example

The following are regular expressions over $\Sigma = \{0, 1\}$:

- $\emptyset$
- $101 + 010$
- $(\epsilon + 10)^*01$

Language of a Regular expression

A regexp defines a language over Σ : the set of words which “match” the expression:

- Concatenation = sequences of expressions
- Union = choice of expressions
- Star = 0 or more occurrences of an expression

Example

The following words match $(000 + 10)^*01$:

- 01
- 101001
- 000101000001

Language of a Regular Expression

Formally, given a regexp, E , over Σ , we define $L(E) \subseteq \Sigma^*$ recursively as follows:

- If $E = \emptyset$ then $L(E) = \emptyset$
- If $E = \epsilon$ then $L(E) = \{\lambda\}$
- If $E = a$ where $a \in \Sigma$ then $L(E) = \{a\}$
- If $E = E_1 E_2$, then $L(E) = L(E_1) \cdot L(E_2)$
- If $E = E_1 + E_2$, then $L(E) = L(E_1) \cup L(E_2)$
- If $E = E_1^*$ then $L(E) = (L(E_1))^*$

Example

$$L(010 + 101) = ?$$

$$L((\epsilon + 10)^* 01) = ?$$

Language of a Regular Expression

Formally, given a regexp, E , over Σ , we define $L(E) \subseteq \Sigma^*$ recursively as follows:

- If $E = \emptyset$ then $L(E) = \emptyset$
- If $E = \epsilon$ then $L(E) = \{\lambda\}$
- If $E = a$ where $a \in \Sigma$ then $L(E) = \{a\}$
- If $E = E_1 E_2$, then $L(E) = L(E_1) \cdot L(E_2)$
- If $E = E_1 + E_2$, then $L(E) = L(E_1) \cup L(E_2)$
- If $E = E_1^*$ then $L(E) = (L(E_1))^*$

Example

$$L(010 + 101) = \{010, 101\}$$

$$L((\epsilon + 10)^* 01) = ?$$

Language of a Regular Expression

Formally, given a regexp, E , over Σ , we define $L(E) \subseteq \Sigma^*$ recursively as follows:

- If $E = \emptyset$ then $L(E) = \emptyset$
- If $E = \epsilon$ then $L(E) = \{\lambda\}$
- If $E = a$ where $a \in \Sigma$ then $L(E) = \{a\}$
- If $E = E_1 E_2$, then $L(E) = L(E_1) \cdot L(E_2)$
- If $E = E_1 + E_2$, then $L(E) = L(E_1) \cup L(E_2)$
- If $E = E_1^*$ then $L(E) = (L(E_1))^*$

Example

$$L(010 + 101) = \{010, 101\}$$

$$L((\epsilon + 10)^* 01) = \{01, 1001, 101001, \dots\}$$

Regular expressions vs NfAs

Theorem (Kleene's theorem)

- For any regular expression E , $L(E)$ is a regular language.
- For any regular language L , there is a regular expression E such that $L = L(E)$

Proof of Kleene's theorem

Given E , $L(E)$ is a regular language. Proof by induction on E .

Proof of Kleene's theorem

Given E , $L(E)$ is a regular language. Proof by induction on E .

Given L , find E such that $L = L(E)$

- Let $L_{q,q'}^X = \{w \in \Sigma^* : q \xrightarrow{w}^* q' \text{ with all intermediate states in } X\}$
- Define $E_{q,q'}^X$ such that $L(E_{q,q'}^X) = L_{q,q'}^X$:
 - When $q = q'$: $E_{q,q'}^\emptyset = \epsilon + a_1 + a_2 + \dots + a_k$ where $q \xrightarrow{a_i} q$
 - When $q \neq q'$: $E_{q,q'}^\emptyset = \emptyset + a_1 + a_2 + \dots + a_k$ where $q \xrightarrow{a_i} q'$
 - For $X \neq \emptyset$:

$$E_{q,q'}^X = \underbrace{E_{q,q'}^{X-\{r\}}}_{(1)} + \underbrace{E_{q,r}^{X-\{r\}} \cdot (E_{r,r}^{X-\{r\}})^* \cdot E_{r,q'}^{X-\{r\}}}_{(2)}$$

- The required expression is then $E = \sum_{q_0 \in F} E_{q_0, q}^Q$

Notes (Week 7 Wednesday)

These are the notes I wrote during the lecture.

Transition systems:

- a set of states,
- a transition relation describing how you can move between states

A **run** of a transition system:

- a sequence of states you can visit by following transitions

A **maximal** run is a run that cannot be extended.

Some (but not all) important info about a transition system is captured by its set of runs

What sort of things might you ask about a transition system?

What **properties** does it have.

Where a property of a transition system is a set of runs that includes all the runs of that transition system.

Property: "Termination"

read: the set of all terminating runs

There are two kinds of properties:

- safety properties "something bad never happens"
- liveness properties "something good will happen"

Neat result (Alpern & Schneider):

Every property is the intersection of a safety property and a liveness property

The program will allocate exactly 100MB

this is the intersection of:

- the program will allocate at least 100MB (liveness)
- the program will allocate at most 100MB (safety)

Safety and liveness properties have completely different proof techniques

Find an **invariant**:

Something that is true initially,
 stays true after every transition,
 and is false when the "bad thing" happens

Preconditions: what's true initially

Postconditions: what we want to be true finally

Total correctness =

if the precondition is true initially,
 the program (transition system) **will**
 reach a final state that satisfies the
 postcondition

Total correctness =

partial correctness (safety)
 + termination (liveness)

Partial correctness =

if the precondition is true initially,
 then **if** the program (transition system)
 terminates, the final state satisfies the
 postcondition

Once we have a **program semantics**, we'll have
 a systematic way of extracting such transition
 systems from program text in a principled way.
 For now, we're winging it.

$$\psi \equiv rx^y = m^n$$

if y is even and

$$\psi(x, y, r)$$

then

$$\psi(x^2, y/2r)$$

if y is even and

$$rx^y = m^n$$

then

$$r(x^2)^{(y/2)} = m^n$$

To show termination for exponentiation
 we need to find a measure

$$f : \mathbb{N} \times \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$$

$$f(x, y, r) = y$$

Show: whenever $p \rightarrow q$, $f(p) > f(q)$

You may notice a connection
to well-founded orders

A measure is a special case of a
well-founded order

A wfo is a poset $(S, <)$
with no infinitely descending chains.

A measure gives you a wfo:
if f is a measure on S , then
 $(S, <)$ is a poset
where
 $x < y$ iff $f(x) < f(y)$

In *most* cases, finding a measure works.
Some transition systems terminate
despite having no measure.

Convention:

An arrow labelled a, b where $a, b \in \Sigma$
Is shorthand for two arrows.
One labelled a and one labelled b .

The *language* of an automaton
is the set of words that cause
the automaton to terminate an
accepting state

10 is not in the language

$L_A(q)$ = the set of words that takes us
from q to a final state

If you start from a DFA A ,
and construct A' by inverting the
set of final states

then $L(A') = L(A)^c$

One state: q

$\delta(q, a) = q$
final states: $\{\}$

x means "either this symbol was an A ,
or index-out-of-bounds"

In a DFA, a word w is accepted if by consuming w , we *must* reach an accepted state.

In an NFA, a word w is accepted if by consuming w , we *may* reach an accepted state.

Execute an automaton by tracking not which state we're in but, which set of states we *could* be in

NFA:s don't add any expressive power.

Tute (Week 7)

Transition systems

Question 1

There are 102 coins on a table: 98 showing heads and 4 showing tails. You can do one of two actions:

- Flip over any 10 coins
 - If there are n heads showing, add $n + 1$ coins, all showing tails.
1. Model this as a transition system, clearly defining the states and transitions.
 2. Show how to get to a state with exactly one tail showing.
 3. Show that you cannot get to a state with exactly one head showing.
 4. Are the properties "you can get to a state with exactly one tail showing" and "you cannot get to a state with exactly one head showing" safety, liveness or reachability properties, respectively?

1. States are pairs of natural numbers representing the number of heads and number of tails. Transitions are determined by the rules, e.g. $(98, 4) \rightarrow (92, 10)$ (flipping 8 heads and 2 tails).

Formally, the *states* are $\mathbb{N} \times \mathbb{N}$ and there is a *transition* from (h, t) to (h', t') iff either:

- $h' = h + 10 - 2m$ and $t' = t - 10 + 2m$ for some $m \in [0, 10]$ (flip m heads and $10 - m$ tails), or
 - $h' = h$ and $t' = t + h + 1$ (add $h + 1$ tails)
2. $(98, 4) \rightarrow (88, 14) \rightarrow (78, 24) \rightarrow \dots \rightarrow (8, 94) \rightarrow (8, 103) \rightarrow (10, 101) \rightarrow (20, 91) \rightarrow \dots (110, 1)$
where $(8, 103) \rightarrow (10, 101)$ is obtained by flipping 4 heads and 6 tails.
 3. The parity of the number of heads is a preserved invariant. So $(1, x)$ is not reachable from $(98, 4)$.
 4. "You can get to a state with exactly one tail showing" is a reachability property. "You cannot get to a state with exactly one head showing" is a safety property.

Automata

Question 1

Let $\Sigma = \{0, 1\}$. Give DFAs over Σ that accept the following languages:

1. All words except the empty word.
2. All words containing at least two 0s and at most one 1.
3. All words containing contains an even number of 0s, or exactly two 1s.

With 2 and 3 the best approach is to consider what information needs to be ``stored" – this is the information we keep in the states of the DFA.

1.  tute7_sol11.png
2. You can think of the "x axis" as representing the number of 0s, and the "y axis" as representing the number of 1s.  tute7_sol21.png
3. You can think of the "x axis" as representing the number of 1s, and the "y axis" as representing the parity of the 0s.  tute7_sol31.png

Question 2

Consider the following DFA:



1. Describe the language accepted by the automaton
2. Give an NFA with 4 states that accepts the same language

1. The set of all words that start and end with the same symbol.
2.  tute7_sol41.png

Question 3

Are regular languages closed under word reversal? That is, if L is a regular language and L' is the language obtained by reversing all the words in L , is L' regular?

Yes. Take a DFA for L . Reverse all the transitions (note this will necessarily make it an NFA). Add a new start state with ϵ transitions to all the final states of the

original automaton. Set the new final state to be the original start state. The language accepted by this NFA will be L' .

Question 4

Give regular expressions for each of the following subsets of $\{a, b\}^*$.

1. All words that contain an even number of a symbols.
2. All words that contain an odd number of b symbols.
3. All words containing an even number of a symbols or an odd number of b symbols.

1. E.g. $b^*(ab^*ab^*)^*$ or $(b + ab^*a)^*$
2. E.g. $(a^*ba^*b)^*(a^*ba^*)$ or $a^*b(a + ba^*b)^*$
3. Use union (+) to combine (a) and (b).

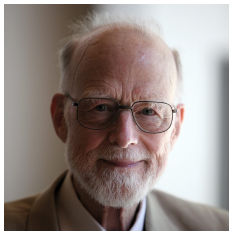
COMP2111 Week 8

Term 1, 2023

Hoare Logic

Sir Tony Hoare

- Pioneer in formal verification
- Invented: Quicksort,
- the null reference (called it his “billion dollar mistake”)
- CSP (formal specification language), and
- Hoare Logic



Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Imperative Programming

imperō

Definition

Imperative programming is where programs are described as a series of *statements* or commands to manipulate mutable *state* or cause externally observable *effects*.

States may take the form of a *mapping* from variable names to their values, or even a model of a CPU state with a memory model (for example, in an *assembly language*).

$\mathcal{L}$: A simple imperative programming language

Consider the vocabulary of basic arithmetic:

- Constant symbols: $0, 1, 2, \dots$
- Function symbols: $+, *, \dots$
- Predicate symbols: $<, \leq, \geq, |, \dots$

$\mathcal{L}$: A simple imperative programming language

Consider the vocabulary of basic arithmetic:

- Constant symbols: $0, 1, 2, \dots$
- Function symbols: $+, *, \dots$
- Predicate symbols: $<, \leq, \geq, |, \dots$
- An **(arithmetic) expression** is a term over this vocabulary.

$\mathcal{L}$: A simple imperative programming language

Consider the vocabulary of basic arithmetic:

- Constant symbols: $0, 1, 2, \dots$
- Function symbols: $+, *, \dots$
- Predicate symbols: $<, \leq, \geq, |, \dots$
- An **(arithmetic) expression** is a term over this vocabulary.
- A **boolean expression** is a predicate formula over this vocabulary.

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

Sequencing: $P; Q$

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

Sequencing: $P; Q$

Conditional: if g then P else Q fi

where g is a boolean expression.

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

Sequencing: $P; Q$

Conditional: if g then P else Q fi

where g is a boolean expression.

While: while g do P od

Factorial in $\mathcal{L}$

Example

```
 $i := 0;$   
 $m := 1;$   
while  $i < N$  do  
   $i := i + 1;$   
   $m := m * i$   
od
```

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Hoare Logic

We'll define a *Hoare Logic* for $\mathcal{L}$ to allow us to prove properties of our program.

We write a *Hoare triple* judgement as:

$$\{\varphi\} P \{\psi\}$$

Where φ and ψ are logical formulae about states, called *assertions*, and $P \in \mathcal{L}$. This triple states that if the program P terminates successfully from a starting state satisfying the *precondition* φ , then the final state will satisfy the *postcondition* ψ .

Hoare triple: Examples

Example

$$\{(x = 0)\} x := 1 \{(x = 1)\}$$

Hoare triple: Examples

Example

$$\{(x = 0)\} x := 1 \{(x = 1)\}$$
$$\{(x = 499)\} x := x + 1 \{(x = 500)\}$$

Hoare triple: Examples

Example

$$\{(x = 0)\} x := 1 \{(x = 1)\}$$

$$\{(x = 499)\} x := x + 1 \{(x = 500)\}$$

$$\{(x > 0)\} y := 0 - x \{(y < 0) \wedge (x \neq y)\}$$

Hoare triple: Factorial Examples

Example

```
{N ≥ 0}  
i := 0;  
m := 1;  
while i < N do  
  i := i + 1;  
  m := m * i  
od  
{m = N!}
```

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Motivation

Question

We know what we want informally; how do we establish when a triple is valid?

Motivation

Question

We know what we want informally; how do we establish when a triple is valid?

- Develop a semantics, OR

Hoare logic consists of one axiom and four inference rules for deriving Hoare triples.

Motivation

Question

We know what we want informally; how do we establish when a triple is valid?

- Develop a semantics, OR
- Derive the triple in a syntactic manner (i.e. Hoare proof)

Hoare logic consists of one axiom and four inference rules for deriving Hoare triples.

Assignment

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{assign})$$

Intuition:

If x has property φ *after* executing the assignment; then e must have property φ *before* executing the assignment

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$
$$\{ \quad \quad \} x := y \{(x = y)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$
$$\{(y = y)\} x := y \{(x = y)\}$$

Assignment: Example

Example

$\{(y = 0)\} x := y \{(x = 0)\}$

$\{(y = y)\} x := y \{(x = y)\}$

$\{ \quad \quad \} x := 1 \{(x < 2)\}$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$

$$\{(y = y)\} x := y \{(x = y)\}$$

$$\{(1 < 2)\} x := 1 \{(x < 2)\}$$

$$\{(y = 3)\} x := y \{(x > 2)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$

$$\{(y = y)\} x := y \{(x = y)\}$$

$$\{(1 < 2)\} x := 1 \{(x < 2)\}$$

$$\{(y = 3)\} x := y \{(x > 2)\} \quad \textit{Problem!}$$

Sequence

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Intuition:

If the postcondition of P matches the precondition of Q we can sequentially combine the two program fragments

Sequence: Example

Example

$$\frac{\{ \quad \} x := 0 \{ \quad \} \quad \{ \quad \} y := 0 \{ (x = y) \}}{\{ \quad \} x := 0; y := 0 \{ (x = y) \}} \quad (\text{seq})$$

Sequence: Example

Example

$$\frac{\{ \quad \} x := 0 \{ (x = 0) \} \quad \{ (x = 0) \} y := 0 \{ (x = y) \}}{\{ \quad \} x := 0; y := 0 \{ (x = y) \}} \quad (\text{seq})$$

Sequence: Example

Example

$$\frac{\{(0 = 0)\} x := 0 \{(x = 0)\} \quad \{(x = 0)\} y := 0 \{(x = y)\}}{\{(0 = 0)\} x := 0; y := 0 \{(x = y)\}} \quad (\text{seq})$$

Conditional

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Intuition:

- When a conditional is executed, either P or Q will be executed.
- For the postcondition ψ to be established, *either* branch must terminate in a state satisfying ψ .

While

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Intuition:

- φ is a **loop invariant**. It must be both a pre- and postcondition of P , so that sequences of P s can be run together.
- If the while loop terminates, g cannot hold.

Consequence

There is one more rule, called the *rule of consequence*, that we need to insert ordinary logical reasoning into our Hoare logic proofs:

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Consequence

There is one more rule, called the *rule of consequence*, that we need to insert ordinary logical reasoning into our Hoare logic proofs:

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Intuition:

- Adding assertions to the precondition makes it more likely the postcondition will be reached
- Removing assertions from the postcondition makes it more likely the postcondition will be reached
- If you can reach the postcondition initially, then you can reach it in the more likely scenario

Back to Assignment Example

Example

$\{(y = 3)\} x := y \{(x > 2)\}$ *Problem!*

Back to Assignment Example

Example

$\{(y = 3)\} x := y \{(x > 2)\}$ *Problem!*

$\{(y > 2)\} x := y \{(x > 2)\} (assign)$

Back to Assignment Example

Example

$\{(y = 3)\} x := y \{(x > 2)\}$ *Problem!*

$\{(y = 3)\} x := y \{(x > 2)\} (assign, cons)$
 $\{(y > 2)\} x := y \{(x > 2)\} (assign)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$
 $m := 1;$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

od

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$
 $m := 1;$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$
 $\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$$\begin{array}{l}
 \{N \geq 0\} \\
 \quad i := 0; \\
 \quad m := 1; \\
 \{m = i! \wedge N \geq 0\} \\
 \text{while } i < N \text{ do} \\
 \quad i := i + 1; \\
 \quad m := m \times i \\
 \text{od } \{m = i! \wedge N \geq 0 \wedge i = N\} \\
 \{m = N!\}
 \end{array}
 \qquad
 \begin{array}{l}
 \frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \\
 \\
 \frac{}{\{\varphi[x := e]\} x := e \{\varphi\}} \\
 \\
 \frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \\
 \\
 \frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}} \\
 \\
 \frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}
 \end{array}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$i := i + 1;$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$\{m \times (i + 1) = (i + 1)! \wedge N \geq 0\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$$\begin{array}{l}
 \{N \geq 0\} \\
 \quad i := 0; \\
 \quad m := 1; \\
 \{m = i! \wedge N \geq 0\} \\
 \text{while } i < N \text{ do } \{m = i! \wedge N \geq 0 \wedge i < N\} \\
 \quad \{m \times (i + 1) = (i + 1)! \wedge N \geq 0\} \\
 \quad i := i + 1; \\
 \quad \{m \times i = i! \wedge N \geq 0\} \\
 \quad m := m \times i \\
 \quad \{m = i! \wedge N \geq 0\} \\
 \text{od } \{m = i! \wedge N \geq 0 \wedge i = N\} \\
 \{m = N!\}
 \end{array}$$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1; \{m = i! \wedge N \geq 0\}$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$\{m \times (i + 1) = (i + 1)! \wedge N \geq 0\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{N ≥ 0}
    i := 0;
    {1 = i! ∧ N ≥ 0} m := 1; {m = i! ∧ N ≥ 0}
    {m = i! ∧ N ≥ 0}
    while i < N do {m = i! ∧ N ≥ 0 ∧ i < N}
        {m × (i + 1) = (i + 1)! ∧ N ≥ 0}
        i := i + 1;
        {m × i = i! ∧ N ≥ 0}
        m := m × i
        {m = i! ∧ N ≥ 0}
    od {m = i! ∧ N ≥ 0 ∧ i = N}
    {m = N!}
    
```

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$$\begin{array}{l}
 \{N \geq 0\} \\
 \quad i := 0; \{1 = i! \wedge N \geq 0\} \\
 \{1 = i! \wedge N \geq 0\} \quad m := 1; \{m = i! \wedge N \geq 0\} \\
 \{m = i! \wedge N \geq 0\} \\
 \text{while } i < N \text{ do } \{m = i! \wedge N \geq 0 \wedge i < N\} \\
 \quad \{m \times (i + 1) = (i + 1)! \wedge N \geq 0\} \\
 \quad i := i + 1; \\
 \quad \{m \times i = i! \wedge N \geq 0\} \\
 \quad m := m \times i \\
 \quad \{m = i! \wedge N \geq 0\} \\
 \text{od } \{m = i! \wedge N \geq 0 \wedge i = N\} \\
 \{m = N!\}
 \end{array}$$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{N ≥ 0}
{1 = 0! ∧ N ≥ 0} i := 0; {1 = i! ∧ N ≥ 0}
{1 = i! ∧ N ≥ 0} m := 1; {m = i! ∧ N ≥ 0}
{m = i! ∧ N ≥ 0}
while i < N do {m = i! ∧ N ≥ 0 ∧ i < N}
  {m × (i + 1) = (i + 1)! ∧ N ≥ 0}
  i := i + 1;
  {m × i = i! ∧ N ≥ 0}
  m := m × i
  {m = i! ∧ N ≥ 0}
od {m = i! ∧ N ≥ 0 ∧ i = N}
{m = N!}
    
```

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Practice Exercise

Example

```
 $m := 1;$   
 $n := 1;$   
 $i := 1;$   
while  $i < N$  do  
   $t := m;$   
   $m := n;$   
   $n := m + t;$   
   $i := i + 1$   
od
```

Practice Exercise

Example

```
 $m := 1;$   
 $n := 1;$   
 $i := 1;$   
while  $i < N$  do  
   $t := m;$   
   $m := n;$   
   $n := m + t;$   
   $i := i + 1$   
od
```

- What does this $\mathcal{L}$ program P compute?
- What is a valid Hoare triple $\{\varphi\}P\{\psi\}$ of this program?
- Prove using the inference rules and consequence axiom that this Hoare triple is valid.

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Semantics

Nope. That's a topic for another lecture.

Notes (Week 8 Monday)

These are the notes I wrote during the lecture.

DFA (deterministic finite automata)

NFA (non-deterministic finite automata)

In common:

- a finite set of states
- a transition relation between the states, labelled with symbols from an alphabet Σ
- what they do is accept or reject words
 - there's a starting state, and a set of final state
- a word is accepted iff there is a sequence of transitions that consumes the word, leading from the initial to a final state.

In DFA:s the transition relation is a function for each state and label, there's exactly one outgoing transition.

In NFA:s:

- the transition relation is a relation
 - For each state and label, there's 0 or more outgoing transitions
- you may have transitions with *no* label. these can be taken without consuming a character

Show that regular languages are closed under various operators (complement, union, ...)

To that, we can do one of two things:

- show how to build an NFA for e.g. $A \cup B$, given NFA:s for A and B
- show how to build a DFA for e.g. $A \cup B$, given DFA:s for A and B

Complement is regular because,
for any DFA with language L,

if we invert the set of final states,
we get a DFA the accepts L^C

```
void main() {
```

```
}
```

```
[void,main,(,),{,}] <-- list of *tokens*
```

```
L(010 + 101) =
L(010) ∪ L(101) =
L(0)·(L(10)) ∪ L(1)·L(01) =
... =
{0}·{1}·{0} ∪ {1}·{0}·{1} =
{010, 101}
```

```
resumption: 11:23
```

```
// 3 + 2 = 8
x = 3;
// x + 2 = 8
y = x + 2
// y = 8
```

Not all programming is structured around
manipulating state (imperative programming)
Other paradigms exist

Declarative programming,
Functional programming
^ the program is specified as a *function*
that given *values* as input,
computes an output value

```
f(0) = 0
f(n) = n + f(n-1)
^in functional programming languages,
code tends to look like that
```

skip a statement that does nothing
 does nothing = doesn't change
 the state

```
x := x
```

if statements look odd

if g then	if(g){
P	P
else	} else {
Q	Q
fi	}

we're forced to write the else clause

if you want to write a program without
an else clause

if g then P else skip fi

abort a program that always fails
 never terminates successfully

while true do skip od

^ this encoding assumes that we don't
care about the difference between
failure ("the program crashes")
divergence ("the program runs forever")

$\{\phi\}$ P $\{\psi\}$
pre prog post

A Hoare triple is a *specification* of the
intended behaviour of P.

If the initial state
satisfies the precondition ϕ ,
and if P terminates,
then the final state satisfies ψ .

^ note: this is the same thing as
partial correctness from w7

```
// 3 + 2 = 8
x = 3;
y = x + 2
// y = 8
```

^ what I wrote represents a Hoare triple!

$\{3 + 2 = 8\}$ $x:=3;$ $y:=x+2$ $\{y=8\}$

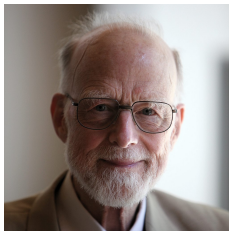
COMP2111 Week 8

Term 1, 2023

Hoare Logic

Sir Tony Hoare

- Pioneer in formal verification
- Invented: Quicksort,
- the null reference (called it his “billion dollar mistake”)
- CSP (formal specification language), and
- Hoare Logic



Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Imperative Programming

imperō

Definition

Imperative programming is where programs are described as a series of *statements* or commands to manipulate mutable *state* or cause externally observable *effects*.

States may take the form of a *mapping* from variable names to their values, or even a model of a CPU state with a memory model (for example, in an *assembly language*).

$\mathcal{L}$: A simple imperative programming language

Consider the vocabulary of basic arithmetic:

- Constant symbols: $0, 1, 2, \dots$
- Function symbols: $+, *, \dots$
- Predicate symbols: $<, \leq, \geq, |, \dots$

$\mathcal{L}$: A simple imperative programming language

Consider the vocabulary of basic arithmetic:

- Constant symbols: $0, 1, 2, \dots$
- Function symbols: $+, *, \dots$
- Predicate symbols: $<, \leq, \geq, |, \dots$
- An **(arithmetic) expression** is a term over this vocabulary.

$\mathcal{L}$: A simple imperative programming language

Consider the vocabulary of basic arithmetic:

- Constant symbols: $0, 1, 2, \dots$
- Function symbols: $+, *, \dots$
- Predicate symbols: $<, \leq, \geq, |, \dots$
- An **(arithmetic) expression** is a term over this vocabulary.
- A **boolean expression** is a predicate formula over this vocabulary.

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

Sequencing: $P; Q$

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

Sequencing: $P; Q$

Conditional: if g then P else Q fi

where g is a boolean expression.

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

Sequencing: $P; Q$

Conditional: if g then P else Q fi

where g is a boolean expression.

While: while g do P od

Factorial in $\mathcal{L}$

Example

```
 $i := 0;$   
 $m := 1;$   
while  $i < N$  do  
   $i := i + 1;$   
   $m := m * i$   
od
```

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Hoare Logic

We are going to define what's called a *Hoare Logic* for $\mathcal{L}$ to allow us to prove properties of our program.

We write a *Hoare triple* judgement as:

$$\{\varphi\} P \{\psi\}$$

Where φ and ψ are logical formulae about states, called *assertions*, and P is a program. This triple states that if the program P terminates and it successfully evaluates from a starting state satisfying the *precondition* φ , then the result state will satisfy the *postcondition* ψ .

Hoare triple: Examples

Example

$$\{(x = 0)\} x := 1 \{(x = 1)\}$$

Hoare triple: Examples

Example

$$\{(x = 0)\} x := 1 \{(x = 1)\}$$
$$\{(x = 499)\} x := x + 1 \{(x = 500)\}$$

Hoare triple: Examples

Example

$$\{(x = 0)\} x := 1 \{(x = 1)\}$$

$$\{(x = 499)\} x := x + 1 \{(x = 500)\}$$

$$\{(x > 0)\} y := 0 - x \{(y < 0) \wedge (x \neq y)\}$$

Hoare triple: Factorial Examples

Example

```
 $\{N \geq 0\}$   
 $i := 0;$   
 $m := 1;$   
while  $i < N$  do  
   $i := i + 1;$   
   $m := m * i$   
od  
 $\{m = N!\}$ 
```

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Motivation

Question

We know what we want informally; how do we establish when a triple is valid?

Motivation

Question

We know what we want informally; how do we establish when a triple is valid?

- Develop a semantics, OR

Hoare logic consists of one axiom and four inference rules for deriving Hoare triples.

Motivation

Question

We know what we want informally; how do we establish when a triple is valid?

- Develop a semantics, OR
- Derive the triple in a syntactic manner (i.e. Hoare proof)

Hoare logic consists of one axiom and four inference rules for deriving Hoare triples.

Assignment

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{assign})$$

Intuition:

If x has property φ *after* executing the assignment; then e must have property φ *before* executing the assignment

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$
$$\{ \quad \quad \} x := y \{(x = y)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$
$$\{(y = y)\} x := y \{(x = y)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$
$$\{(y = y)\} x := y \{(x = y)\}$$
$$\{ \quad \quad \} x := 1 \{(x < 2)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$

$$\{(y = y)\} x := y \{(x = y)\}$$

$$\{(1 < 2)\} x := 1 \{(x < 2)\}$$

$$\{(y = 3)\} x := y \{(x > 2)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$

$$\{(y = y)\} x := y \{(x = y)\}$$

$$\{(1 < 2)\} x := 1 \{(x < 2)\}$$

$$\{(y = 3)\} x := y \{(x > 2)\} \quad \textit{Problem!}$$

Sequence

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Intuition:

If the postcondition of P matches the precondition of Q we can sequentially combine the two program fragments

Sequence: Example

Example

$$\frac{\{ \quad \} x := 0 \{ \quad \} \quad \{ \quad \} y := 0 \{ (x = y) \}}{\{ \quad \} x := 0; y := 0 \{ (x = y) \}} \quad (\text{seq})$$

Sequence: Example

Example

$$\frac{\{ \quad \} x := 0 \{ (x = 0) \} \quad \{ (x = 0) \} y := 0 \{ (x = y) \}}{\{ \quad \} x := 0; y := 0 \{ (x = y) \}} \quad (\text{seq})$$

Sequence: Example

Example

$$\frac{\{(0 = 0)\} x := 0 \{(x = 0)\} \quad \{(x = 0)\} y := 0 \{(x = y)\}}{\{(0 = 0)\} x := 0; y := 0 \{(x = y)\}} \quad (\text{seq})$$

Conditional

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Intuition:

- When a conditional is executed, either P or Q will be executed.
- If ψ is a postcondition of the conditional, then it must be a postcondition of *both* branches
- Likewise, if φ is a precondition of the conditional, then it must be a precondition of both branches
- Which branch gets executed depends on g , so we can assume g to be a precondition of P and $\neg g$ to be a precondition of Q .

While

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Intuition:

- φ is a **loop invariant**. It must be both a pre- and postcondition of P , so that sequences of P s can be run together.
- If the while loop terminates, g cannot hold.

Consequence

There is one more rule, called the *rule of consequence*, that we need to insert ordinary logical reasoning into our Hoare logic proofs:

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Consequence

There is one more rule, called the *rule of consequence*, that we need to insert ordinary logical reasoning into our Hoare logic proofs:

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Intuition:

- Adding assertions to the precondition makes it more likely the postcondition will be reached
- Removing assertions from the postcondition makes it more likely the postcondition will be reached
- If you can reach the postcondition initially, then you can reach it in the more likely scenario

Back to Assignment Example

Example

$\{(y = 3)\} x := y \{(x > 2)\}$ *Problem!*

Back to Assignment Example

Example

$\{(y = 3)\} x := y \{(x > 2)\}$ *Problem!*

$\{(y > 2)\} x := y \{(x > 2)\} (assign)$

Back to Assignment Example

Example

$\{(y = 3)\} x := y \{(x > 2)\}$ *Problem!*

$\{(y = 3)\} x := y \{(x > 2)\} (assign, cons)$
 $\{(y > 2)\} x := y \{(x > 2)\} (assign)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$
 $m := 1;$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

od

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$
 $m := 1;$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$
 $\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$
 $m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$i := i + 1;$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$\{m \times (i + 1) = (i + 1)! \wedge N \geq 0\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$$\begin{array}{l}
 \{N \geq 0\} \\
 \quad i := 0; \\
 \quad m := 1; \\
 \{m = i! \wedge N \geq 0\} \\
 \text{while } i < N \text{ do } \{m = i! \wedge N \geq 0 \wedge i < N\} \\
 \quad \{m \times (i + 1) = (i + 1)! \wedge N \geq 0\} \\
 \quad i := i + 1; \\
 \quad \{m \times i = i! \wedge N \geq 0\} \\
 \quad m := m \times i \\
 \quad \{m = i! \wedge N \geq 0\} \\
 \text{od } \{m = i! \wedge N \geq 0 \wedge i = N\} \\
 \{m = N!\}
 \end{array}$$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1; \{m = i! \wedge N \geq 0\}$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$\{m \times (i + 1) = (i + 1)! \wedge N \geq 0\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{N ≥ 0}
    i := 0;
    {1 = i! ∧ N ≥ 0} m := 1; {m = i! ∧ N ≥ 0}
    {m = i! ∧ N ≥ 0}
    while i < N do {m = i! ∧ N ≥ 0 ∧ i < N}
        {m × (i + 1) = (i + 1)! ∧ N ≥ 0}
        i := i + 1;
        {m × i = i! ∧ N ≥ 0}
        m := m × i
        {m = i! ∧ N ≥ 0}
    od {m = i! ∧ N ≥ 0 ∧ i = N}
    {m = N!}
    
```

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$$\begin{array}{l}
 \{N \geq 0\} \\
 \quad i := 0; \{1 = i! \wedge N \geq 0\} \\
 \{1 = i! \wedge N \geq 0\} \quad m := 1; \{m = i! \wedge N \geq 0\} \\
 \{m = i! \wedge N \geq 0\} \\
 \text{while } i < N \text{ do } \{m = i! \wedge N \geq 0 \wedge i < N\} \\
 \quad \{m \times (i + 1) = (i + 1)! \wedge N \geq 0\} \\
 \quad i := i + 1; \\
 \quad \{m \times i = i! \wedge N \geq 0\} \\
 \quad m := m \times i \\
 \quad \{m = i! \wedge N \geq 0\} \\
 \text{od } \{m = i! \wedge N \geq 0 \wedge i = N\} \\
 \{m = N!\}
 \end{array}$$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{N ≥ 0}
{1 = 0! ∧ N ≥ 0} i := 0; {1 = i! ∧ N ≥ 0}
{1 = i! ∧ N ≥ 0} m := 1; {m = i! ∧ N ≥ 0}
{m = i! ∧ N ≥ 0}
while i < N do {m = i! ∧ N ≥ 0 ∧ i < N}
  {m × (i + 1) = (i + 1)! ∧ N ≥ 0}
  i := i + 1;
  {m × i = i! ∧ N ≥ 0}
  m := m × i
  {m = i! ∧ N ≥ 0}
od {m = i! ∧ N ≥ 0 ∧ i = N}
{m = N!}
    
```

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Practice Exercise

Example

```
 $m := 1;$   
 $n := 1;$   
 $i := 1;$   
while  $i < N$  do  
   $t := m;$   
   $m := n;$   
   $n := m + t;$   
   $i := i + 1$   
od
```

Practice Exercise

Example

```
 $m := 1;$   
 $n := 1;$   
 $i := 1;$   
while  $i < N$  do  
   $t := m;$   
   $m := n;$   
   $n := m + t;$   
   $i := i + 1$   
od
```

- What does this $\mathcal{L}$ program P compute?
- What is a valid Hoare triple $\{\varphi\}P\{\psi\}$ of this program?
- Prove using the inference rules and consequence axiom that this Hoare triple is valid.

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Recall

If R and S are binary relations, then the **relational composition** of R and S , $R; S$ is the relation:

$$R; S := \{(a, c) : \exists b \text{ such that } (a, b) \in R \text{ and } (b, c) \in S\}$$

If $R \subseteq A \times B$ is a relation, and $X \subseteq A$, then the **image of X under R** , $R(X)$ is the subset of B defined as:

$$R(X) := \{b \in B : \exists a \text{ in } X \text{ such that } (a, b) \in R\}.$$

Informal semantics

Hoare logic gives a proof of $\{\varphi\} P \{\psi\}$, that is: $\vdash \{\varphi\} P \{\psi\}$
(axiomatic semantics)

How do we determine when $\{\varphi\} P \{\psi\}$ is **valid**, that is:
 $\models \{\varphi\} P \{\psi\}$?

Informal semantics

Hoare logic gives a proof of $\{\varphi\} P \{\psi\}$, that is: $\vdash \{\varphi\} P \{\psi\}$
(axiomatic semantics)

How do we determine when $\{\varphi\} P \{\psi\}$ is **valid**, that is:
 $\models \{\varphi\} P \{\psi\}$?

If φ holds in a state of some computational model
then ψ holds in the state reached after a successful execution of P .

Informal semantics: Programs

What is a program?

Informal semantics: Programs

What is a program?

A function mapping system states to system states

Informal semantics: Programs

What is a program?

A partial function mapping system states to system states

Informal semantics: Programs

What is a program?

A relation between system states

Informal semantics: States

What is a state of a computational model?

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
- Abstract: from a mathematical perspective

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective
 - The pre-/postcondition predicates *hold* in a state
 - ⇒ States are **logical interpretations** (Model + Environment)

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective
 - The pre-/postcondition predicates *hold* in a state
 - ⇒ States are **logical interpretations** (Model + Environment)
 - There is only one model of interest: standard interpretations of arithmetical symbols

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective
 - The pre-/postcondition predicates *hold* in a state
 - ⇒ States are **logical interpretations** (Model + Environment)
 - There is only one model of interest: standard interpretations of arithmetical symbols
 - ⇒ States are fully determined by **environments**
 - ⇒ States are functions that map variables to values

Informal semantics: **States**

State space (ENV)

$x \leftarrow 0$
 $y \leftarrow 0$
 $z \leftarrow 0$

$x \leftarrow 3$
 $y \leftarrow 2$
 $z \leftarrow 1$

$x \leftarrow 1$
 $y \leftarrow 1$
 $z \leftarrow 1$

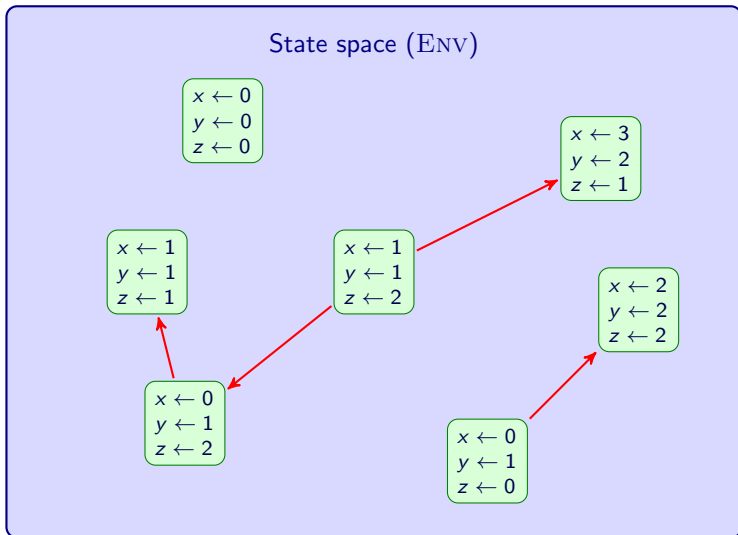
$x \leftarrow 1$
 $y \leftarrow 1$
 $z \leftarrow 2$

$x \leftarrow 2$
 $y \leftarrow 2$
 $z \leftarrow 2$

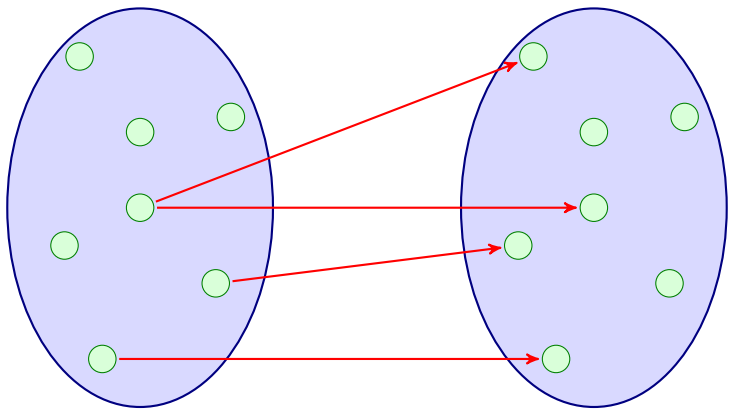
$x \leftarrow 0$
 $y \leftarrow 1$
 $z \leftarrow 2$

$x \leftarrow 0$
 $y \leftarrow 1$
 $z \leftarrow 0$

Informal semantics: **States** and **Programs**



Informal semantics: **States** and **Programs**



Semantics for $\mathcal{L}$

An **environment** or **state** is a function from variables to numeric values. We denote by $\mathbf{ENV}$ the set of all environments.

NB

An environment, η , assigns a numeric value $\llbracket e \rrbracket^\eta$ to all expressions e , and a boolean value $\llbracket b \rrbracket^\eta$ to all boolean expressions b .

Semantics for $\mathcal{L}$

An **environment** or **state** is a function from variables to numeric values. We denote by ENV the set of all environments.

NB

An environment, η , assigns a numeric value $\llbracket e \rrbracket^\eta$ to all expressions e , and a boolean value $\llbracket b \rrbracket^\eta$ to all boolean expressions b .

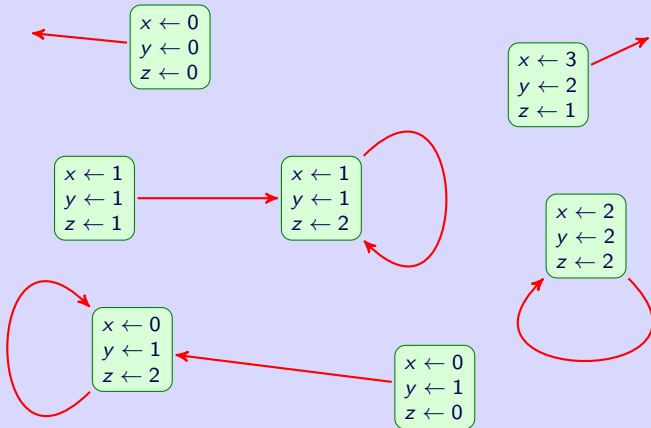
Given a program P of $\mathcal{L}$, we define $\llbracket P \rrbracket$ to be a **binary relation** on ENV in the following manner...

Assignment

$(\eta, \eta') \in \llbracket x := e \rrbracket$ if, and only if $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

Assignment: $\llbracket z := 2 \rrbracket$

State space (ENV)



Sequencing

$$\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$$

where, on the RHS, ; is relational composition.

Conditional, first attempt

$$\llbracket \text{if } b \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \begin{cases} \llbracket P \rrbracket & \text{if } \llbracket b \rrbracket^\eta = \text{true} \\ \llbracket Q \rrbracket & \text{otherwise.} \end{cases}$$

Detour: Predicates as programs

A boolean expression b defines a subset (or unary relation) of ENV :

$$\langle b \rangle = \{\eta : \llbracket b \rrbracket^\eta = \text{true}\}$$

This can be extended to a binary relation (i.e. a program):

$$\llbracket b \rrbracket = \{(\eta, \eta) : \eta \in \langle b \rangle\}$$

Detour: Predicates as programs

A boolean expression b defines a subset (or unary relation) of ENV :

$$\langle b \rangle = \{\eta : \llbracket b \rrbracket^\eta = \text{true}\}$$

This can be extended to a binary relation (i.e. a program):

$$\llbracket b \rrbracket = \{(\eta, \eta) : \eta \in \langle b \rangle\}$$

Intuitively, b corresponds to the program

if b then skip else $\perp$ fi

Conditional, better attempt

$$\llbracket \text{if } b \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket b; P \rrbracket \cup \llbracket \neg b; Q \rrbracket$$

While

while b do P od

- Do 0 or more executions of P while b holds
- Terminate when b does not hold

While

while b do P od

- Do 0 or more executions of $(b; P)$
- Terminate with an execution of $\neg b$

While

while b do P od

- Do 0 or more executions of $(b; P)$
- Terminate with an execution of $\neg b$

How to do “0 or more” executions of $(b; P)$?

Transitive closure

Given a binary relation $R \subseteq E \times E$, the *transitive closure* of R , R^* is defined to be the limit of the sequence

$$R^0 \cup R^1 \cup R^2 \dots$$

where

- $R^0 = \Delta$, the diagonal relation
- $R^{n+1} = R^n; R$

NB

- R^* is the smallest transitive relation which contains R
- Related to the Kleene star operation seen in languages: Σ^*

Transitive closure

Given a binary relation $R \subseteq E \times E$, the *transitive closure* of R , R^* is defined to be the limit of the sequence

$$R^0 \cup R^1 \cup R^2 \dots$$

where

- $R^0 = \Delta$, the diagonal relation
- $R^{n+1} = R^n; R$

NB

- R^* is the *smallest transitive relation* which contains R
- Related to the Kleene star operation seen in languages: Σ^*

Technically, R^* is the **least-fixed point** of $f(X) = X \cup X; R$

While

$$\llbracket \text{while } b \text{ do } P \text{ od} \rrbracket = \llbracket b; P \rrbracket^*; \llbracket \neg b \rrbracket$$

- Do 0 or more executions of $(b; P)$
- Conclude with an execution of $\neg b$

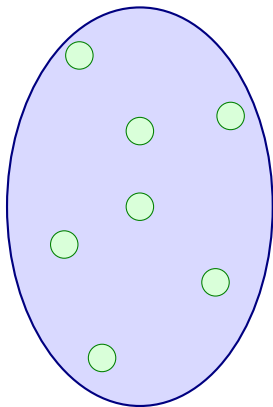
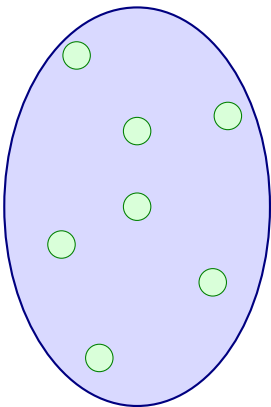
Validity

A Hoare triple is **valid**, written $\models \{\varphi\} P \{\psi\}$ if

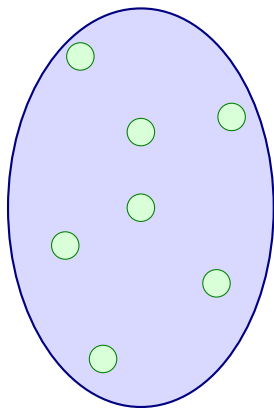
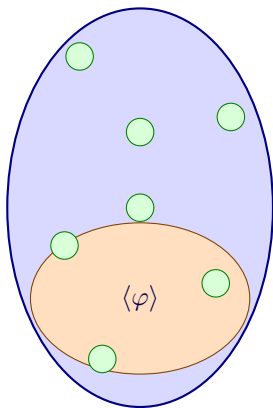
$$\llbracket P \rrbracket(\langle \varphi \rangle) \subseteq \langle \psi \rangle.$$

That is, the relational image under $\llbracket P \rrbracket$ of the set of states where φ holds is contained in the set of states where ψ holds.

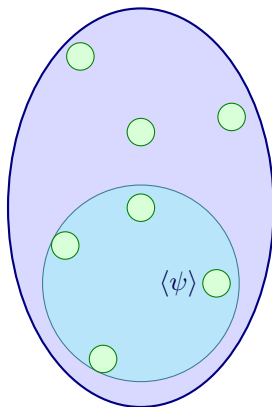
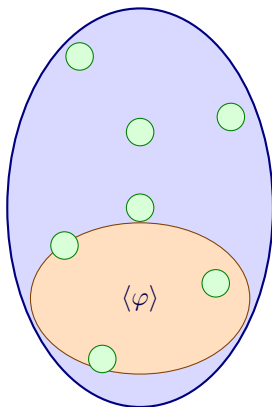
Validity



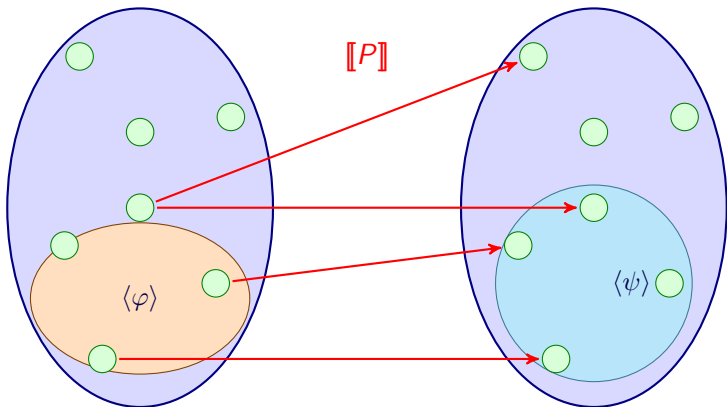
Validity



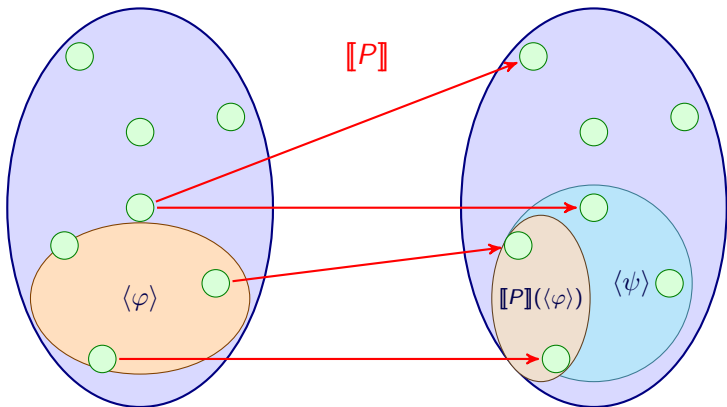
Validity



Validity



Validity



Soundness of Hoare Logic

Hoare Logic is **sound** with respect to the semantics given. That is,

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (a):

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ If $A \subseteq B$ then $R(A) \subseteq R(B)$
- Ⓑ $R(A) \cup S(A) = (R \cup S)(A)$
- Ⓒ $R(S(A)) = (S; R)(A)$

Proof (a):

$$\begin{aligned} y \in R(A) &\Leftrightarrow \exists x \in A \text{ such that } (x, y) \in R \\ &\Rightarrow \exists x \in B \text{ such that } (x, y) \in R \\ &\Leftrightarrow y \in R(B) \end{aligned}$$

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (b):

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ If $A \subseteq B$ then $R(A) \subseteq R(B)$
- Ⓑ $R(A) \cup S(A) = (R \cup S)(A)$
- Ⓒ $R(S(A)) = (S; R)(A)$

Proof (b):

$$\begin{aligned} y \in R(A) \cup S(A) &\Leftrightarrow y \in R(A) \text{ or } y \in S(A) \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, y) \in R \text{ or } \exists x \in A \text{ s.t. } (x, y) \in S \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, y) \in R \text{ or } (x, y) \in S \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, y) \in (R \cup S) \\ &\Leftrightarrow y \in (R \cup S)(A) \end{aligned}$$

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (c):

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ If $A \subseteq B$ then $R(A) \subseteq R(B)$
- Ⓑ $R(A) \cup S(A) = (R \cup S)(A)$
- Ⓒ $R(S(A)) = (S; R)(A)$

Proof (c):

$$\begin{aligned} z \in R(S(A)) &\Leftrightarrow \exists y \in S(A) \text{ s.t. } (y, z) \in R \\ &\Leftrightarrow \exists x \in A, y \in S(A) \text{ s.t. } (x, y) \in S \text{ and } (y, z) \in R \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, z) \in (S; R) \\ &\Leftrightarrow z \in (S; R)(A) \end{aligned}$$

Some results on relational images

Corollary

If $R(A) \subseteq A$ then $R^(A) \subseteq A$*

Some results on relational images

Corollary

If $R(A) \subseteq A$ then $R^(A) \subseteq A$*

Proof:

Some results on relational images

Corollary

If $R(A) \subseteq A$ then $R^(A) \subseteq A$*

Proof:

$$R(A) \subseteq A \Rightarrow R^{i+1}(A) = R^i(R(A)) \subseteq R^i(A)$$

$$\Rightarrow R^{i+1}(A) \subseteq R(A) \subseteq A$$

$$\text{So } R^*(A) = \left(\bigcup_{i=0}^{\infty} R^i \right) (A)$$

$$= \bigcup_{i=0}^{\infty} R^i(A)$$

$$\subseteq A$$

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Proof:

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Proof:

By induction on the structure of the proof.

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Recall: $(\eta, \eta'') \in \llbracket x := e \rrbracket$ if and only if $\eta'' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$,

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \ x := e \ \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \ x := e \ \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Recall: $(\eta, \eta'') \in \llbracket x := e \rrbracket$ if and only if $\eta'' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$,

So $\llbracket x := e \rrbracket(\eta) \in \langle \varphi \rangle$ for all $\eta \in \langle \varphi[e/x] \rangle$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Recall: $(\eta, \eta'') \in \llbracket x := e \rrbracket$ if and only if $\eta'' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$,

So $\llbracket x := e \rrbracket(\eta) \in \langle \varphi \rangle$ for all $\eta \in \langle \varphi[e/x] \rangle$

So $\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

So: $\llbracket P; Q \rrbracket(\langle\varphi\rangle) = \llbracket Q \rrbracket(\llbracket P \rrbracket(\langle\varphi\rangle))$ (see **Lemma 1(c)**)

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

So: $\llbracket P; Q \rrbracket(\langle \varphi \rangle) = \llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle))$ (see **Lemma 1(c)**)

By IH: $\llbracket P \rrbracket(\langle \varphi \rangle) \subseteq \langle \psi \rangle$ and $\llbracket Q \rrbracket(\langle \psi \rangle) \subseteq \langle \rho \rangle$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

So: $\llbracket P; Q \rrbracket(\langle \varphi \rangle) = \llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle))$ (see Lemma 1(c))

By IH: $\llbracket P \rrbracket(\langle \varphi \rangle) \subseteq \langle \psi \rangle$ and $\llbracket Q \rrbracket(\langle \psi \rangle) \subseteq \langle \rho \rangle$

So: $\llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle)) \subseteq \llbracket Q \rrbracket(\langle \psi \rangle) \subseteq \langle \rho \rangle$ (see Lemma 1(a))

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- a) $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- b) $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- Ⓐ $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- Ⓑ $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Proof (a):

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- Ⓐ $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- Ⓑ $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Proof (a):

$$\begin{aligned}\eta' \in \llbracket \varphi \rrbracket(X) &\Leftrightarrow \exists \eta \in X \text{ s.t. } (\eta, \eta') \in \llbracket \varphi \rrbracket \\ &\Leftrightarrow \exists \eta \in X \text{ s.t. } \eta = \eta' \text{ and } \eta \in \langle \varphi \rangle \\ &\Leftrightarrow \eta' \in X \cap \langle \varphi \rangle\end{aligned}$$

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- Ⓐ $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- Ⓑ $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Proof (b):

$$\langle \varphi \wedge \psi \rangle = \langle \varphi \rangle \cap \langle \psi \rangle = \llbracket \varphi \rrbracket(\langle \psi \rangle)$$

$$\begin{aligned} \text{So } R(\langle \varphi \wedge \psi \rangle) &= R(\llbracket \varphi \rrbracket(\langle \psi \rangle)) \\ &= (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle) \quad (\text{see Lemma 1(b)}) \end{aligned}$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle)$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\begin{aligned} & \llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle) \\ &= \llbracket g; P \rrbracket(\langle \varphi \rangle) \cup \llbracket \neg g; Q \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(b)}) \end{aligned}$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\begin{aligned} & \llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle) \\ &= \llbracket g; P \rrbracket(\langle \varphi \rangle) \cup \llbracket \neg g; Q \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(b)}) \\ &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) \cup \llbracket Q \rrbracket(\langle \neg g \wedge \varphi \rangle) \quad (\text{see Lemma 2(b)}) \end{aligned}$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\begin{aligned} & \llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle) \\ &= \llbracket g; P \rrbracket(\langle \varphi \rangle) \cup \llbracket \neg g; Q \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(b)}) \\ &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) \cup \llbracket Q \rrbracket(\langle \neg g \wedge \varphi \rangle) \quad (\text{see Lemma 2(b)}) \\ &\subseteq \langle \psi \rangle \quad (\text{by IH}) \end{aligned}$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that
 $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that
 $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\llbracket g; P \rrbracket(\langle \varphi \rangle) = \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) \quad (\text{see Lemma 2(b)})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

$$\text{So } \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket(\langle \varphi \rangle) = \llbracket \neg g \rrbracket(\llbracket g; P \rrbracket^*(\langle \varphi \rangle)) \quad (\text{see Lemma 1(c)})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

$$\begin{aligned} \text{So } \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket(\langle \varphi \rangle) &= \llbracket \neg g \rrbracket(\llbracket g; P \rrbracket^*(\langle \varphi \rangle)) && (\text{see Lemma 1(c)}) \\ &\subseteq \llbracket \neg g \rrbracket(\langle \varphi \rangle) && (\text{see Lemma 1(a)}) \end{aligned}$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

$$\begin{aligned} \text{So } \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket(\langle \varphi \rangle) &= \llbracket \neg g \rrbracket(\llbracket g; P \rrbracket^*(\langle \varphi \rangle)) && (\text{see Lemma 1(c)}) \\ &\subseteq \llbracket \neg g \rrbracket(\langle \varphi \rangle) && (\text{see Lemma 1(a)}) \\ &= \langle \neg g \wedge \varphi \rangle && (\text{see Lemma 2(a)}) \end{aligned}$$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} \textcolor{blue}{P} \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} \textcolor{blue}{P} \{\psi'\}} \quad (\text{cons})$$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Observe: If $\varphi' \rightarrow \varphi$ then $\langle \varphi' \rangle \subseteq \langle \varphi \rangle$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Observe: If $\varphi' \rightarrow \varphi$ then $\langle \varphi' \rangle \subseteq \langle \varphi \rangle$

$$\llbracket P \rrbracket(\langle \varphi' \rangle) \subseteq \llbracket P \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(a)})$$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Observe: If $\varphi' \rightarrow \varphi$ then $\langle \varphi' \rangle \subseteq \langle \varphi \rangle$

$$\begin{aligned} \llbracket P \rrbracket(\langle \varphi' \rangle) &\subseteq \llbracket P \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(a)}) \\ &\subseteq \langle \psi \rangle && (\text{IH}) \\ &\subseteq \langle \psi' \rangle \end{aligned}$$

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

⇒ There are true statements that do not have a proof.

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

- ⇒ There are true statements that do not have a proof.
- ⇒ Because of (cons) there are valid triples that result from valid, but unprovable, consequences.

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

- ⇒ There are true statements that do not have a proof.
- ⇒ Because of (cons) there are valid triples that result from valid, but unprovable, consequences.
- ⇒ Hoare Logic is not complete.

Relative completeness of Hoare Logic

Theorem (Relative completeness of Hoare Logic)

With an oracle that decides the validity of predicates,

if $\models \{\varphi\} P \{\psi\}$ then $\vdash \{\varphi\} P \{\psi\}$.

Need to know for this course

- Write programs in $\mathcal{L}$.
- Give proofs using the Hoare logic rules (full and outline)
- Definition of $\llbracket \cdot \rrbracket$
- Definition of composition and transitive closure

Notes (Week 8 Monday)

These are the notes I wrote during the lecture.

We introduced the toy programming language $\mathcal{L}$.

We use P, Q to range over programs, $P \in \mathcal{L}$

The P programs are inductively defined by this grammar:

```
P ::=  x := e      (assignment)
      |  P; Q      (seq)
      |  if b then P else Q fi  (if)
      |  while b do P od      (while)
```

^ this is called a BNF (Backus-Naur Form) grammar

The syntax is defined by a bunch of clauses separated by |

Each clause is one clause of an inductive definition

| $P; Q$ means

If P is a program, and Q is a program, then $P; Q$

But it connects to the language of predicate logic as follows:

```
in  x:=e    e is a term over some vocabulary
    if b ...    b is a predicate over the same vocabulary
    while b ...
```

But mostly, we'll assume a vocabulary of integer operations and predicates on integers with their standard meaning
+, -, *, ≤, ...

If you want programs that have more stuff than just ints, (arrays, floats, ...) just chuck it in the vocab

$\{\phi\} P \{\psi\}$

ϕ the precondition is a predicate about states

P the program $P \in \mathcal{L}$

ψ the postcondition is a predicate states

The Hoare triple means:

If ϕ is true in the initial state,
and we run P from that state,
then if we terminate,
 ψ holds in the final state

Two questions:

1. How do you prove a Hoare triple?
2. What does a Hoare triple mean? (formally)

Notice the symmetry with (predicate) logic.

1. How do you prove a formula to be valid? (natural deduction)
2. What do formulas mean? (models, interpretations, $\models$, ..)

$\vdash$ "provability"
 $\models$ "truth"

$\vdash \phi$ " ϕ is provable using natural deduction"
 $\models \phi$ " ϕ is true in every possible model+interpretation"

To prove a Hoare triple, we use the inference rules of Hoare logic.

$\vdash \{\phi\} P \{\psi\}$ "this Hoare triple is provable using the inference rules of Hoare logic"
 $\models \{\phi\} P \{\psi\}$ "this Hoare triple is true about every possible execution of P"

We make this distinction for two reasons:

- It might not be the case that Hoare logic is powerful enough to prove everything that's true
- It might be that Hoare logic is defined so that it proves false things.

$$\frac{A \quad B}{A \wedge B} \text{ (conjI)}$$

If I have a proof of A, and a proof of B,
 I can invoke conjI to get a proof of $A \wedge B$

The (long-awaited, long-teased) inference rules!!

```
// 3 = z
x := 3
// x = z
```

$$\frac{}{\{\phi[x:=e]\} \ x:=e \ \{\phi\}} \text{ (assign)}$$

```
// 3 = 3
z := 3;
// 3 = z
x := 3
// x = z
```

$$\frac{\frac{}{\{3 = 3\} \ z:=3 \ \{3 = z\}} \text{ (assign)} \quad \frac{}{\{3 = z\} \ x:=3 \ \{x = z\}} \text{ (assign)}}{\{3 = 3\} \ z:=3; \ x:=3 \ \{x = z\}} \text{ (seq)}$$

$\{\phi\} P \{\rho\} \quad \{\rho\} Q \{\psi\}$

$$\frac{}{\{\phi\} P; Q \{\psi\}} \text{ (seq)}$$

We're going to execute either P or Q (well, duh)
But we don't necessarily know which one.

ϕ might not imply either b or $\neg b$

$$\frac{\{\phi \wedge b\} P \{\psi\} \quad \{\phi \wedge \neg b\} Q \{\psi\}}{\{\phi\} \text{ if } b \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \text{ (if)}$$

issue: if $\phi = (x=7)$,
then $\phi[x:=y] = 7=y$

the assignment rule lets us prove

$\{y=7\} x:=y \{x=7\}$

$y=7$ and $\top \wedge y = 7$ are different (but equivalent) formulas

What we need is the ability to "massage" pre and postconditions
to fit into other rules: rule of consequence

$$\frac{\frac{}{\{y=7\} x:=y \{x=7\}} \text{ (assign)}}{\{\top \wedge y = 7\} x := y \{x=7\}} \text{ (conseq)} \quad \frac{\frac{}{\{7=7\} x:=7 \{x=7\}} \text{ (assign)}}{\{\top \wedge y \neq 7\} x:=7 \{x=7\}} \text{ (conseq)}$$

$$\frac{}{\{\top\} \text{ if } y = 7 \text{ then } x := y \text{ else } x:=7 \{x=7\}} \text{ (if)}$$

- We can predict *exactly* what a program will do without running it
- Even though there's infinitely many possible states, we only have to do the argument once (c.f. testing)
- The real win comes from the while rule

$$\frac{\{b \wedge \phi\} P \{\phi\}}{\{\phi\} \text{ while } b \text{ do } P \text{ od } \{\neg b \wedge \phi\}} \text{ (while)}$$

ϕ is called a *loop invariant*.

A loop invariant is:

- true in the initial state
- if it's true before a loop iteration, then it's true after the loop iteration

Intuition is:

if something is true initially,
and running the loop doesn't falsify it,
then it must be true after the last loop iteration

// we want to compute $X*Y$, where x,y are positive
// we're going to pretend we don't have a multiplication operator

```

{⊤}
{0 + X*(Y-0) = X*Y}
x := 0;
{x + X*(Y-0) = X*Y}
y := 0;
{x + X*(Y-y) = X*Y}
while y ≠ Y do
  {y ≠ Y ∧ x + X*(Y-y) = X*Y}
  {x + X*(Y-y) = X*Y}
  {x+x + X*(Y-(y+1)) = X*Y}
  x := x+x;
  {x + X*(Y-(y+1)) = X*Y}
  y := y+1
  {x + X*(Y-y) = X*Y}
od
{y = Y ∧ x + X*(Y-y) = X*Y}
{x=X*Y}

```

$$\frac{\frac{\frac{}{} \text{(while)}}{\dots} \text{(conseq)}}{\{⊤\} \ x=0; \ y=0 \ \{x + X*(Y-y) = X*Y\} \quad \{x + X*(Y-y) = X*Y\} \ \text{while } y \neq Y \ \text{do } \dots \ \text{od } \{x=X*Y\}}{\{⊤\} \ x=0; \ y=0; \ \text{while } y \neq Y \ \text{do } \dots \ \text{od } \{x=X*Y\}} \text{(seq once)}$$

- In Hoare logic, no matter how many loop iterations the program runs, we only reason about the loop body once

$$\frac{\phi \Rightarrow \phi' \quad \{\phi'\} P \{\Psi'\} \quad \Psi' \Rightarrow \Psi}{\{\phi\} P \{\psi\}} \text{(conseq)}$$

$\{x=2\} P \{x \text{ is odd}\}$
 ^ in order to prove that, it should suffice to prove
 $\{x \text{ is even}\} P \{x=3\}$

If I've proven a Hoare triple,
 this should imply similar Hoare triples
 with more specific preconditions
 and more vague postconditions

Now we know how to prove things in Hoare logic.

$\vdash \{\phi\} P \{\psi\}$

means "is provable by the inference rules of Hoare logic"

$\models \{\phi\} P \{\psi\}$

means "the Hoare triple holds about every possible execution of the program"

To define $\models$, we need to give a semantics to $\mathcal{L}$.

Use S to denote the set of all states.

1. Define a transition system to describe how $\mathcal{L}$ programs execute step-by-step (operational semantics)

- For convenience, assume "skip" is a primitive in $\mathcal{L}$
- Define an unlabelled transition system, where
 - the set of states is $S \times \mathcal{L}$
 - the transitions are defined inductively by a bunch of inference rules:

$\llbracket e \rrbracket s$ "the value of the expression e in state s "
 $s[x:=v]$ means "the state s , but with the value of x updated to be v "

$$\frac{}{(s, x:=e) \rightarrow (s[x:=\llbracket e \rrbracket s], \text{skip})}$$

$$(s, P) \rightarrow (s', P')$$

$$\frac{}{(s, P; Q) \rightarrow (s', P; Q)}$$

$$\frac{}{(s, \text{skip}; Q) \rightarrow (s, Q)}$$

$$\llbracket b \rrbracket s = \text{false}$$

$$\frac{}{(s, \text{while } b \text{ do } P \text{ od}) \rightarrow (s, \text{skip})}$$

$$\llbracket b \rrbracket s = \text{true}$$

$$\frac{}{(s, \text{while } b \text{ do } P \text{ od}) \rightarrow (s, P; \text{while } b \text{ do } P \text{ od})}$$

...similar rules for if

Now, we can use this transition system to define $\models$

$\models \{\phi\} P \{\psi\}$ is true iff

For all state s and s' such that:

$\llbracket \phi \rrbracket s = \text{true}$, and
 $(s, P) \rightarrow^* (s', \text{skip})$,
 then $\llbracket \psi \rrbracket s' = \text{true}$

2. Define a function $\llbracket \cdot \rrbracket$ which maps $\mathcal{L}$ programs to relations between initial and final states.
 (denotational semantics)

$$\llbracket \cdot \rrbracket : \mathcal{L} \rightarrow (S \times S)$$

$(s, s') \in \llbracket P \rrbracket$ this means that running P from s can result in final state s'

$$\llbracket x := e \rrbracket = \{(s, s') \mid s' = s[x:=\llbracket e \rrbracket s]\}$$

```

[[P; Q]] = [[P]]; [[Q]]
[[if b then P else Q fi]] =
  ((b); [[P]]) ∪ ((¬b); [[Q]])
[[while b do P od]]
  ((b); [[P]])*; (¬b)

```

$\models \{\phi\} P \{\psi\}$ holds iff
 Whenever $(s, s') \in [[P]]$ and $\models \phi s$,
 then $\models \psi s'$

It doesn't matter whether we define $\models$ in terms of the denotational or operational semantics, they are both equivalent.

Here $;$ is overloaded. It's sequential composition on programs, and it's also **relational composition** on relations

For if, the following will help:

```

(b) = {(s, s) | [[b]]s = true}
(b), intuitively means
  "do nothing, but abort if b is false"

```

A **state** is a mapping from variable names to values.

– Hoare logic is sound

$\vdash \{\phi\} P \{\psi\}$ implies $\models \{\phi\} P \{\psi\}$

– Hoare logic is complete-ish

The rule of consequence might not be powerful enough
 to prove all implications (Gödel's fault)
 Therefore, Hoare logic is incomplete on a technicality

If we pretend that all valid implications are provable,
 then it's complete (relative completeness)

Tute (Week 8)

Software and Systems Modelling

This tutorial is themed around RFC 677, in which Paul Johnson and Robert Thomas outline a method for maintaining consistent replicas of a distributed database. To the best of my knowledge, it is the first published solution to this problem, which has only grown in importance as distributed systems have taken over the world. These days though, there are more modern algorithms to do the job.

Here is my attempt to summarise the salient points:

1. The system consists of n independent *database management processes* (DBMPs) with unique PIDs. They communicate with each other using asynchronous message passing over reliable channels with in-order message delivery.
2. Each DBMP maintains a local copy of the database, as a mapping from keys $k \in \mathbf{K}$ to pairs $(v, (t, p))$ of *values* $v \in \mathbf{V}$ and *timestamps* $(t, p) \in (\mathbb{N} \times \mathbb{N})$. In the initial state, all the DBMPs have identical local copies.
3. Each DBMP with PID i maintains a logical clock $C_i \in \mathbb{N}$ that is incremented with every action performed by the DBMP. Clocks are not synchronized between different DBMP:s.
4. At any point, a DBMP with PID i may initiate a *modification request* to update the database, requesting a new value v to be assigned to key k . When it does so, it sends a message $(k, v, (C_i, i))$ to every other DBMP.
5. When a process receives (or initiates) a modification request for key k , it compares the timestamp of the request with the timestamp of the pre-existing entry for k in its local database. If the timestamp of the modification is lexicographically larger, the database entry is updated with the new (value, timestamp); otherwise, the modification is discarded.

A PID is a process ID, a natural number that uniquely identifies a DBMP.

A pair (n, m) is lexicographically larger than (n', m') if either $n > n'$, or if $n = n'$ and $m > m'$. This is similar to how words are sorted in a dictionary.

Question 1

Sketch a formalisation of points 1-5 as a labelled transition system. (Don't do the whole thing — you'll get bogged down in technical details.)

There's many ways to do this. Here's one.

Let K and V denote the sets of keys and values, respectively.

The state is a set of local states for each DBMP, where each DBMP has

- A $PID \in \mathbb{N}$
- A local clock value $\in \mathbb{N}$
- A database db , which is a function $K \rightarrow V \times (\mathbb{N} \times \mathbb{N})$
- A list of incoming messages $\in \text{List}(K \times V \times (\mathbb{N} \times \mathbb{N}))$. (Let's not worry about how to formalise lists in set theory here).

The incoming messages aren't physically part of the DBMP, of course — they're out there in the network somewhere. Nonetheless, it's convenient to model it as a queue.

Then, in our labelled transition system $(S, \rightarrow)$ we have

$$S = \square(\mathbb{N} \times \mathbb{N} \times (K \rightarrow V \times (\mathbb{N} \times \mathbb{N})) \times \text{List}(K \times V \times (\mathbb{N} \times \mathbb{N})))\$$$

This model comes with some caveats. For example, we don't want S to contain more than one element with the same PID, but we haven't forbidden it. To achieve this, we could phrase S as a partial function from PIDs instead.

We'll define the transition relation using the following inference rules.

The first inference rule represents a DBMP receiving, and accepting, a modification request:

$$\frac{db(k) = (v', (ck'', pid'')) \quad (ck'', pid'') < (ck', pid') \quad \forall p \in \text{procs}. pid \neq p_1}{(pid, ck, db, (k, v, (ck', pid'))) :: \text{msgs} \cup \text{procs} \rightarrow (pid, ck + 1, db[k \mapsto (v, (ck', pid'))], \text{msgs} \cup \text{procs})}$$

The next inference rule represents a process rejecting a modification request:

$$\frac{db(k) = (v', (ck'', pid'')) \quad (ck'', pid'') > (ck', pid') \quad \forall p \in \text{procs}. pid \neq p_1}{(pid, ck, db, (k, v, (ck', pid'))) :: \text{msgs} \cup \text{procs} \rightarrow (pid, ck + 1, db, \text{msgs}) \cup \text{procs}}$$

The final inference rule represents a process creating a modification request:

$$\frac{}{(pid, ck, db, \text{msgs}) \cup \text{procs} \rightarrow (pid, ck + 1, db, \text{msgs} + [(k, v, (ck, pid))]) \cup (pid, ck, db, \text{msgs} + [(k, v, (ck, pid))]) : (pid, ck, db, \text{msgs}) \in \text{procs}}$$

Note that this rule doesn't actually perform the local update, instead it sends the modification request to itself as well as to all the other processes. This is a deliberate simplification, to avoid having two inference rules doing essentially the same work.

There is one more aspect not captured by these inference rules: the rules for accepting and rejecting requests always take the first (oldest) element from the message list. However, it's possible for a message other than the oldest to arrive first, if its DBMP of origin is different. To model this, we would have to allow messages to be consumed from not just the first element in the list, under certain circumstances.

In the above, $+$ denotes list append, and $::$ denotes list cons (inserting an element to the front of a list).

Question 2

If you read the RFC, you'll find several details that the summary above completely ignores, and at least one point where the summary chooses a particular interpretation of a piece of text. See how many you can spot.

For one thing, the summary above only considers modification requests originating from what the RFC calls assignments. Selection, creation and deletion requests are completely ignored.

As for ambiguities, consider the following quote:

"When the assignment is initiated (by a person or process) the DBMP involved makes the change locally, and creates a copy of the modified entry and an associated list of DBMPs to which the change must be sent."

When that says "make the change locally", does it mean make the change no matter what? Or does it mean make the change, unless the entry we already have has a higher timestamp?

We chose the latter for a number of reasons.

1. The algorithm doesn't satisfy eventual consistency if timestamps are ignored for local updates: a DBMP may then make a local update that every other process rejects.
2. The following quote comes later, and seems to suggest interpretation 2:

"When a DBMP receives an assignment modification (either locally or from another DBMP) it compares the timestamp of the modification with the timestamp of the copy of the entry in its database and keeps whichever is more "recent" as defined by the ordering given above"

That quote suggests, but doesn't make 100% clear, that interpretation #2 is intended depending on what exactly is meant by a local modification.

Another point: a literal reading of the text seems to suggest modification requests are always sent out when a local update is attempted, even one that will clearly be rejected because the DBMP of origin already had a more recent entry. That's what we went with. But there are at least two other readings possible:

1. Modification requests are only sent to others if it would have a higher timestamp than the existing entry in our local database, i.e., if our local update would succeed.
2. If a local modification fails, an update is still sent to others— but with the existing entry in our database, not with our proposed modification.

Question 3

This algorithm guarantees that if two modification requests have the same timestamp, the modification requests are equal. How?

If two modification requests $(k, v, (n, pid))$ and $(k', v', (n, pid))$ have equal timestamps, then (because PIDs are unique) they must originate from the same DBMP. Because clocks are incremented with every action performed, the two requests must also originate with the same action. The only action that sends messages is clause 4 above, which sends identical messages.

Question 4

The algorithm is eventually consistent, in the following sense: if modifications cease and all outstanding messages have been received and acted upon, all DBMPs will have identical local copies of the database.

Define an invariant that can be used to prove eventual consistency. The invariant must:

- be true in the initial state,
- be preserved by the acts of sending and receiving modification requests, and
- imply eventual consistency once there are no more in-flight messages.

Prove these points.

The invariant can be stated something like this. For every key k there is a unique v associated with the lexicographically highest timestamp (v, ts) of a message or database entry involving k . For every DBMP i that does *not* have (v, ts) associated with k , there is an in-flight message to i containing the modification request (k, v, ts) .

If there are no in-flight messages, this invariant implies that every DBMP must have identical databases. Proof by contradiction: if not, there must a DBMP that has an entry with a lower timestamp than the maximum for some key k . This DBMP must have an inbound modification request, which contradicts the invariant.

It remains to show that the invariant is preserved by actions 4 and 5, which is relatively straightforward.

Question 5

While the algorithm as presented above is *eventually* consistent it is not *causally* consistent. This means that there are executions where a modification request m_1 is discarded in favour of another modification request m_2 , even if the DBMP where m_1 originated was aware of the request m_2 before m_1 fired.

Give an example execution where a causally inconsistent behaviour occurs.

Suppose there are three DBMPs with PIDs 0, 1 and 2. Initially they all map the key k to the value `"hi"`. Then consider the following interleaving:

1. DBMP 0 sends request $(k, \text{"hi"}, (5, 0))$, and increments its clock value from 5 to 6.
2. DBMP 1 and 2 receive this message from 0, and update their local databases accordingly.
3. DBMP 1 sends request $(k, \text{"hej"}, (3, 1))$, and increments its clock value from 3 to 4.
4. DBMP 2 receives $(k, \text{"hej"}, (3, 1))$, and discards the update because $3 < 5$.

This is causally inconsistent because, even though the out-of-synch clocks say otherwise, the modification request from DBMP 1 demonstrably happens after that of DBMP 0. Since sending of the former precedes receipt of the latter, there may even be a direct causal link. It could be that DBMP 1 decided to send its request only because of the request it received from DBMP 0.

If you're interested in how to think about causality in distributed systems, and how to set up the clocks so that they respect causality, the canonical reference is Leslie Lamport. But that's beyond the scope of the course.

COMP2111 Week 9

Term 1, 2023

Hoare Logic

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic
- Handling termination
- Adding non-determinism

Aims

We've seen how to use Hoare logic to verify programs.

But how do we know that Hoare logic *works*? Do we need to take the rules on faith? Or can we prove that it works?

Aims

We've seen how to use Hoare logic to verify programs.

But how do we know that Hoare logic *works*? Do we need to take the rules on faith? Or can we prove that it works?

We've already asked (and answered) a similar question about a different logic (natural deduction).

Informal semantics

Hoare logic gives a proof of $\{\varphi\} P \{\psi\}$, that is: $\vdash \{\varphi\} P \{\psi\}$
(axiomatic semantics)

What does it mean for $\{\varphi\} P \{\psi\}$ to be **valid**, that is:
 $\models \{\varphi\} P \{\psi\}$?

Informal semantics

Hoare logic gives a proof of $\{\varphi\} P \{\psi\}$, that is: $\vdash \{\varphi\} P \{\psi\}$
(axiomatic semantics)

What does it mean for $\{\varphi\} P \{\psi\}$ to be **valid**, that is:
 $\models \{\varphi\} P \{\psi\}$?

We need a *semantics* for $\mathcal{L}$.

Informal semantics

Hoare logic gives a proof of $\{\varphi\} P \{\psi\}$, that is: $\vdash \{\varphi\} P \{\psi\}$
(axiomatic semantics)

What does it mean for $\{\varphi\} P \{\psi\}$ to be **valid**, that is:
 $\models \{\varphi\} P \{\psi\}$?

We need a *semantics* for $\mathcal{L}$.

We *could* use the LTS semantics of $\mathcal{L}$ from Week 8. We will use a *denotational* style instead, similar to Assignment 1 Problem 1 but systematic.

Informal semantics: Programs

We know (from Assignment 1 Problem 1) that programs can be modelled as *relations* between initial and final states.

Informal semantics: States

What is a state?

Informal semantics: States

What is a state?

Two approaches:

- Concrete: from a physical perspective
- Abstract: from a mathematical perspective

Informal semantics: States

What is a state?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective

Informal semantics: States

What is a state?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective
 - The pre-/postcondition predicates *hold* in a state
 - ⇒ States are **logical interpretations** (Model + Environment)

Informal semantics: States

What is a state?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective
 - The pre-/postcondition predicates *hold* in a state
 - ⇒ States are **logical interpretations** (Model + Environment)
 - There is only one model of interest: standard interpretations of arithmetical symbols

Informal semantics: States

What is a state?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective
 - The pre-/postcondition predicates *hold* in a state
 - ⇒ States are **logical interpretations** (Model + Environment)
 - There is only one model of interest: standard interpretations of arithmetical symbols
 - ⇒ States are fully determined by **environments**
 - ⇒ States are functions that map variables to values

Informal semantics: **States**

State space (ENV)

$x \leftarrow 0$
 $y \leftarrow 0$
 $z \leftarrow 0$

$x \leftarrow 3$
 $y \leftarrow 2$
 $z \leftarrow 1$

$x \leftarrow 1$
 $y \leftarrow 1$
 $z \leftarrow 1$

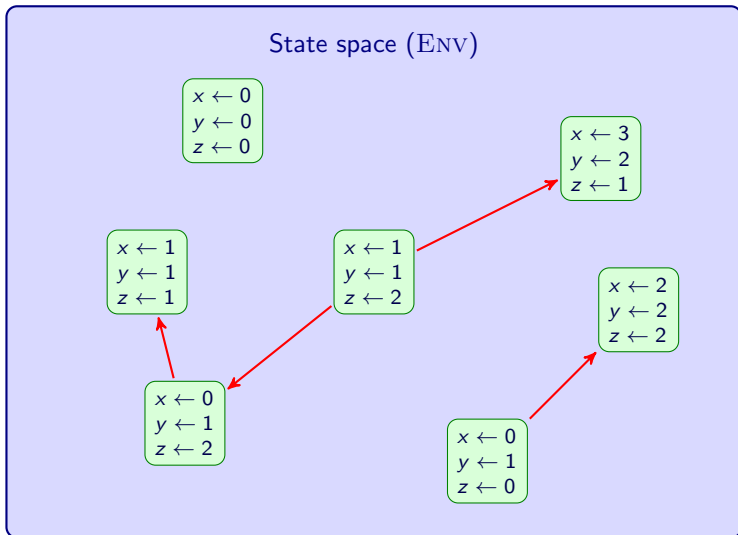
$x \leftarrow 1$
 $y \leftarrow 1$
 $z \leftarrow 2$

$x \leftarrow 2$
 $y \leftarrow 2$
 $z \leftarrow 2$

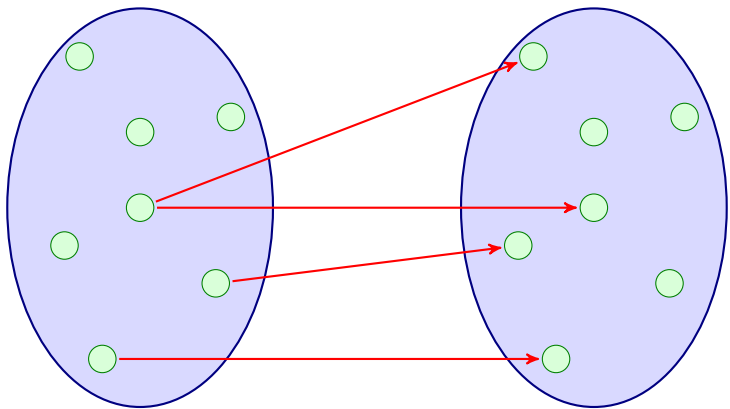
$x \leftarrow 0$
 $y \leftarrow 1$
 $z \leftarrow 2$

$x \leftarrow 0$
 $y \leftarrow 1$
 $z \leftarrow 0$

Informal semantics: **States** and **Programs**



Informal semantics: **States** and **Programs**



Semantics for $\mathcal{L}$

An **environment** or **state** is a function from variables to (numeric) values. We denote by ENV the set of all environments.

NB

An environment, η , assigns a numeric value $\llbracket e \rrbracket^\eta$ to all expressions e , and a boolean value $\llbracket b \rrbracket^\eta$ to all boolean expressions b .

Semantics for $\mathcal{L}$

An **environment** or **state** is a function from variables to (numeric) values. We denote by ENV the set of all environments.

NB

An environment, η , assigns a numeric value $\llbracket e \rrbracket^\eta$ to all expressions e , and a boolean value $\llbracket b \rrbracket^\eta$ to all boolean expressions b .

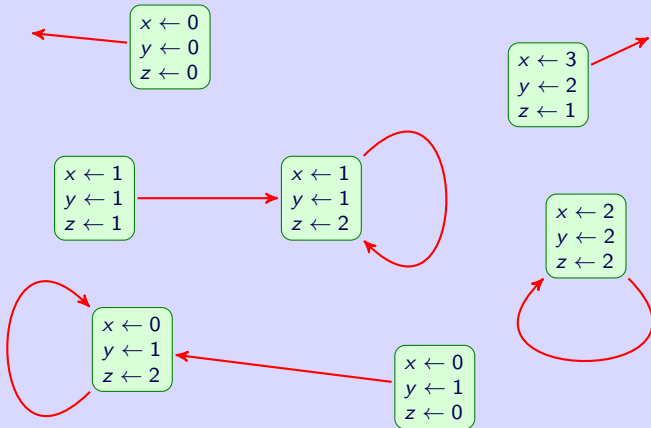
Given a program P of $\mathcal{L}$, we define $\llbracket P \rrbracket$ to be a **binary relation** on ENV in the following manner...

Assignment

$(\eta, \eta') \in \llbracket x := e \rrbracket$ if, and only if $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

Assignment: $\llbracket z := 2 \rrbracket$

State space (ENV)



Recall

If R and S are binary relations, then the **relational composition** of R and S , $R; S$ is the relation:

$$R; S := \{(a, c) : \exists b \text{ such that } (a, b) \in R \text{ and } (b, c) \in S\}$$

If $R \subseteq A \times B$ is a relation, and $X \subseteq A$, then the **image of X under R** , $R(X)$ is the subset of B defined as:

$$R(X) := \{b \in B : \exists a \in X \text{ such that } (a, b) \in R\}.$$

Sequencing

$$\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$$

where, on the RHS, ; is relational composition.

Conditional, first attempt

$$\llbracket \text{if } b \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \begin{cases} \llbracket P \rrbracket & \text{if } \llbracket b \rrbracket^\eta = \text{true} \\ \llbracket Q \rrbracket & \text{otherwise.} \end{cases}$$

Conditional, first attempt

$$\llbracket \text{if } b \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \begin{cases} \llbracket P \rrbracket & \text{if } \llbracket b \rrbracket^\eta = \text{true} \\ \llbracket Q \rrbracket & \text{otherwise.} \end{cases}$$

We'd like to avoid mentioning η on the LHS, so this won't do.

Detour: Predicates as programs

A boolean expression b defines a subset (or unary relation) of ENV :

$$\langle b \rangle = \{\eta : \llbracket b \rrbracket^\eta = \text{true}\}$$

This can be extended to a binary relation (i.e. a program):

$$\llbracket b \rrbracket = \{(\eta, \eta) : \eta \in \langle b \rangle\}$$

Detour: Predicates as programs

A boolean expression b defines a subset (or unary relation) of ENV :

$$\langle b \rangle = \{\eta : \llbracket b \rrbracket^\eta = \text{true}\}$$

This can be extended to a binary relation (i.e. a program):

$$\llbracket b \rrbracket = \{(\eta, \eta) : \eta \in \langle b \rangle\}$$

Intuitively, b corresponds to the program

if b then skip else abort fi

Conditional, better attempt

$$\llbracket \text{if } b \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket b; P \rrbracket \cup \llbracket \neg b; Q \rrbracket$$

While

while b do P od

- Do 0 or more executions of P while b holds
- Terminate when b does not hold

While

while b do P od

- Do 0 or more executions of $(b; P)$
- Terminate with an execution of $\neg b$

While

while b do P od

- Do 0 or more executions of $(b; P)$
- Terminate with an execution of $\neg b$

How to do “0 or more” executions of $(b; P)$?

Reflexive and transitive closure

Given a binary relation $R \subseteq E \times E$, the *transitive closure* of R , R^* is defined inductively by the following rules:

$$\frac{x \in E}{x R^* x}$$

$$\frac{x R y \quad y R^* z}{x R^* z}$$

NB

- $R; R^* \subseteq R^*$.

While

$$\llbracket \text{while } b \text{ do } P \text{ od} \rrbracket = \llbracket b; P \rrbracket^*; \llbracket \neg b \rrbracket$$

- Do 0 or more executions of $(b; P)$
- Conclude with an execution of $\neg b$

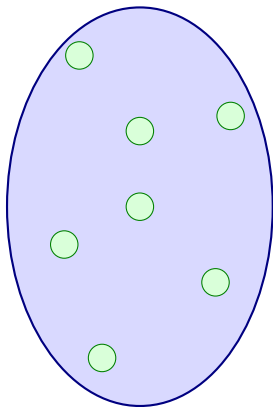
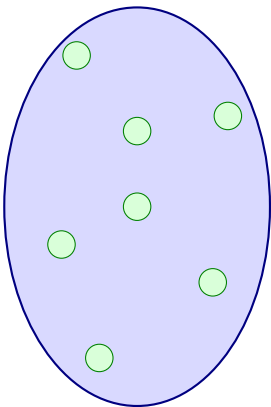
Validity

A Hoare triple is **valid**, written $\models \{\varphi\} P \{\psi\}$ if

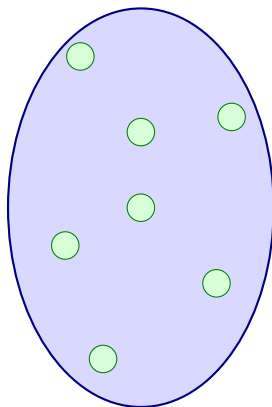
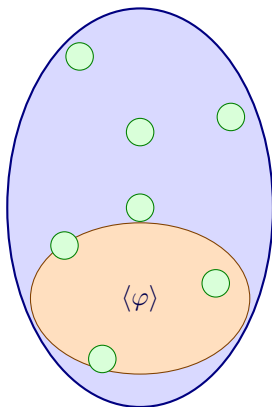
$$\llbracket P \rrbracket(\langle \varphi \rangle) \subseteq \langle \psi \rangle.$$

That is, the relational image under $\llbracket P \rrbracket$ of the set of states where φ holds is contained in the set of states where ψ holds.

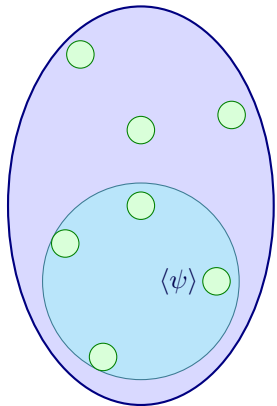
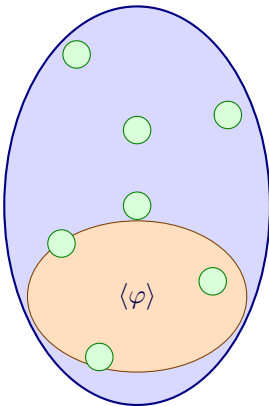
Validity



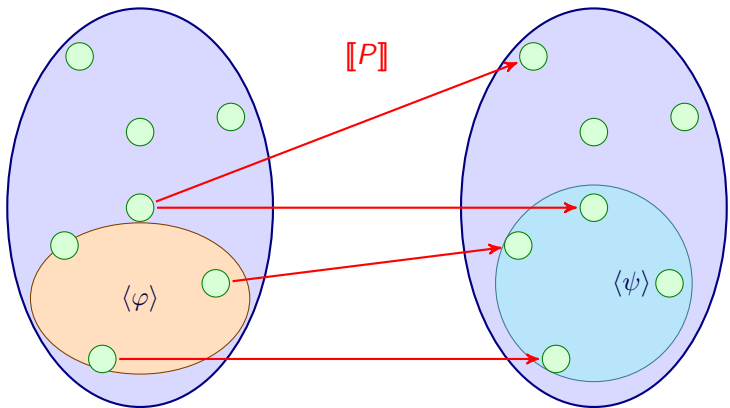
Validity



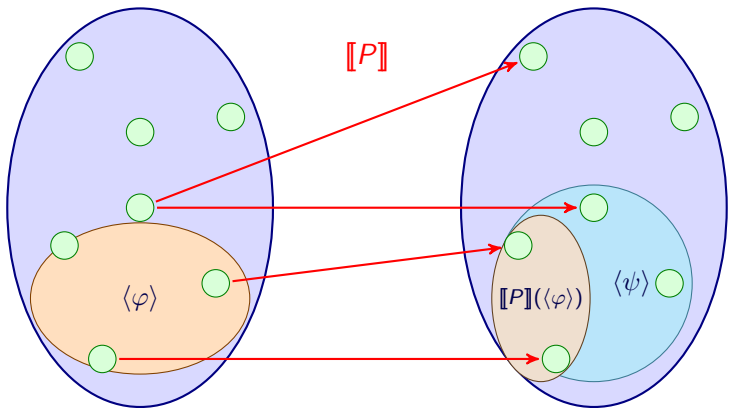
Validity



Validity



Validity



Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

⇒ There are true statements that do not have a proof.

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

- ⇒ There are true statements that do not have a proof.
- ⇒ Because of (cons) there are valid triples that result from valid, but unprovable, consequences.

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

- ⇒ There are true statements that do not have a proof.
- ⇒ Because of (cons) there are valid triples that result from valid, but unprovable, consequences.
- ⇒ Hoare Logic is not complete.

Relative completeness of Hoare Logic

Theorem (Relative completeness of Hoare Logic)

With an oracle that decides the validity of predicates,

$$\text{if } \models \{\varphi\} P \{\psi\} \text{ then } \vdash \{\varphi\} P \{\psi\}.$$

Intuitively: Hoare logic is no more incomplete than the logic used to express the pre- and postconditions.

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic
- Handling termination
- Adding non-determinism

Termination

Hoare triples for partial correctness:

$$\{\varphi\} P \{\psi\}$$

Asserts ψ holds *if* P terminates.

That's just a safety property. Let's add liveness!

Termination

Hoare triples for partial correctness:

$$\{\varphi\} P \{\psi\}$$

Asserts ψ holds *if* P terminates.

That's just a safety property. Let's add liveness!

Hoare triples for total correctness:

$$[\varphi] P [\psi]$$

Asserts:

If φ holds at a starting state, and P is executed;
then P will terminate and ψ will hold in the resulting state.

Warning

Termination is hard!

- Algorithmic limitations (e.g. Halting problem)

Warning

Termination is hard!

- Algorithmic limitations (e.g. Halting problem)
- Mathematical limitations

Example

COLLATZ

while $n > 1$ do

 if $n \% 2 = 0$

 then

$n := n / 2$

 else

$n := 3 * n + 1$

 fi

od

Total correctness

How can we show:

$$[(m \geq 0) \wedge (n > 0)] \text{Pow } [r = n^m]?$$

Total correctness

How can we show:

$$[(m \geq 0) \wedge (n > 0)] \text{Pow } [r = n^m]?$$

Use **Hoare Logic for total correctness**:

- (ass), (seq), (cond), and (cons) rules all the same
- Modified (loop) rule

Rules for total correctness

$$\frac{}{[\varphi[e/x]] \ x := e \ [\varphi]} \quad (\text{ass})$$

$$\frac{[\varphi] \ P \ [\psi] \quad [\psi] \ Q \ [\rho]}{[\varphi] \ P; Q \ [\rho]} \quad (\text{seq})$$

$$\frac{[\varphi \wedge g] \ P \ [\psi] \quad [\varphi \wedge \neg g] \ Q \ [\psi]}{[\varphi] \ \text{if } g \text{ then } P \ \text{else } Q \ \text{fi} \ [\psi]} \quad (\text{if})$$

$$\frac{\varphi' \rightarrow \varphi \quad [\varphi] \ P \ [\psi] \quad \psi \rightarrow \psi'}{[\varphi'] \ P \ [\psi']} \quad (\text{cons})$$

Terminating while loops

$\{\varphi\}$ while b do P od $\{\psi\}$

Partial correctness:

Find an invariant I such that:

- $\varphi \rightarrow I$ (establish)
- $\{I \wedge b\} P \{I\}$ (maintain)
- $(I \wedge \neg b) \rightarrow \psi$ (conclude)

Terminating while loops

$[\varphi] \text{ while } b \text{ do } P \text{ od } [\psi]$

Partial correctness:

Find an invariant I such that:

- $\varphi \rightarrow I$ (establish)
- $[I \wedge b] P [I]$ (maintain)
- $(I \wedge \neg b) \rightarrow \psi$ (conclude)

Show termination:

Find a **variant** v such that:

- $(I \wedge b) \rightarrow v > 0$ (positivity)
- $[I \wedge b \wedge v = N] P [v < N]$ (progress)

Loop rule for total correctness

$$\frac{[\varphi \wedge g \wedge (v = N)] \textcolor{blue}{P} [\varphi \wedge (v < N)] \quad (\varphi \wedge g) \rightarrow (v > 0)}{[\varphi] \textcolor{blue}{\text{while } g \text{ do } P \text{ od } [\varphi \wedge \neg g]}} \quad (\text{loop})$$

Termination for Pow

Pow	
	$\{\text{init}: (m \geq 0) \wedge (n > 0)\}$
	$\{(1 = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$r := 1;$	$\{(r = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$i := 0;$	
	$\{\text{Inv}\}$
while $i < m$ do	$\{\text{Inv} \wedge (i < m)$
	$\{(r * n = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init}\}$
$r := r * n;$	$\{(r = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init}\}$
$i := i + 1$	$\{\text{Inv}$
od	$\{\text{Inv} \wedge (i \geq m)\}$
	$\{r = n^m\}$

What is a suitable variant?

Termination for Pow

Pow	
	$\{\text{init}: (m \geq 0) \wedge (n > 0)\}$
	$\{(1 = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$r := 1;$	$\{(r = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$i := 0;$	
	$\{\text{Inv}\}$
while $i < m$ do	$\{\text{Inv} \wedge (i < m)$
	$\{(r * n = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init}\}$
$r := r * n;$	$\{(r = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init}\}$
$i := i + 1$	$\{\text{Inv}$
od	$\{\text{Inv} \wedge (i \geq m)\}$
	$\{r = n^m\}$

What is a suitable variant? $v := (m - i)$

Termination for Pow

Pow	
	$\{\text{init}: (m \geq 0) \wedge (n > 0)\}$
	$\{(1 = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$r := 1;$	$\{(r = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$i := 0;$	
	$\{\text{Inv}\}$
while $i < m$ do	$\{\text{Inv} \wedge (i < m) \wedge (v = N)\}$
	$\{(r * n = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init}\}$
$r := r * n;$	$\{(r = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init}\}$
$i := i + 1$	$\{\text{Inv}\}$
od	$\{\text{Inv} \wedge (i \geq m)\}$
	$\{r = n^m\}$

What is a suitable variant? $v := (m - i)$

Termination for Pow

Pow	
	$\{\text{init}: (m \geq 0) \wedge (n > 0)\}$
	$\{(1 = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$r := 1;$	$\{(r = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$i := 0;$	
	$\{\text{Inv}\}$
while $i < m$ do	$\{\text{Inv} \wedge (i < m) \wedge (v = N)\}$
	$\{(r * n = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init}\}$
$r := r * n;$	$\{(r = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init}\}$
$i := i + 1$	$\{\text{Inv} \wedge (v < N)\}$
od	$\{\text{Inv} \wedge (i \geq m)\}$
	$\{r = n^m\}$

What is a suitable variant? $v := (m - i)$

Termination for Pow

Pow	
	$\{\text{init}: (m \geq 0) \wedge (n > 0)\}$
	$\{(1 = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$r := 1;$	$\{(r = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$i := 0;$	
	$\{\text{Inv}\}$
while $i < m$ do	$\{\text{Inv} \wedge (i < m) \wedge (v = N)\}$
	$\{(r * n = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init}\}$
$r := r * n;$	$\{(r = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init} \wedge (v = N)\}$
$i := i + 1$	$\{\text{Inv} \wedge (v < N)\}$
od	$\{\text{Inv} \wedge (i \geq m)\}$
	$\{r = n^m\}$

What is a suitable variant? $v := (m - i)$

Termination for Pow

Pow	
	$\{\text{init}: (m \geq 0) \wedge (n > 0)\}$
	$\{(1 = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$r := 1;$	$\{(r = n^0) \wedge (0 \leq m) \wedge \text{init}\}$
$i := 0;$	
	$\{\text{Inv}\}$
while $i < m$ do	$\{\text{Inv} \wedge (i < m) \wedge (v = N)\}$
	$\{(r * n = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init} \wedge (v = N)\}$
$r := r * n;$	$\{(r = n^{i+1}) \wedge (i + 1 \leq m) \wedge \text{init} \wedge (v = N)\}$
$i := i + 1$	$\{\text{Inv} \wedge (v < N)\}$
od	$\{\text{Inv} \wedge (i \geq m)\}$
	$\{r = n^m\}$

What is a suitable variant? $v := (m - i)$

Additional proof obligations

init: $(m \geq 0) \wedge (n > 0)$

Inv: $(r = n^i) \wedge (i \leq m) \wedge \text{init}$

$v : m - i$

- $\text{Inv} \wedge (i < m) \rightarrow (v > 0)$
- $[v = N] i := i + 1 [v < N]$

Additional proof obligations

Total correctness Hoare logic is designed to prove partial correctness and termination at the same time.

You can also do them separately:

- 1 Prove a partial correctness Hoare triple.
- 2 Find a variant for every loop.

Doing it completely separate isn't always possible: sometimes, termination depends on the invariant.

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic
- Handling termination
- Adding non-determinism

Non-determinism

Non-determinism involves the computational model branching into one of several directions.

Any branch can happen (decision is not under our control).

Non-determinism

Why add non-determinism?

- More general than deterministic behaviour
- Sometimes useful for modelling interaction (c.f. coffee machines).
- Useful for abstraction (abstracted code is easier to reason about)

$\mathcal{L}^+$: a simple language with non-determinism

We relax the Conditional and Loop commands in $\mathcal{L}$ to give us non-deterministic behaviour.

The programs of $\mathcal{L}^+$ are defined as:

Assign: $x := e$, where x is a variable and e is an expression

Predicate: φ , where φ is a predicate

Sequence: $P; Q$, where P and Q are programs

$\mathcal{L}^+$: a simple language with non-determinism

We relax the Conditional and Loop commands in $\mathcal{L}$ to give us non-deterministic behaviour.

The programs of $\mathcal{L}^+$ are defined as:

Assign: $x := e$, where x is a variable and e is an expression

Predicate: φ , where φ is a predicate

Sequence: $P; Q$, where P and Q are programs

Choice: $P + Q$, where P and Q are programs; intuitively, make a non-deterministic choice between P and Q

$\mathcal{L}^+$: a simple language with non-determinism

We relax the Conditional and Loop commands in $\mathcal{L}$ to give us non-deterministic behaviour.

The programs of $\mathcal{L}^+$ are defined as:

Assign: $x := e$, where x is a variable and e is an expression

Predicate: φ , where φ is a predicate

Sequence: $P; Q$, where P and Q are programs

Choice: $P + Q$, where P and Q are programs; intuitively, make a non-deterministic choice between P and Q

Loop: P^* , where P is a program; intuitively, loop for a non-deterministic number of iterations

$\mathcal{L}^+$: a simple language with non-determinism

We relax the Conditional and Loop commands in $\mathcal{L}$ to give us non-deterministic behaviour.

The programs of $\mathcal{L}^+$ are defined as:

Assign: $x := e$, where x is a variable and e is an expression

Predicate: φ , where φ is a predicate

Sequence: $P; Q$, where P and Q are programs

Choice: $P + Q$, where P and Q are programs; intuitively, make a non-deterministic choice between P and Q

Loop: P^* , where P is a program; intuitively, loop for a non-deterministic number of iterations

$$P :: (x := e) \mid \varphi \mid P_1; P_2 \mid P_1 + P_2 \mid P_1^*$$

$\mathcal{L}^+$: a simple language with non-determinism

$$P :: (x := e) \mid \varphi \mid P_1; P_2 \mid P_1 + P_2 \mid P_1^*$$

NB

$\mathcal{L}$ can be defined in $\mathcal{L}^+$ by defining:

- if b then P else Q od $= (b; P) + (\neg b; Q)$
- while b do P od $= (b; P)^*; \neg b$

Example

Example

A program in $\mathcal{L}^+$ that non-deterministically checks if $(x \vee y) \wedge (\neg x \vee \neg z) \wedge (\neg y \vee z)$ is satisfiable:

SAT
$(x := 0) + (x := 1);$ $(y := 0) + (y := 1);$ $(z := 0) + (z := 1);$

Example

Example

A program in $\mathcal{L}^+$ that non-deterministically checks if $(x \vee y) \wedge (\neg x \vee \neg z) \wedge (\neg y \vee z)$ is satisfiable:

SAT
$(x := 0) + (x := 1);$ $(y := 0) + (y := 1);$ $(z := 0) + (z := 1);$ $\text{if}((x = 1) \vee (y = 1)) \wedge$ $((x = 0) \vee (z = 0)) \wedge$ $((y = 0) \vee (z = 1))$

Example

Example

A program in $\mathcal{L}^+$ that non-deterministically checks if $(x \vee y) \wedge (\neg x \vee \neg z) \wedge (\neg y \vee z)$ is satisfiable:

SAT
$(x := 0) + (x := 1);$ $(y := 0) + (y := 1);$ $(z := 0) + (z := 1);$ if $((x = 1) \vee (y = 1)) \wedge$ $((x = 0) \vee (z = 0)) \wedge$ $((y = 0) \vee (z = 1))$ then $r := 1$ else $r := 0$ fi

The formula is satisfiable if SAT *could* set r to 1.

Proof rules

Hoare logic rules are cleaner:

$$\frac{\{\varphi\} P \{\psi\} \quad \{\varphi\} Q \{\psi\}}{\{\varphi\} P + Q \{\psi\}} \quad (\text{choice})$$

$$\frac{\{\varphi\} P \{\varphi\}}{\{\varphi\} P^* \{\varphi\}} \quad (\text{loop})$$

Semantics

Semantics is as for $\mathcal{L}$, except:

$$\llbracket P + Q \rrbracket = \llbracket P \rrbracket \cup \llbracket Q \rrbracket \qquad \llbracket P^* \rrbracket = \llbracket P \rrbracket^*$$

Bonus slides

What follows is a proof that Hoare logic is sound.

We most likely won't have time to do any of this in the lectures.

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (a):

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ If $A \subseteq B$ then $R(A) \subseteq R(B)$
- Ⓑ $R(A) \cup S(A) = (R \cup S)(A)$
- Ⓒ $R(S(A)) = (S; R)(A)$

Proof (a):

$$\begin{aligned} y \in R(A) &\Leftrightarrow \exists x \in A \text{ such that } (x, y) \in R \\ &\Rightarrow \exists x \in B \text{ such that } (x, y) \in R \\ &\Leftrightarrow y \in R(B) \end{aligned}$$

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (b):

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ If $A \subseteq B$ then $R(A) \subseteq R(B)$
- Ⓑ $R(A) \cup S(A) = (R \cup S)(A)$
- Ⓒ $R(S(A)) = (S; R)(A)$

Proof (b):

$$\begin{aligned} y \in R(A) \cup S(A) &\Leftrightarrow y \in R(A) \text{ or } y \in S(A) \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, y) \in R \text{ or } \exists x \in A \text{ s.t. } (x, y) \in S \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, y) \in R \text{ or } (x, y) \in S \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, y) \in (R \cup S) \\ &\Leftrightarrow y \in (R \cup S)(A) \end{aligned}$$

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (c):

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ If $A \subseteq B$ then $R(A) \subseteq R(B)$
- Ⓑ $R(A) \cup S(A) = (R \cup S)(A)$
- Ⓒ $R(S(A)) = (S; R)(A)$

Proof (c):

$$\begin{aligned} z \in R(S(A)) &\Leftrightarrow \exists y \in S(A) \text{ s.t. } (y, z) \in R \\ &\Leftrightarrow \exists x \in A, y \in S(A) \text{ s.t. } (x, y) \in S \text{ and } (y, z) \in R \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, z) \in (S; R) \\ &\Leftrightarrow z \in (S; R)(A) \end{aligned}$$

Some results on relational images

Corollary

If $R(A) \subseteq A$ then $R^(A) \subseteq A$*

Reformulated: assuming $R(A) \subseteq A$, $x \in A$, and $x R^* y$, prove $y \in A$.

Proof is by induction on the derivation of $x R^* y$.

- (B) Trivial when $x = y$.
- (I) We know that $x \in A$, $x R y$ and $y R^* z$. Because $R(A) \subseteq A$, we have $y \in A$. By the induction hypothesis, $z \in A$.

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Proof:

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Proof:

By induction on the structure of the proof.

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} x := e \{\varphi\}} \quad (\text{ass})$$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Recall: $(\eta, \eta'') \in \llbracket x := e \rrbracket$ if and only if $\eta'' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$,

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Recall: $(\eta, \eta'') \in \llbracket x := e \rrbracket$ if and only if $\eta'' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$,

So $\llbracket x := e \rrbracket(\eta) \in \langle \varphi \rangle$ for all $\eta \in \langle \varphi[e/x] \rangle$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Recall: $(\eta, \eta'') \in \llbracket x := e \rrbracket$ if and only if $\eta'' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$,

So $\llbracket x := e \rrbracket(\eta) \in \langle \varphi \rangle$ for all $\eta \in \langle \varphi[e/x] \rangle$

So $\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

So: $\llbracket P; Q \rrbracket(\langle \varphi \rangle) = \llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle))$ (see **Lemma 1(c)**)

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

So: $\llbracket P; Q \rrbracket(\langle \varphi \rangle) = \llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle))$ (see **Lemma 1(c)**)

By IH: $\llbracket P \rrbracket(\langle \varphi \rangle) \subseteq \langle \psi \rangle$ and $\llbracket Q \rrbracket(\langle \psi \rangle) \subseteq \langle \rho \rangle$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

So: $\llbracket P; Q \rrbracket(\langle \varphi \rangle) = \llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle))$ (see Lemma 1(c))

By IH: $\llbracket P \rrbracket(\langle \varphi \rangle) \subseteq \langle \psi \rangle$ and $\llbracket Q \rrbracket(\langle \psi \rangle) \subseteq \langle \rho \rangle$

So: $\llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle)) \subseteq \llbracket Q \rrbracket(\langle \psi \rangle) \subseteq \langle \rho \rangle$ (see Lemma 1(a))

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- a) $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- b) $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- Ⓐ $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- Ⓑ $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Proof (a):

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- Ⓐ $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- Ⓑ $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Proof (a):

$$\begin{aligned}\eta' \in \llbracket \varphi \rrbracket(X) &\Leftrightarrow \exists \eta \in X \text{ s.t. } (\eta, \eta') \in \llbracket \varphi \rrbracket \\ &\Leftrightarrow \exists \eta \in X \text{ s.t. } \eta = \eta' \text{ and } \eta \in \langle \varphi \rangle \\ &\Leftrightarrow \eta' \in X \cap \langle \varphi \rangle\end{aligned}$$

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- Ⓐ $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- Ⓑ $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Proof (b):

$$\langle \varphi \wedge \psi \rangle = \langle \varphi \rangle \cap \langle \psi \rangle = \llbracket \varphi \rrbracket(\langle \psi \rangle)$$

$$\begin{aligned} \text{So } R(\langle \varphi \wedge \psi \rangle) &= R(\llbracket \varphi \rrbracket(\langle \psi \rangle)) \\ &= (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle) \quad (\text{see Lemma 1(b)}) \end{aligned}$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle)$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\begin{aligned} & \llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle) \\ &= \llbracket g; P \rrbracket(\langle \varphi \rangle) \cup \llbracket \neg g; Q \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(b)}) \end{aligned}$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\begin{aligned} & \llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle) \\ &= \llbracket g; P \rrbracket(\langle \varphi \rangle) \cup \llbracket \neg g; Q \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(b)}) \\ &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) \cup \llbracket Q \rrbracket(\langle \neg g \wedge \varphi \rangle) \quad (\text{see Lemma 2(b)}) \end{aligned}$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\begin{aligned} & \llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle) \\ &= \llbracket g; P \rrbracket(\langle \varphi \rangle) \cup \llbracket \neg g; Q \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(b)}) \\ &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) \cup \llbracket Q \rrbracket(\langle \neg g \wedge \varphi \rangle) \quad (\text{see Lemma 2(b)}) \\ &\subseteq \langle \psi \rangle \quad (\text{by IH}) \end{aligned}$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that
 $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that
 $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\llbracket g; P \rrbracket(\langle \varphi \rangle) = \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) \quad (\text{see Lemma 2(b)})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

$$\text{So } \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket(\langle \varphi \rangle) = \llbracket \neg g \rrbracket(\llbracket g; P \rrbracket^*(\langle \varphi \rangle)) \quad (\text{see Lemma 1(c)})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

$$\begin{aligned} \text{So } \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket(\langle \varphi \rangle) &= \llbracket \neg g \rrbracket(\llbracket g; P \rrbracket^*(\langle \varphi \rangle)) && (\text{see Lemma 1(c)}) \\ &\subseteq \llbracket \neg g \rrbracket(\langle \varphi \rangle) && (\text{see Lemma 1(a)}) \end{aligned}$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

$$\begin{aligned} \text{So } \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket(\langle \varphi \rangle) &= \llbracket \neg g \rrbracket(\llbracket g; P \rrbracket^*(\langle \varphi \rangle)) && (\text{see Lemma 1(c)}) \\ &\subseteq \llbracket \neg g \rrbracket(\langle \varphi \rangle) && (\text{see Lemma 1(a)}) \\ &= \langle \neg g \wedge \varphi \rangle && (\text{see Lemma 2(a)}) \end{aligned}$$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} \textcolor{blue}{P} \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} \textcolor{blue}{P} \{\psi'\}} \quad (\text{cons})$$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Observe: If $\varphi' \rightarrow \varphi$ then $\langle \varphi' \rangle \subseteq \langle \varphi \rangle$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Observe: If $\varphi' \rightarrow \varphi$ then $\langle \varphi' \rangle \subseteq \langle \varphi \rangle$

$$\llbracket P \rrbracket(\langle \varphi' \rangle) \subseteq \llbracket P \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(a)})$$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Observe: If $\varphi' \rightarrow \varphi$ then $\langle \varphi' \rangle \subseteq \langle \varphi \rangle$

$$\begin{aligned} \llbracket P \rrbracket(\langle \varphi' \rangle) &\subseteq \llbracket P \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(a)}) \\ &\subseteq \langle \psi \rangle && (\text{IH}) \\ &\subseteq \langle \psi' \rangle \end{aligned}$$

Notes (Week 9 Monday)

These are the notes I wrote during the lecture.

Today:

- Total correctness Hoare logic
- Hoare logic + non-determinism

Then, player's choice:

1. Cover semantics of Hoare logic
a bit more slowly
2. Catch up on quiz reviews
3 votes!
3. Sneak peek at next Monday's topic: lambda calculus
4. Past exams (but that's planned for W10)

So far, we've done **partial** correctness Hoare logic

$\{\phi\} P \{\psi\}$

"P will never terminate with the wrong result"

If ϕ holds in the initial state, and we run P,
then **if** P terminates, then ψ holds.

Fun fact:

The program

while true do skip od

Satisfies every (partial correctness) Hoare triple.

$\{I \wedge b\} P \{I\}$

_____ (while)

$\{I\} \text{ while } b \text{ do } P \{I \wedge \neg b\}$

.....

_____ $\{\top \wedge \top\} \text{ skip } \{\top\}$

_____ While

$\{\top\} \text{ while true do skip } \{\top \wedge \perp\}$

_____ Conseq

$\{\top\} \text{ while true do skip } \{\perp\}$

We know that this Hoare triple is valid:

$$\models \{\top\} \text{ while true do skip } \{\perp\}$$

This is the weakest possible Hoare triple,
it implies every other Hoare triple
by the rule of consequence.

$$\frac{\varphi \Rightarrow \varphi' \quad \{\varphi'\} P \{\psi'\} \quad \psi' \Rightarrow \psi}{\{\varphi\} P \{\psi\}} \text{ (conseq)}$$

With partial correctness, we want a Hoare logic

$$[\varphi] P [\psi] \quad \text{total correctness Hoare triple}$$

total correctness

$$= \text{partial correctness (safety)} + \text{termination (liveness)}$$

If φ holds in the initial state,
then P *will* terminate, and the final state
satisfies ψ

Aside: the square-bracket notation
is not universally understood

$$\begin{array}{ll} \mathcal{L} := & x := e \quad \text{(assign)} \\ & | \quad P; Q \quad \text{(seq)} \\ & | \quad \text{if } b \text{ then } P \text{ else } Q \text{ fi} \quad \text{(if)} \\ & | \quad \text{while } b \text{ do } P \text{ od} \quad \text{(while)} \end{array}$$

$$\frac{}{[\varphi[e/x]] \ x:=e \ [\varphi]} \text{ (assign)}$$

Q: what does $\varphi[e/x]$ mean?

A: Notation for substitution

Replace all free occurrences of x with e
in φ

$$\varphi[x:=e] \quad \varphi\{e/x\} \quad \varphi[e/x]$$

$$\frac{[\varphi] P [\rho] \quad [\rho] Q [\psi]}{[\varphi] P; Q [\psi]} \text{ (seq)}$$

$$\frac{[\varphi \wedge b] P [\psi] \quad [\varphi \wedge \neg b] Q [\psi]}{} \text{ (if)}$$

$$[\phi] \text{ if } b \text{ then } P \text{ else } Q \text{ fi } [\psi]$$

$$\frac{\phi \Rightarrow \phi' \quad [\phi'] P [\psi'] \quad \psi' \Rightarrow \psi}{[\phi] P [\psi]} \quad (\text{conseq})$$

While rule:

previously, we required an **invariant**
Invariant, in English: something unchanging
Something that's true when we start,
and stays true for every loop iteration.

To establish termination, we'll want a **variant**
Variant, in English: something changing.
Essentially the same thing as a measure.
An integer value computed from the state,
that decreases with every loop iteration,
and has a lower bound
(for convenience, we're using 0 as this bound)

$$\frac{[I \wedge b \wedge v = N] P [I \wedge v < N] \quad I \wedge b \Rightarrow v > 0}{[I] \text{ while } b \text{ do } P \text{ od } [I \wedge \neg b]} \quad (\text{while})$$

Here, N is a constant (not a program variable)
doesn't have to be a specific number
v is an integer-valued expression
in the sense of predicate logic terms

```
[x, y = 0  ∧  N non-negative]
[y + (N-x)*M = N*M ∧ N non-negative ∧ x ≤ N]
while x < N do
  [y + (N-x)*M = N*M ∧ N non-negative ∧ x ≤ N ∧ x < N ∧ N-x = Z]
  x := x+1;
  y := y+M
  [y + (N-x)*M = N*M ∧ N non-negative ∧ x ≤ N ∧ N-x < Z]
od
[y + (N-x)*M = N*M ∧ N non-negative ∧ x ≤ N ∧ x ≥ N]
[y = N*M]
```

We need to add a variant to the above
x changes with every loop iteration, but increases
y may or may not change

We can pick:

v := N-x

```

[x,y = 0  ∧  N non-negative]
[...]
while x < N do
  [...] ∧ x < N ∧ N-x = Z
  [...] ∧ N-(x+1) < Z
  x := x+1;
  [...] ∧ N-x < Z
  y := y+M
  [...] ∧ N-x < Z
od
[...] ∧ x ≥ N
[y = N*M]

```

We end up needing to show

$N-x = Z \quad \text{implies} \quad N-(x+1) < Z$

That's not hard.

All that remains is the lower bound bit.

We need to show that invariant + loop guard true implies
that $N-x \geq 0$

We need to show that this:

$y + (N-x)*M = N*M \wedge N \text{ non-negative} \wedge x \leq N \wedge x < N$

Implies:

$N-x \geq 0$

The semantics of $P \in \mathcal{L}$, written $\llbracket P \rrbracket$,
is a relation between initial and final states.

This seems overkill for our purposes:

program in $\mathcal{L}$ are deterministic, so why not use
a partial function instead?

We didn't do this because having relations works
better with non-determinism.

You can think of it in two ways:

- there's more than one way for a program to execute
- there are parts of the program that are underspecified

Think non-determinism $\approx$ underspecification

For the non-determinism we'll introduce

a language $\mathcal{L}^+$ that does have non-determinism.

The intention behind this language is to allow
us to present "programs" that aren't necessarily
implementable as written, but that can be refined
into implementable programs.

- Underspecification (non-determinism)
- Magic (a command that magically makes things true)

```

 $\mathcal{L}^+ ::=$    $x := e$       (assign)
           |   $P; Q$        (sequential composition)
           |   $P + Q$       (choice, or "alternative composition")
           |   $P^*$         (repetition)
           |   $\varphi$       (assert)

```

φ think "assert φ "
 this allows two interpretations:
 - abort unless φ holds
 - perform a miracle that makes φ true

under the second interpretation,
 the miracle represents a "hole" in our program,
 to make it implementable, we need to insert
 real code that actually establishes φ somehow,
 but we might want to abstract away from
 how exactly that's implemented

Repetition P^* means:

- run P zero or more times (analogous to Kleene star, or RTC)

$P + Q$ "do either P or Q "

- this is a program that non-deterministically chooses
 to behave either as P or Q , according to its own whims
 - specifies a family of concrete programs, where
 in the implementation, the implementor can choose
 to either do P or Q

Roll a six-sided die

$(x:=1) + (x:=2) + (x:=3) + (x:=4) + (x:=5) + (x:=6)$

Q: When moving from $\mathcal{L}$ to $\mathcal{L}^+$, where did if and while go?

A: we don't particularly need them anymore

```

if b then P else Q      (b; P) + ( $\neg$ b; Q)
while b do P od         (b; P)*;  $\neg$ b

```

```

quicksort(xs):
  choose a pivot  $x \in xs$ ; // non-determinism
  ys := everything  $\leq x$  in xs;
  zs := everything  $> x$  in xs;
  return(quicksort(ys) + quicksort(zs))

```

The point: any concrete implementation needs to have an
 actual specified, executable way to choose a pivot.

But: for the purposes of reasoning about whether quicksort is correct (in the sense that it sorts), we don't care.
 Any method of choosing the pivot leads to a correct sorting algorithm.

By modelling "choose a pivot" as a non-deterministic choice, we can verify many possible quicksort algorithms at the same time.

Q: What's the semantics of $\mathcal{L}^+$?

$$\llbracket P + Q \rrbracket = \llbracket P \rrbracket \cup \llbracket Q \rrbracket$$

$$\llbracket P^* \rrbracket = \llbracket P \rrbracket^* \quad (\text{note the first } * \text{ is program syntax,} \\ \text{the second star is reflexive-transitive closure} \\ \text{of relations})$$

$$\llbracket \varphi \rrbracket = \langle \varphi \rangle$$

$$\langle \varphi \rangle = \{ (s, s) \mid \llbracket \varphi \rrbracket s \}$$

Q: What are the Hoare rules?

$$\frac{\{I\} P \{I\}}{\{I\} P^* \{I\}} \quad (\text{repetition})$$

$$\frac{\{ \varphi \} P \{ \psi \} \quad \{ \varphi \} Q \{ \psi \}}{\{ \varphi \} P + Q \{ \psi \}} \quad (\text{alternative})$$

(The rule for asserts: saved for the quiz)

$$(x > y) [x := x + y]$$

$$\equiv x + y > y$$

$$\equiv x > 0$$

$$\{ (x > y) [x := x + y] \} x := x + y \{ x > y \}$$

A transition system is a tuple $(S, \rightarrow)$

– S is a set of states

– $\rightarrow \subseteq S \times S$

An invariant is something whose truth is preserved by every transition.

Observation: $\top$ is an invariant of every transition system.

Some optional components:

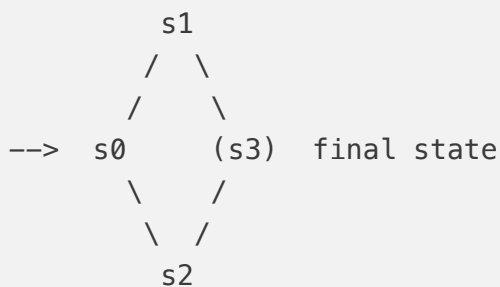
– A start state

- A set of final states
- Transition labels

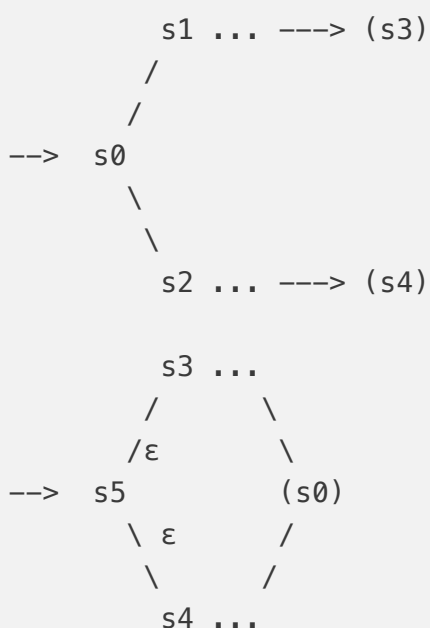
Here's a transition system:

$(\{s_0, s_1\}, s_0, \emptyset)$

A non-deterministic transition system is one where some state have more than one successor state



Suppose L is a regular language, with the following NFA:



Intuition: you can make an NFA that accepts the reverse language, by reversing all arrow, making the initial state final, and adding an initial state with ϵ -transitions to all previously final states

Tute (Week 9)

David Gries' coffee beans

A bag of coffee beans contains beans which are either black or white. Take any two beans from the bag at random: if they have different colours, throw away the white bean, but put the black bean back into the bag. If on the other hand they have the same colour, throw away both beans; but then add (back) a new white bean into the bag (from our infinite supply of extra white beans on the side). Repeat this process until there's only one bean left.

1. Suppose you know how many beans there are of each colour initially. From that, can you predict the colour of the last bean?
2. Write down a program in the language $\square^+$ that models this process by updating two variables b (number of black beans) and w (number of white beans). Use the non-deterministic choice operator $+$ to model the act of picking two beans at random.
3. What is the *variant* of the program you wrote just above?
4. Write a loop *invariant* that can be used to verify that your prediction (from the first question) is correct.

1. The parity of the number of black beans is an invariant. Hence, if we had an even number of black beans initially, the final bean will be white; if we had an odd number, it will be black.
2. This uses the encoding of **while** presented in the lectures. That is, **while** b **do** P **od** should be understood as shorthand for $(b; P)^*; \neg b$.

```
while ( $b + w > 1$ ) do
  ( $b > 1; b := b - 2; w := w + 1$ )
  + ( $b > 0 \wedge w > 0; w := w - 1$ )
  + ( $w > 1; w := w - 1$ )
od
```

Each loop iteration is a non-deterministic choice between drawing two black beans, two different beans, or two white beans. Each choice is guarded by a check that makes sure we have enough beans to actually make this draw.

3. The variant is $b + w$.

4. An invariant is:

$$b \bmod 2 = N \wedge b \geq 0 \wedge w \geq 0$$

where N is an integer constant that doesn't depend on any of the program variables.

You can also mention the specific parity, which means you'd technically need two separate Hoare logic proofs for the even and odd cases, respectively.

$$\text{odd}(b) \wedge b \geq 0 \wedge w \geq 0$$

Extending our Language

Suppose we wanted to extend $\square$ with a "repeat" loop

do s until b

that runs the loop body s at least once: it checks the guard of the loop only *after* each iteration of the loop body, exiting the loop if the guard is true.

1. Propose a translation of this repeat loop into existing $\square$ constructs.
2. Validate your translation by showing that the following Hoare logic rule for the repeat loop is derivable for your translation:

$$\frac{\{\phi\} s \{\phi\}}{\{\phi\} \text{ do } s \text{ until } e \{\phi \wedge e\}} \quad .$$

1. s ; **while**($\neg b$) **do** s **od**

2. This proof tree elides the rule names, as well as a couple of implications of the form $A \rightarrow A$ from the rule of consequence:

$$\frac{\{\phi\} s \{\phi\} \quad \frac{\frac{\phi \wedge \neg b \rightarrow \phi \quad \{\phi\} s \{\phi\}}{\{\phi \wedge \neg b\} s \{\phi\}} \quad \frac{\{\phi\} \text{ while}(\neg b) \text{ do } s \text{ od } \{\phi \wedge \neg e\}}{\{\phi\} \text{ while}(\neg b) \text{ do } s \text{ od } \{\phi \wedge e\}} \quad \phi \wedge \neg e \rightarrow \phi \wedge e}{\{\phi\} s; \text{ while}(\neg b) \text{ do } s \text{ od } \{\phi \wedge e\}}$$

Program algebra

Program algebra is the manipulation of program *texts* in a way that preserves their meaning, and (hopefully) leads to a better or more understandable program than

before. An example is the equality between these two programs:

$$\llbracket x := 1; y := 2 \rrbracket = \llbracket y := 2; x := 1 \rrbracket$$

For these two programs, at least sequential composition commutes. But it is not the case, in general, that,

$$\llbracket P; Q \rrbracket = \llbracket Q; P \rrbracket$$

1. Find a counterexample to the above formula; or, in other words, show that sequential composition is not commutative.
2. Show that sequential composition is associative.
3. What is the identity element of sequential composition? Or in other words, what is a program P such that for all Q ,

$$\llbracket P; Q \rrbracket = \llbracket Q; P \rrbracket = \llbracket Q \rrbracket$$

4. Are the following algebraic laws true in general or not?

$$\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket P; \text{while } g \text{ do } P \text{ od} \rrbracket$$

$$\llbracket \text{if } g \text{ then } P; Q \text{ else } P; R \text{ fi} \rrbracket = \llbracket P; \text{if } g \text{ then } Q \text{ else } R \text{ fi} \rrbracket$$

$$\llbracket \text{while } g \text{ do } (\text{while } g \text{ do } P \text{ od}) \text{ od} \rrbracket = \llbracket \text{while } g \text{ do } P \text{ od} \rrbracket$$

$$\llbracket \text{while } g \text{ do } (\text{while } g' \text{ do } P \text{ od}) \text{ od} \rrbracket = \llbracket \text{while } g \wedge g' \text{ do } P \text{ od} \rrbracket$$

1. For example, let $P = x := 1$ and $Q = x := 2$. The final value of x will be different depending on which one happens last.

2. Step by step, as follows:

$$\begin{aligned} \llbracket (P; Q); R \rrbracket &= \llbracket P; Q \rrbracket; \llbracket R \rrbracket && \text{(definition)} \\ &= (\llbracket P \rrbracket; \llbracket Q \rrbracket); \llbracket R \rrbracket && \text{(definition)} \\ &= \llbracket P \rrbracket; (\llbracket Q \rrbracket; \llbracket R \rrbracket) && \text{(assoc. of relational composition)} \\ &= \llbracket P \rrbracket; \llbracket Q; R \rrbracket && \text{(definition)} \\ &= \llbracket P; (Q; R) \rrbracket && \text{(definition)} \end{aligned}$$

3. The identity element of sequential composition is **skip**. The semantics of **skip** is the identity relation, so the identity laws follow from the fact that the identity relation is the left and right identity of relational composition.

4. As follows:

- It is *not* the case that

$$\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket P; \text{while } g \text{ do } P \text{ od} \rrbracket$$

because the two exhibit different behaviour if g is false initially.

- It does *not* hold that

$$\llbracket \text{if } g \text{ then } P; Q \text{ else } P; R \text{ fi} \rrbracket = \llbracket P; \text{if } g \text{ then } Q \text{ else } R \text{ fi} \rrbracket$$

because executing P may change the truth value of g . If P cannot change the truth value of g , it becomes provable.

- It does hold that

$$\llbracket \text{while } g \text{ do (while } g \text{ do } P \text{ od) od} \rrbracket = \llbracket \text{while } g \text{ do } P \text{ od} \rrbracket$$

Two key facts about the relational operators (not proved here) are used in its proof:

$$(R^*)^* = R^*$$

$$(\langle g \rangle; (\langle g \rangle; R)^*)^* = (\langle g \rangle; R)^*$$

- It is *not* the case that

$$\llbracket \text{while } g \text{ do (while } g' \text{ do } P \text{ od) od} \rrbracket = \llbracket \text{while } g \wedge g' \text{ do } P \text{ od} \rrbracket$$

Consider the case where $g = \top$ and $g' = \perp$. The former will loop forever, but the latter will not.

COMP2111 Week 10
Term 1, 2023
Lambda Calculus and Higher-Order Logic

Week 10

Today λ -calculus.

Friday Wrapup and exam prep.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.
- When the time starts, you will receive the exam questions in your UNSW inbox.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.
- When the time starts, you will receive the exam questions in your UNSW inbox.
- Submission via pdf through give.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.
- When the time starts, you will receive the exam questions in your UNSW inbox.
- Submission via pdf through give.
- You can ask questions on Ed during the exam. I sleep at night, so try to ask early.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.
- When the time starts, you will receive the exam questions in your UNSW inbox.
- Submission via pdf through give.
- You can ask questions on Ed during the exam. I sleep at night, so try to ask early.
- The intended workload is < 4 hours; the 24 hours are just to give you flexibility. We can't stop you from using more than 4 hours...

Detour: program evaluation

```
int f (int n) {  
    if n = 0 then 1 else 0  
}
```

Let's evaluate this function step by step:

$f(0)$ $\longrightarrow$

Detour: program evaluation

```
int f (int n) {  
    if n = 0 then 1 else 0  
}
```

Let's evaluate this function step by step:

$f(0)$	$\longrightarrow$ (function application)
if $0 = 0$ then 1 else 0	$\longrightarrow$

Function application: substitute the argument into the body.

Detour: program evaluation

```
int f (int n) {  
    if n = 0 then 1 else 0  
}
```

Let's evaluate this function step by step:

$f(0)$	→ (function application)
if $0 = 0$ then 1 else 0	→ (equality comparison)
if true then 1 else 0	→

Function application: substitute the argument into the body.

Detour: program evaluation

```
int f (int n) {  
    if n = 0 then 1 else 0  
}
```

Let's evaluate this function step by step:

$f(0)$	→ (function application)
if $0 = 0$ then 1 else 0	→ (equality comparison)
if true then 1 else 0	→ (if true)
1	

Function application: substitute the argument into the body.

Detour: program evaluation

The λ -calculus formalises this kind of step-by-step evaluation of program expressions...

Detour: program evaluation

The λ -calculus formalises this kind of step-by-step evaluation of program expressions...

...for a tiny language where the *only* operation is function application. Nonetheless, λ -calculus is Turing complete. How can that be?

λ -calculus

Alonzo Church

- lived 1903–1995
- supervised people like Alan Turing, Stephen Kleene
- famous for Church-Turing thesis, lambda calculus, first undecidability results
- invented λ calculus in 1930's



λ -calculus

Alonzo Church

- lived 1903–1995
- supervised people like Alan Turing, Stephen Kleene
- famous for Church-Turing thesis, lambda calculus, first undecidability results
- invented λ calculus in 1930's



λ -calculus

- originally meant as foundation of mathematics
- important applications in theoretical computer science
- foundation of computability and functional programming

untyped λ -calculus

Basic intuition:

instead of	$f(x) = x + 5$
write	$f = \lambda x. x + 5$

untyped λ -calculus

Basic intuition:

instead of $f(x) = x + 5$
write $f = \lambda x. x + 5$

- a term

untyped λ -calculus

Basic intuition:

instead of $f(x) = x + 5$
write $f = \lambda x. x + 5$

$\lambda x. x + 5$

- a term
- a nameless function

untyped λ -calculus

Basic intuition:

instead of $f(x) = x + 5$
write $f = \lambda x. x + 5$

$\lambda x. x + 5$

- a term
- a nameless function
- that adds 5 to its parameter

Function Application

For applying arguments to functions

instead of	$f(a)$
write	$f\ a$

Function Application

For applying arguments to functions

instead of $f(a)$
write $f\ a$

Example: $(\lambda x. x + 5)\ a$

Function Application

For applying arguments to functions

instead of $f(a)$
write $f\ a$

Example: $(\lambda x. x + 5)\ a$

Evaluating: in $(\lambda x. t)\ a$ replace x by a in t
(computation!)

Function Application

For applying arguments to functions

instead of $f(a)$
write $f\ a$

Example: $(\lambda x. x + 5)\ a$

Evaluating: in $(\lambda x. t)\ a$ replace x by a in t
(computation!)

Example: $(\lambda x. x + 5)\ (a + b)$ evaluates to $(a + b) + 5$

That's it!

That's it!

Now Formally

Syntax

Terms: $t ::= v \mid c \mid (t\ t) \mid (\lambda x. t)$
 $v, x \in V, \quad c \in C, \quad V, C$ sets of names

Syntax

Terms: $t ::= v \mid c \mid (t\ t) \mid (\lambda x. t)$

$v, x \in V, \quad c \in C, \quad V, C$ sets of names

- v, x variables
- c constants
- $(t\ t)$ application
- $(\lambda x. t)$ abstraction

Conventions

- leave out parentheses where possible
- list variables instead of multiple λ

Example: instead of $(\lambda y. (\lambda x. (x y)))$ write $\lambda y x. x y$

Conventions

- leave out parentheses where possible
- list variables instead of multiple λ

Example: instead of $(\lambda y. (\lambda x. (x y)))$ write $\lambda y x. x y$

Rules:

- list variables: $\lambda x. (\lambda y. t) = \lambda x y. t$
- application binds to the left: $x y z = (x y) z \neq x (y z)$
- abstraction binds to the right:
 $\lambda x. x y = \lambda x. (x y) \neq (\lambda x. x) y$
- leave out outermost parentheses

Getting used to the Syntax

Example:

$\lambda x y z. x z (y z) =$

Getting used to the Syntax

Example:

$\lambda x y z. x z (y z) =$

$\lambda x y z. (x z) (y z) =$

Getting used to the Syntax

Example:

$\lambda x y z. x z (y z) =$

$\lambda x y z. (x z) (y z) =$

$\lambda x y z. ((x z) (y z)) =$

Getting used to the Syntax

Example:

$\lambda x y z. x z (y z) =$

$\lambda x y z. (x z) (y z) =$

$\lambda x y z. ((x z) (y z)) =$

$\lambda x. \lambda y. \lambda z. ((x z) (y z)) =$

Getting used to the Syntax

Example:

$\lambda x y z. x z (y z) =$

$\lambda x y z. (x z) (y z) =$

$\lambda x y z. ((x z) (y z)) =$

$\lambda x. \lambda y. \lambda z. ((x z) (y z)) =$

$(\lambda x. (\lambda y. (\lambda z. ((x z) (y z)))))$

Encoding numbers

Remember Peano arithmetic?

$$\begin{aligned}0 &\equiv Z \\1 &\equiv S(Z) \\2 &\equiv S(S(Z))\end{aligned}$$

In the λ -calculus we have *Church numerals*:

$$\begin{aligned}0 &\equiv \lambda f x. x \\1 &\equiv \lambda f x. f x \\2 &\equiv \lambda f x. f(f x)\end{aligned}$$

Encoding numbers

$$0 \equiv \lambda f x. x$$

$$1 \equiv \lambda f x. f\ x$$

$$2 \equiv \lambda f x. f(f\ x)$$

- We encode a number n a function...

Encoding numbers

$$0 \equiv \lambda f x. x$$

$$1 \equiv \lambda f x. f\ x$$

$$2 \equiv \lambda f x. f(f\ x)$$

- We encode a number n a function...
- ..that accepts two arguments, f and x ...

Encoding numbers

$$0 \equiv \lambda f x. x$$

$$1 \equiv \lambda f x. f\ x$$

$$2 \equiv \lambda f x. f(f\ x)$$

- We encode a number n a function...
- ..that accepts two arguments, f and x ...
- ..and returns the result of applying f to x n times.

Encoding numbers

$$0 \equiv \lambda f x. x$$

$$1 \equiv \lambda f x. f\ x$$

$$2 \equiv \lambda f x. f(f\ x)$$

Here's the successor function (given m return $m + 1$):

$$\lambda m. \lambda f x. f\ (m\ f\ x)$$

Encoding numbers

$$0 \equiv \lambda f x. x$$

$$1 \equiv \lambda f x. f\ x$$

$$2 \equiv \lambda f x. f(f\ x)$$

Here's addition (given m and n return $m + 1$):

$$\lambda m n. \lambda f x. m\ f\ (n\ f\ x)$$

Encoding booleans

<code>true</code>	$\equiv$	$\lambda x y. x$
<code>false</code>	$\equiv$	$\lambda x y. y$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

<code>true</code>	$\equiv$	$\lambda x\ y. x$
<code>false</code>	$\equiv$	$\lambda x\ y. y$
$\neg b$	$\equiv$	

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

<code>true</code>	$\equiv$	$\lambda x y. x$
<code>false</code>	$\equiv$	$\lambda x y. y$
$\neg b$	$\equiv$	$\lambda b. b \text{ false true}$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

<code>true</code>	$\equiv$	$\lambda x y. x$
<code>false</code>	$\equiv$	$\lambda x y. y$
$\neg b$	$\equiv$	$\lambda b. b \text{ false true}$
<code>if z then x else y</code>	$\equiv$	

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

<code>true</code>	$\equiv$	$\lambda x y. x$
<code>false</code>	$\equiv$	$\lambda x y. y$
$\neg b$	$\equiv$	$\lambda b. b \text{ false true}$
<code>if z then x else y</code>	$\equiv$	$\lambda z x y. z x y$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

<code>true</code>	$\equiv$	$\lambda x y. x$
<code>false</code>	$\equiv$	$\lambda x y. y$
$\neg b$	$\equiv$	$\lambda b. b \text{ false true}$
<code>if z then x else y</code>	$\equiv$	$\lambda z x y. z x y$
<code>isZero(n)</code>	$\equiv$	

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

<code>true</code>	$\equiv$	$\lambda x y. x$
<code>false</code>	$\equiv$	$\lambda x y. y$
$\neg b$	$\equiv$	$\lambda b. b \text{ false true}$
<code>if z then x else y</code>	$\equiv$	$\lambda z x y. z x y$
<code>isZero(n)</code>	$\equiv$	$\lambda n. n (\lambda x. \text{false}) \text{ true}$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Detour revisited

```
int f (int n) {  
    if n = 0 then 1 else 0  
}
```

This program can be encoded in λ -calculus as follows:

```
 $\lambda n.$   
  ( $n$  ( $\lambda x. (\lambda y. y)$ ) ( $\lambda x y. x$ ))  
    ( $\lambda f x. f x$ )  
    ( $\lambda f x. x$ )
```

Detour revisited

```
int f (int n) {  
    if n = 0 then 1 else 0  
}
```

This program can be encoded in λ -calculus as follows:

```
 $\lambda n.$   
  ( $n$  ( $\lambda x. (\lambda y. y)$ ) ( $\lambda x y. x$ ))  if isZero( $n$ ) then  
    ( $\lambda f x. f x$ )  
    ( $\lambda f x. x$ )
```


Detour revisited

```
int f (int n) {  
    if n = 0 then 1 else 0  
}
```

This program can be encoded in λ -calculus as follows:

```
 $\lambda n.$   
  ( $n$  ( $\lambda x. (\lambda y. y)$ ) ( $\lambda x y. x$ )) if isZero( $n$ ) then  
    ( $\lambda f x. f x$ ) 1  
    ( $\lambda f x. x$ )
```

Detour revisited

```
int f (int n) {  
    if n = 0 then 1 else 0  
}
```

This program can be encoded in λ -calculus as follows:

```
 $\lambda n.$   
  ( $n$  ( $\lambda x. (\lambda y. y)$ ) ( $\lambda x y. x$ )) if isZero( $n$ ) then  
    ( $\lambda f x. f x$ ) 1  
    ( $\lambda f x. x$ ) else 0
```

Computation

Intuition: replace parameter by argument
this is called β -reduction

Example

$$(\lambda x y. f (y x)) \ 5 \ (\lambda x. x) \longrightarrow_{\beta}$$

Computation

Intuition: replace parameter by argument
this is called β -reduction

Example

$$(\lambda x y. f (y x)) \ 5 \ (\lambda x. x) \longrightarrow_{\beta}$$

$$(\lambda y. f (y 5)) \ (\lambda x. x) \longrightarrow_{\beta}$$

Computation

Intuition: replace parameter by argument
this is called β -reduction

Example

$$(\lambda x y. f (y x)) \ 5 \ (\lambda x. x) \longrightarrow_{\beta}$$

$$(\lambda y. f (y \ 5)) \ (\lambda x. x) \longrightarrow_{\beta}$$

$$f ((\lambda x. x) \ 5) \longrightarrow_{\beta}$$

Computation

Intuition: replace parameter by argument
this is called β -reduction

Example

$$(\lambda x y. f (y x)) \ 5 \ (\lambda x. x) \longrightarrow_{\beta}$$

$$(\lambda y. f (y \ 5)) \ (\lambda x. x) \longrightarrow_{\beta}$$

$$f \ ((\lambda x. x) \ 5) \longrightarrow_{\beta}$$

$$f \ 5$$

Defining Computation

β reduction:

$$\begin{array}{c} \frac{}{(\lambda x. s) t \longrightarrow_{\beta} s[x \leftarrow t]} \qquad \frac{s \longrightarrow_{\beta} s'}{(s t) \longrightarrow_{\beta} (s' t)} \\[2ex] \frac{t \longrightarrow_{\beta} t'}{(s t) \longrightarrow_{\beta} (s t')} \qquad \frac{s \longrightarrow_{\beta} s'}{\lambda x. s \longrightarrow_{\beta} \lambda x. s'} \end{array}$$

Defining Computation

β reduction:

$$\frac{}{(\lambda x. s) t \longrightarrow_{\beta} s[x \leftarrow t]} \qquad \frac{s \longrightarrow_{\beta} s'}{(s t) \longrightarrow_{\beta} (s' t)}$$
$$\frac{t \longrightarrow_{\beta} t'}{(s t) \longrightarrow_{\beta} (s t')} \qquad \frac{s \longrightarrow_{\beta} s'}{\lambda x. s \longrightarrow_{\beta} \lambda x. s'}$$

Still to do: define $s[x \leftarrow t]$

Defining Substitution

Easy concept. Small problem: variable capture.

Example: $(\lambda x. x z)[z \leftarrow x]$

Defining Substitution

Easy concept. Small problem: variable capture.

Example: $(\lambda x. x z)[z \leftarrow x]$

We do **not** want: $(\lambda x. x x)$ as result.

What do we want?

Defining Substitution

Easy concept. Small problem: variable capture.

Example: $(\lambda x. x z)[z \leftarrow x]$

We do **not** want: $(\lambda x. x x)$ as result.

What do we want?

In $(\lambda y. y z)[z \leftarrow x] = (\lambda y. y x)$ there would be no problem.

So, solution is: rename bound variables.

Free Variables

Bound variables: in $(\lambda x. t)$, x is a bound variable.

Free Variables

Bound variables: in $(\lambda x. t)$, x is a bound variable.

Free variables FV of a term:

$$FV(x) = \{x\}$$

$$FV(c) = \{\}$$

$$FV(s\ t) = FV(s) \cup FV(t)$$

$$FV(\lambda x. t) = FV(t) \setminus \{x\}$$

Example: $FV(\lambda x. (\lambda y. (\lambda x. x) y) y\ x)$

Free Variables

Bound variables: in $(\lambda x. t)$, x is a bound variable.

Free variables FV of a term:

$$FV(x) = \{x\}$$

$$FV(c) = \{\}$$

$$FV(s\ t) = FV(s) \cup FV(t)$$

$$FV(\lambda x. t) = FV(t) \setminus \{x\}$$

Example: $FV(\lambda x. (\lambda y. (\lambda x. x) y) y\ x) = \{y\}$

Free Variables

Bound variables: in $(\lambda x. t)$, x is a bound variable.

Free variables FV of a term:

$$FV(x) = \{x\}$$

$$FV(c) = \{\}$$

$$FV(st) = FV(s) \cup FV(t)$$

$$FV(\lambda x. t) = FV(t) \setminus \{x\}$$

Example: $FV(\lambda x. (\lambda y. (\lambda x. x) y) y x) = \{y\}$

Term t is called **closed** if $FV(t) = \{\}$

Free Variables

Bound variables: in $(\lambda x. t)$, x is a bound variable.

Free variables FV of a term:

$$FV(x) = \{x\}$$

$$FV(c) = \{\}$$

$$FV(s\ t) = FV(s) \cup FV(t)$$

$$FV(\lambda x. t) = FV(t) \setminus \{x\}$$

Example: $FV(\lambda x. (\lambda y. (\lambda x. x) y) y\ x) = \{y\}$

Term t is called **closed** if $FV(t) = \{\}$

The substitution example, $(\lambda x. x\ z)[z \leftarrow x]$, is problematic because the bound variable x is a free variable of the replacement term “ x ”.

Substitution

$$x [x \leftarrow t] = t$$

$$y [x \leftarrow t] = y$$

$$c [x \leftarrow t] = c$$

if $x \neq y$

$$(s_1 \ s_2) [x \leftarrow t] =$$

Substitution

$$\begin{array}{ll} x [x \leftarrow t] & = t \\ y [x \leftarrow t] & = y \\ c [x \leftarrow t] & = c \end{array} \quad \text{if } x \neq y$$

$$(s_1 \ s_2) [x \leftarrow t] = (s_1[x \leftarrow t] \ s_2[x \leftarrow t])$$

$$(\lambda x. s) [x \leftarrow t] =$$

Substitution

$$\begin{array}{ll} x [x \leftarrow t] & = t \\ y [x \leftarrow t] & = y \\ c [x \leftarrow t] & = c \end{array} \quad \text{if } x \neq y$$

$$(s_1 \ s_2) [x \leftarrow t] = (s_1[x \leftarrow t] \ s_2[x \leftarrow t])$$

$$(\lambda x. s) [x \leftarrow t] = (\lambda x. s)$$

$$(\lambda y. s) [x \leftarrow t] =$$

Substitution

$$\begin{array}{ll} x [x \leftarrow t] & = t \\ y [x \leftarrow t] & = y \\ c [x \leftarrow t] & = c \end{array} \quad \text{if } x \neq y$$

$$(s_1 \ s_2) [x \leftarrow t] = (s_1[x \leftarrow t] \ s_2[x \leftarrow t])$$

$$\begin{array}{ll} (\lambda x. s) [x \leftarrow t] & = (\lambda x. s) \\ (\lambda y. s) [x \leftarrow t] & = (\lambda y. s[x \leftarrow t]) \\ (\lambda y. s) [x \leftarrow t] & = \end{array} \quad \text{if } x \neq y \text{ and } y \notin FV(t)$$

Substitution

$$\begin{array}{ll} x [x \leftarrow t] & = t \\ y [x \leftarrow t] & = y \\ c [x \leftarrow t] & = c \end{array} \quad \text{if } x \neq y$$

$$(s_1 \ s_2) [x \leftarrow t] = (s_1[x \leftarrow t] \ s_2[x \leftarrow t])$$

$$\begin{array}{ll} (\lambda x. s) [x \leftarrow t] & = (\lambda x. s) \\ (\lambda y. s) [x \leftarrow t] & = (\lambda y. s[x \leftarrow t]) \\ (\lambda y. s) [x \leftarrow t] & = (\lambda z. s[y \leftarrow z][x \leftarrow t]) \end{array} \quad \begin{array}{l} \text{if } x \neq y \text{ and } y \notin FV(t) \\ \text{if } x \neq y \\ \text{and } z \notin FV(t) \cup FV(s) \end{array}$$

Substitution Example

$$(x \ (\lambda x. x) \ (\lambda y. z \ x))[x \leftarrow y]$$

Substitution Example

$$\begin{aligned} & (x \ (\lambda x. x) \ (\lambda y. z \ x))[x \leftarrow y] \\ = & (x[x \leftarrow y]) \ ((\lambda x. x)[x \leftarrow y]) \ ((\lambda y. z \ x)[x \leftarrow y]) \end{aligned}$$

Substitution Example

$$\begin{aligned} & (x \ (\lambda x. x) \ (\lambda y. z \ x))[x \leftarrow y] \\ = & (x[x \leftarrow y]) \ ((\lambda x. x)[x \leftarrow y]) \ ((\lambda y. z \ x)[x \leftarrow y]) \\ = & y \ (\lambda x. x) \ (\lambda y'. z \ y) \end{aligned}$$

α Conversion

Bound names are irrelevant:

$\lambda x. x$ and $\lambda y. y$ denote the same function.

α conversion:

$s =_{\alpha} t$ means $s = t$ up to renaming of bound variables.

α Conversion

Bound names are irrelevant:

$\lambda x. x$ and $\lambda y. y$ denote the same function.

α **conversion:**

$s =_{\alpha} t$ means $s = t$ up to renaming of bound variables.

Formally:

$$\frac{y \notin FV(t)}{(\lambda x. t) \longrightarrow_{\alpha} (\lambda y. t[x \leftarrow y])}$$

$$\frac{s \longrightarrow_{\alpha} s'}{(s t) \longrightarrow_{\alpha} (s' t)}$$

$$\frac{t \longrightarrow_{\alpha} t'}{(s t) \longrightarrow_{\alpha} (s t')}$$

$$\frac{s \longrightarrow_{\alpha} s'}{(\lambda x. s) \longrightarrow_{\alpha} (\lambda x. s')}$$

α Conversion

Bound names are irrelevant:

$\lambda x. x$ and $\lambda y. y$ denote the same function.

α **conversion:**

$s =_{\alpha} t$ means $s = t$ up to renaming of bound variables.

Formally:

$$\frac{y \notin FV(t)}{(\lambda x. t) \rightarrow_{\alpha} (\lambda y. t[x \leftarrow y])}$$

$$\frac{s \rightarrow_{\alpha} s'}{(s t) \rightarrow_{\alpha} (s' t)}$$

$$\frac{t \rightarrow_{\alpha} t'}{(s t) \rightarrow_{\alpha} (s t')}$$

$$\frac{s \rightarrow_{\alpha} s'}{(\lambda x. s) \rightarrow_{\alpha} (\lambda x. s')}$$

$$s =_{\alpha} t \quad \text{iff} \quad s \rightarrow_{\alpha}^* t$$

($\rightarrow_{\alpha}^*$ = transitive, reflexive closure of $\rightarrow_{\alpha}$ = multiple steps)

α Conversion

Examples:

$x (\lambda x y. x y)$

α Conversion

Examples:

$$\begin{aligned} & x (\lambda x y. x y) \\ =_{\alpha} & x (\lambda y x. y x) \end{aligned}$$

α Conversion

Examples:

$$\begin{aligned} & x (\lambda x y. x y) \\ =_{\alpha} & x (\lambda y x. y x) \\ =_{\alpha} & x (\lambda z y. z y) \end{aligned}$$

α Conversion

Examples:

$$\begin{aligned} & x (\lambda x y. x y) \\ =_{\alpha} & x (\lambda y x. y x) \\ =_{\alpha} & x (\lambda z y. z y) \\ \neq_{\alpha} & z (\lambda z y. z y) \end{aligned}$$

α Conversion

Examples:

$$\begin{aligned} & x (\lambda x y. x y) \\ =_{\alpha} & x (\lambda y x. y x) \\ =_{\alpha} & x (\lambda z y. z y) \\ \neq_{\alpha} & z (\lambda z y. z y) \\ \neq_{\alpha} & z (\lambda x x. x x) \end{aligned}$$

Back to β

We have defined β reduction: $\longrightarrow_{\beta}$

Some notation and concepts:

- **β conversion:** $s =_{\beta} t$ iff $\exists n. s \longrightarrow_{\beta}^* n \wedge t \longrightarrow_{\beta}^* n$

Back to β

We have defined β reduction: $\longrightarrow_{\beta}$

Some notation and concepts:

- β **conversion**: $s =_{\beta} t$ iff $\exists n. s \longrightarrow_{\beta}^* n \wedge t \longrightarrow_{\beta}^* n$
- t is **reducible** if there is an s such that $t \longrightarrow_{\beta} s$

Back to β

We have defined β reduction: $\longrightarrow_{\beta}$

Some notation and concepts:

- β **conversion**: $s =_{\beta} t$ iff $\exists n. s \longrightarrow_{\beta}^* n \wedge t \longrightarrow_{\beta}^* n$
- t is **reducible** if there is an s such that $t \longrightarrow_{\beta} s$
- $(\lambda x. s) t$ is called a **redex** (reducible expression)

Back to β

We have defined β reduction: $\longrightarrow_{\beta}$

Some notation and concepts:

- β **conversion**: $s =_{\beta} t$ iff $\exists n. s \longrightarrow_{\beta}^* n \wedge t \longrightarrow_{\beta}^* n$
- t is **reducible** if there is an s such that $t \longrightarrow_{\beta} s$
- $(\lambda x. s) t$ is called a **redex** (reducible expression)
- t is reducible iff it contains a redex

Back to β

We have defined β reduction: $\longrightarrow_{\beta}$

Some notation and concepts:

- β **conversion**: $s =_{\beta} t$ iff $\exists n. s \longrightarrow_{\beta}^* n \wedge t \longrightarrow_{\beta}^* n$
- t is **reducible** if there is an s such that $t \longrightarrow_{\beta} s$
- $(\lambda x. s) t$ is called a **redex** (reducible expression)
- t is reducible iff it contains a redex
- if it is not reducible, t is in **normal form**

Does every λ term have a normal form?

Example:

$$(\lambda x. x x) (\lambda x. x x) \longrightarrow_{\beta}$$

Does every λ term have a normal form?

Example:

$$\begin{aligned} (\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \\ (\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \end{aligned}$$

Does every λ term have a normal form?

No!

Example:

$$\begin{aligned}(\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \\(\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \\(\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \dots\end{aligned}$$

Does every λ term have a normal form?

No!

Example:

$$\begin{aligned}(\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \\(\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \\(\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \dots\end{aligned}$$

$$\text{(but: } (\lambda x y. y) ((\lambda x. x x) (\lambda x. x x)) \longrightarrow_{\beta} \lambda y. y \text{)}$$

Does every λ term have a normal form?

No!

Example:

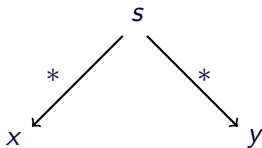
$$\begin{aligned}(\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \\(\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \\(\lambda x. x x) (\lambda x. x x) &\longrightarrow_{\beta} \dots\end{aligned}$$

$$\text{(but: } (\lambda x y. y) ((\lambda x. x x) (\lambda x. x x)) \longrightarrow_{\beta} \lambda y. y \text{)}$$

λ calculus is not terminating

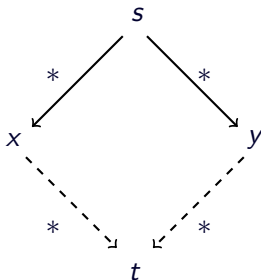
β reduction is confluent

Confluence: $s \longrightarrow_{\beta}^* x \wedge s \longrightarrow_{\beta}^* y \implies \exists t. x \longrightarrow_{\beta}^* t \wedge y \longrightarrow_{\beta}^* t$



β reduction is confluent

Confluence: $s \longrightarrow_{\beta}^* x \wedge s \longrightarrow_{\beta}^* y \implies \exists t. x \longrightarrow_{\beta}^* t \wedge y \longrightarrow_{\beta}^* t$



Order of reduction does not matter for result
Normal forms in λ calculus are unique

β reduction is confluent

Example:

$(\lambda x. y. y) ((\lambda x. x x) a)$

$(\lambda x. y. y) ((\lambda x. x x) a)$

β reduction is confluent

Example:

$$(\lambda x y. y) ((\lambda x. x x) a) \longrightarrow_{\beta} (\lambda x y. y) (a a)$$

$$(\lambda x y. y) ((\lambda x. x x) a) \longrightarrow_{\beta} \lambda y. y$$

β reduction is confluent

Example:

$$(\lambda x y. y) ((\lambda x. x x) a) \longrightarrow_{\beta} (\lambda x y. y) (a a) \longrightarrow_{\beta} \lambda y. y$$

$$(\lambda x y. y) ((\lambda x. x x) a) \longrightarrow_{\beta} \lambda y. y$$

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

`true` $\equiv \lambda x y. x$

`false` $\equiv \lambda x y. y$

`if` $\equiv \lambda z x y. z x y$

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

$\text{true} \equiv \lambda x y. x$

$\text{false} \equiv \lambda x y. y$

$\text{if} \equiv \lambda z x y. z x y$

$\text{if true } x y \longrightarrow_{\beta}^* x$

$\text{if false } x y \longrightarrow_{\beta}^* y$

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

$\text{true} \equiv \lambda x y. x$

$\text{false} \equiv \lambda x y. y$

$\text{if} \equiv \lambda z x y. z x y$

$\text{if true } x y \longrightarrow_{\beta}^* x$

$\text{if false } x y \longrightarrow_{\beta}^* y$

Now, not, and, or, etc is easy:

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

$\text{true} \equiv \lambda x y. x$

$\text{if true } x y \longrightarrow_{\beta}^* x$

$\text{false} \equiv \lambda x y. y$

$\text{if false } x y \longrightarrow_{\beta}^* y$

$\text{if} \equiv \lambda z x y. z x y$

Now, not, and, or, etc is easy:

$\text{not} \equiv \lambda x. \text{if } x \text{ false true}$

$\text{and} \equiv \lambda x y. \text{if } x y \text{ false}$

$\text{or} \equiv \lambda x y. \text{if } x \text{ true } y$

Fix Points

$$(\lambda x f. f (x x f)) (\lambda x f. f (x x f)) t \longrightarrow_{\beta}$$

Fix Points

$$\begin{aligned} & (\lambda x f. f (x x f)) (\lambda x f. f (x x f)) t \longrightarrow_{\beta} \\ & (\lambda f. f ((\lambda x f. f (x x f)) (\lambda x f. f (x x f)) f)) t \longrightarrow_{\beta} \end{aligned}$$

Fix Points

$$\begin{aligned} & (\lambda x. f. f (x \times f)) \ (\lambda x. f. f (x \times f)) \ t \longrightarrow_{\beta} \\ & (\lambda f. f \ ((\lambda x. f. f (x \times f)) \ (\lambda x. f. f (x \times f)) \ f)) \ t \longrightarrow_{\beta} \\ & t \ ((\lambda x. f. f (x \times f)) \ (\lambda x. f. f (x \times f)) \ t) \end{aligned}$$

Fix Points

$$\begin{aligned} & (\lambda x f. f (x x f)) (\lambda x f. f (x x f)) t \longrightarrow_{\beta} \\ & (\lambda f. f ((\lambda x f. f (x x f)) (\lambda x f. f (x x f)) f)) t \longrightarrow_{\beta} \\ & t ((\lambda x f. f (x x f)) (\lambda x f. f (x x f)) t) \end{aligned}$$

$$\begin{aligned} \mu &= (\lambda x f. f (x x f)) (\lambda x f. f (x x f)) \\ \mu t &\longrightarrow_{\beta} t (\mu t) \longrightarrow_{\beta} t (t (\mu t)) \longrightarrow_{\beta} t (t (t (\mu t))) \longrightarrow_{\beta} \dots \end{aligned}$$

Fix Points

$$\begin{aligned} & (\lambda x f. f (x x f)) (\lambda x f. f (x x f)) t \longrightarrow_{\beta} \\ & (\lambda f. f ((\lambda x f. f (x x f)) (\lambda x f. f (x x f)) f)) t \longrightarrow_{\beta} \\ & t ((\lambda x f. f (x x f)) (\lambda x f. f (x x f)) t) \end{aligned}$$

$$\begin{aligned} \mu &= (\lambda x f. f (x x f)) (\lambda x f. f (x x f)) \\ \mu t &\longrightarrow_{\beta} t (\mu t) \longrightarrow_{\beta} t (t (\mu t)) \longrightarrow_{\beta} t (t (t (\mu t))) \longrightarrow_{\beta} \dots \end{aligned}$$

$(\lambda x f. f (x x f)) (\lambda x f. f (x x f))$ is Turing's fix point operator

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

- **Russell** (1901): Paradox $R \equiv \{X | X \notin X\}$
- **Church** (1930): λ calculus as logic, true, false, $\wedge$, ... as λ terms

Russell's paradox in λ -calculus:

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

- **Russell** (1901): Paradox $R \equiv \{X | X \notin X\}$
- **Church** (1930): λ calculus as logic, true, false, $\wedge$, ... as λ terms

Russell's paradox in λ -calculus:

you can write $R \equiv \lambda x. \text{not } (x\ x)$

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

- **Russell** (1901): Paradox $R \equiv \{X | X \notin X\}$
- **Church** (1930): λ calculus as logic, true, false, $\wedge$, ... as λ terms

Russell's paradox in λ -calculus:

you can write $R \equiv \lambda x. \text{not } (x\ x)$
and get $(R\ R) =_{\beta} \text{not } (R\ R)$

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

- **Russell** (1901): Paradox $R \equiv \{X | X \notin X\}$
- **Church** (1930): λ calculus as logic, true, false, $\wedge$, ... as λ terms

Russell's paradox in λ -calculus:

you can write $R \equiv \lambda x. \text{not } (x \ x)$

and get $(R \ R) =_{\beta} \text{not } (R \ R)$

because $(R \ R) = (\lambda x. \text{not } (x \ x)) \ R \longrightarrow_{\beta} \text{not } (R \ R)$

Summary of what we learned so far

- λ calculus syntax
- free variables, substitution
- α conversion
- β reduction
- β reduction is confluent
- λ calculus is very expressive (Turing complete)
- λ calculus is inconsistent (as a logic)

Exercise

- Reduce $(\lambda x. y (\lambda v. x v)) (\lambda y. v y)$ to β normal form.

λ calculus is inconsistent

Can find term R such that $R R =_{\beta} \text{not}(R R)$

λ calculus is inconsistent

Can find term R such that $R R =_{\beta} \text{not}(R R)$

Solution: rule out ill-formed terms by using types.
(Church 1940)

Introducing types

Idea: assign a type to each “sensible” λ term.

Examples:

Introducing types

Idea: assign a type to each “sensible” λ term.

Examples:

- for *term t has type α* write $t :: \alpha$

Introducing types

Idea: assign a type to each “sensible” λ term.

Examples:

- for *term* t has type α write $t :: \alpha$
- if x has type α then $\lambda x. x$ is a function from α to α
Write: $(\lambda x. x) :: \alpha \Rightarrow \alpha$

Introducing types

Idea: assign a type to each “sensible” λ term.

Examples:

- for *term* t has type α write $t :: \alpha$
- if x has type α then $\lambda x. x$ is a function from α to α
Write: $(\lambda x. x) :: \alpha \Rightarrow \alpha$
- for $s\ t$ to be sensible:
 s must be a function
 t must be right type for parameter
 If $s :: \alpha \Rightarrow \beta$ and $t :: \alpha$ then $(s\ t) :: \beta$

That's it!

That's it!

Now Formally

Syntax for $\lambda^{\rightarrow}$

Terms: $t ::= v \mid c \mid (t\ t) \mid (\lambda x. t)$
 $v, x \in V, \quad c \in C, \quad V, C$ sets of names

Types: $\tau ::= b \mid \nu \mid \tau \Rightarrow \tau$
 $b \in \{\text{bool}, \text{int}, \dots\}$ base types
 $\nu \in \{\alpha, \beta, \dots\}$ type variables

$$\alpha \Rightarrow \beta \Rightarrow \gamma \quad = \quad \alpha \Rightarrow (\beta \Rightarrow \gamma)$$

Syntax for $\lambda^{\rightarrow}$

Terms: $t ::= v \mid c \mid (t\ t) \mid (\lambda x. t)$
 $v, x \in V, \quad c \in C, \quad V, C$ sets of names

Types: $\tau ::= b \mid \nu \mid \tau \Rightarrow \tau$
 $b \in \{\text{bool}, \text{int}, \dots\}$ base types
 $\nu \in \{\alpha, \beta, \dots\}$ type variables

$$\alpha \Rightarrow \beta \Rightarrow \gamma \quad = \quad \alpha \Rightarrow (\beta \Rightarrow \gamma)$$

Context Γ :

Γ : function from variable and constant names to types.

Syntax for $\lambda^{\rightarrow}$

Terms: $t ::= v \mid c \mid (t\ t) \mid (\lambda x. t)$
 $v, x \in V, \quad c \in C, \quad V, C$ sets of names

Types: $\tau ::= b \mid \nu \mid \tau \Rightarrow \tau$
 $b \in \{\text{bool}, \text{int}, \dots\}$ base types
 $\nu \in \{\alpha, \beta, \dots\}$ type variables

$$\alpha \Rightarrow \beta \Rightarrow \gamma = \alpha \Rightarrow (\beta \Rightarrow \gamma)$$

Context Γ :

Γ : function from variable and constant names to types.

Term t has type τ in context Γ : $\Gamma \vdash t :: \tau$

Examples

$$\Gamma \vdash (\lambda x. x) :: \alpha \Rightarrow \alpha$$

Examples

$$\Gamma \vdash (\lambda x. x) :: \alpha \Rightarrow \alpha$$
$$[y \leftarrow \text{int}] \vdash y :: \text{int}$$

Examples

$$\Gamma \vdash (\lambda x. x) :: \alpha \Rightarrow \alpha$$
$$[y \leftarrow \text{int}] \vdash y :: \text{int}$$
$$[z \leftarrow \text{bool}] \vdash (\lambda y. y) z :: \text{bool}$$

Examples

$$\Gamma \vdash (\lambda x. x) :: \alpha \Rightarrow \alpha$$
$$[y \leftarrow \text{int}] \vdash y :: \text{int}$$
$$[z \leftarrow \text{bool}] \vdash (\lambda y. y) z :: \text{bool}$$
$$[] \vdash \lambda f x. f x :: (\alpha \Rightarrow \beta) \Rightarrow \alpha \Rightarrow \beta$$

Examples

$$\Gamma \vdash (\lambda x. x) :: \alpha \Rightarrow \alpha$$

$$[y \leftarrow \text{int}] \vdash y :: \text{int}$$

$$[z \leftarrow \text{bool}] \vdash (\lambda y. y) z :: \text{bool}$$

$$[] \vdash \lambda f x. f x :: (\alpha \Rightarrow \beta) \Rightarrow \alpha \Rightarrow \beta$$

A term t is **well typed** or **type correct**
if there are Γ and τ such that $\Gamma \vdash t :: \tau$

Type Checking Rules

Variables:

$$\overline{\Gamma \vdash x :: \Gamma(x)}$$

Type Checking Rules

Variables:

$$\overline{\Gamma \vdash x :: \Gamma(x)}$$

Application:

$$\frac{}{\Gamma \vdash (t_1 \ t_2) :: \tau}$$

Type Checking Rules

Variables:

$$\overline{\Gamma \vdash x :: \Gamma(x)}$$

Application:

$$\frac{\Gamma \vdash t_1 :: \tau_2 \Rightarrow \tau \quad \Gamma \vdash t_2 :: \tau_2}{\Gamma \vdash (t_1 \ t_2) :: \tau}$$

Type Checking Rules

Variables: $\overline{\Gamma \vdash x :: \Gamma(x)}$

Application:
$$\frac{\Gamma \vdash t_1 :: \tau_2 \Rightarrow \tau \quad \Gamma \vdash t_2 :: \tau_2}{\Gamma \vdash (t_1 \ t_2) :: \tau}$$

Abstraction:
$$\overline{\Gamma \vdash (\lambda x. \ t) :: \tau_x \Rightarrow \tau}$$

Type Checking Rules

Variables:
$$\overline{\Gamma \vdash x :: \Gamma(x)}$$

Application:
$$\frac{\Gamma \vdash t_1 :: \tau_2 \Rightarrow \tau \quad \Gamma \vdash t_2 :: \tau_2}{\Gamma \vdash (t_1 \ t_2) :: \tau}$$

Abstraction:
$$\frac{\Gamma[x \leftarrow \tau_x] \vdash t :: \tau}{\Gamma \vdash (\lambda x. \ t) :: \tau_x \Rightarrow \tau}$$

What about β reduction?

What about β reduction?

Definition of β reduction stays the same.

What about β reduction?

Definition of β reduction stays the same.

Fact: Well typed terms stay well typed after β reduction

Formally: $\Gamma \vdash s :: \tau \wedge s \longrightarrow_{\beta} t \implies \Gamma \vdash t :: \tau$

What about β reduction?

Definition of β reduction stays the same.

Fact: Well typed terms stay well typed after β reduction

Formally: $\Gamma \vdash s :: \tau \wedge s \longrightarrow_{\beta} t \implies \Gamma \vdash t :: \tau$

This property is called **subject reduction**

Exercise

Derive the normal form by β -reducing the following term:

$$(\lambda f\ x. f\ x)(\lambda y. y)\ z$$

What about termination?

What about termination?

β reduction in $\lambda^{\rightarrow}$ always terminates.



(Alan Turing, 1942)

What about termination?

β reduction in $\lambda^{\rightarrow}$ always terminates.



(Alan Turing, 1942)

- $=_{\beta}$ is decidable

To decide if $s =_{\beta} t$, reduce s and t to normal form (always exists, because $\longrightarrow_{\beta}$ terminates), and compare result.

What about termination?

β reduction in $\lambda^{\rightarrow}$ always terminates.



(Alan Turing, 1942)

- $=_{\beta}$ is decidable

To decide if $s =_{\beta} t$, reduce s and t to normal form (always exists, because $\rightarrow_{\beta}$ terminates), and compare result.

- $=_{\alpha\beta}$ is decidable

We have learned so far...

- Simply typed lambda calculus: $\lambda^{\rightarrow}$
- Typing rules for $\lambda^{\rightarrow}$, type variables, type contexts
- β -reduction in $\lambda^{\rightarrow}$ satisfies subject reduction
- β -reduction in $\lambda^{\rightarrow}$ always terminates

Defining Higher Order Logic

What is Higher Order Logic?

- **Propositional Logic:**
 - no quantifiers
 - all variables have type bool

What is Higher Order Logic?

- **Propositional Logic:**
 - no quantifiers
 - all variables have type bool
- **First Order Logic:**
 - quantification over values, but not over functions and predicates,
 - terms and formulas syntactically distinct

What is Higher Order Logic?

- **Propositional Logic:**

- no quantifiers
- all variables have type bool

- **First Order Logic:**

- quantification over values, but not over functions and predicates,
- terms and formulas syntactically distinct

- **Higher Order Logic:**

- quantification over everything, including predicates
- consistency by types
- formula = term of type bool
- definition built on $\lambda^{\rightarrow}$ with certain default types and constants

Defining Higher Order Logic

Default types:

Defining Higher Order Logic

Default types:

`bool`

Defining Higher Order Logic

Default types:

bool $- \Rightarrow -$

Defining Higher Order Logic

Default types:

bool

$- \Rightarrow -$

ind

Defining Higher Order Logic

Default types:

bool $_ \Rightarrow _$ ind

- **bool**
- $\Rightarrow$ sometimes called *fun*

Defining Higher Order Logic

Default types:

bool $_ \Rightarrow _$ ind

- **bool**
- $\Rightarrow$ sometimes called *fun*

Default Constants:

Defining Higher Order Logic

Default types:

`bool` `_  $\Rightarrow$  _` `ind`

- `bool`
- `$\Rightarrow$` sometimes called *fun*

Default Constants:

`=` `::` `$\alpha \Rightarrow \alpha \Rightarrow bool$`

Higher Order Abstract Syntax

Problem: Define syntax for binders like $\forall$ and $\exists$

Higher Order Abstract Syntax

Problem: Define syntax for binders like $\forall$ and $\exists$

One approach: $\forall :: \text{var} \Rightarrow \text{term} \Rightarrow \text{bool}$

Drawback: need to think about substitution, α conversion again.

Higher Order Abstract Syntax

Problem: Define syntax for binders like $\forall$ and $\exists$

One approach: $\forall :: var \Rightarrow term \Rightarrow bool$

Drawback: need to think about substitution, α conversion again.

But: Already have binder, substitution, α conversion in host language

λ

Higher Order Abstract Syntax

Problem: Define syntax for binders like $\forall$ and $\exists$

One approach: $\forall :: \text{var} \Rightarrow \text{term} \Rightarrow \text{bool}$

Drawback: need to think about substitution, α conversion again.

But: Already have binder, substitution, α conversion in host language

λ

So: Use λ to encode all other binders.

Higher Order Abstract Syntax

Example:

$ALL :: (\alpha \Rightarrow bool) \Rightarrow bool$

HOAS

usual syntax

Higher Order Abstract Syntax

Example:

$ALL :: (\alpha \Rightarrow bool) \Rightarrow bool$

HOAS

usual syntax

$ALL (\lambda x. x = 2)$

Higher Order Abstract Syntax

Example:

$ALL :: (\alpha \Rightarrow bool) \Rightarrow bool$

HOAS

usual syntax

$ALL (\lambda x. x = 2)$

$\forall x. x = 2$

Higher Order Abstract Syntax

Example:

$ALL :: (\alpha \Rightarrow bool) \Rightarrow bool$

HOAS

usual syntax

$ALL (\lambda x. x = 2)$

$\forall x. x = 2$

$ALL P$

Higher Order Abstract Syntax

Example:

$ALL :: (\alpha \Rightarrow bool) \Rightarrow bool$

HOAS

usual syntax

$ALL (\lambda x. x = 2)$

$\forall x. x = 2$

$ALL P$

$\forall x. P\ x$

Back to HOL

Base: $bool, \Rightarrow, ind$ =

And the rest is

Back to HOL

Base: $bool, \Rightarrow, ind$ =

And the rest is definitions:

$\text{True} \equiv$

$P \wedge Q \equiv$

$P \longrightarrow Q \equiv$

$\text{All } P \equiv$

$\text{Ex } P \equiv$

$\text{False} \equiv$

$\neg P \equiv$

$P \vee Q \equiv$

Back to HOL

Base: $bool, \Rightarrow, ind$ =

And the rest is definitions:

$\text{True} \equiv (\lambda x. x) = (\lambda x. x)$

$P \wedge Q \equiv$

$P \longrightarrow Q \equiv$

$\text{All } P \equiv$

$\text{Ex } P \equiv$

$\text{False} \equiv$

$\neg P \equiv$

$P \vee Q \equiv$

Back to HOL

Base: $bool, \Rightarrow, ind$ =

And the rest is definitions:

$\text{True} \equiv (\lambda x. x) = (\lambda x. x)$

$P \wedge Q \equiv \lambda p q. ((\lambda f. f \ p \ q) = (\lambda f. f \ \text{True} \ \text{True}))$

$P \longrightarrow Q \equiv$

$\text{All } P \equiv$

$\text{Ex } P \equiv$

$\text{False} \equiv$

$\neg P \equiv$

$P \vee Q \equiv$

Back to HOL

Base: $bool, \Rightarrow, ind$ =

And the rest is definitions:

$True \equiv (\lambda x. x) = (\lambda x. x)$

$P \wedge Q \equiv \lambda p q. ((\lambda f. f p q) = (\lambda f. f True True))$

$P \longrightarrow Q \equiv \lambda p q. ((p \wedge q) = p)$

$All P \equiv$

$Ex P \equiv$

$False \equiv$

$\neg P \equiv$

$P \vee Q \equiv$

Back to HOL

Base: $bool, \Rightarrow, ind$ =

And the rest is definitions:

$\text{True} \equiv (\lambda x. x) = (\lambda x. x)$

$P \wedge Q \equiv \lambda p q. ((\lambda f. f p q) = (\lambda f. f \text{True} \text{True}))$

$P \longrightarrow Q \equiv \lambda p q. ((p \wedge q) = p)$

$\text{All } P \equiv P = (\lambda x. \text{True})$

$\text{Ex } P \equiv \forall Q. (\forall x. P x \longrightarrow Q) \longrightarrow Q$

$\text{False} \equiv \forall P. P$

$\neg P \equiv P \longrightarrow \text{False}$

$P \vee Q \equiv \forall R. (P \longrightarrow R) \longrightarrow (Q \longrightarrow R) \longrightarrow R$

In conclusion

That was HOL from scratch! But we skipped some steps.

For example, what did we assume about $=$?

Anyhow, the resulting logic is consistent. (How do we know that?)

We have learned so far ...

- HOL
- Higher Order Abstract Syntax
- Defining HOL

More on HOL in COMP4161.

Notes (Week 10 Monday)

These are the notes I wrote during the lecture.

This lecture is too long. Anything I don't get to, won't come on the exam.

Not every program that has a valid total correctness triple will **always** terminate. It **will** terminate if the precondition holds, but not necessarily if it's false.

Here's a valid total correctness triple:

$$\vdash [\perp] \text{ while true do skip od } [\top]$$

$$\{\phi\} P \{\psi\} \quad \{\phi\} Q \{\psi\}$$

$$\{\phi\} P + Q \{\psi\}$$

Formulas ϕ as part of $\mathcal{L}^+$ programs

- In partial correctness,
 - Perform a miracle that causes ϕ to be true
- In total correctness,
 - Abort unless ϕ is true

$$\{\phi\} \psi \{\phi \wedge \psi\}$$

...after we execute ψ , ψ is (somehow) true!

What then does $\perp$ mean?

- In partial correctness,
 - Make everything true!
- In total correctness world, abort

$P + \perp$	Either execute P, or make everything true
$\perp; P$	Make everything true, then execute P
$P + \top$	Either execute P, or make true true (aka do nothing)
$\top; P$	Make true true, then execute P

Give function calls meaning by using substitution

```
int f(int n) {
  if n = 0 then 1 else 0
}
```

Performing the function call

$f(0)$

The first step is to enter the body of the function
 Replace the function call with the body, but with
 all occurrences of the parameter(s) replaced
 by the argument(s)

Parameters: placeholders in the definition

Arguments: what you call the function with

$f(0)$

→

$(\text{if } n = 0 \text{ then } 1 \text{ else } 0)[n:=0]$

In the λ -calculus, you can write functions.

You can call functions,

but there's no data (other than functions)

(higher-order functions)

A computational model is Turing-complete

if it can represent every computable function.

What is a computable function?

Intuitively, a mathematical function (from $\mathbb{N}$ s $\mathbb{N}$ s)

such that an algorithm that is guaranteed to terminate
 can implement it

$(\lambda x. e)$ function abstraction
 a function with no name (anonymous function)
 with parameter x
 and body e

Function application

$f \ a$ function f applied to argument a

Reduction (computation),

because there's only functions,

there's only one computation step you can do:

call functions

$(\lambda x. x + 5) \ 3 \rightarrow (x + 5)[x:=3] = 3 + 5$

Unless we get to like slide 120,
the difference between variables and constants
doesn't really matter

$$\lambda y. x. x y$$

is sugar for

$$\lambda y. \lambda x. x y$$

$x y$	call x with argument y
$\lambda x. x y$	accepts an argument x , then calls that x with argument y
$\lambda y. \lambda x. x y$	accepts an argument y , and returns a function that accepts an argument x , then calls x with argument y

a function that accepts two arguments,
and calls the second with the first

Convention "currying" (named for Haskell Curry)

We don't have multi-argument functions.

But we don't need multi-argument functions.

Idea: encode $f(x,y) = x^y$
as $f = \lambda x. \lambda y. x^y$

We split the tuple and accept each component separately

The idea:

$$(\lambda x y. x^y) 3 4$$

I want this to reduce to

$$3^4$$

The way to get that is

$$\begin{aligned} & ((\lambda x y. x^y) 3) 4 \\ & \rightarrow (\lambda y. 3^y) 4 \\ & \rightarrow 3^4 \end{aligned}$$

Notice that

$$(\lambda x y. x^y) (3 4)$$

Is the wrong thing: 3 applied to 4 is probably not what we want

We only have functions,

but we want to encode data somehow to show that we can do useful computation.

In λ -calculus, it be what it do

Intuition: encode data in terms of its function,
 encode data as the thing we can do with it

What is a number? Ask Giuseppe Peano $Z, S Z, S(S Z)$

Peano gives a way to **construct** the natural numbers

..but here, we'll focus on how to **destruct** the natural numbers
 in other words: how to use them

When you have a number n , what do you with it?

idea: we use n to iterate some task n times (for-loop)

A Church numeral accepts two arguments:

$\lambda f x. \dots$

The first argument f represents the task to be repeated.

The second argument represents 0 , represents the initial object
 that the task should be performed on.

$0 = \lambda f x. x$

$1 = \lambda f x. f x$

$2 = \lambda f x. f(f x)$

$3 = \dots$

...so we can have numbers, but can we do stuff with the numbers?

The successor function $f(x) = x+1$ S

$\lambda n. \dots$ takes a number as input

$\lambda f x. \dots$ needs to return a number, which is a binary function
 $f(n f x)$

Addition $(\lambda x y. x + y)$

$(+) = \lambda n m. \lambda f x. n f (m f x)$

Let's do $1+1$. $(+) 1 1$

$(+) 1 1$

$= (\lambda n m. \lambda f x. n f (m f x)) 1 1$

$\rightarrow (\lambda m. \lambda f x. 1 f (m f x)) 1$

$\rightarrow \lambda f x. 1 f (1 f x)$

$= \lambda f x. 1 f ((\lambda f x. f x) f x)$

$\rightarrow \lambda f x. 1 f ((\lambda x. f x) x)$

$\rightarrow \lambda f x. 1 f (f x)$

$= \lambda f x. (\lambda f x. f x) f (f x)$

$\rightarrow \lambda f x. (\lambda x. f x) (f x)$

```
→ λf x. f (f x)
= 2
```

My goal here is not to convince you that you'd
 want to program in the λ -calculus
 The goal is to convince you that, in principle, you can

What do you *do* with booleans?

A: you choose a branch based on whether it's true or false
 (if, while)

We encode booleans as binary functions that return
 either the first or the second argument

```
true = λx y. x
false = λx y. y
```

A boolean is essentially the same thing as an if-then-else

Whenever you see

```
b x y
```

Think:

```
if b then x else y
```

```
¬ = λb. b false true
```

```
∧ = λp q. p q false
```

```
p q false    read    "if p then q else false"
```

We can have booleans and nats. Can we do loops?

Or more clearly: unbounded repetition

Normal form.

A λ -calculus term is in *normal form* if it contains no β -redex.

A β -redex is a β -reducible subterm.

β -reduction is the fancy name for "apply a function call"

```
(λx. t) t' →β t[x:=t']
```

A λ term is in normal form if there are no more unevaluated
 function calls left.

The (vanilla) λ -calculus allows you to perform β -reductions
 anywhere in a term, in any order.

$$(\lambda x. (\lambda x. t) x) ((\lambda x. y) z)$$

This term has three β -redexes.

So we have three choices of where to reduce:

- reduce the argument before the top-level function call
 - eager evaluation, call-by-value
- perform the function call **before** evaluating the argument
 - call-by-name (e.g. Haskell)
- start evaluating inside the function before we call the function
 - compiler optimisations

Theorem:

if a lambda-term has a normal form, it's unique.

To rephrase "can we have loops" more formally, we might ask:

Does **every** λ -term have a normal form?

$f(x) = f(x-1) + 1$ $\leftarrow$ we can't do that,
because functions don't have names

But we **can** encode it.

First: a useless example of a term with no normal form.

$$(\lambda x. x x) (\lambda x. x x) \mapsto_{\beta} (x x)[x := \lambda x. x x] =$$

$$(\lambda x. x x) (\lambda x. x x)$$

That shows us that we can have unproductive loops. But what about loops (or at least recursive functions) that actually do stuff?

It just so happens that λ -calculus has a number of

fixpoint combinators

Y-combinator

Turing's fixpoint combinator T

$$Y f \mapsto_{\beta} f(Y f) \mapsto_{\beta} f(f(Y f)) \mapsto_{\beta} f(f(f(Y f)))$$

Imagine f is something like

"if n is 0 then return otherwise keep going"

$$Y = \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))$$

$$(\lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))) g$$

$$\mapsto_{\beta} (\lambda x. g(x x)) (\lambda x. g(x x))$$

$$\mapsto_{\beta} g((\lambda x. g(x\ x)) (\lambda x. g(x\ x)))$$

$$\equiv g(Y\ g)$$

Fix points are nice, because they allow us to encode loops
 But they're also the reason why λ -calculus fails as a foundation of logic

$$Y\ \neg = \neg(Y\ \neg)$$

In other words, there exists a term that equals its own negation

The solution (that we didn't do, but see COMP3151, COMP4161) is to add a type system to the λ -calculus

One last convention: the choice of placeholders doesn't matter.

$$(\lambda x. x) = (\lambda y. y)$$

Intuition: functions are equal (formally, α -equivalent) if they differ only in choice of placeholders for λ -bound arguments

This means that you're allowed to (and should) rename stuff to avoid variable capture before β -reducing

$$(\lambda y\ x. x+y)\ (3+x) =_{\alpha} (\lambda y\ z. z+y)\ (3+x)$$

$$\mapsto_{\beta} \lambda z. z + (3+x)$$

Q: Is α -equivalence computable?

A: Yes. by a (mostly straightforward) syntax walk

System Modelling and Design

COMP2111



Johannes Åman Pohjola

Formal

System Modelling and Design COMP2111



Johannes Åman Pohjola

We'll learn to

model systems in a way that's unambiguous and mathematically precise.

We'll be able to

say what it means for a system to satisfy its specification,
and prove that it does so.

We'll need

a substantial toolbox of discrete math and formal logic.

Don't worry; we'll teach it, not assume it.

We'll learn to

model systems in a way that's unambiguous and mathematically precise.

We'll be able to

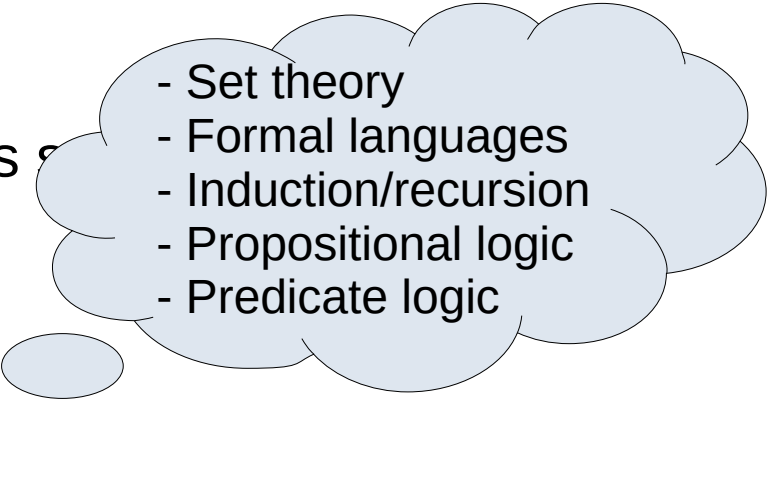
say what it means for a system to satisfy its spec

and prove that it does so.

We'll need

a substantial toolbox of discrete math and formal logic.

Don't worry; we'll teach it, not assume it.

- 
- Set theory
 - Formal languages
 - Induction/recursion
 - Propositional logic
 - Predicate logic

We'll learn to

model systems in a way that's unambiguous and mathematically precise.

We'll be able to

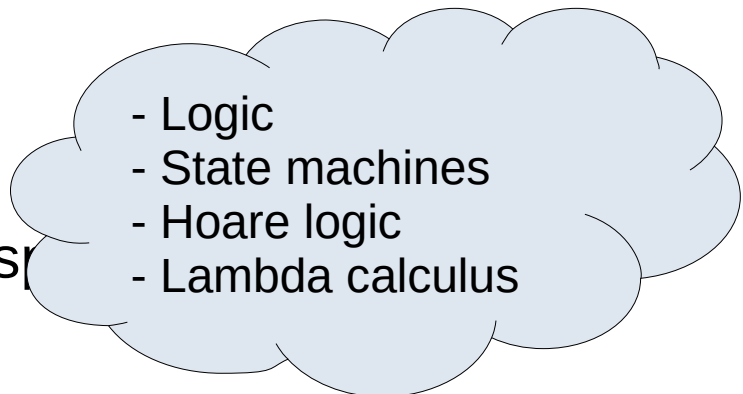
say what it means for a system to satisfy its spec

and prove that it does so.

We'll need

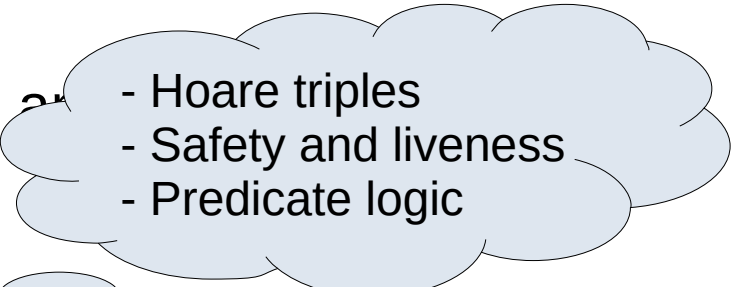
a substantial toolbox of discrete math and formal logic.

Don't worry; we'll teach it, not assume it.

- 
- Logic
 - State machines
 - Hoare logic
 - Lambda calculus

We'll learn to

model systems in a way that's unambiguous and mathematically precise.

- 
- Hoare triples
 - Safety and liveness
 - Predicate logic

We'll be able to

say what it means for a system to satisfy its specification,

and prove that it does so.

We'll need

a substantial toolbox of discrete math and formal logic.

Don't worry; we'll teach it, not assume it.

We'll learn to

model systems in a way that's unambiguous and mathematically precise.

We'll be able to

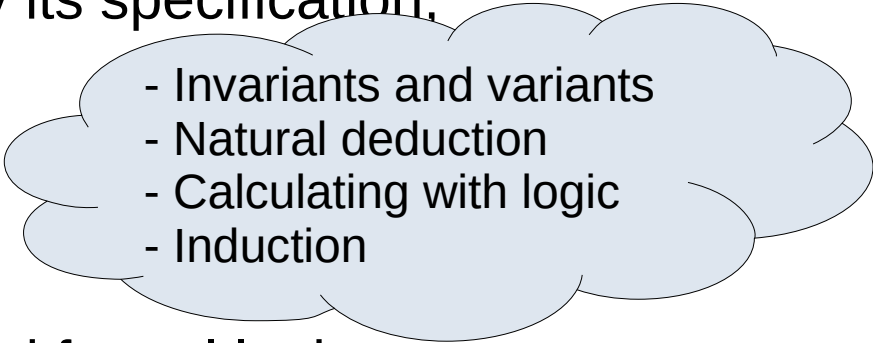
say what it means for a system to satisfy its specification,

and prove that it does so.

We'll need

a substantial toolbox of discrete math and formal logic.

Don't worry; we'll teach it, not assume it.

- 
- Invariants and variants
 - Natural deduction
 - Calculating with logic
 - Induction

Non-examples

September 1981

Transmission Control Protocol
Functional Specification

3.9. Event Processing

The processing depicted in this section is an example of one possible implementation. Other implementations may have slightly different processing sequences, but they should differ from those in this section only in detail, not in substance.

The activity of the TCP can be characterized as responding to events. The events that occur can be cast into three categories: user calls, arriving segments, and timeouts. This section describes the processing the TCP does in response to each of the events. In many cases the processing required depends on the state of the connection.

Events that occur:

User Calls

OPEN
SEND
RECEIVE
CLOSE
ABORT
STATUS

Arriving Segments

SEGMENT ARRIVES

Timeouts

USER TIMEOUT
RETRANSMISSION TIMEOUT
TIME-WAIT TIMEOUT

The model of the TCP/user interface is that user commands receive an immediate return and possibly a delayed response via an event or pseudo interrupt. In the following descriptions, the term "signal" means cause a delayed response.

Error responses are given as character strings. For example, user commands referencing connections that do not exist receive "error: connection not open".

Please note in the following that all arithmetic on sequence numbers, acknowledgment numbers, windows, et cetera, is modulo 2^{32} the size of the sequence number space. Also note that " $=<$ " means less than or equal to (modulo 2^{32}).

Non-examples

This RFC is a specification in English.

Natural language specs tend to have:

- Ambiguities
- Room for interpretation
- Important details in the writer's head absent from actual text.

September 1981

Transmission Control Protocol
Functional Specification

3.9. Event Processing

The processing depicted in this section is an example of one possible implementation. Other implementations may have slightly different processing sequences, but they should differ from those in this section only in detail, not in substance.

The activity of the TCP can be characterized as responding to events. The events that occur can be cast into three categories: user calls, arriving segments, and timeouts. This section describes the processing the TCP does in response to each of the events. In many cases the processing required depends on the state of the connection.

Events that occur:

User Calls

OPEN
SEND
RECEIVE
CLOSE
ABORT
STATUS

Arriving Segments

SEGMENT ARRIVES

Timeouts

USER TIMEOUT
RETRANSMISSION TIMEOUT
TIME-WAIT TIMEOUT

The model of the TCP/user interface is that user commands receive an immediate return and possibly a delayed response via an event or pseudo interrupt. In the following descriptions, the term "signal" means cause a delayed response.

Error responses are given as character strings. For example, user commands referencing connections that do not exist receive "error: connection not open".

Please note in the following that all arithmetic on sequence numbers, acknowledgment numbers, windows, et cetera, is modulo 2^{32} the size of the sequence number space. Also note that " $=<$ " means less than or equal to (modulo 2^{32}).

Non-examples

This RFC is a specification in English.

Natural language specs tend to have:

- Ambiguities
- Room for interpretation
- Important details in the writer's head absent from actual text.

You've experienced this yourself
now, in Assignment 2.

3.9. Event Processing

The processing depicted in this section is an example of one possible implementation. Other implementations may have slightly different processing sequences, but they should differ from those in this section only in detail, not in substance.

The activity of the TCP can be characterized as responding to events. The events that occur can be cast into three categories: user calls, arriving segments, and timeouts. This section describes the processing the TCP does in response to each of the events. In many cases the processing required depends on the state of the connection.

Events that occur:

User Calls

OPEN
SEND
RECEIVE
CLOSE
ABORT
STATUS

Arriving Segments

SEGMENT ARRIVES

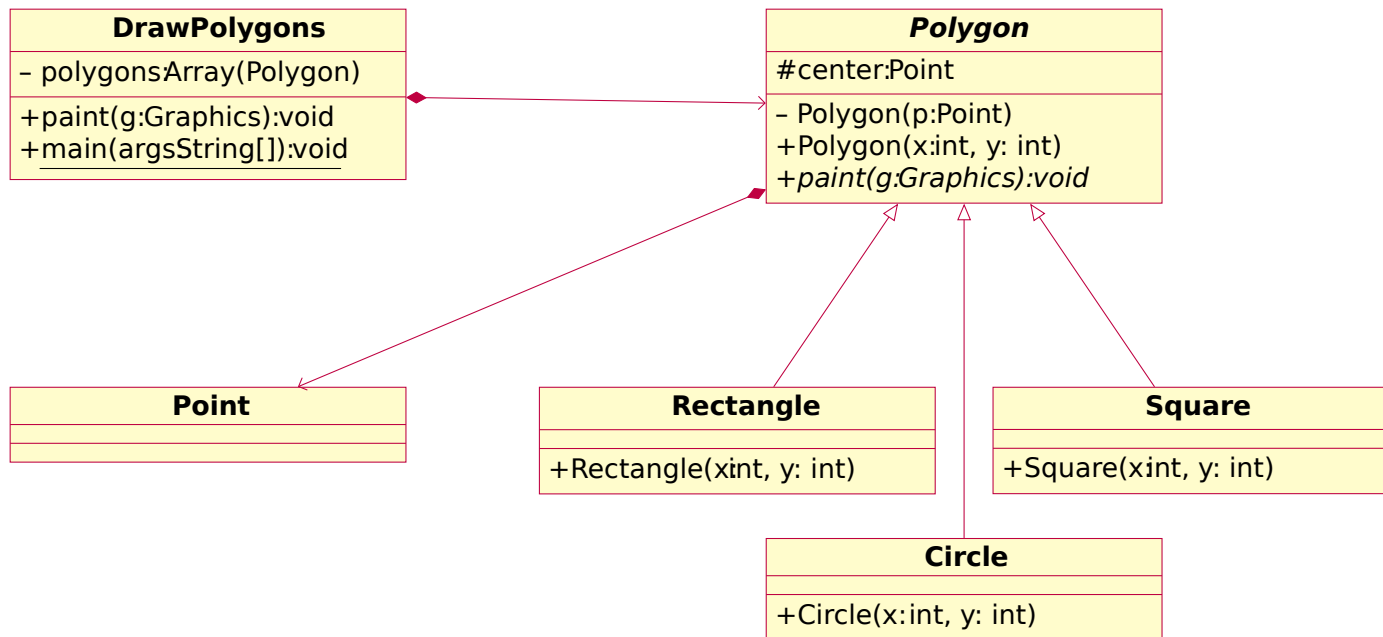
Timeouts

Commands receive an
an event or
term "signal"

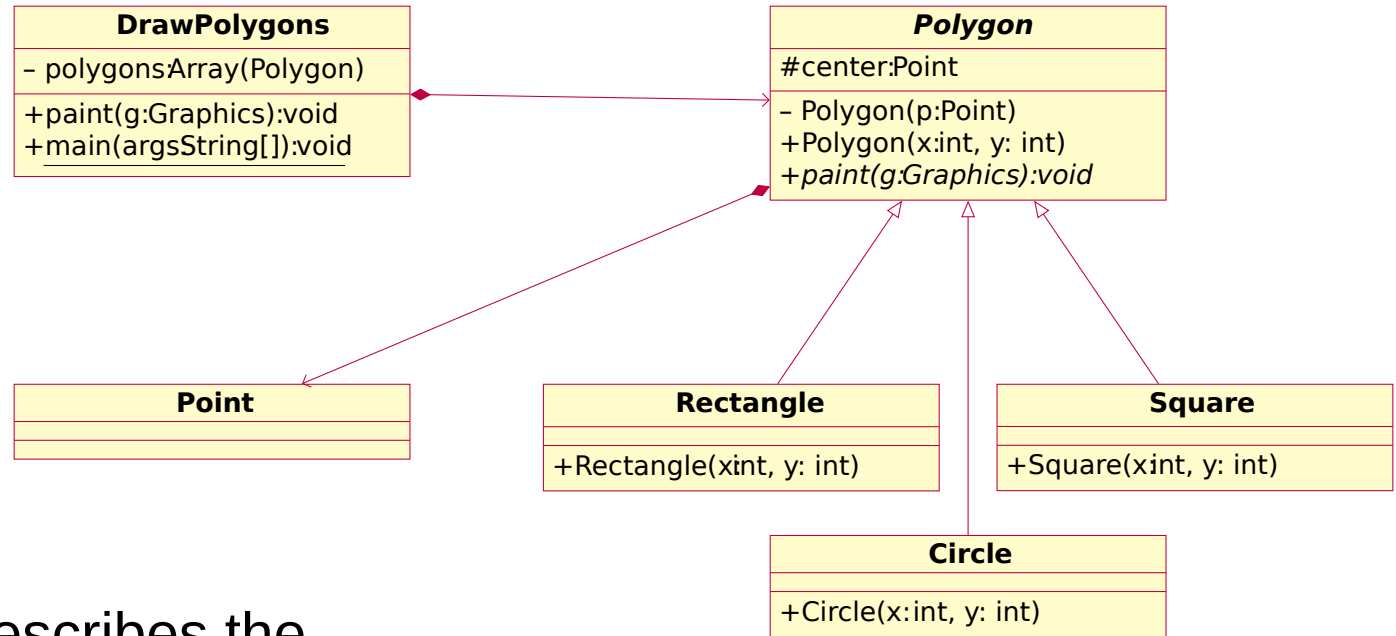
For example, user
not exist receive "error:

The following that all arithmetic on sequence numbers,
segment numbers, windows, et cetera, is modulo 2^{32} the size
of the sequence number space. Also note that " $=$ " means less than or
equal to (modulo 2^{32}).

Non-examples



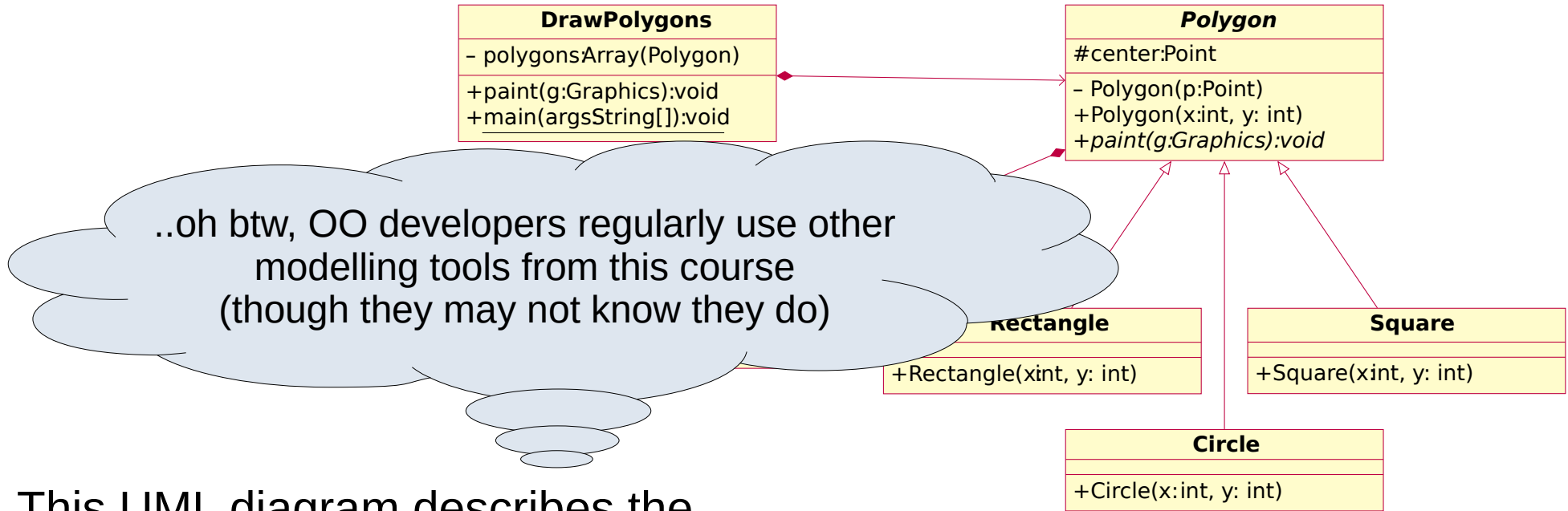
Non-examples



This UML diagram describes the *structure* of the system, not its behaviour.

Could we *give* it meaning using the tools from this course?

Non-examples



This UML diagram describes the *structure* of the system, not its behaviour.

From Java's library documentation

max

```
public static <T extends Object & Comparable<? super T>> T max(Collection<? extends T> coll)
```

Returns the maximum element of the given collection, according to the *natural ordering* of its elements. All elements in the collection must implement the `Comparable` interface. Furthermore, all elements in the collection must be *mutually comparable* (that is, `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the collection).

This method iterates over the entire collection, hence it requires time proportional to the size of the collection.

Type Parameters:

T - the class of the objects in the collection

Parameters:

coll - the collection whose maximum element is to be determined.

Returns:

the maximum element of the given collection, according to the *natural ordering* of its elements.

Throws:

`ClassCastException` - if the collection contains elements that are not *mutually comparable* (for example, strings and integers).

`NoSuchElementException` - if the collection is empty.

See Also:

`Comparable`

From Java's library documentation

max

```
public static <T extends Object & Comparable<? super T>> T max(Collection<? extends T> coll)
```

Returns the maximum element of the given collection, according to the *natural ordering* of its elements. All elements in the collection must implement the `Comparable` interface. Furthermore, all elements in the collection must be *mutually comparable* (that is, `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the collection).

This method iterates over the entire collection, hence it requires time proportional to the size of the collection.

Type Parameters:

T - the class of the objects in the collection

Parameters:

coll - the collection whose maximum element is to be determined.

Returns:

the maximum element of the given collection, according to the *natural ordering* of its elements.

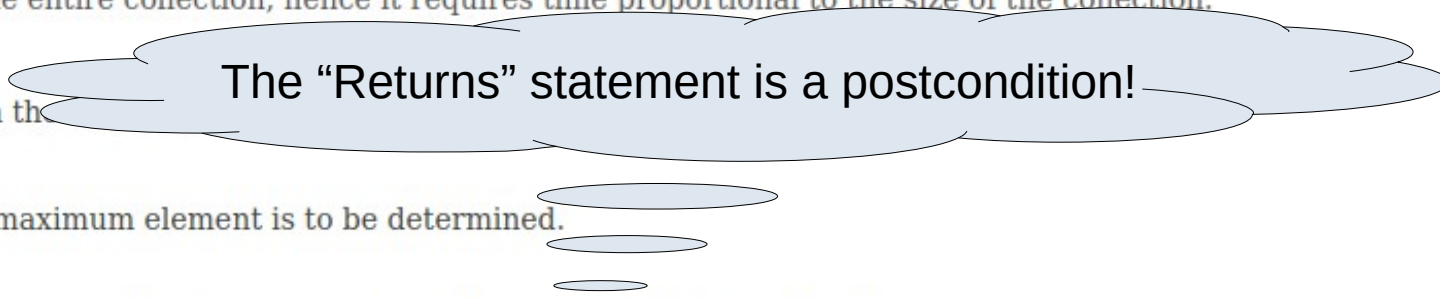
Throws:

`ClassCastException` - if the collection contains elements that are not *mutually comparable* (for example, strings and integers).

`NoSuchElementException` - if the collection is empty.

See Also:

`Comparable`



The "Returns" statement is a postcondition!

From Java's library documentation

max

```
public static <T extends Object & Comparable<? super T>> T max(Collection<? extends T> coll)
```

Returns the maximum element of the given collection, according to the *natural ordering* of its elements. All elements in the collection must implement the `Comparable` interface. Furthermore, all elements in the collection must be *mutually comparable* (that is, `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the collection).

This method iterates over the entire collection, hence it requires time proportional to the size of the collection.

Type Parameters:

T - the class of the objects in the collection

Parameters:

coll - the collection

Returns:

the maximum element of the collection, according to the *natural ordering* of its elements.

Throws:

`ClassCastException` - if the collection contains elements that are not *mutually comparable* (for example, strings and integers).

`NoSuchElementException` - if the collection is empty.

See Also:

`Comparable`

The “Throws” statement is a (negated) precondition!

From Java's library documentation

max

In OO design this is called *design by contract*, not Hoare triples, but it's the same thing.

You know how to *prove* this function is correct: Hoare logic.

Formal methods have found bugs in Java's standard library before.

Maybe you'll find the next bug? :)

`coll` - the collection whose maximum element is to be determined.

Returns:

the maximum element of the given collection, according to the *natural ordering* of its elements.

Throws:

`ClassCastException` - if the collection contains elements that are not *mutually comparable* (for example, strings and integers).

`NoSuchElementException` - if the collection is empty.

See Also:

`Comparable`

Practice exam

Exam coming up soon: May 3rd 8AM - May 4th 8AM

The 2021 exam is available for practice on the course website.

It's mostly representative of what you should expect. Any topic covered in the lectures and tutes may come up.

Related courses

COMP3151: Foundations of Concurrency

COMP3161: Concepts of Programming Languages

COMP3153: Algorithmic Program Verification

COMP4161: Advanced Topics in Software Verification

COMP9020: Foundations of Computer Science

COMP6721: (In-)formal Methods

That's all folks!

Thanks for attending the course.

Please don't forget to do the myExperience survey.

Come talk to me if you're interested in thesis projects :)

Notes (Week 10 Wednesday)

These are the notes I wrote during the lecture. Most of this is going over Sample Exam 2 from the sidebar.

The exam is open-book,
 – you can use any **passive** resource,
 but **active** resources are limited.

You can read the course website,
 your favourite book,
 your favourite website

You can't actively solicit answers,
 don't ask on Chegg, stackoverflowm
 ...

Exception:
 – You can ask on Ed, if you make the thread initially private.

Q1

```
0 = λf x. x
1 = λf x. f x
2 = λf x. f(f x)
```

```
true = λt f. t
false = λt f. f
```

A: $\lambda f x. x$
 represents both 0 and false.

Note: we don't differentiate
 between ** α -equivalent** terms

α -equivalence means

"equal up to choice of
 placeholders for bound
 variables"

Q2

CNF: a formula that is a
 conjunction of disjunction
 of atoms:

```
( _ v _ ) ∧ ( _ v _ v _ )
```

An atom is a variable,

or a negated variable

For CNF:

for each row where ϕ is false,
write a disjunction that captures
"how do we avoid this line"

$(A \vee B \vee \neg C) \wedge$
 $(A \vee \neg B \vee \neg C) \wedge$
 $(\neg A \vee B \vee C) \wedge$
 $(\neg A \vee \neg B \vee C)$

^ Done!

Remember: the CNF is not necessarily
unique, we could present
other variants.

Q3

Prove $(R^*)^* \subseteq R^*$

We have an inductively defined
relation R^* , and that means
we can use structural induction
over the inference rules.

Here's the induction principle:

If $x R^* y$, and if:
 Base cases: $\forall x. \phi(x, x)$
 and
 Inductive: $\forall x y z.$
 if $\phi(y, z)$
 and $x R y$
 then $\phi(x, z)$
 then
 $\phi(x, y)$

This induction principle lets us
prove things of the form:

$x R^* y \Rightarrow \phi(x, y)$

Theorem RTC_idem:

$(R^*)^* \subseteq R^*$

Proof

By definition of $\subseteq$, it suffices
to prove

$\forall x, y \in E.$
 $x (R^*)^* y \Rightarrow x R^* y$

Now we can use induction on

$$x (R^*)^* y$$

Base case: prove $x R^* x$
 immediate from the definition

Inductive case:

assume

(i) $x R^* y$

(ii) $x R^* z$ (IH)

It follows by transitivity of R^*
 that $x R^* z$.

QED

Q4

```

|-
1 | | B
2 | |-
3 | | | ¬B
4 | | |-
5 | | | ⊥      ¬-E 1,3
6 | | ¬¬B      ¬-I 3-5
7 | B → ¬¬B    →-I 1-6

```

Here I chose Fitch-style proof
 because I like it.

You can also do proof trees,
 or tables.

Q5

Sentence: formula with no free
 variables

(a) $(0 < 0) \wedge \neg(0 < 0)$

(b) $\exists x y z.$
 $\neg(x=y) \wedge \neg(x=z) \wedge \neg(y=z)$

Here's a tempting (but wrong) answer:

$$\exists x. x \neq 0 \wedge x \neq 1$$

Q: Why is this wrong?

A: 0 and 1 might denote the same
 element.

Here's a tempting (but wrong) fix:

$$0 \neq 1 \wedge \exists x. x \neq 0 \wedge x \neq 1$$

Q: Why is this wrong?

A: It's possible for the domain to have three elements, even if 0 and 1 happen to denote the same one.

Ex: domain: {a,b,c}

model of 0: a

model of 1: a

(c)

Observation:

the reals have numbers between 0 and 1, the ints don't

$$\neg \exists x. 0 < x \wedge x < 1$$

Q6

(a)

```
x y z  →  x y z
          x z y
          y x z
          y z x
          z x y **
          z y x
```

P =

```
skip
+
swap(y,z)
+
swap(x,y)
+
swap(x,y); swap(y,z)
+
swap(x,y); swap(x,z)
+
swap(x,z)
```

(b)

```
{x = X ∧ y = Y ∧ z = Z}
P
{
  sort[x,y,z] = sort[X,Y,Z]
}
```

^ the first sketch with sets doesn't account for multiplicity

(c)

Suppose $\models \{\phi\} P \{\psi\}$

By definition of $\models$, it holds that
if $\forall s s'$.

$(s, s') \in \llbracket P \rrbracket$

and $\llbracket \phi \rrbracket s = \text{true}$, then $\llbracket \psi \rrbracket s$

Note: $\llbracket P \rrbracket$ is the union

of the semantics of every
branch of the +

$\llbracket P \rrbracket = \llbracket \text{skip} \rrbracket \cup \llbracket \text{swap}(x, y) \rrbracket$
 $\cup \dots$

$\llbracket \text{skip} \rrbracket = I$ (the identity relation)

Therefore, as a special case,
if $\forall s s'$.

$(s, s') \in \llbracket \text{skip} \rrbracket$

and $\llbracket \phi \rrbracket s = \text{true}$, then $\llbracket \psi \rrbracket s$

...and that's exactly the definition
of $\models \{\phi\} \text{skip} \{\psi\}$

Basic idea:

if a Hoare triple holds about a
program, it must hold for every
possible behaviour of that program.

A possible behaviour of P is skip .

Therefore, the triple is valid
for skip too.

General:

If $\llbracket P \rrbracket \subseteq \llbracket Q \rrbracket$

Then every triple about Q is
also valid about P .

- (d) Any Hoare triple we write
also holds about skip .
The second requirement is
not a property of skip .
Therefore, no Hoare triple
can capture the second requirement.

(e)

$\{(s, s'):$
 $\text{sort}[s(x), s(y), s(z)] =$
 $\text{sort}[s'(x), s'(y), s'(z)] \wedge$
 $s \text{ and } s' \text{ agree on all variables}$
 $\text{other than } x, y, z\}$
 $\subseteq \llbracket P \rrbracket$

$S = \text{Pow } \{q_0, q_1, q_2\}$
 $A = \{s \in S \mid s \in q_1\}$

(a)

$\{q_0\} \xrightarrow{-0} \{q_0, q_1, q_2\}$
 $\{q_0\} \xrightarrow{-1} \{\}$
 $\{q_0, q_1, q_2\} \xrightarrow{-0} \{q_0, q_1, q_2\}$
 $\{q_0, q_1, q_2\} \xrightarrow{-1} \{q_2\}$

..from those states, no further transitions are possible

(b) is just A above

$\{q_1\}$
 $\{q_0, q_1\}$
 $\{q_1, q_2\}$
 $\{q_0, q_1, q_2\}$

Q8

(a) $s(y) = 1$
 $s(z) = 2$
 $s(b) = \top$
 $s(a) = \perp$

If I use rule (3) from this state,
the invariant gets violated

(e) (a--d) asks about a safety property

The presentation here is
(deliberately) obscure
The goal is to make sure
both flags are not true.

This is a lock implementation.
Lamport's bakery algorithm.

Q: for part a could u do $(\text{swap}(x,y) + \text{swap}(y,x) + \text{swap}(x,z) ; \text{swap}(x,y) + \text{swap}(y,x) + \text{swap}(x,z))$

A: Yes.

Tute (Week 10)

Church Numerals

This question will use the Church encoding of the natural numbers, as seen in the lectures. Here's a quick refresher:

$$\begin{aligned} 0 &= \lambda f x. x \\ 1 &= \lambda f x. f x \\ 2 &= \lambda f x. f(f x) \\ &\dots \end{aligned}$$

$$(+)=\lambda n m f x. n f (m f x)$$

1. You might expect that (+) is commutative, in the sense that (+) a b and (+) b a reduce to the same value for all a and b. That is not the case. Can you think of a counterexample? *Hint*: try feeding it things that aren't Church numerals.
2. How would you define multiplication of Church numerals in the lambda calculus?
3. How would you define exponentiation?
4. How would you define the predecessor function on Church numerals? In day-to-day mathematics this function could be defined as

$$\begin{aligned} \text{pre}(0) &= 0 \\ \text{pre}(n) &= n - 1 \quad \text{if } n > 0 \end{aligned}$$

Where the first clause is to prevent us from falling outside the natural numbers. This is sometimes called saturating subtraction.

This question is harder than the rest. You may want to save it for last.

5. Given a predecessor function as defined in the previous question, can you define saturating subtraction (where $n - m = 0$ if $m > n$)?
6. Can you define a function which, given two numbers, returns true iff they are equal?

Recall the Church encoding of the booleans:

$$\begin{aligned} \text{true} &= \lambda x y. x \\ \text{false} &= \lambda x y. y \end{aligned}$$

1. We have

$$(+) (\lambda f x. f) (\lambda f x. x x) \mapsto_{\beta}^* \lambda f x. f$$

but if we commute the arguments we get

$$(+) (\lambda f x. x x) (\lambda f x. f) \mapsto_{\beta}^* \lambda f x. f f$$

2. $(*) = \lambda n m. n ((+) m) 0$

3. $\text{exp} = \lambda n m. m ((* n) 1)$

4. $\text{pre} = \lambda n f x. (n (\lambda m g. g (m f)) (\lambda g. x)) (\lambda x. x)$

What's going on here? The intention is that the innermost expression

$n (\lambda m g. g (m f)) (\lambda g. x)$ is a copy of the numeral $n f x$, but with the outermost f abstracted. This will evaluate to an expression of the form $\lambda g. f^{n-1} x$. Instantiating g to be the identity function yields the predecessor of n , as desired.

5. $(-) = \lambda n m. m \text{ pre } n$

6. $\lambda n m. ((-) n m) (\lambda x. \text{false}) (((-) m n) (\lambda x. \text{false}) \text{true})$

The idea is that n equals m holds iff $n - m = 0$ and $m - n = 0$.

Fixpoint combinators

We briefly saw Turing's fixpoint combinator in the lectures, a lambda-calculus term which we will denote μ . Its exact definition is not important right now: what matters is that it satisfies

$$\mu f \mapsto_{\beta}^* f(\mu f)$$

1. Use μ to define a function which computes the n :th triangular number.

2. Use μ to define a function which computes the n :th Fibonacci number.

Hint: your function should take the form μf for some f that you'll have to write. Your f will be a function that takes two arguments: the first argument represents a continuation (intuitively, a function that you can call to initiate a recursive call) and the second argument will be the number n . You may find it helpful to use the auxiliary functions from the previous question.

1. $\mu (\lambda c n. (\text{eq } n 0) 0 ((+) n (c (\text{pre } n))))$

This is reminiscent of the following recursive function definition:

$$c(n) =$$

if $n = 0$ **then**
0
else
 $n + c(n - 1)$

2. $\mu (\lambda c n. (eq\ n\ 0)\ 0\ ((eq\ n\ 1)\ 1\ ((+)\ (c\ (pre\ (pre\ n)))\ (c\ (pre\ n)))))$

This is reminiscent of the following recursive function definition:

$$c(n) =$$

if $n = 0$ **then**
0
else if $n = 1$ **then**
1
else
 $c(n - 2) + c(n - 1)$